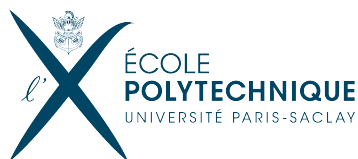


INF442  
Traitement des données massives



Frank Nielsen

# Table des matières

|          |                                                                                                                                         |           |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>I</b> | <b>Introduction au HPC avec MPI</b>                                                                                                     | <b>1</b>  |
| <b>1</b> | <b>Le calcul haute performance (le HPC)</b>                                                                                             | <b>3</b>  |
| 1.1      | Qu'est-ce que le Calcul Haute Performance ? . . . . .                                                                                   | 3         |
| 1.2      | Pourquoi le HPC ? . . . . .                                                                                                             | 4         |
| 1.3      | Les grandes données : les quatre V du <i>Big Data</i> . . . . .                                                                         | 5         |
| 1.4      | Paradigmes de programmation parallèle . . . . .                                                                                         | 5         |
| 1.5      | Granularité du parallélisme : petits grains et gros grains . . . . .                                                                    | 5         |
| 1.6      | L'architecture pour le HPC : mémoire et réseau . . . . .                                                                                | 6         |
| 1.7      | L'accélération (le <i>speed-up</i> ) . . . . .                                                                                          | 11        |
| 1.7.1    | L'accélération, l'efficacité et l'extensibilité . . . . .                                                                               | 11        |
| 1.7.2    | La loi d'Amdahl (taille des données fixée) . . . . .                                                                                    | 11        |
| 1.7.3    | La loi de Gustafson : tailles des données évoluant avec le nombre de proces-<br>seurs ( <i>scale speed-up</i> ) . . . . .               | 13        |
| 1.7.4    | Extensibilité et iso-efficacité . . . . .                                                                                               | 14        |
| 1.7.5    | Simulation de machines parallèles sur une machine séquentielle . . . . .                                                                | 14        |
| 1.7.6    | Le BigData et les systèmes de fichiers parallèles . . . . .                                                                             | 15        |
| 1.8      | Huit fausses idées sur le HPC . . . . .                                                                                                 | 15        |
| 1.9      | * Pour en savoir plus : notes, références et discussion . . . . .                                                                       | 17        |
| 1.10     | En résumé : ce qu'il faut retenir ! . . . . .                                                                                           | 17        |
| 1.11     | Exercices . . . . .                                                                                                                     | 18        |
| <b>2</b> | <b>Introduction à l'interface MPI : Message Passing Interface</b>                                                                       | <b>21</b> |
| 2.1      | L'interface MPI pour la programmation parallèle : communication par messages . . .                                                      | 21        |
| 2.2      | Modèles de programmation parallèle, fils de calcul et processus . . . . .                                                               | 22        |
| 2.3      | Les communications collectives . . . . .                                                                                                | 23        |
| 2.3.1    | Quatre opérations collectives de base . . . . .                                                                                         | 23        |
| 2.3.2    | Communications point à point bloquantes et non-bloquantes . . . . .                                                                     | 25        |
| 2.3.3    | Les situations de blocage ( <i>deadlocks</i> ) . . . . .                                                                                | 28        |
| 2.3.4    | Quelques hypothèses sur la concurrence : calculs locaux et communications<br>recouvrantes . . . . .                                     | 31        |
| 2.3.5    | Communications uni-directionnelles et bi-directionnelles . . . . .                                                                      | 32        |
| 2.3.6    | Les calculs globaux en MPI : les opérations de réduction ( <i>reduce</i> ) et de<br>(somme) préfixe parallèle ( <i>scan</i> ) . . . . . | 32        |
| 2.3.7    | * Les groupes de communication : les communicators . . . . .                                                                            | 35        |

|          |                                                                                                                                                                                                    |           |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| 2.4      | Les barrières de synchronisation : points de ralliement des processus . . . . .                                                                                                                    | 36        |
| 2.4.1    | Un exemple de synchronisation en MPI pour mesurer le temps d'exécution . .                                                                                                                         | 36        |
| 2.4.2    | Le modèle BSP : Bulk Synchronous Parallel . . . . .                                                                                                                                                | 39        |
| 2.5      | Prise en main de l'API MPI . . . . .                                                                                                                                                               | 39        |
| 2.5.1    | Le traditionnel "Hello World" avec l'API MPI en C++ . . . . .                                                                                                                                      | 39        |
| 2.5.2    | Programmer en MPI avec l'API en C . . . . .                                                                                                                                                        | 41        |
| 2.5.3    | * MPI avec l'API de Boost en C++ . . . . .                                                                                                                                                         | 42        |
| 2.6      | * Utiliser MPI avec OpenMP . . . . .                                                                                                                                                               | 43        |
| 2.7      | La syntaxe en MPI des principales opérations de communication . . . . .                                                                                                                            | 45        |
| 2.7.1    | Syntaxe MPI pour la diffusion, diffusion personnalisée ( <i>scatter</i> ), le rassemblement ( <i>gather</i> ), la réduction ( <i>reduce</i> ), et la réduction totale ( <i>Allreduce</i> ) . . . . | 46        |
| 2.7.2    | Autre opérations de communication/calculs globaux moins courants . . . . .                                                                                                                         | 47        |
| 2.8      | Communication sur l'anneau avec MPI . . . . .                                                                                                                                                      | 49        |
| 2.9      | L'ordonnanceur de tâches SLURM . . . . .                                                                                                                                                           | 50        |
| 2.10     | Quelques exemples de programmes parallèles en MPI et leurs accélérations . . . . .                                                                                                                 | 52        |
| 2.10.1   | Le produit matrice-vecteur . . . . .                                                                                                                                                               | 53        |
| 2.10.2   | Calcul de $\pi$ approché par une simulation Monte-Carlo . . . . .                                                                                                                                  | 56        |
| 2.10.3   | Calcul approché du volume d'une molécule par intégration stochastique . . .                                                                                                                        | 60        |
| 2.11     | * Pour en savoir plus : notes, références et discussion . . . . .                                                                                                                                  | 65        |
| 2.12     | En résumé : ce qu'il faut retenir ! . . . . .                                                                                                                                                      | 66        |
| 2.13     | Exercices . . . . .                                                                                                                                                                                | 66        |
| <b>3</b> | <b>La topologie des réseaux d'interconnexion</b> . . . . .                                                                                                                                         | <b>67</b> |
| 3.1      | Réseaux statiques/dynamiques et réseaux logiques/physiques . . . . .                                                                                                                               | 67        |
| 3.2      | Réseaux d'interconnexions : modélisation par des graphes . . . . .                                                                                                                                 | 67        |
| 3.3      | Caractéristiques des topologies . . . . .                                                                                                                                                          | 68        |
| 3.3.1    | Degré et diamètre d'un graphe . . . . .                                                                                                                                                            | 69        |
| 3.3.2    | Connexité et bisection . . . . .                                                                                                                                                                   | 69        |
| 3.3.3    | Critères pour une bonne topologie du réseau . . . . .                                                                                                                                              | 69        |
| 3.4      | Les topologies usuelles : les réseaux statiques simples . . . . .                                                                                                                                  | 70        |
| 3.4.1    | La clique : le graphe complet $K$ . . . . .                                                                                                                                                        | 70        |
| 3.4.2    | L'étoile, l'anneau et l'anneau cordal . . . . .                                                                                                                                                    | 70        |
| 3.4.3    | Grilles et tores . . . . .                                                                                                                                                                         | 71        |
| 3.4.4    | Le cube 3D et les cycles connectés en cube (CCC) . . . . .                                                                                                                                         | 71        |
| 3.4.5    | Arbres et arbres élargis ( <i>fat trees</i> ) . . . . .                                                                                                                                            | 71        |
| 3.5      | La topologie de l'hypercube et le code de Gray . . . . .                                                                                                                                           | 73        |
| 3.5.1    | Construction récursive de l'hypercube . . . . .                                                                                                                                                    | 73        |
| 3.5.2    | Numérotage des nœuds avec le code de Gray . . . . .                                                                                                                                                | 74        |
| 3.5.3    | Génération d'un code de Gray en C++ . . . . .                                                                                                                                                      | 76        |
| 3.5.4    | Convertir les codes de Gray en codes binaires équivalents, et vice versa . . . .                                                                                                                   | 77        |
| 3.5.5    | * Produit cartésien de graphes (opérateur noté $\otimes$ ) . . . . .                                                                                                                               | 77        |
| 3.6      | Algorithmes de communication sur les topologies . . . . .                                                                                                                                          | 78        |
| 3.6.1    | Les communications sur l'anneau . . . . .                                                                                                                                                          | 78        |
| 3.6.2    | Un algorithme de diffusion sur l'hypercube : communications arborescentes .                                                                                                                        | 87        |
| 3.7      | Plongements de topologies dans d'autres topologies . . . . .                                                                                                                                       | 88        |
| 3.8      | * Topologies régulières complexes . . . . .                                                                                                                                                        | 91        |

|          |                                                                                      |            |
|----------|--------------------------------------------------------------------------------------|------------|
| 3.9      | * Réseaux d'interconnexions sur la puce . . . . .                                    | 92         |
| 3.10     | * Pour en savoir plus : notes, références et discussion . . . . .                    | 95         |
| 3.11     | En résumé : ce qu'il faut retenir ! . . . . .                                        | 95         |
| 3.12     | Exercices . . . . .                                                                  | 96         |
| <b>4</b> | <b>Le tri parallèle</b> . . . . .                                                    | <b>99</b>  |
| 4.1      | Le tri en séquentiel : quelques rappels élémentaires . . . . .                       | 99         |
| 4.1.1    | Quelques rappels succincts sur les principaux algorithmes de tri . . . . .           | 99         |
| 4.1.2    | Complexité des algorithmes de tri . . . . .                                          | 100        |
| 4.2      | Le tri parallèle par fusion de listes : MergeSort parallèle . . . . .                | 102        |
| 4.3      | Le tri parallèle par rang : RankSort . . . . .                                       | 102        |
| 4.4      | QuickSort en parallèle : ParallelQuickSort . . . . .                                 | 104        |
| 4.5      | L'algorithme amélioré HyperQuickSort . . . . .                                       | 107        |
| 4.6      | * Le tri parallèle PSRS par échantillonnage régulier ( <i>PSRS</i> ) . . . . .       | 110        |
| 4.7      | * Le tri sur la grille 2D : ShearSort . . . . .                                      | 110        |
| 4.8      | Tri par réseaux de comparaisons . . . . .                                            | 112        |
| 4.9      | * Fusion de listes triées par un circuit de comparateurs . . . . .                   | 115        |
| 4.10     | * Tri récursif de séquences bitoniques . . . . .                                     | 115        |
| 4.11     | * Pour en savoir plus : notes, références et discussion . . . . .                    | 118        |
| 4.12     | En résumé : ce qu'il faut retenir ! . . . . .                                        | 119        |
| 4.13     | Exercices . . . . .                                                                  | 119        |
| <b>5</b> | <b>Algèbre linéaire en parallèle</b> . . . . .                                       | <b>123</b> |
| 5.1      | Algèbre linéaire distribuée . . . . .                                                | 123        |
| 5.1.1    | Algèbre linéaire pour la science des données . . . . .                               | 123        |
| 5.1.2    | Algèbre linéaire classique . . . . .                                                 | 125        |
| 5.1.3    | Le produit matrice-vecteur : $y = Ax$ . . . . .                                      | 127        |
| 5.1.4    | Motifs pour le parallélisme des données matricielles . . . . .                       | 128        |
| 5.2      | Produit matrice-vecteur sur la topologie de l'anneau . . . . .                       | 130        |
| 5.3      | Produit matriciel sur la grille ( <i>outer product algorithm</i> ) . . . . .         | 135        |
| 5.4      | Produit matriciel sur la topologie du tore . . . . .                                 | 135        |
| 5.4.1    | L'algorithme de Cannon . . . . .                                                     | 137        |
| 5.4.2    | L'algorithme de Fox . . . . .                                                        | 139        |
| 5.4.3    | L'algorithme de Snyder . . . . .                                                     | 141        |
| 5.4.4    | Comparatif des trois algorithmes de produit matriciel sur le tore . . . . .          | 143        |
| 5.5      | * Pour en savoir plus : notes, références et discussion . . . . .                    | 143        |
| 5.6      | En résumé : ce qu'il faut retenir ! . . . . .                                        | 145        |
| 5.7      | Exercices . . . . .                                                                  | 145        |
| <b>6</b> | <b>Le modèle de calcul MapReduce</b> . . . . .                                       | <b>147</b> |
| 6.1      | Le défi de pouvoir traiter les BigData rapidement . . . . .                          | 147        |
| 6.2      | Le principe de base de MapReduce . . . . .                                           | 148        |
| 6.2.1    | Processus <i>mappers</i> et processus <i>reducers</i> . . . . .                      | 148        |
| 6.2.2    | * Les fonctions <i>map</i> et <i>reduce</i> dans les langages fonctionnels . . . . . | 148        |
| 6.3      | Typage et le méta-algorithme MapReduce . . . . .                                     | 149        |
| 6.4      | Un exemple complet de programme MapReduce en C++ . . . . .                           | 150        |
| 6.5      | Modèle d'exécution de MapReduce et architecture . . . . .                            | 153        |

|                                                              |                                                                                                            |            |
|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|------------|
| 6.6                                                          | * Utiliser le paradigme MapReduce en MPI avec MR-MPI . . . . .                                             | 155        |
| 6.7                                                          | * Pour en savoir plus : notes, références et discussion . . . . .                                          | 157        |
| 6.8                                                          | En résumé : ce qu'il faut retenir ! . . . . .                                                              | 158        |
| <br><b>II Introduction aux sciences des données avec MPI</b> |                                                                                                            | <b>159</b> |
| <b>7</b>                                                     | <b>Les <math>k</math>-moyennes : le regroupement par partitions</b>                                        | <b>161</b> |
| 7.1                                                          | La recherche exploratoire par le regroupement . . . . .                                                    | 161        |
| 7.1.1                                                        | Le regroupement par partitions . . . . .                                                                   | 162        |
| 7.1.2                                                        | Coût d'un regroupement et regroupement par modèles . . . . .                                               | 163        |
| 7.2                                                          | La fonction de coût des $k$ -moyennes . . . . .                                                            | 163        |
| 7.2.1                                                        | Réécriture de la fonction de coût : regrouper ou séparer les données . . . . .                             | 167        |
| 7.2.2                                                        | Calculabilité : complexité du calcul des $k$ -moyennes . . . . .                                           | 168        |
| 7.3                                                          | L'heuristique locale de Lloyd pour les $k$ -moyennes . . . . .                                             | 169        |
| 7.4                                                          | Initialisation des $k$ -moyennes . . . . .                                                                 | 172        |
| 7.4.1                                                        | Initialisation aléatoire (dite de Forgy) . . . . .                                                         | 172        |
| 7.4.2                                                        | Initialisation avec les $k$ -moyennes globales . . . . .                                                   | 172        |
| 7.4.3                                                        | Initialisation probabiliste garantie avec les $k$ -moyennes++ . . . . .                                    | 172        |
| 7.5                                                          | La quantification de vecteurs et les $k$ -moyennes . . . . .                                               | 174        |
| 7.5.1                                                        | La quantification . . . . .                                                                                | 174        |
| 7.5.2                                                        | * Les minima locaux de Lloyd engendrent des partitions de Voronoï . . . . .                                | 174        |
| 7.6                                                          | Interprétation physique des $k$ -moyennes : décomposition de l'inertie . . . . .                           | 175        |
| 7.7                                                          | Choix du nombre $k$ de groupes : sélection de modèle . . . . .                                             | 176        |
| 7.7.1                                                        | Méthode du coude . . . . .                                                                                 | 177        |
| 7.7.2                                                        | Proportion de la variance expliquée par $k$ . . . . .                                                      | 177        |
| 7.8                                                          | Les $k$ -moyennes sur une grappe de machines pour les grandes données . . . . .                            | 178        |
| 7.9                                                          | Évaluation et comparaisons des partitions obtenues par les méthodes de regroupement                        | 179        |
| 7.9.1                                                        | L'index de Rand . . . . .                                                                                  | 180        |
| 7.9.2                                                        | L'information mutuelle normalisée (NMI) . . . . .                                                          | 181        |
| 7.10                                                         | * Pour en savoir plus : notes, références et discussion . . . . .                                          | 181        |
| 7.11                                                         | En résumé : ce qu'il faut retenir ! . . . . .                                                              | 183        |
| 7.12                                                         | Exercices . . . . .                                                                                        | 183        |
| <b>8</b>                                                     | <b>Le regroupement hiérarchique</b>                                                                        | <b>189</b> |
| 8.1                                                          | Regroupement hiérarchique ascendant et descendant : représentations enracinées par dendrogrammes . . . . . | 189        |
| 8.2                                                          | Stratégies pour définir une bonne distance de chaînage . . . . .                                           | 191        |
| 8.2.1                                                        | Algorithmes pour le regroupement hiérarchique agglomératif . . . . .                                       | 193        |
| 8.2.2                                                        | Choix de la distance de base . . . . .                                                                     | 195        |
| 8.3                                                          | Critère de fusion de Ward et centroïdes . . . . .                                                          | 196        |
| 8.4                                                          | Partitionnements à partir du dendrogramme . . . . .                                                        | 196        |
| 8.5                                                          | Distances ultramétriques et arbres phylogénétiques . . . . .                                               | 198        |
| 8.6                                                          | * Pour en savoir plus : notes, références et discussion . . . . .                                          | 200        |
| 8.7                                                          | En résumé : ce qu'il faut retenir ! . . . . .                                                              | 201        |
| 8.8                                                          | Exercices . . . . .                                                                                        | 201        |

|           |                                                                                      |            |
|-----------|--------------------------------------------------------------------------------------|------------|
| <b>9</b>  | <b>La classification supervisée par les <math>k</math>-Plus Proches Voisins</b>      | <b>205</b> |
| 9.1       | L'apprentissage supervisé . . . . .                                                  | 205        |
| 9.2       | La règle du proche voisin (PPV) . . . . .                                            | 205        |
| 9.2.1     | Optimisation du calcul du plus proche voisin pour la distance euclidienne . .        | 206        |
| 9.2.2     | Règle du PPV et les diagrammes de Voronoï . . . . .                                  | 207        |
| 9.2.3     | La règle des $k$ -Plus Proches Voisins . . . . .                                     | 207        |
| 9.3       | Évaluation de la performance d'un classifieur . . . . .                              | 207        |
| 9.3.1     | Taux d'erreur de classification ( <i>misclassification</i> ) . . . . .               | 207        |
| 9.3.2     | Matrices de confusion et faux positifs/négatifs . . . . .                            | 207        |
| 9.3.3     | Précision, rappel et $F$ -score . . . . .                                            | 209        |
| 9.4       | L'apprentissage statistique et l'erreur minimale bayésienne . . . . .                | 210        |
| 9.4.1     | Estimation non-paramétrique de densités . . . . .                                    | 210        |
| 9.4.2     | L'erreur minimale : la probabilité d'erreur et l'erreur de Bayes . . . . .           | 211        |
| 9.4.3     | Probabilité d'erreur pour la règle du PPV . . . . .                                  | 213        |
| 9.5       | Requêtes des PPVs sur architecture parallèle à mémoire distribuée . . . . .          | 213        |
| 9.6       | * Pour en savoir plus : notes, références et discussion . . . . .                    | 213        |
| 9.7       | En résumé : ce qu'il faut retenir ! . . . . .                                        | 215        |
| 9.8       | Exercices . . . . .                                                                  | 216        |
| <b>10</b> | <b>Optimisation approchée par sous-ensembles noyaux et la réduction de dimension</b> | <b>219</b> |
| 10.1      | Optimisation approchée pour les données volumineuses . . . . .                       | 219        |
| 10.1.1    | Un exemple qui montre le besoin de traiter les grandes dimensions . . . . .          | 219        |
| 10.1.2    | Phénomènes sur les distances en grandes dimensions . . . . .                         | 220        |
| 10.1.3    | Passer des grandes aux petites données ! . . . . .                                   | 220        |
| 10.2      | Définition des sous-ensembles noyaux ( <i>coresets</i> ) . . . . .                   | 220        |
| 10.3      | Sous-ensembles noyaux pour la plus petite boule englobante . . . . .                 | 221        |
| 10.4      | Une heuristique rapide pour approcher la boule englobante minimale . . . . .         | 222        |
| 10.4.1    | * Preuve de convergence . . . . .                                                    | 223        |
| 10.4.2    | * Boule englobante approchée et séparateur linéaire approché . . . . .               | 224        |
| 10.5      | * Sous-ensembles noyaux pour les $k$ -moyennes . . . . .                             | 225        |
| 10.6      | Réduction rapide de la dimension des données . . . . .                               | 225        |
| 10.6.1    | Le fléau des grandes dimensions . . . . .                                            | 225        |
| 10.6.2    | Deux exemples concrets pour illustrer le besoin des grandes dimensions . . .         | 226        |
| 10.6.3    | La réduction de dimension linéaire . . . . .                                         | 226        |
| 10.6.4    | Le théorème de Johnson-Lindenstrauss . . . . .                                       | 227        |
| 10.6.5    | Matrices de projection aléatoire . . . . .                                           | 227        |
| 10.7      | Exercices . . . . .                                                                  | 228        |
| 10.8      | * Pour en savoir plus : notes, références et discussion . . . . .                    | 229        |
| 10.9      | En résumé : ce qu'il faut retenir ! . . . . .                                        | 229        |
| <b>11</b> | <b>Algorithmique parallèle pour les graphes</b>                                      | <b>231</b> |
| 11.1      | Détection de sous-graphes denses dans les (grands) graphes . . . . .                 | 231        |
| 11.1.1    | Définition du problème . . . . .                                                     | 231        |
| 11.1.2    | Complexité du problème et une heuristique gloutonne . . . . .                        | 232        |
| 11.1.3    | Une heuristique facilement parallélisable . . . . .                                  | 236        |
| 11.2      | Tester l'isomorphisme de (petits) graphes . . . . .                                  | 238        |

|            |                                                                                                                     |            |
|------------|---------------------------------------------------------------------------------------------------------------------|------------|
| 11.2.1     | Principes généraux des algorithmes d'énumération . . . . .                                                          | 240        |
| 11.2.2     | L'algorithme d'Ullmann pour tester l'isomorphisme . . . . .                                                         | 240        |
| 11.2.3     | Parallélisation de l'algorithme énumératif . . . . .                                                                | 242        |
| 11.3       | * Pour en savoir plus : notes, références et discussion . . . . .                                                   | 243        |
| 11.4       | En résumé : ce qu'il faut retenir ! . . . . .                                                                       | 243        |
| 11.5       | Exercices . . . . .                                                                                                 | 243        |
| <b>III</b> | <b>C/C++/Shell</b>                                                                                                  | <b>249</b> |
| <b>A</b>   | <b>Le langage C quand on connaît déjà Java</b>                                                                      | <b>251</b> |
| A.1        | Fichiers d'en-tête <code>.h</code> , fichiers de description <code>.c</code> , macros et le préprocesseur . . . . . | 251        |
| A.2        | Allocation mémoire et destruction de tableaux . . . . .                                                             | 254        |
| A.3        | Construction de structures de données avec <code>struct</code> . . . . .                                            | 255        |
| A.4        | Déclaration de fonctions . . . . .                                                                                  | 256        |
| A.5        | Quelques autres spécificités du langage C . . . . .                                                                 | 257        |
| A.6        | Les entrées et sorties illustrées par le tri à bulles . . . . .                                                     | 257        |
| <b>B</b>   | <b>Le langage C++ quand on connaît déjà le C</b>                                                                    | <b>259</b> |
| B.1        | Appel de fonctions : passage par valeurs . . . . .                                                                  | 260        |
| B.2        | Les classes et les objets . . . . .                                                                                 | 260        |
| B.2.1      | Définition d'une classe . . . . .                                                                                   | 261        |
| B.2.2      | L'héritage et les hiérarchies de classe . . . . .                                                                   | 262        |
| B.3        | Le mot clef <code>const</code> dans les méthodes . . . . .                                                          | 263        |
| B.4        | Construction et destruction de tableaux . . . . .                                                                   | 263        |
| B.5        | Surcharge des opérateurs . . . . .                                                                                  | 264        |
| B.6        | La généricité en C++ . . . . .                                                                                      | 264        |
| B.7        | La bibliothèque STL . . . . .                                                                                       | 265        |
| B.8        | Les entrées-sorties en C++ . . . . .                                                                                | 265        |
| B.9        | La bibliothèque <code>Boost</code> pour les matrices ( <code>ublas</code> ) . . . . .                               | 266        |
| B.10       | Quelques autres spécificités du C++ . . . . .                                                                       | 267        |
| <b>C</b>   | <b>Commandes shell pour manipuler les processus et les E/Ss</b>                                                     | <b>269</b> |
| C.1        | Fichier de configuration initiale <code>.bashrc</code> . . . . .                                                    | 269        |
| C.2        | Commandes Unix pipelinées et la redirection des entrées/sorties . . . . .                                           | 270        |
| C.3        | Manipuler les tâches ( <i>jobs</i> ) . . . . .                                                                      | 271        |
| <b>D</b>   | <b>Liste des ordinateurs en salles machines</b>                                                                     | <b>273</b> |

# Préface

Bienvenue dans le cours “Traitement des données massives” (INF442 TMD) !

Dans ce cours de huit blocs, nous allons donner une *initiation* au traitement massif des données grâce au calcul haute performance, plus connu sous l’abréviation anglaise de HPC pour le *High Performance Computing*. Les grandes données (voire les flux de grandes données) abondent, et ces données présentent potentiellement une source très riche d’information à extraire pour pouvoir en tirer profit. C’est une des problématiques du BigData, l’analyse de données volumineuses qui permet aussi de mieux comprendre les phénomènes de masse. Le traitement des BigData a d’ores et déjà des usages variés dans le marketing, le transport, la sécurité, la médecine, etc. Pour pouvoir exploiter les BigData, on doit savoir comment traiter efficacement les données par des algorithmes parallèles.

On distingue deux grandes façons de concevoir ces algorithmes parallèles suivant que l’on considère les architectures à mémoire partagée ou les architectures à mémoire distribuée. Dans une architecture à mémoire partagée comme les processeurs multi-cœurs des ordinateurs et des téléphones portables (*smartphones*), les unités de calcul sont sur une même puce, et on peut facilement programmer des algorithmes parallèles comme le décodage ou le rendu de vidéo en utilisant des fils de calcul (*multi-threading*). C’est donc un parallélisme qui exploite une fine granularité. Les architectures à mémoire distribuée permettent, quant à elle, de considérer des algorithmes qui passent à l’échelle (sont extensibles) en adaptant le nombre de machines utilisées en fonction de la taille des données : c’est le *calcul élastique* qui s’adapte en fonction de la taille des données et des problèmes à traiter. Ces architectures à mémoire distribuée se prêtent très bien au parallélisme à gros grains. De plus, ces architectures à mémoire distribuée sont à la fois modulables et très flexibles car on peut choisir la manière dont ces machines sont reliées entre elles par un réseau d’interconnexion, la topologie du réseau. Ce réseau d’interconnexion peut être statique ou dynamique.

Ce cours est dédié à la conception d’*algorithmes parallèles à mémoire distribuée* en utilisant l’interface MPI : Message Passing Interface. MPI est le standard de référence dans le domaine pour (i) appeler des opérations de communication qui échangent des données contenues dans des messages et (ii) pour effectuer des calculs collaboratifs. Le standard MPI est disponible dans plusieurs interfaces (*bindings* en anglais) suivant les langages de programmation : C, C++, Fortran, Python, R, etc. Nous utiliserons le langage orienté-objet C++ pour écrire les codes en TDs avec l’API en C de MPI.

Les chapitres de ce cours sont organisés en deux grandes parties : (1) une introduction au HPC avec MPI, et (2) une introduction aux sciences des données avec les aspects de parallélisation. On décrit brièvement le contenu de ces deux parties comme suit :

**Partie 1 – Introduction au HPC avec MPI.** — On commence par donner une brève introduction au calcul haute performance, le *HPC*, et on présente la loi d’Amdahl qui caractérise

- l'accélération optimale théorique, puis la loi de Gustfason.
- Puis on décrit les principaux concepts de MPI et son interface : on y introduit les notions de communications bloquantes et non-bloquantes (synchrones/asynchrones), présente les situations de blocage (*deadlocks*), des différents types de communication globale (diffusion, diffusion personnalisée, commérage, etc.)
  - Ensuite au chapitre 3, on présente et discute de l'importance du choix de la topologie pour les réseaux d'interconnexion qui relient les machines entre elles. On distingue notamment le réseau physique matériel du réseau logique conceptuel utilisé par les algorithmes parallèles pour leurs communications et calculs globaux. En particulier, on décrit quelques algorithmes de communication sur la topologie de l'anneau (comme la diffusion pipelinée) et sur la topologie de l'hypercube (en utilisant le code de Gray).
  - Dans le chapitre 4, on présente les principaux algorithmes de tri parallèle sur mémoire distribuée. On verra que l'on peut d'abord paralléliser le célèbre algorithme QuickSort naïvement, puis on introduira de meilleurs algorithmes comme l'HyperQuickSort et enfin l'algorithme PSRS utilisé en pratique.
  - Le chapitre 5 étudie quelques algorithmes parallèles pour l'algèbre linéaire. On présente essentiellement les algorithmes de multiplication matricielle (matrice-vecteur, matrice-matrice) sur les topologies de l'anneau et du tore.
  - Le chapitre 6 introduit le modèle de calcul parallèle MapReduce qui est très populaire (dans sa version *open source*, *Hadoop*). Le modèle MapReduce permet d'utiliser un très grand nombre d'ordinateurs (des milliers voire des centaines de milliers) en utilisant des programmes parallèles simples construits avec seulement deux opérations de base : **map** et **reduce**. MapReduce est aussi un formalisme car son architecture système maître-esclave permet de gérer en temps réel les diverses pannes des machines ou le trafic trop lent sur certaines machines, etc. On explique comment programmer ces types d'algorithmes avec le standard MPI qui lui n'a par défaut aucune tolérance aux pannes.

**Partie 2 – Introduction aux sciences des données.** Cette partie donne une initiation aux *sciences des données* (*data science*) comme suit :

- On commence par traiter des problèmes de regroupement (chapitre 7, le *clustering* faisant partie de l'apprentissage non supervisé) et de regroupement hiérarchique (chapitre 8) pour la découverte de classes.
- Puis on considère la classification supervisée (un des piliers de l'apprentissage par machine) au chapitre 9.
- Dans le chapitre 10, on présente un paradigme qui permet de résoudre des problèmes d'optimisation sur les grandes données (voire aussi des données en très grande dimension) en cherchant des sous-ensembles noyaux de petites tailles pour lesquels leurs optimisations garantissent une approximation pour la totalité des données. C'est une technique fort utile quand applicable, qui permet de passer des BigData au TinyData ! On explique aussi succinctement comment passer des grandes dimensions aux petites dimensions grâce à des matrices aléatoires de projection se basant sur un théorème de Johnson–Lindenstrauss.
- Le dernier chapitre 11 présente quelques algorithmes parallèles sur les graphes. On décrit un algorithme qui cherche des graphes denses (interprétables par exemple comme des communautés sur les graphes de réseaux sociaux), et on présente un algorithme pour tester l'isomorphisme de graphes, un problème dont il est facile de vérifier une solution mais dont la complexité reste inconnue dans le cas général (pas de preuve que ce problème soit NP-complet avec une avancée majeure obtenue en 2015).

À la fin de chaque chapitre, un résumé des *points essentiels* à retenir est fourni et quelques exercices sont proposés pour consolider les connaissances. Les sections avec une astérisque peuvent être sautées en première lecture, et les niveaux de difficulté des exercices sont signalés par leurs nombres d'astérisques. La dernière mise à jour de ce cours (incluant les corrections éventuelles) est disponible sur le site `moodle`<sup>1</sup> de l'école.

L'objectif principal est qu'à la fin de ce cours, vous puissiez concevoir et programmer *par vous même* des algorithmes en parallèle avec MPI dans le langage C++. Un deuxième objectif est de vous montrer la richesse et la variété des thématiques abordées, tant sur le plan théorique que sur le plan pratique, de la science du calcul haute performance, le *Super-Computing*! Pour finir, mentionnons que ce cours est une première *initiation* au HPC et aux sciences des données et que des sujets importants du parallélisme comme l'ordonnancement de tâches, les *nids de boucles* ou les algorithmes parallèles sur les matrices creuses n'y sont pas abordés. Un ouvrage en anglais intitulé *Introduction to HPC with MPI for Data Science* [74] basé sur ce cours a été publié par Springer en 2016. Le site web<sup>2</sup> de cet ouvrage offre en téléchargement les codes sources de nombreux programmes.

Bonne lecture!  
Frank Nielsen  
Avril 2016.

---

1. <https://moodle.polytechnique.fr/>

2. <https://www.lix.polytechnique.fr/~nielsen/HPC4DS/>

Première partie

# Introduction au HPC avec MPI



# Chapitre 1

## Le calcul haute performance (le HPC)

Un résumé des points essentiels à retenir est donné dans §1.10.

### 1.1 Qu'est-ce que le Calcul Haute Performance ?

Le *Calcul Haute Performance* plus connu sous son acronyme *HPC*, dérivé de l'anglais *High Performance Computing*, est un domaine incluant les divers paradigmes de programmation parallèle et ses langages de programmation, les outils logiciels, les systèmes informatiques, les conférences dédiées (ACM/IEEE Super-Computing), bref tout ce qui touche aux sciences et techniques des *super-ordinateurs*.

La liste des 500 premiers super-ordinateurs connus dans le monde est périodiquement mise à jour et rendue publique sur le site Internet.<sup>1</sup> En 2014, c'est le super-ordinateur Tianhe-2 (voie lactée, *MilkyWay-2*) du *National Super Computer Center* à Guangzhou (Chine) qui figure en première position de cette liste. Ce super-ordinateur dispose de 3,12 millions de cœurs (des unités de calcul) délivrant une puissance de calcul de l'ordre de 54,9 Petaflops (où 1 PFlops équivaut à  $10^{15}$  opérations arithmétiques en calcul flottant) et nécessite une source d'énergie électrique de 17,8 Mega Watts (le coût d'1 MW est d'environ 100 euro/heure, soit environ un million d'euro par an). La table 1.1 récapitule les différents ordres de grandeur pour qualifier la puissance de calcul des ordinateurs et leurs capacités de mémoire globale. Nous sommes actuellement dans la course à l'exaflops ( $10^{18}$  flops, 1024 Pflops) que l'on espère pouvoir obtenir vers 2017-2020, puis viendra l'ère du zetaFlops ( $10^{21}$ ) vers 2030 ? et ensuite le yottaFlops ( $10^{24}$ ), etc. Ce classement traditionnel qui compte avant tout la puissance en FLOPS des super-ordinateurs est peu écologique car très gourmande en énergie. Un autre classement, appelé le *green HPC*, s'intéresse plutôt aux scores des super-ordinateurs en *MFlops/W*. Bien que la puissance de calcul soit un critère très important, il faut aussi dans les faits prendre en compte la mémoire vive, la bande passante du réseau, etc.

---

1. <http://www.top500.org/>

| unité     | échelle    | puissance de calcul     | mémoire en octets |
|-----------|------------|-------------------------|-------------------|
| k (kilo)  | $10^3$     | kFLOPS                  | ko (KB)           |
| M (méga)  | $10^6$     | MFLOPS                  | Mo (MB)           |
| G (giga)  | $10^9$     | GFLOPS                  | Go (GB)           |
| T (téra)  | $10^{12}$  | TFLOPS                  | To (TB)           |
| P (péta)  | $10^{15}$  | PFLOPS                  | Po (PB)           |
| E (exa)   | $10^{18}$  | EFLOPS (vers 2017-2020) | Eo (EB)           |
| Z (zéta)  | $10^{21}$  | ZFLOPS                  | Zo (ZB)           |
| Y (yotta) | $10^{24}$  | YFLOPS                  | Yo (YB)           |
| ...       | ...        | ...                     | ...               |
| googol    | $10^{100}$ | googolFLOPS             | googol octets     |
| ...       | ...        | ...                     | ...               |

TABLE 1.1 – Ordres de grandeur caractérisant la puissance des super-ordinateurs : on compte la puissance des super-ordinateurs en flops, c'est-à-dire, en nombre d'opérations arithmétiques à virgule flottante par seconde, et la mémoire globale en octets (*bytes* avec 1 o=1 B=8 bits). Si l'on prend  $1024 = 2^{10}$  (puissance de 2) au lieu de  $1000 = 10^3$  pour le saut entre deux échelles, alors les abréviations deviennent ki ( $2^{10}$ ), Mi ( $2^{20}$ ), Gi ( $2^{30}$ ), Ti ( $2^{40}$ ), Pi ( $2^{50}$ ), Eo ( $2^{60}$ ), Zo ( $2^{70}$ ), et Yi ( $2^{80}$ ). Il y a donc une différence entre 1 Go et 1 Gio (1,073,741,824 octets).

## 1.2 Pourquoi le HPC ?

La première réponse qui vient à l'esprit est que le HPC est utile pour aller plus *vite* et être plus *précis* (notamment dans les simulations, comme pour la météo, la mécanique numérique, ou dans la modélisation de phénomènes complexes). Le HPC permet aussi de pouvoir résoudre de plus *gros problèmes* : soient des problèmes de simulation sur des domaines maillés plus finement, soient des problèmes sur de plus grands jeux de données : les *BigData*.

Par contre, ce qui peut être moins connu est que le HPC est aussi grandement utile et prometteur pour pouvoir économiser de l'énergie : à même puissance flop utilisée, on préférera plus de *processeurs lents* qui consomment moins d'énergie au total ! Enfin, le HPC permet de simplifier des traitements de données car certains algorithmes sont *intrinsèquement parallèles*. En effet, les algorithmes sur les vidéos ou les images calculent souvent des filtres qui sont des opérations qui peuvent se calculer en parallèle pour chaque pixel ou voxel (en imagerie médicale). Dans ce dernier cas, les cartes graphiques (*Graphics Processing Unit*, GPU) qui sont des cartes contenant beaucoup de cœurs<sup>2</sup> permettent non seulement de créer de belles images mais sont aussi utiles pour des calculs généraux : c'est le paradigme GPGPU pour *General Purpose Graphics Processing Unit*.

Citons plus précisément ces quelques cas de figure pour l'utilisation du *Super Computing* :

- utiliser des *modèles* pour faire de la *simulation* parce que sinon c'est trop difficile à construire (souffleries) ou trop cher à faire (crash d'avion/voiture), voire trop lent sur les machines classiques (évolution du climat ou des galaxies), ou trop dangereux à réaliser en pratique (armes nucléaires, drogues, pollutions, épidémies).
- avoir des résultats *rapides* et même sans attente, c'est-à-dire *incrémentaux* (*on-line algorithms*) : certains résultats ont une valeur temporelle comme la météo : celle de demain ne

2. Par exemple, le GPU haut de gamme de nVidia possède 1536 cœurs pour une puissance de calcul de 2,3 TFlops. Le GPU d'AMD Radeon Sky 900 possède quant à lui 3584 cœurs pour une puissance de 1,5 TFlops (double précision).

nous intéresse qui si nous arrivons à la *prédire* avant le lendemain. De même, il est intéressant d'avoir le résultat en *premier* afin de pouvoir prendre des décisions le plus rapidement possible (avant les autres comme dans la prédiction de cours de bourse pour le *trading*).

- traiter les données **massives** comme l'analyse de génomes ou d'une famille de génomes, voire même de détecter s'il existe une vie intelligente extraterrestre (*Search for ExtraTerrestrial Intelligence, SETI*<sup>3</sup>).

### 1.3 Les grandes données : les quatre V du *Big Data*

Le *BigData* est un mot à la mode (*buzzword*) très médiatisé qui cache plusieurs facettes. On peut encore parler plus simplement de traitement des données à grande échelle (*large-scale data processing*). On caractérise le traitement sur les grandes données par les 4 V :

- Volume,
- Variété/ *Variety* (données hétérogènes),
- Vitesse/ *Velocity* (données générées en temps réel par des capteurs),
- Valeur/ *Value* (pas de simulation mais de la *valorisation*).

On estime qu'aujourd'hui le monde produit 2,5 exaoctets ( $10^{18}$ , unité Eo, ou *quintillion* en anglais) de données par jour !

### 1.4 Paradigmes de programmation parallèle

Il existe plusieurs paradigmes de programmation des algorithmes parallèles pour les grandes données (un sous-domaine du HPC). Ces modèles dépendent de la tolérance aux pannes des ordinateurs ou des réseaux. Les deux grands modèles de programmation qui se complètent aujourd'hui sont :

- la programmation avec MPI (abréviation de *Message Passing Interface*) qui a une tolérance nulle aux pannes par défaut mais offre une grande souplesse de programmation,
- la programmation avec *MapReduce* (ou son équivalent dans le logiciel libre *Hadoop*<sup>4</sup>) qui inclut dans son cadre une très grande tolérance mais offre un modèle relativement limité de calcul parallèle comparé au MPI.

### 1.5 Granularité du parallélisme : petits grains et gros grains

On peut concevoir et programmer des algorithmes parallèles avec divers degrés de *granularité*. La granularité se définit par la grosseur des parties de code qui sont parallélisables. La granularité peut être aussi interprétée comme le rapport du temps de calcul sur le temps de communication d'un algorithme parallèle. On classe en *trois grandes catégories* ces algorithmes parallèles en fonction de la taille des grains (la granularité) :

- *grain fin* (*fine-grained parallelism*) : au niveau des variables à l'intérieur d'une même tâche. Les données sont souvent transférées parmi les unités de calcul. À noter que le jeu d'instruction des microprocesseurs usuels appelé x86 (datant de 1978) a été étendu avec de nouvelles extensions (comme MMX, SSE, SSE2, etc.). Beaucoup de ces nouvelles extensions sont des

---

3. <http://www.seti.org/>

4. <http://hadoop.apache.org/>

ajouts d'instructions SIMD (*Streaming SIMD Extensions*<sup>5</sup>). Le parallélisme à fine granularité peut aussi utiliser des morceaux de code sur la carte graphique (le GPU).

- *grain intermédiaire* : au niveau des tâches d'un même programme en utilisant les fils de calcul (*threads*).
- *gros grain* (*coarse-grained parallelism*) : les données sont transférées peu souvent après de gros calculs. La parallélisation à gros grains peut aussi se faire au niveau des programmes avec l'ordonnanceur de tâches qui s'occupe de distribuer les tâches indépendantes sur le cluster d'ordinateurs (la "machine parallèle").

## 1.6 L'architecture pour le HPC : mémoire et réseau

On distingue les machines parallèles à *mémoire partagée* des machines parallèles à *mémoire distribuée*. Typiquement, sur un processeur multi-cœur, tous les cœurs utilisent la même zone mémoire (partagée) : architecture de type *symmetric shared memory multiprocessor* (*SMP*) qui considère les cœurs identiques, les assimilant à des unités de calcul indépendantes. Même si on considère un modèle de calcul parallèle à mémoire partagée, il faut en pratique tenir compte des différents types de mémoire qui sont partagées : registres mémoires<sup>6</sup> localisés dans le processeur et caches dans la mémoire vive (*Random Access Memory*, RAM), les disques durs (*disk arrays*), disques durs "flash" (*Solid-State Drive*, SSD), bandes magnétiques pour les sauvegardes (*back-up tape*), etc.

En pratique, pour obtenir de *bonnes performances* de calcul, il faut tenir compte de la *localité spatiale* lors des accès à la mémoire. C'est-à-dire, on cherche à accéder des emplacements mémoires proches des précédents. Car dans ce cas, les données ont été transférées dans la mémoire cache et leurs accès est beaucoup plus rapide. Par exemple, comparons le code C/C++ de cette double boucle imbriquée (qui calcule le produit matrice-vecteur  $y = A \times x$  quand on a préalablement initialisé  $y$  à 0) :

```
// premier code : 1,45 seconde pour n = 10000
for (int j=0; j<n; ++j)
{
    // les calculs des y[i] se font progressivement.
    // Seulement à la dernière étape de j, quand j = n - 1,
    // on obtient y[0], y[1], ..., y[n-1]
    for (int i=0; i<n; ++i)
        {y[i] += a[i][j] * x[j];}
}
```

avec celui-ci :

```
// deuxième code plus rapide : 0,275 seconde pour n = 10000
for (int i=0; i<n; ++i)
{
    // on calcule itérativement y[0], y[1], ..., y[n-1]
    for (int j=0; j<n; ++j)
        {y[i] += a[i][j] * x[j];}
}
```

En théorie, ces deux codes ont la même complexité : temps quadratique en  $O(n^2)$ . Par contre, en pratique les compilateurs<sup>7</sup> optimisent l'accès mémoire. Sur un ordinateur classique, le premier code aura un temps de calcul de 1,45 seconde pour  $n = 10000$ , alors qu'en compilant avec les options

5. [http://fr.wikipedia.org/wiki/Streaming\\_SIMD\\_Extensions](http://fr.wikipedia.org/wiki/Streaming_SIMD_Extensions)

6. Les registres en nombre limités sont donc à la racine de la hiérarchie mémoire.

7. `g++ -help`, regarder les options `-O`

d'optimisation (`g++ -O`), nous obtenons un temps d'exécution de 0,275 seconde. Le deuxième code calcule les coefficients du vecteur (tableau)  $y$ , un par un, par accumulation successive. Pour ce faire, on accède aux coefficients  $a[i][j]$  de la matrice  $A$  consécutivement en mémoire<sup>8</sup>, et les transferts des données au processeur sont optimisées par anticipation en mettant dans le cache mémoire les données qui seront prochainement accédées. Le premier code, quant à lui, doit attendre la dernière itération sur la boucle  $j$  pour commencer à obtenir les résultats finaux sur les coefficients  $y[0]$ ,  $y[1]$ , ...,  $y[n-1]$  du vecteur  $y$ . Ce premier code accède les coefficients de  $A$  en faisant des sauts successifs de  $n$  cases à chaque calcul de multiplication, et ne bénéficie pas de la localité des données dans le cache mémoire.

Diamétralement opposée à l'architecture à mémoire partagée, nous avons les ordinateurs parallèles à mémoire distribuée qui sont des ordinateurs indépendants reliés entre-eux par un réseau d'interconnexion. La topologie choisie pour le réseau d'interconnexion détermine l'efficacité des communications mais aussi implique un coût matériel en conséquence. La figure 1.4 montre quelques topologies usuelles pour ces réseaux d'interconnexion. Aux deux extrêmes figurent le bus et le graphe complet d'interconnexion. Les échanges de messages peuvent se faire point à point (par exemple, en utilisant l'architecture du bus ou la connectique du réseau complet<sup>9</sup>) ou alors en transitant par des nœuds intermédiaires. En pratique les grappes d'ordinateurs mettent en commun des ressources de calcul très hétérogènes comme le monde la Figure 1.2. Toutefois dans ce cours, afin de mettre en relief les principales notions, nous supposons un cadre théorique idéalisé où chaque nœud consiste en un ordinateur simple (un seul CPU). Ainsi tous les processus seront exécutés sur leur propre nœud distinct.<sup>10</sup>

La figure 1.3 illustre très schématiquement l'évolution des architectures des ordinateurs lors de ces dernières années : avec les procédés lithographiques de gravage des puces de plus en plus petit (de l'ordre de 11 nm en 2015), nous sommes passés progressivement d'une architecture avec plusieurs ordinateurs connectés par un réseau, à un ordinateur ayant plusieurs processeurs sur sa carte mère, puis récemment à un ordinateur ayant un processeur contenant plusieurs cœurs (plus efficace car la connectique est bien optimisée). Malheureusement, cette miniaturisation ne passe pas à l'échelle et pour avoir des systèmes parallèles performants nous devons utiliser des clusters d'ordinateurs où chaque ordinateur peut varier d'une architecture simple (mono-CPU mono-cœur) à une architecture plus complexe pour la performance (par exemple, plusieurs processeurs multi-cœurs communiquant avec plusieurs cartes graphiques).

Les architectures à mémoire distribuée associent un espace mémoire local à chaque processeur : il n'y a donc pas de mémoire globale partagée. Dans ce modèle, l'accès mémoire à un autre processeur est explicitement faite par un échange de messages sur le réseau. La figure 1.1 illustre notre modèle de machines parallèles pour ce cours. C'est le réseau d'interconnexion qui détermine la vitesse d'accès aux données. Trois caractéristiques du réseau sont :

- la *latence* (*latency*) : le temps requis pour initialiser une communication,
- la *bande passante* (*bandwidth*) : la vitesse de transfert des données sur les liens de communication,
- la *topologie* (*topology*) : l'architecture physique du réseau d'interconnexion (par exemple, l'étoile ou la grille).

Il existe plusieurs modèles de programmation parallèle des nœuds. Sur les *super-calculateurs*

8. La mémoire peut être schématiquement représenté par un long ruban 1D de cases mémoires.

9. Le réseau complet est modélisé par un graphe complet : une clique.

10. Cela explique parfois l'utilisation du vocabulaire : nœud ou processeur pour désigner son processus correspondant.

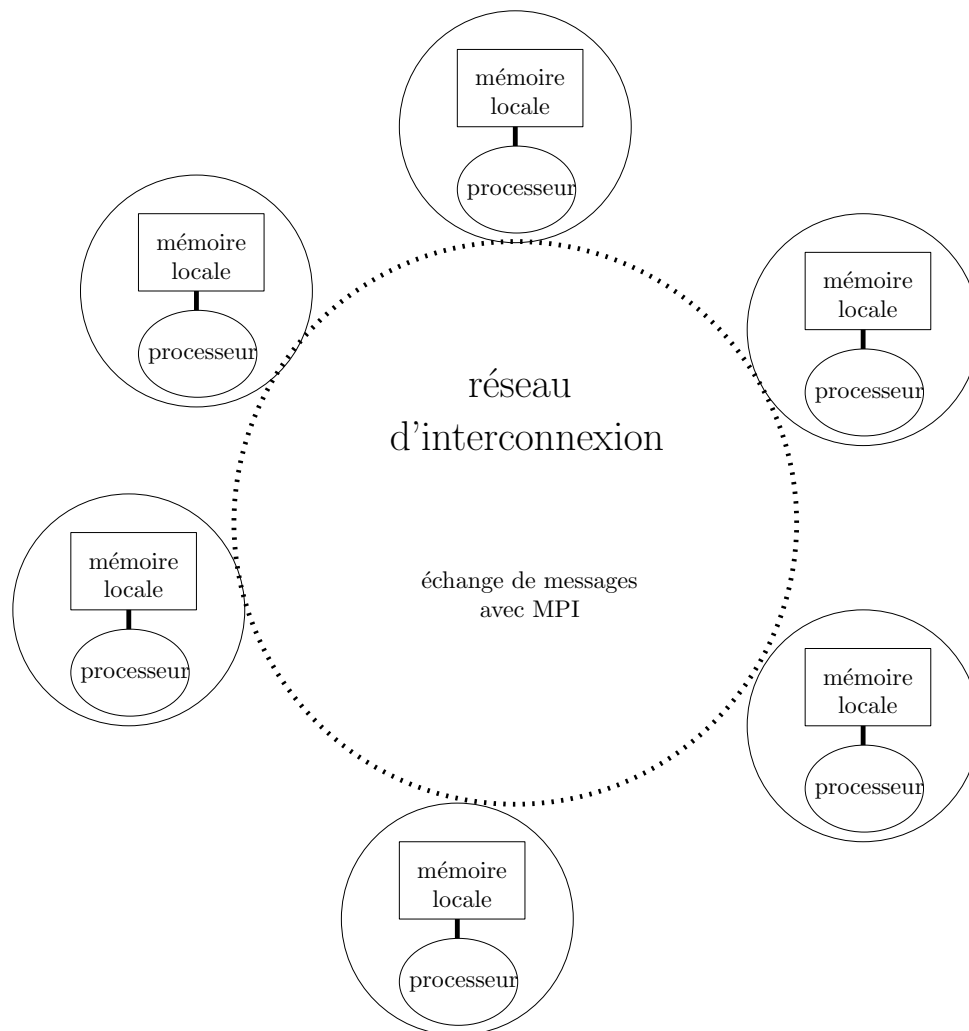


FIGURE 1.1 – Architecture d'une machine parallèle à mémoire distribuée : les nœuds communiquent entre eux par l'envoi et la réception de messages via l'interface standardisée MPI.

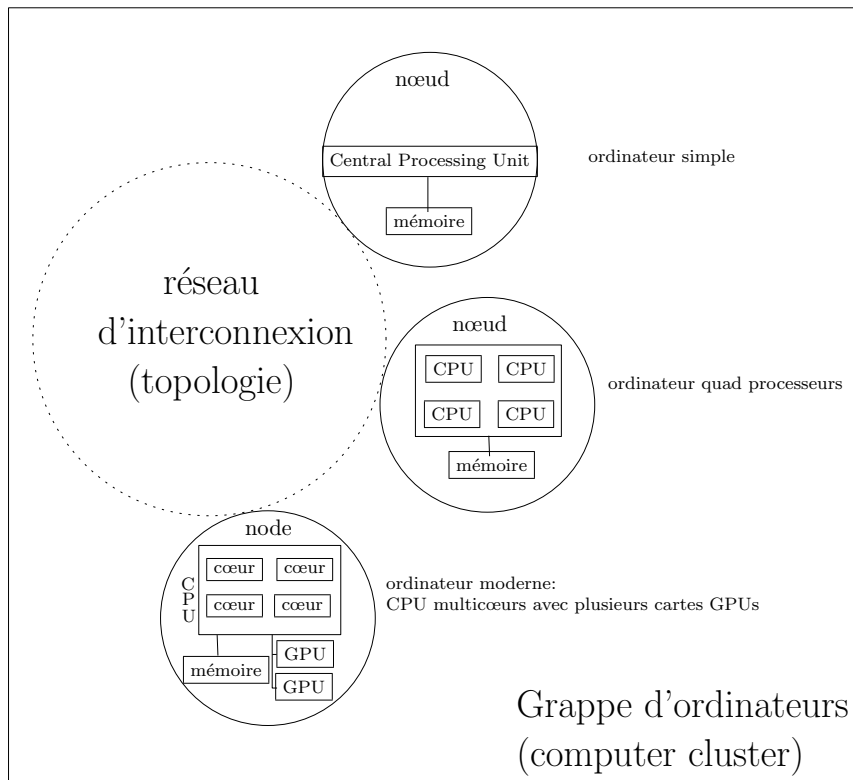


FIGURE 1.2 – Architecture d'une grappe de machines (*computer cluster*, ordinateur parallèle à mémoire distribuée) : les nœuds renferment localement les ressources de calcul et communiquent entre eux grâce au réseau d'interconnexion. Un nœud peut décrire un ordinateur standard simple (un seul CPU), un ordinateur multi-processeurs (plusieurs CPUs), ou alors un ordinateur multi-cœur. En théorie, on idéalise un cluster en considérant tous les nœuds renfermant un ordinateur simple à un seul CPU. Ainsi chaque processus est exécuté sur son propre nœud.

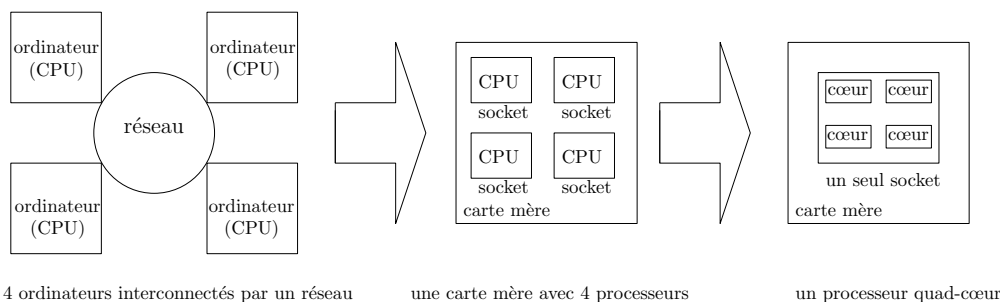


FIGURE 1.3 – Evolution schématisée des architecture des ordinateurs : Un petit nombre d'ordinateurs reliés entre eux par un réseau a d'abord évolué technologiquement en un ordinateur contenant plusieurs processeurs sur sa carte mère, et récemment en un ordinateur avec un processeur multi-cœur.

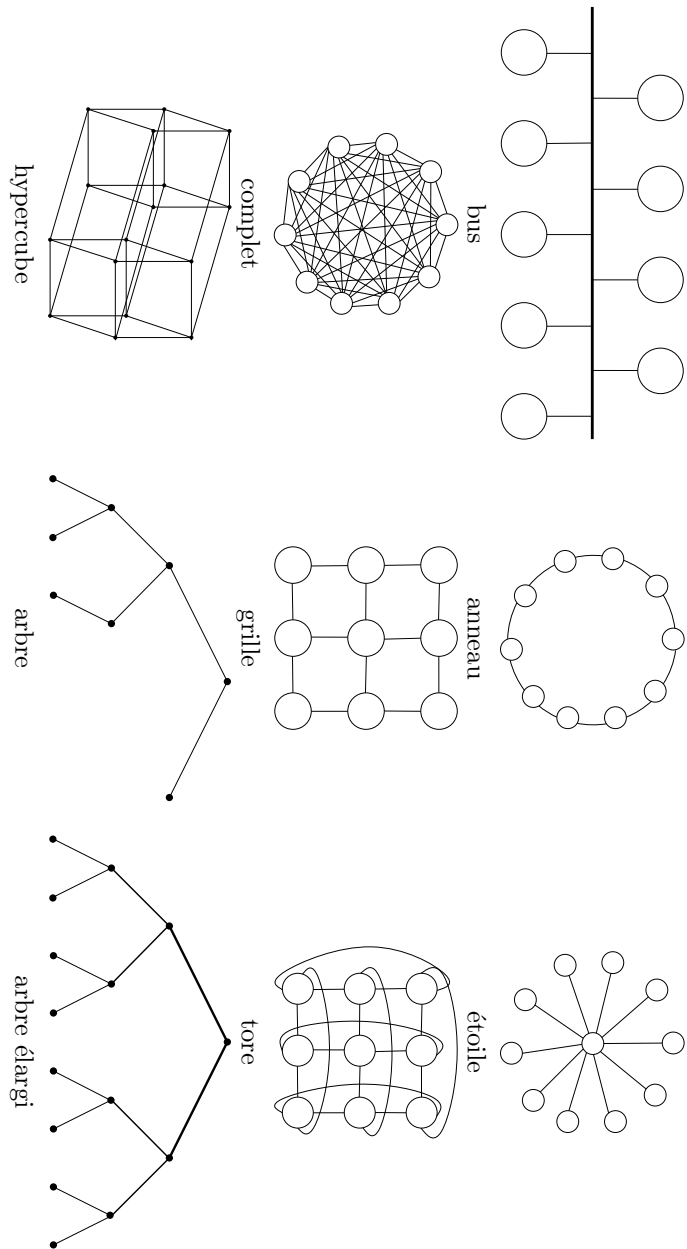


FIGURE 1.4 – Topologies usuelles pour le réseau d'interconnexion dans une architecture de type cluster d'ordinateurs ("machine parallèle") à mémoire distribuée. Les liens de communication peuvent être uni-directionnels ou bi-directionnels.

*vectoriels* (comme les ordinateurs Cray), on utilise le modèle de *programmation vectorielle* (*Single Instruction Multiple Data*, SIMD). Pour les ordinateurs multi-cœur à mémoire partagée, on utilise le *multi-threading* et son interface de programmation standardisée : OpenMP<sup>11</sup> (*Open Multi-Platform shared-memory parallel programming*). Pour les architectures à mémoire distribuée, on utilise le standard MPI<sup>12</sup> (*Message Passing Interface*) pour échanger explicitement des messages contenant des données parmi les processeurs. Enfin, on peut également considérer des modèles de *programmation hybride* qui utilisent le GPU pour certains calculs et ses interfaces dédiées (CUDA, OpenCL, HMPP) ou des modèles de *programmation mixte* où chaque processus MPI utiliserait plusieurs fils de calcul, et des modèles *mixtes et hybrides* (en utilisant donc MPI avec des nœuds multi-cœur équipés de GPU).

## 1.7 L'accélération (le *speed-up*)

### 1.7.1 L'accélération, l'efficacité et l'extensibilité

Soit  $t_{\text{seq}}$  le temps écoulé par le programme séquentiel et  $t_P$  celui passé par le programme parallèle sur  $P$  processeurs. On note  $t_1$  le temps d'exécution par le programme parallèle exécuté en séquentiel avec  $P = 1$  nœud.<sup>13</sup>

On définit les trois quantités suivantes :

- l'*accélération* :  $\text{speedup}(P) = \frac{t_{\text{seq}}}{t_P}$ . Souvent, nous avons  $\frac{t_{\text{seq}}}{t_P} \simeq \frac{t_1}{t_P}$ ,
- l'*efficacité* :  $e = \frac{\text{speedup}(P)}{P} = \frac{t_{\text{seq}}}{P t_P}$  (par rapport au speed-up linéaire),
- l'*extensibilité* ou encore *scalabilité* (traduction littérale de *scalability*) :  $\text{scalability}(O, P) = \frac{t_O}{t_P}$  avec  $O < P$ .

### 1.7.2 La loi d'Amdahl (taille des données fixée)

Gene M. Amdahl (IBM) a caractérisé en 1967 le gain de performance idéal pour une architecture parallèle comme suit : soit  $\alpha_{\text{par}}$  la fraction du code qui peut être parallélisable et  $\alpha_{\text{seq}}$  la fraction de code non parallélisable, c'est-à-dire intrinsèquement séquentiel (*serial code*). Nous avons  $\alpha_{\text{par}} + \alpha_{\text{seq}} = 1$ .

L'accélération (*speed-up*) obtenue par un algorithme parallèle en utilisant  $P$  processeurs (ou nœuds) pour une *taille des données fixée* est (en faisant l'hypothèse raisonnable que  $t_{\text{seq}} = t_1$ ) :

$$\text{speedup}(P) = \frac{t_1}{t_P} = \frac{(\alpha_{\text{par}} + \alpha_{\text{seq}})t_1}{(\alpha_{\text{seq}} + \frac{\alpha_{\text{par}}}{P})t_1} = \frac{1}{\alpha_{\text{seq}} + \frac{\alpha_{\text{par}}}{P}}.$$

Ainsi, lorsque le nombre de processeurs  $P$  tend vers l'infini ( $P \rightarrow \infty$ ), l'accélération est *majorée* par la fraction de code  $\alpha_{\text{seq}} = 1 - \alpha_{\text{par}}$  non-parallélisable comme suit :

$$\lim_{P \rightarrow \infty} \text{speedup}(P) = \frac{1}{\alpha_{\text{seq}}} = \frac{1}{1 - \alpha_{\text{par}}}.$$

La figure 1.5 dessine le graphe de la fonction  $\text{speedup}(P)$  en fonction de  $0 \leq \alpha_{\text{seq}} \leq 1$ . On visualise bien sur la partie extrême droite de ce graphes les accélérations asymptotiques optimales.

11. <http://openmp.org/>

12. <http://www.mpi-forum.org/>

13. Pour faire simple, on suppose dans ce cours que chaque processus tourne sur un processeur distinct qui forme un nœud du réseau de la machine parallèle à mémoire distribuée.

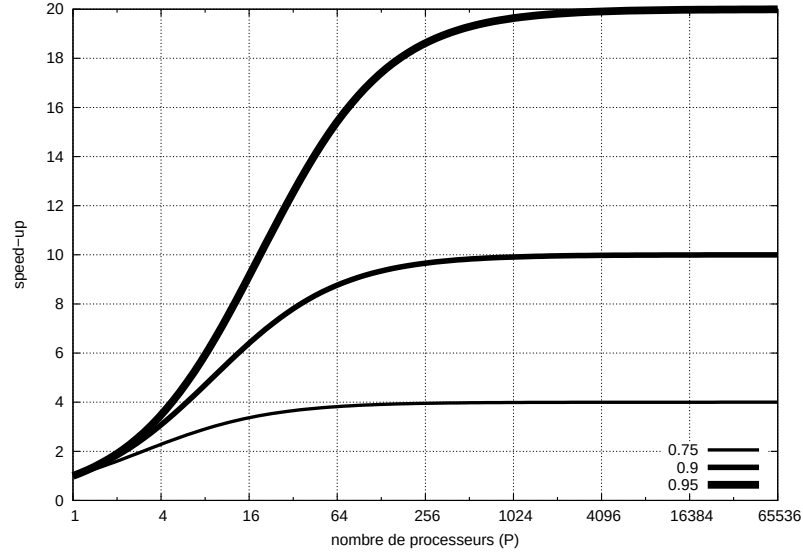


FIGURE 1.5 – Loi d’Amdahl qui caractérise l’accélération en fonction du pourcentage de code parallélisable. Notons l’échelle logarithmique sur l’axe des abscisses. Cette loi démontre que pour un volume de données fixé, l’accélération maximale est bornée par l’inverse de la proportion de code séquentiel.

**Théorème 1.** *La loi d’Amdahl indique que l’accélération optimale d’un programme est asymptotiquement  $\text{speedup} = \frac{1}{\alpha_{\text{seq}}} = \frac{1}{1-\alpha_{\text{par}}}$ , où  $\alpha_{\text{par}}$  est la proportion du programme parallélisable et  $\alpha_{\text{seq}} = 1 - \alpha_{\text{par}}$  la proportion du programme intrinsèquement séquentielle.*

Autrement dit, la loi d’Amdahl montre que le goulot d’étranglement pour paralléliser efficacement un algorithme est sa proportion de code séquentiel (*sequential bottleneck in parallelism*). Quelque soit le nombre de processeurs  $P$  que vous ayez à votre disposition, la limite théorique de l’accélération est l’inverse de la fraction de code séquentiel :  $\text{speedup} = \frac{1}{\alpha_{\text{seq}}}$  (voir la figure 1.6). Notons que l’hypothèse faite est que le volume de données soit fixé. Ce n’est pas une hypothèse forte pour certains programmes qui considère des petites données de taille constante. De plus, nous ne tenons pas en compte la performance en fonction du coût des processeurs. Cette performance décroît fortement lorsque  $P$  croît.

La figure 1.6 illustre le fait que l’accélération maximale soit bornée puisque la partie de code séquentiel est incompressible.

Bien qu’en théorie, l’accélération soit bornée par  $P$ , il se trouve qu’en pratique, on peut parfois mesurer des taux supérieurs! C’est-à-dire, des *accélérations super-linéaires* (*super linear speedup*). Cela peut paraître surprenant mais il n’en est rien car on peut avoir des effets de cache qui sont dus à l’utilisation des différentes mémoires hiérarchiques. Par exemple, lorsqu’on utilise  $P$  machines en parallèle traitant  $n$  données, on peut stocker  $\frac{n}{P}$  données en mémoire vive sur chaque machine alors que sur une seule machine on devrait utiliser le disque dur en plus qui donne un accès de lecture et d’écriture bien plus lent que la mémoire vive (la RAM).

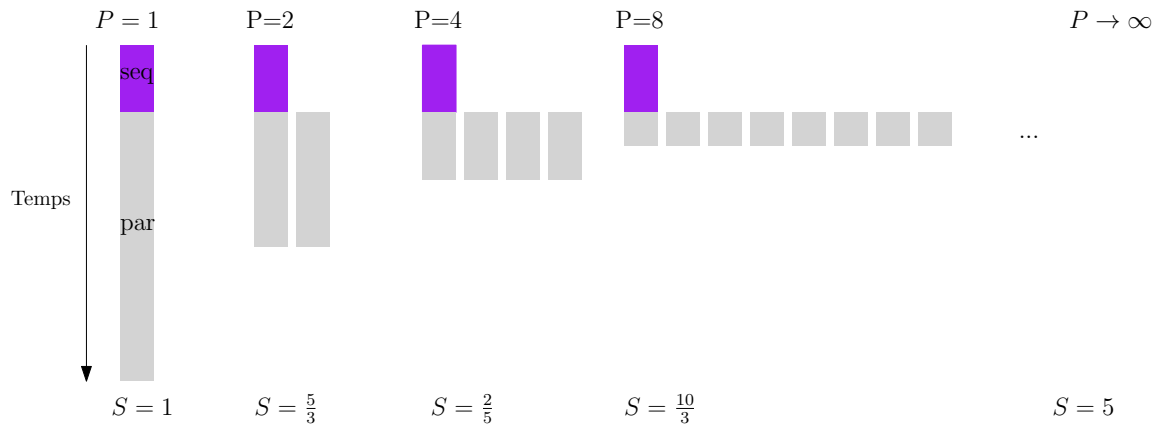


FIGURE 1.6 – La loi d'Amdahl considère l'accélération pour une taille fixe des données, et borne celle-ci par l'inverse de la fraction de code qui est intrinsèquement séquentiel. Ici, l'accélération asymptotique maximale est  $\times 5$ .

### 1.7.3 La loi de Gustafson : tailles des données évoluant avec le nombre de processeurs (*scale speed-up*)

La loi de Gustafson [42] (1988) caractérise un *parallélisme de données*. C'est-à-dire que dans cette analyse, on ne suppose plus que le volume de données soit fixé (comme dans le cas de la loi d'Amdahl), mais plutôt que la taille des données dépende du nombre de processeurs  $P$ . En effet, Gustafson a judicieusement remarqué qu'en pratique les utilisateurs choisissent souvent la taille des données en fonction des ressources informatiques disponibles afin d'obtenir des temps de calcul raisonable.<sup>14</sup> Fixer la taille des données en fonction de la puissance de calcul est communément fait dans l'analyse d'images et de vidéos ou lors de simulations où l'on choisit le pas de la grille. Lorsque l'on dispose de puissance de calcul plus puissante, on augmente ainsi proportionnellement la taille des données.

Soit  $\alpha_{\text{par}}$  la proportion du code parallélisable et  $\alpha_{\text{seq}} = 1 - \alpha_{\text{par}}$  celle du code séquentiel. Puisque l'accélération est le rapport du temps de calcul de l'algorithme séquentiel  $t_{\text{seq}} + Pt_{\text{par}}$  sur le temps de calcul de l'algorithme parallèle  $t_{\text{seq}} + t_{\text{par}}$ , nous avons :

$$\text{speedup}(P) = \frac{t_{\text{seq}} + P \times t_{\text{par}}}{t_{\text{seq}} + t_{\text{par}}}.$$

Puisque  $\frac{t_{\text{seq}}}{t_{\text{seq}} + t_{\text{par}}} = \alpha_{\text{seq}}$  et  $\frac{t_{\text{par}}}{t_{\text{seq}} + t_{\text{par}}} = \alpha_{\text{par}}$ , on en déduit que l'accélération est donnée par :

$$\text{speedup}_{\text{Gustafson}}(P) = \alpha_{\text{seq}} + P \times \alpha_{\text{par}} = \alpha_{\text{seq}} + P \times (1 - \alpha_{\text{seq}}).$$

**Théorème 2.** La loi de Gustafson démontre que l'accélération optimale d'un programme est asymptotiquement  $\text{speedup}(P) = P(1 - \alpha_{\text{seq}})$ , où  $\alpha_{\text{seq}} > 0$  est la proportion du programme séquentiel non parallélisable.

14. En effet, nous ne sommes pas intéressés à lancer un programme qui devrait mettre 100 ans à s'exécuter.

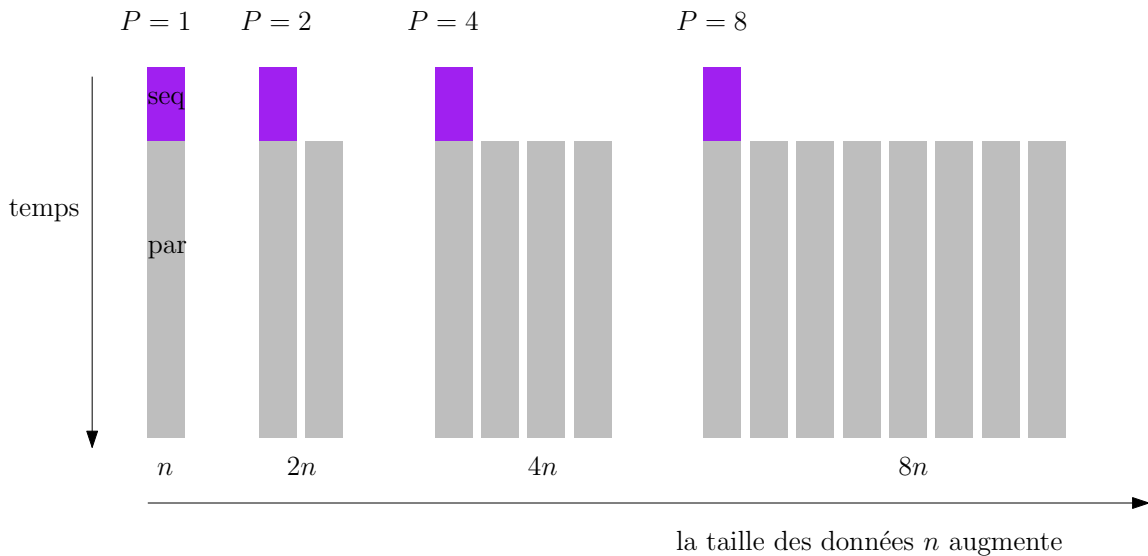


FIGURE 1.7 – Accélération de Gustafson : on considère la taille des données qui augmente en fonction du nombre de processeurs (on fait l'hypothèse que le temps parallèle est constant).

Lorsque la taille des données augmente pour une partie fixe du code séquentiel, l'accélération croît avec le nombre de processeurs. C'est pourquoi l'accélération de Gustafson est encore appelée accélération d'échelle, *scale speed-up*. On observe que la charge de travail (*workload*) augmente avec  $P$  afin de maintenir le temps d'exécution parallèle fixe. Ainsi, la philosophie de Gustafson est de démontrer que la vraie puissance d'un système parallèle se regarde en considérant des jeux de données de plus en plus grands. Ce principe est illustré à la figure 1.7.

#### 1.7.4 Extensibilité et iso-efficacité

On s'intéresse le plus souvent au passage à l'échelle des algorithmes parallèles. En effet, on veut pouvoir tirer partie au mieux des ressources au fur et à mesure que celles-ci deviennent disponibles. Un algorithme parallèle passe à l'échelle, *scalable* en anglais, lorsqu'il est facilement extensible avec  $P$ , le nombre de processeurs. Pour une taille du problème donnée, quand on augmente le nombre de processeurs  $P$ , l'efficacité tend à décroître. Aussi, pour pouvoir conserver un bon speed-up (une bonne accélération) quand  $P$  s'accroît, il est important d'accroître également la taille du problème  $n$  : les valeurs  $n$  et  $P$  sont corrélées.

Cette analyse est celle de l'*iso-efficacité* (*iso-efficiency analysis*). Il se pose alors la question de savoir à quel taux  $\rho$  doit-on augmenter la taille du problème en fonction du nombre d'éléments de calculs (*PEs* : *Processing Elements*) afin de garder l'efficacité constante ? Plus le taux  $\rho$  est petit, meilleur c'est !

#### 1.7.5 Simulation de machines parallèles sur une machine séquentielle

On peut simuler n'importe quel algorithme parallèle sur une architecture séquentielle en exécutant séquentiellement les instructions élémentaires ou les morceaux de code des  $P$  processeurs

$P_1, \dots, P_P$  par étape jusqu'à des *barrières de synchronisation* qui sont des communications bloquantes. Un algorithme parallèle engendre un algorithme séquentiel mais pas nécessairement la réciproque : un algorithme parallèle doit être conçu à la base : il faut penser "informatiquement" au calcul parallèle !

Le fait de pouvoir simuler les algorithmes parallèles sur une machine séquentielle donne plusieurs conséquences en théorie : L'accélération maximale (le speed-up) est en  $O(P)$  où  $P$  est le nombre de processeurs. Le temps de résoudre un problème en parallèle de complexité  $\Omega(c_{\text{seq}})$  en séquentiel est au mieux en  $\Omega(\frac{c_{\text{seq}}}{P})$ . Cela donne donc une borne inférieure.

### 1.7.6 Le BigData et les systèmes de fichiers parallèles

Pour traiter les BigData, on doit lire et sauvegarder de gros fichiers ou énormément de plus petits fichiers : ce sont les opérations d'entrées et de sorties, ou *E/Ss (Input/Output, I/Os)*. Sur le modèle à mémoire distribuée, cette gestion peut-être également faite en parallèle par des E/Ss directement implémentées explicitement dans les programmes (*MPI-IO*), ou bien encore localement par des E/Ss standards explicites sur chaque nœud. Fort heureusement, afin de simplifier ces opérations délicates qui sont très chronophages (*time consuming*), il existe plusieurs *systèmes de fichiers parallèles* qui libèrent les programmeurs de ces tâches explicites : citons le logiciel libre Lustre<sup>15</sup> utilisé par les gros calculateurs, ou bien encore le GPFS<sup>16</sup> (*General Parallel File System*) développé par IBM pour des volumes de données dépassant le péta-octet ( $10^{15}$ ). Le modèle MapReduce, quant à lui, tire partie du système de fichiers GFS (*Google File System*).

## 1.8 Huit fausses idées sur le HPC

L'histoire des systèmes distribués et des réseaux à grande échelle a débuté en 1969 avec le réseau ARPANET qui a donné lieu à Internet. Ensuite, vint en 1973 le protocole de transfert monétaire mondial SWIFT<sup>17</sup> (*Society for Worldwide Interbank Financial Telecommunication*). Après presque un demi-siècle d'expérience, force est de constater qu'il est toujours difficile de concevoir et déployer de gros systèmes distribués. Peter Deutsch (SUN, 1994) et James Gosling (SUN, 1997) ont listé huit fausses idées sur le HPC qui sont :

1. le réseau est fiable,
2. le temps de latence est nul,
3. la bande passante est infinie,
4. le réseau est sûr,
5. la topologie du réseau ne change pas,
6. il y a un et un seul administrateur réseau,
7. le coût de transport est nul,
8. le réseau est homogène.

Nous allons brièvement expliquer la teneur de ces idées préconçues qui ne sont pas valides dans la pratique :

---

15. <http://lustre.opensfs.org/>

16. <http://www-03.ibm.com/systems/platformcomputing/products/gpfs/>

17. [http://www.swift.com/about\\_swift/company\\_information/swift\\_history](http://www.swift.com/about_swift/company_information/swift_history)

1. **Le réseau est fiable.** Les applications critiques doivent être utilisables 7 jours sur 7 et donc avoir une tolérance aux pannes nulle (*zero tolerance*). Mais quand un commutateur<sup>18</sup> (*switch*) tombe en panne, des réactions en cascade s'enchaînent aboutissant à une panne générale qui peuvent créer des dégâts catastrophiques. Afin de se prémunir contre ce risque (du moins le minimiser!), il faut introduire de la *redondance* à la fois dans le matériel et le logiciel, et évaluer le risque en fonction de l'investissement.
2. **Le temps de latence est nul.** Bien que la latence soit bonne dans les LANs (*Local Area Network*), elle devient plus mauvaise dans les WANs (*Wide Area Network*). Ces onze dernières années, la bande passante des réseaux a été multipliée par environ 1500 mais la latence divisée par 10 seulement. La latence est intrinsèquement limitée par la vitesse de la lumière dans les fibres optiques. En effet, considérons le temps d'un *ping*, c'est-à-dire un aller-retour entre deux points extrêmes de la Terre (*rtt*, *round trip time*) : cela demande 0,2 seconde ( $20000 \text{ km} \times 2 \times 5\mu\text{s}/\text{km} = 0,2\text{s}$ , sans compter les calculs!), et 40 milli-seconde entre New York et Los Angeles. Il faut donc être très vigilant à la latence quand on déploie un logiciel en dehors des LANs!
3. **La bande passante est infinie.** Bien que la bande passante augmente, l'information qu'on y fait circuler aussi! (comme les vidéos 4K et très bientôt la 8K) Les algorithmes de routage rencontrent des problèmes de congestion et en pratique on a des problèmes de perte de paquets dans les WANs.
4. **Le réseau est sûr.** Fort heureusement, au vu des récents événements qui défilent dans les journaux, plus personne n'est assez naïf pour le croire! Pire, les attaques augmentent exponentiellement (parfois, plus de 100 par semaine dans les grands groupes industriels). Il faut donc faire des sauvegardes, avoir des solutions de secours/replis, des plans B, des plans C, etc.
5. **La topologie du réseau ne change pas.** Bien que l'on peut supposer une topologie statique lors du développement d'une solution, on ne contrôle pas la topologie ni sur les WANs et ni sur Internet! Cela veut dire que l'on doit considérer la topologie comme étant en changement perpétuel (topologie dynamique) pour les solutions en production.
6. **Il y a un et un seul administrateur réseau.** Même si on suppose que l'administrateur ne tombe jamais malade et puisse répondre à toutes les requêtes dans les plus brefs délais, force est de constater qu'aujourd'hui un expert est un expert dans quelques domaines seulement, et que les architectures et les logiciels multiples demandent dans la pratique beaucoup d'administrateurs car il faut une multitude de compétences.
7. **Le coût de transport est nul.** Bien entendu, le coût d'un réseau en câblage et autres équipements n'est pas gratuit et si l'on veut une bonne bande passante, il faut payer pour un transport qui garantit une qualité de service (*quality of service*, QoS). Aussi, passer du niveau de l'application tournant sur un simple nœud (*single node*) au niveau de l'application répartie sur plusieurs nœuds, demande de savoir faire du transport des données de l'application sur le réseau. Il faut donc sérialiser en bits et en octets des données structurées afin de pouvoir les transmettre entre plusieurs nœuds. C'est un coût logiciel supplémentaire de développement.
8. **Le réseau est homogène.** Les grands parcs informatiques sont hautement hétérogènes en configuration de machines (matériels, systèmes d'exploitation, logiciels) et des interopérabilités sont requises afin de pouvoir utiliser toutes ces ressources harmonieusement.

18. Pourtant le matériel est souvent garanti à vie, avec un temps entre deux pannes estimé à plus de 50000 heures (MTBF, *Mean Time Between Failure*). Le LIX a connu une telle panne en 2014.

## 1.9 \* Pour en savoir plus : notes, références et discussion

Ce chapitre couvre les notions élémentaires du calcul haute performance (le HPC). On retrouve donc les notions abordées dans les principaux ouvrages traitant du parallélisme [60, 50]. La loi d'Amdahl [2] date de 1967, et considère le volume de données constant. Elle fut revisitée par Gustafson [42] en 1988 qui proposa de faire varier le volume de données en fonction du nombre de processeurs. Ces deux lois sont unifiées par la loi générique de Sun et Ni [50] qui repose sur un modèle de mémoire finie. Avec l'avènement des architectures multi-cœurs, la loi d'Amdahl a été étendu pour prendre en considération le nombre de cœurs [47] et des critères d'énergie [96]. Le HPC a pour but de délivrer des solutions *plus efficaces* en distribuant à la fois les calculs et les données : cela permet de faire des plus grandes simulations ou de traiter de plus grands volumes de données (qui ne pourraient pas tenir dans l'espace mémoire d'une seule machine). Un autre avantage (surtout industriel et moins avouable scientifiquement) concerne la *rapidité d'obtention des résultats* pour l'analyse de données : parfois, il est plus "simple" d'implanter un algorithme parallèle naïf plutôt qu'un algorithme séquentiel plus complexe à déboguer. On obtient ainsi les résultats plus rapidement en utilisant un gros cluster de machines (avec des algorithmes parallèles simples et donc rapides à implanter par des ingénieurs<sup>19</sup> développement logiciel) plutôt que de passer beaucoup plus de temps à implanter et tester une solution plus complexe sur une seule machine multi-cœur (coût de développement). Les huit fausses idées sur les systèmes distribués sont décrites dans [81]. Avec l'avènement des BigData, on cherche à traiter le plus grand volume de données le plus rapidement possible : le Big&FastData vise une analyse en temps réel des *flux de données* (comme les *tweets*) minimisant le temps de latence.

## 1.10 En résumé : ce qu'il faut retenir !

Le traitement massif des données utilise les super-ordinateurs dont on mesure la performance en FLOPS (*F*loating-*P*oint *O*peration *P*er *S*econd) : le nombre d'opérations arithmétiques à virgule flottante par seconde. Dans le calcul haute performance (HPC), on distingue les ordinateurs à mémoire partagée qui utilisent les fils de calcul des ordinateurs à mémoire distribuée dont les nœuds de calcul sont reliés par un réseau d'interconnexion. Sur ces architectures à mémoire distribuée, l'échange de données se fait explicitement par l'envoi et la réception de messages via une interface standardisée, la MPI, qui inclut les opérations de communication dont l'efficacité dépend de la topologie du réseau sous-jacent d'interconnexion, des bandes passantes des liens, et des temps de latence. La loi d'Amdahl suppose que le volume de données est fixé, et démontre mathématiquement que l'accélération est bornée asymptotiquement par l'inverse de la proportion de code qui n'est pas parallélisable. La loi de Gustafson, quant à elle, fait varier la charge de travail (*workload*) en fonction du nombre de processeurs (pour cette raison, on appelle cette accélération le *scale speed-up* en anglais) : cette hypothèse suppose que l'on peut augmenter la taille des données, par exemple en maillant plus finement ou alors en prenant de plus grandes images/vidéos (4K, 8K, etc.) Actuellement, nous sommes dans la course à l'exaflops que l'on espère atteindre vers 2017-2020.

Notations :

$n$     taille des données

---

19. En anglais, *Software Development Engineer* (SDE).

|                            |                                                                                                                                                         |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| $P$                        | nombre de processeurs (ou de processus, ou de nœuds)                                                                                                    |
| $t(n)$                     | temps séquentiel pour une taille $n$ , ou $t_s(n)$                                                                                                      |
| $t_P(n)$                   | temps parallèle sur $P$ processeurs pour un volume de $n$ données                                                                                       |
| $t_{\text{seq}}(n)$        | temps séquentiel pour $n$ données                                                                                                                       |
| $t_{\text{par}}(n)$        | temps parallèle pour $n$ données (avec $P$ implicite)                                                                                                   |
| $c_P(n)$                   | coût : travail global effectué par tous les processeurs : $c_P(n) = Pt_P(n)$                                                                            |
| $t_1(n)$                   | temps d'un algorithme parallèle sur un processeur. $t_1(n) \geq t(n)$ , $t_1(n) \approx t(n)$                                                           |
| $S_P(n)$                   | accélération pour $P$ processeurs : $S_P(n) = \frac{t(n)}{t_P(n)}$                                                                                      |
| $E_P(n)$                   | efficacité : $E_P(n) = \frac{t(n)}{c_P(n)} = \frac{S_P(n)}{P} = \frac{t(n)}{Pt_P(n)}$                                                                   |
| $\alpha_{\text{par}}$      | proportion du code parallèle ( $\alpha_{\text{par}} = 1 - \alpha_{\text{seq}}$ )                                                                        |
| $\alpha_{\text{seq}}$      | proportion du code séquentiel ( $\alpha_{\text{seq}} = 1 - \alpha_{\text{par}}$ )                                                                       |
| $S_A(P)$                   | loi d'Amdahl pour l'accélération (volume $n$ fixé) : $\frac{1}{\alpha_{\text{seq}} + \frac{\alpha_{\text{par}}}{P}} \leq \frac{1}{\alpha_{\text{seq}}}$ |
| $S_G(P)$                   | accélération de Gustafson's ( <i>scale speed-up</i> ) : $S_G(P) = \alpha_{\text{seq}} + (1 - \alpha_{\text{seq}})P$                                     |
| $\text{scalability}(O, P)$ | extensibilité générique $\frac{t_O(n)}{t_P(n)}$ avec $O < P$                                                                                            |

## 1.11 Exercices

### Exercice 1 : La loi d'Amdahl

Supposons qu'un programme puisse être parallélisé à 90%. Calculer l'accélération asymptotique en utilisant la loi d'Amdahl. Sachant que le programme séquentiel s'exécute en 10 heures, en déduire le temps critique qu'aucun algorithme parallèle ne puisse battre ? En déduire d'une deuxième façon l'accélération maximale. Mêmes questions avec cette fois 1% du code qui peut être parallélisé.

### Exercice 2 : Accélération et loi d'Amdahl

Soit  $\mathcal{P}$  un programme séquentiel qui est à 75% parallélisable.

- Quelle est l'accélération théorique pour  $P = 3$  processeurs, et  $P = 4$  processeurs ?
- Quelles sont l'accélération et l'efficacité théoriques asymptotiques quand  $P \rightarrow +\infty$  ?

Soit maintenant donné un programme séquentiel  $\mathcal{P} = \mathcal{P}_1; \mathcal{P}_2$  qui est constitué de deux routines consécutives  $\mathcal{P}_1$  et  $\mathcal{P}_2$ . La première routine est parallélisable à 50% et demande un temps  $t_1$ . La seconde routine est parallélisable à 75% et demande un temps  $t_2$ . De plus, pour l'algorithme séquentiel, la seconde routine prend toujours deux fois plus de temps que la première routine :  $t_2 = 2t_1$ .

- Quelle est l'accélération que l'on obtient pour  $P$  processeurs ?
- En déduire une valeur asymptotique de l'accélération quand  $P \rightarrow \infty$ .

On considère un algorithme parallèle sur un jeu de données de taille  $n$  qui a une complexité en temps parallèle pour  $P$  processeurs de  $O_{\parallel} \left( \frac{n^2}{\sqrt{P}} \right)$ . On supposera que les  $n$  données tiennent toutes dans les mémoires locales des processeurs. Vous disposez en pratique d'un cluster de machines avec seulement  $P_0 \ll P$  processeurs pour faire tourner le programme avec  $P$  processus (`mpirun -np P programme`) (avec  $P$  supposé un multiple de  $P_0$  :  $P \bmod P_0 = 0$ ).

1. En simulant un “cluster virtuel” à  $P$  machines sur un “cluster physique” à  $P_0$  machines (en mappant plusieurs processus sur un même processeur), quelle est la complexité du temps de calcul de l’algorithme parallèle avec  $P$  processus sur ce cluster physique à  $P_0$  machines ?

**Exercice 3 :** *Estimation de la fraction du code parallélisable pour la loi d’Amdahl*

Montrez que l’on peut estimer la fraction du code parallélisable en utilisant cette formule :  $\widehat{\alpha_{\text{par}}} = \frac{\frac{1}{\hat{S}} - 1}{\frac{1}{P} - 1}$ , où  $\hat{S}$  est l’accélération mesurée pour  $P$  processeurs. En déduire une formule pour l’accélération maximale connaissant  $\hat{S}$  mesuré pour  $P$  processeurs.

**Exercice 4 :** *Borne supérieure pour la loi d’Amdahl*

Prouvez que pour un nombre arbitraire de processeurs  $P$ , l’accélération est majoré par  $\frac{1}{\alpha_{\text{seq}}}$  avec  $\alpha_{\text{seq}}$  la proportion relative de code séquentiel.



## Chapitre 2

# Introduction à l'interface MPI : Message Passing Interface

Un résumé des points essentiels à retenir est donné dans §2.12.

### 2.1 L'interface MPI pour la programmation parallèle : communication par messages

La programmation d'algorithmes parallèles est beaucoup plus délicate que la programmation d'algorithmes séquentiels (ainsi que son débogage!). En effet, il existe plusieurs modèles de “machines parallèles” avec des types différents de programmation : par exemple,

- les *super-calculateurs vectoriels* avec la programmation *Single Instruction Multiple Data* (SIMD), et l'optimisation de code par opérations pipelinées,
- les *machines multi-cœurs* à mémoire partagée et la programmation avec des fils de calcul (*multi-threads*) sur de la mémoire partagée,
- les *machines interconnectées* par un réseau de connections qui ont une mémoire distribuée et forment une grappe d'ordinateurs : un cluster.

C'est ce dernier modèle de machines parallèles qui nous intéresse : la programmation parallèle avec une *mémoire distribuée*. Chaque ordinateur (processeur) peut exécuter des programmes avec sa propre mémoire locale en coopérant. On distingue la programmation de grappes de petites tailles (disons quelques dizaines/centaines voire milliers d'ordinateurs) qui communiquent par l'envoi et la réception de messages, de la programmation de grappes massives (quelques milliers/centaines voire même des millions d'ordinateurs/processeurs/cœurs) qui exécutent des codes beaucoup plus simples (programmation de type MapReduce/Hadoop) pour des applications plus ciblées traitant de grandes masses de données.

Le standard Message Passing Interface (MPI en raccourci) est une interface de programmation (*Application Programming Interface*, API) qui définit la syntaxe et la sémantique d'une bibliothèque de routines standardisées. Cela permet d'écrire des programmes utilisant des *échanges de messages* contenant les données à transférer. Standardiser l'API a l'avantage de ne pas dépendre du langage sous-jacent. On peut donc utiliser les commandes MPI avec les langages usuels : Java, C, C++,

Fortran<sup>1</sup>, etc. (plusieurs *bindings* de l'API sont disponibles).

Le MPI a historiquement vu le jour lors d'un atelier sur les environnements à mémoire distribuée en 1991. Aujourd'hui, nous utilisons la version 3 (MPI-3) dont la standardisation a été publiée en 2008. Nous utilisons OpenMPI (<http://www.open-mpi.org/>) en salles machine avec le C++. L'interface MPI a donné lieu à un modèle de programmation stable dominant dans le monde du HPC. Les points forts de MPI sont la standardisation des nombreuses opérations de *communications globales* (comme la diffusion, *broadcast*, d'un message à tous les autres processus<sup>2</sup>) et les opérations de *calculs collaboratifs* (comme un *Allreduce* qui calcule la somme cumulée des données distribuées et renvoie le résultat à tous les processus). La complexité de ces opérations dépend de la topologie sous-jacente du réseau d'interconnexion des machines.

## 2.2 Modèles de programmation parallèle, fils de calcul et processus

Les systèmes d'exploitation modernes sont *multi-tâches* : plusieurs applications dites *non-bloquantes* semblent tourner en "même temps". C'est une illusion donnée à l'utilisateur car sur l'unité centrale de calcul (CPU), il ne peut y avoir qu'un morceau de programme qui tourne à la fois. C'est-à-dire, que sur le CPU, un processus est en *cours d'exécution* pendant que les autres processus sont soit bloqués (suspendus, en attente de réveil) soient prêts à être exécutés et attendant leur tour. C'est l'*ordonnanceur* qui a la tâche d'allouer les processus au CPU au fil du temps.

Les CPUs modernes sont dotés de plusieurs cœurs et permettent pour chaque application (processus) d'utiliser plusieurs *fils de calcul* : c'est le *multi-threading* qui donne lieu à l'exécution concurrente des fils. Par exemple, notre navigateur Web permet de visualiser plusieurs pages simultanément dans des onglets : chaque page Web donne lieu à un *fil de calcul* (*thread*) pour récupérer et afficher les données HTML<sup>3</sup>/XML. Les ressources allouées à un processus sont *partagées* entre les différents fils qui le composent, et un processus a au moins un fil qui contient la fonction principale `main`. On peut différencier les processus des fils de calcul comme suit :

- Les fils de calcul d'un même processus partagent la *même zone mémoire*, à la fois pour le code mais aussi pour les données. Il est donc facile d'accéder aux données entre plusieurs fils, mais cela pose certaines difficultés liées éventuellement à l'accès simultané de la mémoire : source de plantage ! Une abstraction théorique est le modèle *PRAM* (pour *Parallel Random-Access Machine*). Sur le modèle PRAM, on distingue les conflits pendant les opérations de lecture et d'écriture. On distingue les sous-modèles EREW (*Exclusive Read Exclusive Write*), CREW (*Concurrent Read Exclusive Write*), et CRCW (*Concurrent Read Concurrent Write*)
- Le modèle de programmation parallèle par fils de calcul est bien adapté aux architectures multi-cœurs d'un même processeur, et permet des applications plus rapides (par exemple, l'encodage de vidéo ou de musique) et des applications non-bloquantes (par exemple, un navigateur web).
- Les processus ont leurs *propres zones mémoires d'adressage*. La communication entre processus doit se faire de façon méthodique, notamment avec le standard MPI.

---

1. Fortran (pour *FORmula TRANslating system*) est souvent utilisé par les physiciens pour les simulations numériques.

2. Si le nombre de machines est égal au nombre de processus, on peut supposer que sur chaque machine tourne un processus. Sinon, on peut aussi avoir plusieurs processus (avec leur propre zone de mémoire non partagée) qui tourne sur une même machine (*single-node mode*).

3. Hypertext Markup Language

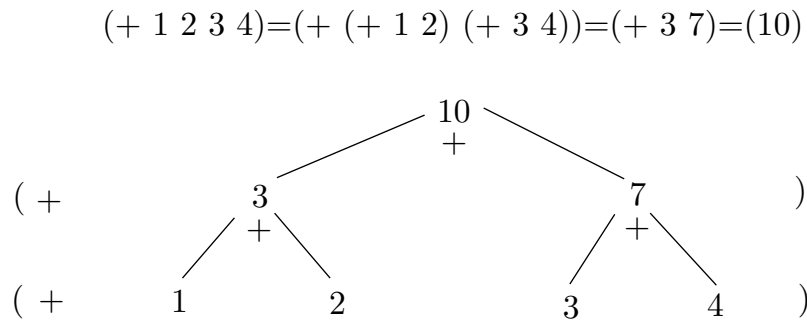


FIGURE 2.1 – Exemple d’un calcul global collaboratif : on calcule la somme cumulative des valeurs d’une variable locale associée à chaque processus. On montre ici l’arbre de réduction effectuée lorsque l’on appelle cette primitive **reduce**.

On distingue aussi la programmation parallèle *SPMD* (*Single Program Multiple Data*) de celle *MPMD* (*Multiple Program Multiple Data*). Notons que l’on peut faire tourner plusieurs processus sur un même processeur (en parallèle si le processeur est multi-cœur) ou sur un ensemble de processeurs reliés entre eux par un réseau d’interconnexion. On peut également répartir les processus sur plusieurs processeurs multi-cœurs (en utilisant la programmation MPI+OpenMP dans ce cas là).

## 2.3 Les communications collectives

Dans un programme MPI, outre les calculs locaux de chaque processus, on a aussi :

- des mouvements de données : par exemple, la diffusion d’un message contenant des données à tous les autres processus (*broadcast* en anglais),
- de la synchronisation par barrières où les processus s’attendent tous avant de pouvoir continuer,
- des calculs globaux : par exemple, une opération de réduction (*reduce* en anglais) qui calcule le minimum d’une variable distribuée  $x$  répartie sur tous les processus, ou alors une somme globale (Figure 2.1).

Les opérations de communication globale concernent tous les processus d’un *groupe de communication*. Par défaut, lors de l’initialisation de MPI, tous les processus se retrouvent dans un même groupe de communication appelé `MPI_COMM_WORLD`.

### 2.3.1 Quatre opérations collectives de base

On distingue les opérations qui *diffusent* des données d’un processus dit *processus racine* à tous les autres, des opérations qui *agrègent* les données réparties sur chaque processus à un processus ou à tous les processus. Le *broadcast* (diffusion, `MPI_Bcast`) diffuse un message à tous les autres. La diffusion personnalisée (*scatter*, `MPI_Scatter`), quant à elle, partitionne les données en morceaux et envoie chaque morceau au processus cible.

Les opérations d’agrégation peuvent soit être de type communication soit de type calcul : Le *gather* reçoit les messages répartis en morceaux sur les différents processus et assemble ces messages

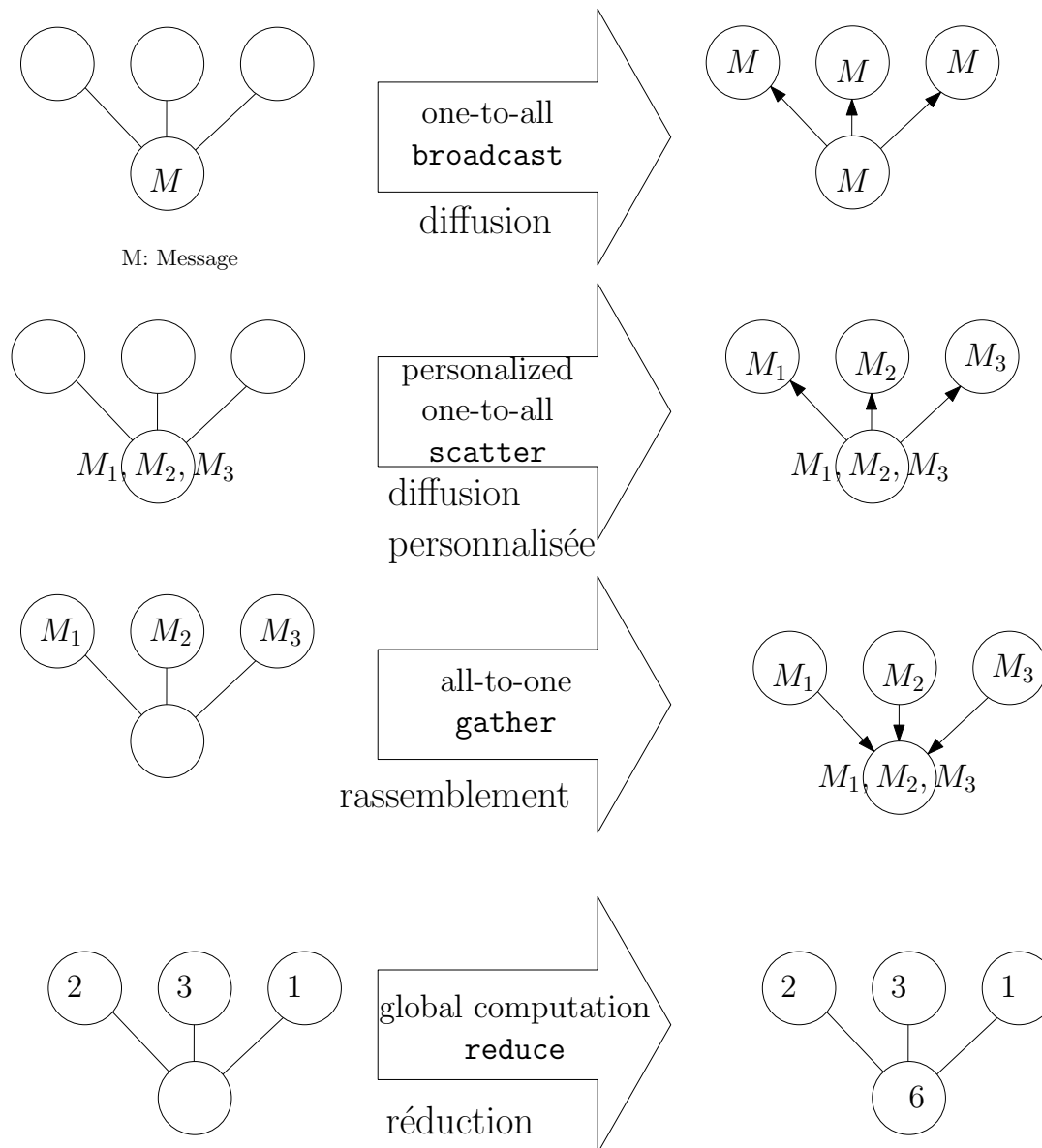


FIGURE 2.2 – Quatre opérations collectives de communication : la diffusion (*broadcast*), la diffusion personnalisée (*scatter*), le rassemblement (*gather*) et la réduction (*reduce*) (ici, avec l'opérateur binaire  $+$ , pour calculer la somme cumulée  $\sum$ ).

| nom        | signification          |
|------------|------------------------|
| MPI_MAX    | maximum                |
| MPI_MIN    | minimum                |
| MPI_SUM    | sum                    |
| MPI_PROD   | product                |
| MPI_LAND   | logical and            |
| MPI_BAND   | bit-wise and           |
| MPI_LOR    | logical or             |
| MPI BOR    | bit-wise or            |
| MPI_LXOR   | logical xor            |
| MPI_BXOR   | bit-wise xor           |
| MPI_MAXLOC | max value and location |
| MPI_MINLOC | min value and location |

TABLE 2.1 – Les opérations de calcul global prédéfinies en MPI pour la réduction.

en un message unique pour le processus cible (inverse de l'opération *scatter*). Le *reduce* (**MPI\_Reduce**) permet de faire un calcul global en réduisant des valeurs par un opérateur binaire commutatif<sup>4</sup> comme la somme (**MPI\_SUM**) ou le produit (**MPI\_PROD**), etc. La liste de tels opérateurs d'agrégation se trouve à la table 2.1. Ces quatre opérations basiques sont résumées graphiquement à la figure 2.2. Enfin, on peut également faire une opération *all-to-all* (*total exchange* avec **MPI\_Alltoall**).

### 2.3.2 Communications point à point bloquantes et non-bloquantes

Notons les deux opérations de communication de base par **send** et **receive** dont la syntaxe est résumée comme suit :

- **send(&data, n, Pdest)** : Envoie un tableau de  $n$  données pointé par l'adresse **&data** au processeur **Pdest**
- **receive(&data,n, Psrc)** : Reçoit  $n$  données à l'adresse tableau pointée par **&data** du processeur **Psrc**

Que se passe-t-il dans cet exemple ?

|                                                   |                                   |                                                                         |                                   |
|---------------------------------------------------|-----------------------------------|-------------------------------------------------------------------------|-----------------------------------|
| <pre> ... a=442; send(&amp;a, 1, P1); a=0; </pre> | <p>Processus <math>P_0</math></p> | <pre> ... receive(&amp;a, 1, P0); cout &lt;&lt; a &lt;&lt; endl; </pre> | <p>Processus <math>P_1</math></p> |
|---------------------------------------------------|-----------------------------------|-------------------------------------------------------------------------|-----------------------------------|

Les communications bloquantes (non-bufferisées) provoquent une situation d'*attente* (*idling*). En effet, le processus envoyeur et le processus receveur doivent s'attendre *mutuellement* : c'est le mode de communication par *rendez-vous* (*hand-shake*). Ce mode permet d'avoir des *communications synchrones* (*synchronous communications*). Cette situation est illustrée à la figure 2.3

4. Un exemple d'opérateur binaire non-commutatif est la division :  $p/q \neq q/p$ .

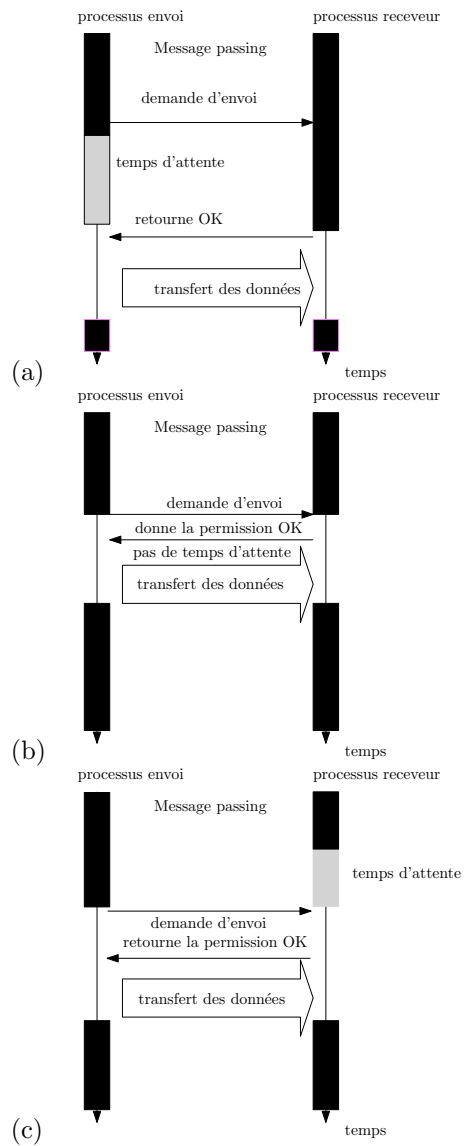


FIGURE 2.3 – Communications bloquantes par rendez-vous : (a) le processus envoyeur attend le “OK” du receveur et provoque une situation d’attente, (b) on cherche à minimiser ces temps d’attente, et (c) configuration où c’est le processus de réception qui se met en attente.

Le programme<sup>5</sup> C ci-dessous montre un exemple élémentaire de communication bloquante en MPI (avec l'API en C) :

Code 1 – Exemple de communication bloquante en MPI : `commbloq442.cpp`

```
// Nom du programme : commbloq442.cpp
// Compilez avec mpic++ commbloq442.cpp -o commbloq442.exe
// Exécutez avec mpirun -np 2 commbloq442.exe

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>
#include <math.h>

int main(int argc, char ** argv)
{
    int myid, numprocs;
    int tag, source, destination, count;
    int buffer;
    MPI_Status status;

    MPI_Init(&argc, &argv);

    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &myid);

    tag=442;
    source=0;
    destination=1;
    count=1;

    if(myid == source){
        buffer=2015;
        MPI_Send(&buffer, count, MPI_INT, destination, tag, MPI_COMM_WORLD);
        printf("Le processeur %d a envoye %d\n", myid, buffer);
    }

    if(myid == destination){
        MPI_Recv(&buffer, count, MPI_INT, source, tag, MPI_COMM_WORLD, &status);
        printf("Le processeur %d a reçu %d\n", myid, buffer);
    }

    MPI_Finalize();
}
```

L'exécution de ce programme renvoie à la console l'affichage suivant :

Le processeur 0 a envoye 2015

Le processeur 1 a reçu 2015

Pour des communications bloquantes, on cherche donc à minimiser les temps d'attente. Plus tard, on verra l'*équilibrage de charge*, le *load balancing*.

On donne la syntaxe d'appel de la primitive `send`<sup>6</sup> en MPI :

5. Avant la standardisation moderne du langage C, on peut trouver des codes C pour lesquels les arguments de procédures sont déclarés après le prototypage des procédures. Par exemple, on peut écrire dans l'ancien style C : `int main(argc, argv) int argc; char *argv[];`

6. Voir le manuel [https://www.open-mpi.org/doc/v1.4/man3/MPI\\_Send.3.php](https://www.open-mpi.org/doc/v1.4/man3/MPI_Send.3.php)

| type MPI           | type dans le langage C |
|--------------------|------------------------|
| MPI_CHAR           | signed char            |
| MPI_SHORT          | signed short int       |
| MPI_INT            | signed int             |
| MPI_LONG           | signed long int        |
| MPI_UNSIGNED_CHAR  | unsigned char          |
| MPI_UNSIGNED_SHORT | unsigned short int     |
| MPI_UNSIGNED       | unsigned int           |
| MPI_UNSIGNED_LONG  | unsigned long int      |
| MPI_FLOAT          | float                  |
| MPI_DOUBLE         | double                 |
| MPI_LONG_DOUBLE    | long double            |
| MPI_BYTE           |                        |
| MPI_PACKED         |                        |

TABLE 2.2 – Types de bases MPI pour l’interface en C.

— Syntaxe en C :

```
#include <mpi.h>
int MPI_Send(void *buf, int count, MPI_Datatype datatype, int dest, int tag,
             MPI_Comm comm)
```

— Syntaxe en C++ (plus mis à jour depuis MPI-2<sup>7</sup>) :

```
#include <mpi.h>
void Comm::Send(const void* buf, int count, const Datatype& datatype, int dest,
                int tag) const
```

Le paramètre `tag` attribue au message un entier (integer) qui est utile pour la filtration et l’appariement des opérations `send/receive`.

Les différents types de données MPI en C sont résumés dans la table 2.2.

### 2.3.3 Les situations de blocage (*deadlocks*)

Utiliser les communications bloquantes permet d’associer les requêtes d’envoi avec celles de réception mais on peut aboutir à des *situations de blocage*<sup>8</sup> (*deadlocks*).

Regardons ce qui se passe dans ce petit exemple :

|                                      |                                      |
|--------------------------------------|--------------------------------------|
| Processus $P_0$                      | Processus $P_1$                      |
| <code>send(&amp;a, 1, P1);</code>    | <code>send(&amp;a, 1, P0);</code>    |
| <code>receive(&amp;b, 1, P1);</code> | <code>receive(&amp;b, 1, P0);</code> |

Le processus  $P_0$  envoyeur attend le message “OK pour envoi” du processus receveur  $P_1$ , mais

7. On préconise donc d’utiliser l’API en C de MPI. Le standard actuel est MPI-3. Voir [http://meetings.mpi-forum.org/MPI\\_3.0\\_main\\_page.php](http://meetings.mpi-forum.org/MPI_3.0_main_page.php)

8. Dans ce cas, soit un signal *time-out* arrête les processus, soit on doit manuellement arrêter les processus en tuant les numéros de processus avec la commande du shell `kill`.

la requête d'envoi de  $P_1$  attend aussi le "OK pour envoi" du processus  $P_0$ . C'est typiquement une situation de blocage. Nous avons présenté ce petit exemple pédagogique simplifié afin de mettre en garde contre les situations de blocage qui peuvent arriver lorsqu'on utilise les communications bloquantes en MPI. En pratique, dans le standard MPI, chaque envoi/reception de message a un tag (attribut entier) et s'adresse à un groupe de communication. D'un point de vue algorithmique, les communications bloquantes sont nécessaires pour assurer la *consistence* (sémantique) des programmes (par exemple, éviter que des messages se croisent) mais elles peuvent donc faire apparaître ces situations indésirables de blocage.

Afin de minimiser ces situations, on peut pré-allouer à chaque processus un espace mémoire dédié pour bufferiser les données : le *buffer des données* (*Data Buffer*, *DB* en anglais, des *zones mémoires tampons*). On envoie les données alors en *deux temps* : le processus qui fait une opération **send** envoie le message sur le buffer des données, et le processeur receveur recopie le buffer des données à l'endroit de sa zone mémoire locale indiquée dans le message par l'adresse `&data`. Cette technique de *communications bufferisées* est implantée soit matériellement soit par un protocole logiciel. Néanmoins, il subsiste toujours une situation de blocage lorsque le buffer de données DB devient plein (état saturé).

De plus, même si on gère bien les **send** d'un côté, le problème des blocages subsiste même avec des communications bufferisées car on a le problème des opérations **receive** bloquantes, illustré par cette configuration :

| Processus $P_0$                      | Processus $P_1$                      |
|--------------------------------------|--------------------------------------|
| <code>receive(&amp;a, 1, P1);</code> | <code>receive(&amp;a, 1, P0);</code> |
| <code>send(&amp;b, 1, P1);</code>    | <code>send(&amp;b, 1, P0);</code>    |

Chaque processeur attend un message avant de pouvoir envoyer son message!

En résumé, les communications bloquantes sont un atout lorsque l'on considère des opérations de communication globale comme la diffusion afin d'assurer le bon ordre de réception des messages mais il faut faire attention aux blocages.

Une solution pour éviter ces blocages est de considérer les primitives **send** et **receive** non bloquantes. On note ces opérations non-bloquantes et non-bufferisées par **Isend** et **Ireceive** en MPI : les *communications asynchrones* (*asynchronous communications*). Dans ce cas, l'envoyeur poste un message "Demande d'envoi" (*pending message*) et continue l'exécution de son programme, et quand le receveur poste un "OK pour envoi", le transfert de données s'effectue (en interne, cela se gère avec les signaux). Quand le transfert des données est fini, un *check status* indique que l'on peut toucher alors aux données sans danger. Le programme en C ci-dessous illustre un tel échange en utilisant l'interface C de MPI. Notons la primitive<sup>9</sup> `MPI_Wait(&request, &status);` qui attend jusqu'à ce que le transfert soit fini et indique si celui-ci a été fructueux ou pas dans la variable d'état **status** :

Code 2 – Exemple de communication non-bloquante `commonbloq442.cpp`

```
// Nom du programme : commonbloq442.cpp
// Compilez avec mpic++ commonbloq442.cpp -o commonbloq442.exe
// Exécutez avec mpirun -np 2 commonbloq442.exe
```

9. [https://www.open-mpi.org/doc/v1.8/man3/MPI\\_Wait.3.php](https://www.open-mpi.org/doc/v1.8/man3/MPI_Wait.3.php)

```

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>
#include <math.h>

int main(int argc, char ** argv)
{
    int myid, numprocs;
    int tag, source, destination, count;
    int buffer;
    MPI_Status status;
    MPI_Request request;

    MPI_Init(&argc, &argv);

    MPI_Comm_size(MPI_COMM_WORLD, &numprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &myid);

    tag=442;
    source=0;
    destination=1;
    count=1;

    request=MPI_REQUEST_NULL;

    if(myid == source){
        buffer=2015;
        MPI_Isend(&buffer, count, MPI_INT, destination, tag, MPI_COMM_WORLD, &request);
    }
    if(myid == destination){
        MPI_Irecv(&buffer, count, MPI_INT, source, tag, MPI_COMM_WORLD, &request);
    }

    printf("attente avec MPI_WAIT...\n");
    MPI_Wait(&request, &status);
    printf("[proc %d] status de MPI_WAIT: %d\n", myid, status);

    if(myid == source){
        printf("Le processeur %d a envoye %d\n", myid, buffer);
    }
    if(myid == destination){
        printf("Le processeur %d a bien reçu %d\n", myid, buffer);
    }
    MPI_Finalize();
}

```

À l'exécution du programme, nous obtenons à la console le résultat suivant :

```

attente avec MPI_WAIT...
attente avec MPI_WAIT...
[proc 0] status de MPI_WAIT: 0
Le processeur 0 a envoye 2015
[proc 1] status de MPI_WAIT: 0
Le processeur 1 a bien reçu 2015

```

On résume les syntaxes d'appel de l'API en C des primitives `Isend` et `Irecv` pour les communications non-bloquantes :

|      | Opérations bloquantes                                                                                 | Opérations non-bloquantes                                                                                                                                |
|------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|
| TB   | <code>send</code> fini après que les données ont été copiées dans la mémoire tampon de communication  | <code>send</code> fini après avoir initialisé le transfert DMA (Direct Memory Access) au tampon. L'opération n'est pas forcément finie après la commande |
| TNB  | <code>send</code> bloquant jusqu'à ce qu'il rencontre l'opération <code>receive</code> correspondante | A définir                                                                                                                                                |
| sens | Sémantique de <code>send</code> et <code>receive</code> par opération correspondante                  | Sémantique doit être explicitement garantie par le programmeur en vérifiant le statut de l'opération                                                     |

TABLE 2.3 – Différents protocoles pour les opérations d'envoi et de réception. TB signifie Transfert Bufferisé et TNB Transfert Non-Bufferisé.

```
int MPI_Isend( void *buf, int count, MPI_Datatype datatype, int dest, int tag,
              MPI_Comm comm, MPI_Request *req )
```

```
int MPI_Irecv( void *buf, int count, MPI_Datatype datatype, int src, int tag,
              MPI_Comm comm, MPI_Request *req )
```

La structure `MPI_Request` est souvent utilisée dans les routines : retourne `*flag=1` si l'opération `*req` est finie, et 0 sinon.

```
int MPI_Test( MPI_Request *req, int *flag, MPI_Status *status )
```

La primitive `MPI_Wait`, quant à elle, attend jusqu'à ce que l'opération associée avec `*req` soit finie.

```
int MPI_Wait( MPI_Request *req, MPI_Status *status )
```

On résume les différents protocoles pour les opérations d'envoi et de réception à la table 2.3.

Ces programmes mettent en lumière les six routines standards de MPI parmi la centaine de procédures offertes :

|                            |                                   |
|----------------------------|-----------------------------------|
| <code>MPI_Init</code>      | Initialisation de la bibliothèque |
| <code>MPI_Finalize</code>  | Termine l'utilisation de MPI      |
| <code>MPI_Comm_size</code> | Donne le nombre de processus      |
| <code>MPI_Comm_rank</code> | Étiquette du processus appelant   |
| <code>MPI_Send</code>      | Envoi un message (bloquant)       |
| <code>MPI_Recv</code>      | Reçoit un message (bloquant)      |

Toutes ces procédures retournent `MPI_SUCCESS` en cas de succès, sinon un code d'erreur lié au type d'erreur. Les types de données et constantes sont préfixés par `MPI_` (voir le fichier d'en-tête `mpi.h`)

### 2.3.4 Quelques hypothèses sur la concurrence : calculs locaux et communications recouvrantes

On peut supposer que les processeurs des nœuds (*Processing Elements, PEs*) peuvent effectuer plusieurs opérations en même temps. Par exemple, un scénario est d'utiliser les communications non-bloquantes (`MPI_IRecv` et `MPI_Isend`) en même temps que le processus effectue un calcul local. Il faut donc que ces trois opérations soient indépendantes. Dans une étape, on ne peut donc

pas envoyer le résultat du calcul et on ne peut pas envoyer (*forwarder*) ce que l'on reçoit. En algorithmique parallèle, on note ces opérations concurrentes en pseudo-code grâce la double barre `||` :

`IRecv || ISend || Calcul_Local`

### 2.3.5 Communications uni-directionnelles et bi-directionnelles

La *communication uni-directionnelle* (*one-way communication*) comme son nom l'indique n'autorise que les communications dans un seul sens : soit on envoie un message soit on en reçoit un (`MPI_Send` / `MPI_Recv`). Dans une *communication bi-directionnelle*, on peut communiquer dans les deux sens (*two-way communication*) : en MPI, cela se fait par l'appel de la primitive `MPI_Sendrecv`<sup>10</sup>.

### 2.3.6 Les calculs globaux en MPI : les opérations de réduction (*reduce*) et de (somme) préfixe parallèle (*scan*)

En MPI, on peut faire des *calculs globaux* comme calculer la somme cumulée  $V = \sum_{i=0}^{P-1} v_i$  où  $v_i$  est une variable locale stockée dans la mémoire distribuée du processus  $P_i$ . Le résultat de ce calcul global  $V$  est alors disponible dans la mémoire locale du processus qui a effectué cet appel de l'opération de réduction (**reduce**) : le processus racine (ou processus appelant). On décrit ci-dessous comment appeler cette primitive **reduce** avec l'interface<sup>11</sup> en C de MPI (*C binding*) :

```
#include <mpi.h>

int MPI_Reduce ( // Reduce routine
void* sendBuffer, // Address of local val
void* recvBuffer, // Place to receive into
int count, // No. of elements
MPI_Datatype datatype, // Type of each element
MPI_Op op, // MPI operator
int root, // Process to get result
MPI_Comm comm // MPI communicator
);
```

Les opérations de réduction prédéfinies en MPI se font grâce à la sélection d'un mot clef parmi cette liste (voir aussi le tableau 2.1) :

10. [https://www.open-mpi.org/doc/v1.8/man3/MPI\\_Sendrecv.3.php](https://www.open-mpi.org/doc/v1.8/man3/MPI_Sendrecv.3.php)

11. Voir en ligne [https://www.open-mpi.org/doc/v1.5/man3/MPI\\_Reduce.3.php](https://www.open-mpi.org/doc/v1.5/man3/MPI_Reduce.3.php)

|            |                                                  |
|------------|--------------------------------------------------|
| MPI_MAX    | maximum                                          |
| MPI_MIN    | minimum                                          |
| MPI_SUM    | somme $\sum$                                     |
| MPI_PROD   | produit $\prod$                                  |
| MPI_LAND   | et logique                                       |
| MPI_BAND   | et bit à bit, <i>bitwise and</i>                 |
| MPI_LOR    | ou logique                                       |
| MPI_BOR    | ou bit à bit                                     |
| MPI_LXOR   | xor logique                                      |
| MPI_BXOR   | xor bit à bit                                    |
| MPI_MAXLOC | valeur maximale et position de l'élément maximal |
| MPI_MINLOC | valeur minimale et position de l'élément minimal |

Par exemple, à titre d'illustration, considérons le calcul de la factorielle par une opération de réduction avec l'opérateur MPI\_PROD ( $\times$ ) :

Code 3 – La factorielle avec une opération de réduction : `factoriellempireduce442.cpp`

```
// Nom du programme : factoriellempireduce442.cpp
// Compilez avec mpic++ factoriellempireduce442.cpp -o factoriellempireduce442.exe
// Exécutez avec mpirun -np 8 factoriellempireduce442

#include <stdio.h>
#include "mpi.h"

int main(int argc, char *argv[]) {
    int i, moi, nprocs;
    int nombre, globalFact=-1, localFact;

    MPI_Init(&argc, &argv);

    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &moi);

    nombre=moi+1;
    MPI_Reduce(&nombre, &globalFact, 1, MPI_INT, MPI_PROD, 0, MPI_COMM_WORLD);
    if (moi==0)
        {printf("factorielle avec reduce pour %d processus = %d\n", nprocs,
            globalFact);}

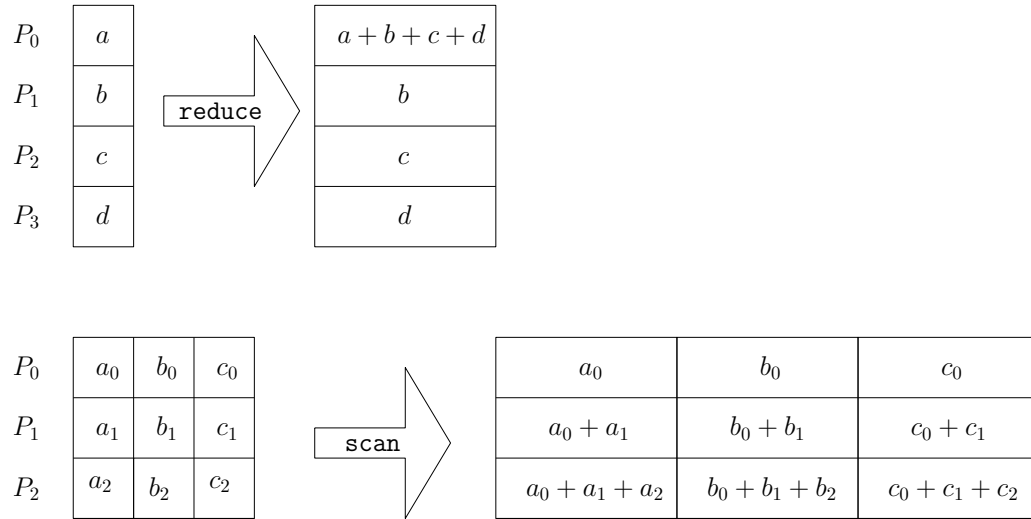
    localFact=1; for(i=0; i<nprocs; i++) {localFact*=i+1;}
    if (moi==0)
        {printf("factorielle locale : %d\n", localFact);}

    MPI_Finalize();
}
```

L'exécution de ce programme renvoie sur la console le résultat suivant :

```
factorielle avec reduce pour 8 processus = 40320
factorielle locale : 40320
```

En MPI, on peut aussi construire son propre type de données et définir l'opérateur binaire associatif  $\oplus$  pour la réduction.

FIGURE 2.4 – Visualisation d’une opération de réduction **reduce** et de préfixe parallèle (**scan**).

Un deuxième type d’opération de calcul global sont les *opérations de préfixe* en parallèle (*parallel prefix* ou encore préfixe somme) : **scan** (*scan*). Une opération de type **scan** calcule les réductions partielles sur les données des processeurs.

La syntaxe d’appel en MPI est la suivante :

```
int MPI_Scan( void *sendbuf, void *recvbuf, int count, MPI_Datatype datatype, MPI_Op
              op, MPI_Comm comm )
```

L’appel de cette primitive permet de faire une *somme préfixe* (*prefix sum*) sur les données de **sendbuf** sur *chaque processus* avec le résultat disponible à l’adresse mémoire **recvbuf**.

Par exemple, voici le résultat d’une somme préfixe pour quatre processeurs pour la variable locale  $v_i$  :

| processus        | $P_0$ | $P_1$       | $P_2$           | $P_3$                |
|------------------|-------|-------------|-----------------|----------------------|
| entrée ( $v_i$ ) | 1     | 2           | 3               | 4                    |
| sortie           | 1     | 3 (= 1 + 2) | 6 (= 1 + 2 + 3) | 10 (= 1 + 2 + 3 + 4) |

Pour un opérateur binaire et commutatif  $\oplus$ , le processus  $P_i$  aura donc le résultat du calcul  $\bigoplus_{j=0}^i v_j$ . Il s’agit donc de calculer toutes les réductions partielles. La figure 2.4 illustre la différence entre ces deux opérations de calcul global : le **reduce** et le **scan**. Notons que l’opération *scan* peut se faire sur un tableau.

```
MPI_Scan( vals, cumsum, 4, MPI_INT, MPI_SUM, MPI_COMM_WORLD )
```

Nous avons présenté la syntaxe de **reduce** et **scan** avec l’API MPI en C car depuis le standard MPI-2, l’interface en C++ de MPI n’est plus supportée (et ne dispose donc pas des dernières possibilités). En pratique, on programme ainsi souvent en C++ (langage orienté objet) en faisant des appels à la bibliothèque MPI via l’interface en C (le C n’est pas un langage orienté objet et manipule des structures de données définies par le mot clef **struct**, voir l’annexe A).

Ces opérations de calculs globaux sont souvent implémentées en utilisant des arbres recouvrants de la topologie du réseau d'interconnexion.

### 2.3.7 \* Les groupes de communication : les communicators

En MPI, les *communicators* permettent de regrouper les nœuds-processus en divers groupes de communication. Chaque processus inclus dans un **communicator** a un rang associé. Par défaut, `MPI_COMM_WORLD` inclut tous les  $P$  processus, avec le rang variant de 0 à  $P - 1$ . Pour connaître le nombre de processus dans un communicator et savoir son rang dans son communicator, on utilise les primitives MPI `int MPI_Comm_size(MPI_Comm comm, int *size)` et `int MPI_Comm_rank(MPI_Comm comm, int *size)`.

Voici un exemple qui montre comment utiliser les *communicateurs* (*communicators*) :

Code 4 – Créer un groupe de communication en MPI : `groupecom442.cpp`

```
// Nom du programme : groupecom442.cpp
// Compilez avec mpic++ groupecom442.cpp -o groupecom442.exe
// Exécutez avec mpirun -np 12 groupecom442.exe
// Programme adapté de la source
// https://www.nics.tennessee.edu/files/pdf/MPI_WORKSHOP/group.c

#include "mpi.h"
#include <stdio.h>
#define NPROCS 12

int main(int argc, char ** argv)
{
    int rank, new_rank, sendbuf, recvbuf, numtasks;
    int ranks1[6]={0,1,2,3,4,5}, ranks2[6]={6,7,8,9,10,11};
    MPI_Group orig_group, new_group;
    MPI_Comm new_comm;

    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // Taille du groupe de communication MPI_COMM_WORLD
    MPI_Comm_size(MPI_COMM_WORLD, &numtasks);

    if (numtasks != NPROCS) {
        MPI_Finalize();
        exit(0);
    }

    sendbuf = rank;

    // Récupère le groupe original
    MPI_Comm_group(MPI_COMM_WORLD, &orig_group);

    // Divise les processus en deux groupes distincts
    if (rank < NPROCS/2)
        MPI_Group_incl(orig_group, NPROCS/2, ranks1, &new_group);
    else
        MPI_Group_incl(orig_group, NPROCS/2, ranks2, &new_group);

    // Créer un nouveau communicator
```

```

MPI_Comm_create(MPI_COMM_WORLD, new_group, &new_comm);

MPI_Allreduce(&sendbuf, &recvbuf, 1, MPI_INT, MPI_SUM, new_comm);

    // Rang dans le nouveau communicator
    MPI_Group_rank (new_group, &new_rank);
    printf("rank= %d newrank= %d recvbuf= %d\n",rank,new_rank,recvbuf);

MPI_Finalize();
}

```

`MPI_Comm_create` est une opération **collective**. Tous les processus de l'ancien groupe de communication doivent l'appeler, même ceux qui ne feront pas partie du nouveau communicateur.

L'exécution de ce programme produit le résultat suivant dans la console :

```

rank= 1 newrank= 1 recvbuf= 15
rank= 2 newrank= 2 recvbuf= 15
rank= 3 newrank= 3 recvbuf= 15
rank= 5 newrank= 5 recvbuf= 15
rank= 6 newrank= 0 recvbuf= 51
rank= 7 newrank= 1 recvbuf= 51
rank= 10 newrank= 4 recvbuf= 51
rank= 0 newrank= 0 recvbuf= 15
rank= 4 newrank= 4 recvbuf= 15
rank= 8 newrank= 2 recvbuf= 51
rank= 9 newrank= 3 recvbuf= 51
rank= 11 newrank= 5 recvbuf= 51

```

Les *communicators* sont très intéressants pour créer des topologies virtuelles (voir le chapitre 3).

## 2.4 Les barrières de synchronisation : points de ralliement des processus

Dans le modèle de parallélisme à gros grains, les processeurs effectuent des calculs locaux indépendamment les uns des autres. Ils peuvent ensuite s'attendre mutuellement à une barrière (en MPI `MPI_Barrier`), dite *barrière de synchronisation*, et décider de s'envoyer des messages pour s'échanger des données, puis poursuivre leurs calculs respectifs, etc (Figure 2.5).

### 2.4.1 Un exemple de synchronisation en MPI pour mesurer le temps d'exécution

Par exemple, illustrons comment mesurer le temps de calcul parallèle d'un programme MPI en utilisant une barrière de synchronisation. On utilise la fonction<sup>12</sup> `MPI_Wtime` pour mesurer le temps sous MPI. Voici un programme de type maître/esclave pour mesurer le temps d'un programme parallèle :

12. [https://www.open-mpi.org/doc/v1.4/man3/MPI\\_Wtime.3.php](https://www.open-mpi.org/doc/v1.4/man3/MPI_Wtime.3.php)

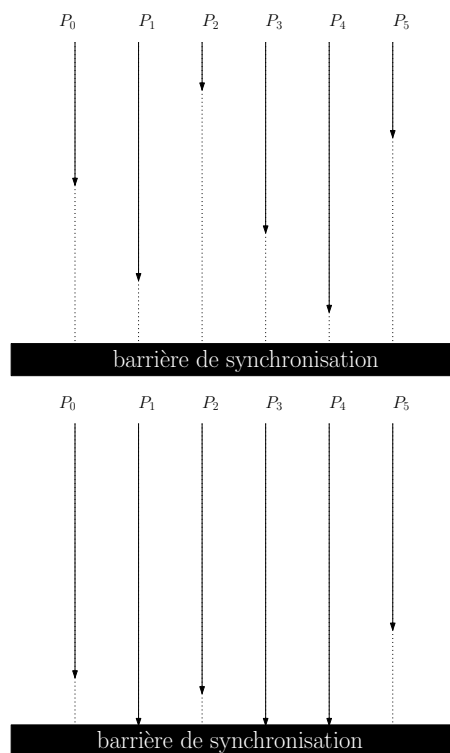


FIGURE 2.5 – Illustration conceptuelle d’une barrière de synchronisation : tous les processus s’attendent à la barrière de synchronisation avant de poursuivre leur exécution.

Code 5 – Mesure le temps d'un programme en MPI : `mesuretemps442.cpp`

```
// Nom du programme : mesuretemps442.cpp
// Compilez avec mpic++ mesuretemps442.cpp -o mesuretemps442.exe
// Exécutez avec mpirun -np 4 mesuretemps442.exe

#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>

int calculLocal442(int monRang)
{
    // on simule ici un calcul factice!
    sleep(2*monRang+1);
    return 0;
}

int main(int argc, char** argv)
{
    int monRang;
    double s_time, l_time, g_time;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &monRang);

    // barrière de synchronisation : tout le monde s'attend ici !
    MPI_Barrier(MPI_COMM_WORLD);
    s_time = MPI_Wtime();
    calculLocal442(monRang);
    l_time = MPI_Wtime();

    MPI_Barrier(MPI_COMM_WORLD);
    g_time = MPI_Wtime();

    printf("Processus %d: Temps local = %lf, Temps global = %lf\n", monRang, l_time
        - s_time, g_time-s_time);

    MPI_Finalize();
    return 0;
}
```

Un exemple d'exécution donne en sortie :

```
Processus 1: Temps local = 3.001023, Temps global = 7.001117
Processus 0: Temps local = 1.001967, Temps global = 7.001096
Processus 2: Temps local = 5.001050, Temps global = 7.001100
Processus 3: Temps local = 7.001109, Temps global = 7.001118
```

Si l'on relance avec 16 processus (`mpirun -np 16 mesuretemps442.exe`), on obtient cette sortie sur la console :

```
Processus 6: Temps local = 13.001811, Temps global = 31.001227
Processus 14: Temps local = 29.002159, Temps global = 31.001227
Processus 2: Temps local = 5.000655, Temps global = 31.001249
Processus 4: Temps local = 9.001789, Temps global = 31.001300
Processus 10: Temps local = 21.001001, Temps global = 31.001247
Processus 12: Temps local = 25.001137, Temps global = 31.001296
Processus 0: Temps local = 1.001600, Temps global = 31.001547
Processus 8: Temps local = 17.001928, Temps global = 31.001550
Processus 7: Temps local = 15.001944, Temps global = 31.001662
Processus 13: Temps local = 27.001220, Temps global = 31.001678
Processus 15: Temps local = 31.001244, Temps global = 31.001611
Processus 1: Temps local = 3.001736, Temps global = 31.001712
Processus 9: Temps local = 19.001082, Temps global = 31.001720
Processus 11: Temps local = 23.001174, Temps global = 31.001720
Processus 3: Temps local = 7.000810, Temps global = 31.001746
Processus 5: Temps local = 11.001830, Temps global = 31.001676
```

Notons que l'on peut aussi utiliser `MPI_Reduce()` pour calculer le minimum des temps et le maximum des temps des processus, mais cela demande alors de rajouter une étape de réduction par calcul global.

### 2.4.2 Le modèle BSP : Bulk Synchronous Parallel

Un modèle de programmation pour les algorithmes parallèles est le *Bulk Synchronous Parallel* (BSP). Ce modèle abstrait BSP conçu par Leslie Valiant de l'université d'Harvard dans les années 1980 permet d'aider la conception des algorithmes parallèles en considérant trois étapes fondamentales dans les algorithmes parallèles :

- étape de calculs concurrentiels : les processeurs calculent localement de façon asynchrone et ces calculs peuvent recouvrir des opérations de communication,
- étape de communication : les processus s'échangent les données entre eux,
- étape de la barrière de synchronisation : quand un processus atteint une barrière de synchronisation, il attend que tous les autres processus atteignent cette barrière avant de recommencer une nouvelle *super-étape* (*super step*).

Une bibliothèque logicielle BSPonMPI<sup>13</sup> permet d'utiliser ce paradigme de programmation aisément en MPI.

## 2.5 Prise en main de l'API MPI

On montre trois façons d'utiliser la spécification MPI en salles machines suivant la bibliothèque de liaison (*binding*) utilisée (C++/C/Boost).

### 2.5.1 Le traditionnel "Hello World" avec l'API MPI en C++

Considérons le programme `programme442.cpp` suivant :

Code 6 – Le "Hello World" en MPI `programme442.cpp`

```
// Nom du programme : programme442.cpp
// Compilez avec mpic++ programme442.cpp -o programme442.exe
// Exécutez avec mpirun -np 4 programme442.exe

# include <iostream>
using namespace std;

# include "mpi.h"

int main ( int argc, char *argv[] )
{
    int id, p, name_len;
    char processor_name[MPI_MAX_PROCESSOR_NAME];

    // Initialise MPI.
    MPI::Init ( argc, argv );

    // Retourne le nombre de processus
    p = MPI::COMM_WORLD.Get_size ( );
```

13. <http://bsponmpi.sourceforge.net/>

```

// Donne le rang du processus courant
id = MPI::COMM_WORLD.Get_rank ( );
// et puis son nom aussi
MPI_Get_processor_name(processor_name, &name_len);

// Affiche un message de bienvenue
cout << " Processeur " << processor_name<<" ID="<<id << " bienvenue en INF442
!'\n";

// On termine proprement MPI
MPI::Finalize ( );

return 0;
}

```

La compilation se fait par :

```
mpic++ programme442.cpp -o programme442
```

Si l'option -o n'est pas mise, le compilateur écrira dans un fichier nommé par défaut : `a.out`. Une fois la compilation finie, on exécute son programme sur sa machine localement (ici `hollande`) :

```

[hollande MPI]$ mpirun -n programme442
Processeur hollande.polytechnique.fr ID=3 bienvenue en INF442!'
Processeur hollande.polytechnique.fr ID=0 bienvenue en INF442!'
Processeur hollande.polytechnique.fr ID=1 bienvenue en INF442!'
Processeur hollande.polytechnique.fr ID=2 bienvenue en INF442!'

```

Si on était sur une autre machine locale comme `essonne`, la sortie devient :

```

[essonne MPI]$ mpic++ programme442.cpp -o programme442.exe
[essonne MPI]$ mpirun -np 4 programme442.exe
Processeur essonne.polytechnique.fr ID=1 bienvenue en INF442 !'
Processeur essonne.polytechnique.fr ID=2 bienvenue en INF442 !'
Processeur essonne.polytechnique.fr ID=0 bienvenue en INF442 !'
Processeur essonne.polytechnique.fr ID=3 bienvenue en INF442 !'

```

Notons que sur la console, on voit les messages affichés dans un ordre qui reflète l'ordre des exécutions par les différentes unités de calcul de la commande `cout`. Ainsi, si on exécute de nouveau ce programme, l'ordre d'arrivée sur la console peut être différent.

On peut exécuter sur plusieurs machines des salles informatiques. Pour avoir les droits d'accès, faire d'abord<sup>14</sup> un `kinit` et se logger une fois en entrant son mot de passe (LDAP) sur deux machines : Par exemple, `angleterre` et `autriche`. Ensuite, on exécute le programme compilé MPI avec la commande `mpirun` comme suit :

```

[hollande MPI]$ mpirun -np 5 -host angleterre,autriche programme442
Processeur autriche.polytechnique.fr ID=1 bienvenue en INF442!'
Processeur autriche.polytechnique.fr ID=3 bienvenue en INF442!'
Processeur angleterre.polytechnique.fr ID=0 bienvenue en INF442!'
Processeur angleterre.polytechnique.fr ID=2 bienvenue en INF442!'
Processeur angleterre.polytechnique.fr ID=4 bienvenue en INF442!'

```

14. Cela permet d'initialiser les clefs Kerberos et d'éviter de taper son Mot De Passe (MDP) à chaque fois.

La commande `mpirun` est un alias pour `orterun`.

On accède à la liste des bibliothèques connues par MPI en tapant dans la console :

```
>mpic++ --showme:libs
mpi_cxx mpi open-rte open-pal dl nsl util m dl
```

On peut rajouter une bibliothèque comme ceci :

```
export LIBS=${LIBS}:/usr/local/boost-1.39.0/include/boost-1_39
```

Puis compiler avec la commande :

```
mpic++ -c t.cpp -I$LIBS
```

Le mieux est de mettre par défaut votre configuration dans votre fichier local `.bashrc`. À tout moment, après avoir édité votre fichier de configuration, vous pouvez ressourcer celui-ci afin que votre fenêtre Shell reconnaisse les derniers changements (sinon il faudra ouvrir une nouvelle fenêtre).

```
source ~/.bashrc
```

Voilà! La porte vous est ouverte pour utiliser la puissance de calcul des 169 machines<sup>15</sup> des salles informatiques de l'école! Mais attention, cela vous responsabilise aussi car ces ordinateurs constituent une ressource commune partagée par de nombreux utilisateurs.

## 2.5.2 Programmer en MPI avec l'API en C

Le petit programme ci-dessous vous montre comment définir un programme de type maître-esclave :

Code 7 – Programme de type maître-esclave en MPI : `archimaitreesclave442.cpp`

```
// Nom du programme : archimaitreesclave442.cpp
// Compilez avec mpic++ archimaitreesclave442.cpp -o archimaitreesclave442.exe
// Exécutez avec mpirun -np 2 archimaitreesclave442.exe

#include <stdio.h>
#include "mpi.h"

void master()
{printf("Je suis le processus master\n");}

void slave()
{printf("Je suis le processus esclave\n");}

int main (int argc, char **argv)
{
    int myrank, size;
    MPI_Init (&argc, &argv);
    MPI_Comm_rank (MPI_COMM_WORLD, &myrank);
    MPI_Comm_size( MPI_COMM_WORLD, &size );

    if (!myrank)
```

---

15. La liste des noms de ces machines est donnée en appendice.

```

        {master ();}
    else
        {slave ();}
    MPI_Finalize ();
    return (1);
}

```

Voici deux cas d'exécution avec 2 et 3 processus demandés (la machine possède un CPU à 8 cœurs) :

```

[essonne MPI]$ mpirun -np 2 archimaitreeslave442.exe
Je suis le processus master
Je suis le processus esclave
[essonne MPI]$ mpirun -np 3 archimaitreeslave442.exe
Je suis le processus esclave
Je suis le processus esclave
Je suis le processus master

```

Notez que l'interface MPI utilisée est différente de notre premier programme "Hello World". En effet, ici nous utilisons l'interface C de MPI. Il se trouve que l'interface C est la plus communément utilisée et mise à jour régulièrement alors que celle en C++ offre moins de possibilités. Nous préconisons donc d'utiliser celle en C, quitte à mélanger du code C++ avec du code C (pour la bonne cause!).

### 2.5.3 \* MPI avec l'API de Boost en C++

Boost<sup>16</sup> est une bibliothèque en C++ très utile pour manipuler les matrices, les graphes, etc. Cette bibliothèque Boost possède aussi sa propre façon d'utiliser le standard MPI, mais ne contient qu'une partie des fonctionnalités de MPI.

```

/usr/local/openmpi-1.8.3/bin/mpic++ -I/usr/local/boost-1.56.0/include/
-L/usr/local/boost-1.56.0/lib/ -lboost_mpi -lboost_serialization monprog442.cpp
-o monprog442

```

Code 8 – Utiliser MPI avec la bibliothèque Boost : `mpiboostex442.cpp`

```

// Nom du programme : mpiboostex442.cpp
// Compilez avec mpic++ mpiboostex442.cpp -o mpiboostex442.exe
// Exécutez avec mpirun -np 2 mpiboostex442.exe

#include <boost/mpi/environment.hpp>
#include <boost/mpi/communicator.hpp>
#include <iostream>
namespace mpi = boost::mpi;

int main()
{
    mpi::environment env;
    mpi::communicator world;
    std::cout << "Je suis le processus " << world.rank() << " sur " << world.size
        () << "." << std::endl;
}

```

16. <http://www.boost.org/>

```
    return 0;
}
```

On conseille d'éditer son fichier `.bashrc` dans son répertoire locale, et d'y ajouter l'alias suivant :

```
alias mpiboost='/usr/local/openmpi-1.8.3/bin/mpic++ -I/usr/local/
    boost-1.56.0/include/ -L/usr/local/boost-1.56.0/lib/ -lboost_mpi
    -lboost_serialization '
```

Cela permet simplement de compiler en tapant :

```
mpiboost mpiboostex442.cpp -o mpiboostex442.exe
```

## 2.6 \* Utiliser MPI avec OpenMP

*OpenMP*<sup>17</sup> est une autre interface de programmation pour les applications (*Application Programming Interface, API*) pour la programmation parallèle avec *mémoire partagée*. OpenMP est multi-plateforme et offre une interface flexible en C/C++/Fortran. OpenMP est typiquement utilisée lorsque l'on veut programmer un algorithme en parallèle qui utilise l'ensemble des cœurs d'un même processeur (mémoire partagée). On peut utiliser OpenMP avec MPI afin de tirer partie de la mémoire partagée entre les cœurs d'un même processeur et de la mémoire distribuée entre plusieurs processeurs. Voici un petit programme (le fameux "Hello World") qui utilise à la fois MPI et OpenMP :

Code 9 – Programme qui combine à la fois MPI et OpenMP (multi-fil) : `mpiopenmpex442.cpp`

```
// Nom du programme : mpiopenmpex442.cpp
// Compilez avec mpic++ -fopenmp mpiopenmpex442.cpp -o mpiopenmpex442.exe
// Exécutez avec mpirun -np 4 mpiopenmpex442.exe

#include <mpi.h>
#include <omp.h>
#include <stdio.h>

int main (int nargs, char** args)
{
    int rank, nprocs, thread_id, nthreads;
    int name_len;
    char processor_name[MPI_MAX_PROCESSOR_NAME];

    MPI_Init (&nargs, &args);
    MPI_Comm_size (MPI_COMM_WORLD, &nprocs);
    MPI_Comm_rank (MPI_COMM_WORLD, &rank);
    MPI_Get_processor_name(processor_name, &name_len);

    #pragma omp parallel private(thread_id, nthreads)
    {
        thread_id = omp_get_thread_num ();
        nthreads = omp_get_num_threads ();
        printf("Je suis le fil nr.%d (sur %d) sur le processus MPI nr.%d (sur of %d)
            [%s]\n",

```

---

17. <http://openmp.org/wp/>

```

    thread_id, nthreads, rank, nprocs, processor_name);
}

MPI_Finalize ();
return 0;
}

```

On le compile en salles machines avec l'option `-fopenmp` :

```
mpic++ -fopenmp testmpiopenmp.cpp -o testmp.exe
```

On exécute le programme en tapant à la ligne de commande du shell :

```
mpirun -np 2 -host royce,simca testmp.exe
```

```

[royce ~]$ mpirun -np 2 -host royce,simca dmp.exe
Je suis le fil nr.0 (sur 8) sur le processus MPI nr.1 (sur of 2) [simca.polytechnique.fr]
Je suis le fil nr.1 (sur 8) sur le processus MPI nr.1 (sur of 2) [simca.polytechnique.fr]
Je suis le fil nr.5 (sur 8) sur le processus MPI nr.1 (sur of 2) [simca.polytechnique.fr]
Je suis le fil nr.4 (sur 8) sur le processus MPI nr.1 (sur of 2) [simca.polytechnique.fr]
Je suis le fil nr.3 (sur 8) sur le processus MPI nr.1 (sur of 2) [simca.polytechnique.fr]
Je suis le fil nr.7 (sur 8) sur le processus MPI nr.1 (sur of 2) [simca.polytechnique.fr]
Je suis le fil nr.0 (sur 8) sur le processus MPI nr.0 (sur of 2) [royce.polytechnique.fr]
Je suis le fil nr.1 (sur 8) sur le processus MPI nr.0 (sur of 2) [royce.polytechnique.fr]
Je suis le fil nr.5 (sur 8) sur le processus MPI nr.0 (sur of 2) [royce.polytechnique.fr]
Je suis le fil nr.4 (sur 8) sur le processus MPI nr.0 (sur of 2) [royce.polytechnique.fr]
Je suis le fil nr.7 (sur 8) sur le processus MPI nr.0 (sur of 2) [royce.polytechnique.fr]
Je suis le fil nr.2 (sur 8) sur le processus MPI nr.0 (sur of 2) [royce.polytechnique.fr]
Je suis le fil nr.3 (sur 8) sur le processus MPI nr.0 (sur of 2) [royce.polytechnique.fr]
Je suis le fil nr.6 (sur 8) sur le processus MPI nr.0 (sur of 2) [royce.polytechnique.fr]
Je suis le fil nr.2 (sur 8) sur le processus MPI nr.1 (sur of 2) [simca.polytechnique.fr]
Je suis le fil nr.6 (sur 8) sur le processus MPI nr.1 (sur of 2) [simca.polytechnique.fr]

```

Nous voyons que nous avons 8 cœurs sur ces deux machines hôtes. On note l'ordre d'arrivée des messages sur la console. Une nouvelle exécution donnera un ordre d'affichage différent. Ne choisissons pas toujours les mêmes machines ! On peut se souvenir de quelques noms de machines en se souvenant aussi des groupes (salles) auxquels ils appartiennent : salle 30 (pays), salle 31 (oiseaux), salle 32 (orchidées), salle 33 (départements français), salle 34 (poissons), salle 35 (os du corps humain), salle 36 (voitures). Aussi, on verra bientôt que l'ordonnanceur SLURM s'occupe d'allouer les ressources d'un cluster automatiquement aux programmes MPI.

Pour un programme MPI MPMD (*Multiple Program Multiple Data*), il est utile d'utiliser la construction `switch/case` du langage C/C++ comme l'illustre cet exemple ci-dessous.

Code 10 – Exemple de programme *Multiple Program Multiple Data* en MPI : `mpimpmd442.cpp`

```

// Nom du programme : mpimpmd442.cpp
// Compilez avec mpic++ mpimpmd442.cpp -o mpimpmd442.exe
// Exécutez avec mpirun -np 8 mpimpmd442.exe

#include <mpi.h>
#include <stdio.h>

// MPMD : Multiple Program Multiple Data
void Prog0()
{printf("Je suis P0 et j'ai du travail sur mes propres donnees !\n");}

void Prog1()
{printf("Je suis P1 et j'ai du travail sur mes propres donnees !\n");}

void Prog2()
{printf("Je suis P2 et j'ai du travail sur mes propres donnees !\n");}

```

```
int main(int argc, char** argv) {
    int rang, nprocs;
    int message=-1;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &rang);

    switch(rang){
        case 0 :
            Prog0();
            break;
        case 1 :
            Prog1();
            break;
        case 2 :
            Prog2();
            break;

        default :
            printf("Je suis %d et j'ai pas de travail !\n",rang);
    }

    MPI_Finalize();
    return 0;
}
```

Voici un résultat d'exécution de ce programme :

```
[france MPI]$ mpirun -np 4 mpimpmd442.exe
Je suis P0 et j'ai du travail sur mes propres donnees !
Je suis P2 et j'ai du travail sur mes propres donnees !
Je suis 3 et j'ai pas de travail !
Je suis P1 et j'ai du travail sur mes propres donnees !
```

## 2.7 La syntaxe en MPI des principales opérations de communication

On rappelle les principales opérations de communications qui sont des opérations globales s'effectuant sur un communicateur :

- la diffusion (un vers tous) et la réduction (tous vers un, interprétable comme la diffusion inverse),
- la diffusion personnalisée (des messages différents pour tous, *scatter*),
- le rassemblement (*gather* ou encore *all-to-one*),
- calculs globaux comme les sommes préfixes,
- la communication de tous vers tous (la diffusion totale, *total exchange*),
- la communication de tous vers tous personnalisée (messages différents pour chaque processus, le commérage),
- etc.

### 2.7.1 Syntaxe MPI pour la diffusion, diffusion personnalisée (*scatter*), le rassemblement (*gather*), la réduction (*reduce*), et la réduction totale (*Allreduce*)

- la diffusion (*broadcast*) : `MPI_Bcast`<sup>18</sup>  

```
int MPI_Bcast(void *buffer, int count,
MPI_Datatype datatype,
int root, MPI_Comm comm)
```

Attention, tout les processus doivent appeler `MPI_Bcast` !

Code 11 – Exemple de diffusion en MPI : `simplebroadcast442.cpp`

```
// Nom du programme : simplebroadcast442.cpp
// Compilez avec mpic++ simplebroadcast442.cpp -o simplebroadcast442.exe
// Exécutez avec mpirun -np 8 simplebroadcast442.exe

#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank;
    int n;
    const int root=0;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank == root) {n = 442; }
    printf("[%d]: avant Bcast, n=%d\n", rank, n);

    // Attention : tout le monde doit appeller Bcast!
    MPI_Bcast(&n, 1, MPI_INT, root, MPI_COMM_WORLD);
    printf("[%d]: apres Bcast, n=%d\n", rank, n);

    MPI_Finalize();
    return 0;
}

[essonne MPI]$ mpirun -np 8 simplebroadcast442.exe
[7]: avant Bcast, n=-1078927288
[0]: avant Bcast, n=442
[0]: apres Bcast, n=442
[2]: avant Bcast, n=-1081611048
[2]: apres Bcast, n=442
[4]: avant Bcast, n=-1074500424
[4]: apres Bcast, n=442
[5]: avant Bcast, n=-1075968680
[6]: avant Bcast, n=-1080181944
[6]: apres Bcast, n=442
[1]: avant Bcast, n=-1079497448
[1]: apres Bcast, n=442
```

18. [https://www.open-mpi.org/doc/v1.5/man3/MPI\\_Bcast.3.php](https://www.open-mpi.org/doc/v1.5/man3/MPI_Bcast.3.php)

```
[5]: apres Bcast, n=442
[3]: avant Bcast, n=-1074122568
[3]: apres Bcast, n=442
[7]: apres Bcast, n=442
```

- la diffusion personnalisée (*scatter*) : `MPI_Scatter`<sup>19</sup>

```
int MPI_Scatter(void *sendbuf, int sendcount, MPI_Datatype
sendtype, void *recvbuf, int recvcount,
MPI_Datatype recvtype, int root, MPI_Comm comm)
```

- le rassemblement (*gather*) : `MPI_Gather`<sup>20</sup>

```
int MPI_Gather(void *sendbuf, int sendcount, MPI_Datatype
sendtype, void *recvbuf, int recvcount,
MPI_Datatype recvtype, int root, MPI_Comm comm)
```

- la réduction (*reduce*) : `MPI_Reduce`<sup>21</sup>

```
int MPI_Reduce(void *sendbuf, void *recvbuf, int count,
MPI_Datatype datatype, MPI_Op op, int root, MPI_Comm comm)
```

- la réduction totale (*Allreduce* ou encore *réduction généralisée*) : `MPI_Allreduce`<sup>22</sup>

```
int MPI_Allreduce(void *sendbuf, void *recvbuf, int count,
MPI_Datatype datatype, MPI_Op op, MPI_Comm comm)
```

Ces opérations sont décrites visuellement à la figure 2.6.

## 2.7.2 Autre opérations de communication/calculs globaux moins courants

- la *somme préfixe* (*prefix sum*) considère une opération associative binaire  $\oplus : +, \times, \max, \min$  et on calcule pour  $0 \leq k \leq P - 1$ , la somme stockée sur  $P_k$  :

$$S_k = M_0 \oplus M_1 \oplus \dots \oplus M_k$$

Les  $P$  messages  $M_k$  sont supposés stockés sur la mémoire locale du processus  $P_k$ .

- la *réduction all-to-all* se définit grâce à une opération associative  $\oplus : +, \times, \max, \min$  et donne, quant à elle, en sortie :

$$M_r = \bigoplus_{i=0}^{P-1} M_{i,r}.$$

On a  $P^2$  messages  $M_{r,k}$  pour  $0 \leq r, k \leq P - 1$ , et les messages  $M_{r,k}$  sont stockés localement sur  $P_r$ .

19. [https://www.open-mpi.org/doc/v1.5/man3/MPI\\_Scatter.3.php](https://www.open-mpi.org/doc/v1.5/man3/MPI_Scatter.3.php)

20. [https://www.open-mpi.org/doc/v1.5/man3/MPI\\_Gather.3.php](https://www.open-mpi.org/doc/v1.5/man3/MPI_Gather.3.php)

21. [https://www.open-mpi.org/doc/v1.5/man3/MPI\\_Reduce.3.php](https://www.open-mpi.org/doc/v1.5/man3/MPI_Reduce.3.php)

22. [https://www.open-mpi.org/doc/v1.5/man3/MPI\\_Allreduce.3.php](https://www.open-mpi.org/doc/v1.5/man3/MPI_Allreduce.3.php)

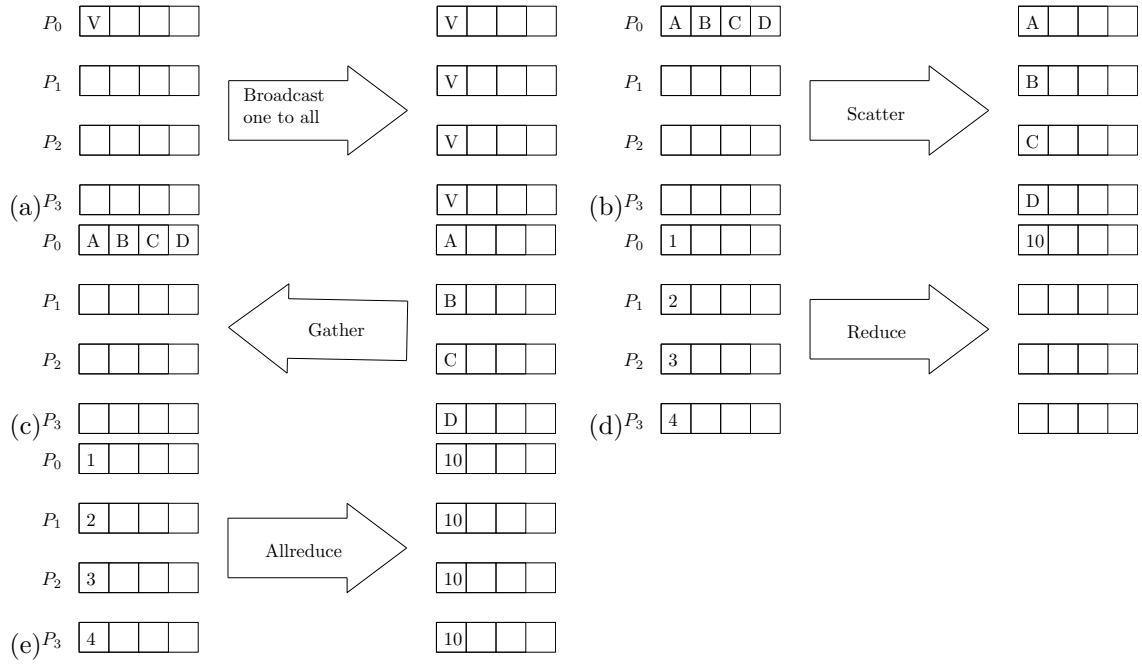


FIGURE 2.6 – Les opérations standard de communication et de calculs globaux en MPI : (a) la diffusion, (b) la diffusion personnalisée (*scatter*), (c) le rassemblement (*gather*), (d) la réduction (*reduce*) et (e) la réduction totale (*Allreduce*).

- la *transposition*, un all-to-all personnalisé qui effectue une transposition des messages sur les processus :

| $P_0$     | $P_1$     | $P_2$     | $P_3$     |                                      | $P_0$     | $P_1$     | $P_2$     | $P_3$     |
|-----------|-----------|-----------|-----------|--------------------------------------|-----------|-----------|-----------|-----------|
| $M_{0,3}$ | $M_{1,3}$ | $M_{2,3}$ | $M_{3,3}$ | $\xrightarrow{\text{transposition}}$ | $M_{3,0}$ | $M_{3,1}$ | $M_{3,2}$ | $M_{3,3}$ |
| $M_{0,2}$ | $M_{1,2}$ | $M_{2,2}$ | $M_{3,2}$ |                                      | $M_{2,0}$ | $M_{2,1}$ | $M_{2,2}$ | $M_{2,3}$ |
| $M_{0,1}$ | $M_{1,1}$ | $M_{2,1}$ | $M_{3,1}$ |                                      | $M_{1,0}$ | $M_{1,1}$ | $M_{1,2}$ | $M_{1,3}$ |
| $M_{0,0}$ | $M_{1,0}$ | $M_{2,0}$ | $M_{3,0}$ |                                      | $M_{0,0}$ | $M_{0,1}$ | $M_{0,2}$ | $M_{0,3}$ |

On a  $P^2$  messages  $M_{r,k}$  avec les  $P$  messages  $M_{r,k}$  stockés sur  $P_r$ , et en sortie après la transposition, on a  $M_{r,k}$  stockés sur  $P_k$  pour tout  $0 \leq k \leq P - 1$ . Cette opération est pratique pour inverser des matrices blocs distribuées sur une topologie de grille ou de tore, par exemple.

- le *shift circulaire* comme son nom l'indique, effectue un déplacement global des messages comme suit :

$$M_0 \quad M_1 \quad M_2 \quad M_3 \xrightarrow{\text{shift circulaire}} M_3 \quad M_0 \quad M_1 \quad M_2$$

Les  $P$  messages  $M_k$  sont stockés localement et le message  $M_{(k-1) \bmod P}$  est stocké sur  $P_k$  en sortie.

## 2.8 Communication sur l'anneau avec MPI

Dans le chapitre 5, on considère des algorithmes parallèles pour la multiplication matricielle avec les topologies de l'anneau et du tore. On illustre ici en MPI la communication en anneau avec les primitives `send` et `receive` de communications bloquantes :

Code 12 – Programme MPI pour la communication sur l'anneau : `commanneau442.cpp`

```
// Nom du programme : commanneau442.cpp
// Compilez avec mpic++ commanneau442.cpp -o commanneau442.exe
// Exécutez avec mpirun -np 8 commanneau442.exe

#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, nprocs;
    int message=-1;
    int Pstart=2, Pend=6;

    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    if (rank==0)
        {printf("Envoi sur l'anneau d'un message de P%d a P%d",Pstart,Pend);}

    // On simule avec des communications bloquantes par lien,
    // l'envoi d'une message de Pstart a Pend sur l'anneau orienté
    if (rank==Pstart) {
```

```

        // je suis l'envoyeur, je prépare mon message et je l'envoie
message=442;
MPI_Send(&message, 1, MPI_INT, Pstart+1, 0, MPI_COMM_WORLD);
    }
    else
    {
        if (rank==Pend)
        { // Je suis le destinataire, et je reçois seulement
            MPI_Recv(&message, 1, MPI_INT, Pend-1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE
                );
        } else if ((rank>Pstart)&&(rank<Pend))
        { // Je suis un nœud intermédiaire de la topologie de l'anneau
            MPI_Recv(&message, 1, MPI_INT, rank-1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE
                );
            printf("Je suis le proc. P%d. Recu le message de %d et le renvoie a %d\n",
                rank, rank-1,rank+1);
            MPI_Send(&message, 1, MPI_INT, rank+1, 0, MPI_COMM_WORLD);
        }
        else
        {printf("%d: Je ne fais rien moi !\n",rank);}
    }

    printf("Proc. %d message=%d\n", rank, message);

    MPI_Finalize();
    return 0;
}

```

On obtient à l'exécution de ce programme, le résultat suivant à la console :

```

mpirun -np 8 commanneau442.exe
Envoi sur l'anneau d'un message de P2 a P60: Je ne fais rien moi !
Proc. 0 message=-1
1: Je ne fais rien moi !
Proc. 1 message=-1
7: Je ne fais rien moi !
Proc. 7 message=-1
Proc. 2 message=442
Je suis le proc. P3. Recu le message de 2 et le renvoie a 4
Proc. 3 message=442
Je suis le proc. P4. Recu le message de 3 et le renvoie a 5
Proc. 4 message=442
Je suis le proc. P5. Recu le message de 4 et le renvoie a 6
Proc. 5 message=442
Proc. 6 message=442

```

## 2.9 L'ordonnanceur de tâches SLURM

SLURM<sup>23</sup> (*Simple Linux Utility for Resource Management*) est un utilitaire simple pour gérer les ressources. SLURM permet d'arbitrer les différentes requêtes des utilisateurs qui partagent

23. Disponible à <https://computing.llnl.gov/linux/slurm/>

les ressources en s'occupant d'ordonnancer une liste de tâches à exécuter (*pending jobs*). SLURM alloue l'accès aux différents nœuds d'une grappe (*cluster*) et gère les ressources de cette grappe. SLURM permet ainsi de lancer les tâches qui peuvent être exécutées en parallèle et s'occupe des Entrées/Sorties (E/Ss), Input-Outputs (I/Os) en anglais, des signaux, etc.

Les tâches (*jobs*) des utilisateurs sont soumis *en ligne* directement à SLURM par des commandes dans le shell, et ces tâches sont ordonnancées sur le principe premier entré premier servi : le modèle FIFO (*First-In First-Out*). Un script qui définit les jobs *par paquets* (*batch*) peut aussi être soumis à SLURM en option. Les quatre commandes principales pour l'utilisateur sont :

- **sinfo** : affiche les informations générales du système,
- **srun** : soumet ou initialise un job,
- **scancel** : lance un signal ou annule un job,
- **squeue** : affiche les informations sur les jobs du système (avec en visualisation R pour *Running* et PD pour *Pending*),
- **scontrol** : outil administrateur pour établir ou modifier la configuration.

Un tutoriel pour l'utilisation de SLURM se trouve en ligne<sup>24</sup>. Dans les salles informatiques, on a installé quatre clusters indépendants qui couvrent le parc informatique des 169 machines comme suit :

| Cluster/Partition par défaut | salles info      | nœuds    | contrôleur |
|------------------------------|------------------|----------|------------|
| Debug                        | salle info 30    | 19 nœuds | allemagne  |
| SN252                        | salle info 35/36 | 50 nœuds | acromion   |
| SN253                        | salle info 30/31 | 50 nœuds | albatros   |
| SN254                        | salle info 33/34 | 50 nœuds | ain        |

Loggons nous sur la machine **malte** et tapons la commande **sinfo** pour avoir les informations liées à son cluster d'appartenance :

```
[malte ~]$ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
Test*      up           15:00    19   idle
allemagne, angleterre, autriche, belgique, espagne, finlande, france,
groenland,
hollande, hongrie, irlande, islande, lituanie, malte, monaco, pologne,
portugal, roumanie, suede
```

Lançons une commande qui exécute sur 5 nœuds (hôtes) le programme **hostname** (programme qui écrit à la console le nom de la machine sur lequel le processus tourne) avec un maximum de 2 tâches par nœud :

```
[malte ~]$ srun -n 5 --ntasks-per-node=2 hostname
angleterre.polytechnique.fr
autriche.polytechnique.fr
allemagne.polytechnique.fr
allemagne.polytechnique.fr
angleterre.polytechnique.fr
```

On peut aussi utiliser SLURM pour lancer une commande shell qui exécute un programme **monProgramme MPI** :

24. <http://slurm.schedmd.com/tutorials.html>

```
[malte ~]$ cat test.sh
#!/bin/bash
LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/openmpi-1.8.3/lib/:
/usr/local/boost-1.56.0/lib/
/usr/local/openmpi-1.8.3/bin/mpiexec -np 4 ./monProgramme

[malte ~]$ srun -p Test -n 25 --label test.sh
09: I am process 1 of 4.
09: I am process 0 of 4.
...
01: I am process 0 of 4.
01: I am process 2 of 4.
05: I am process 2 of 4.
05: I am process 3 of 4.
```

Le tableau ci-dessous résume les principales commandes de SLURM.

|                       |                                                  |
|-----------------------|--------------------------------------------------|
| <code>salloc</code>   | allocation des ressources                        |
| <code>sbatch</code>   | pour passer les “batch”                          |
| <code>sbcst</code>    | dispatch les fichiers sur les nœuds d’allocation |
| <code>scancel</code>  | annuler le “batch” en cours                      |
| <code>scontrol</code> | interface de contrôle pour SLURM                 |
| <code>sdiag</code>    | pour récupérer l’état d’exécution                |
| <code>sinfo</code>    | état de cluster                                  |
| <code>squeue</code>   | état de file d’attente                           |
| <code>srun</code>     | exécution d’un “job”                             |
| <code>sstat</code>    | état d’exécution                                 |
| <code>strigger</code> | gestion des triggers                             |
| <code>svew</code>     | interface pour visualiser le cluster             |

## 2.10 Quelques exemples de programmes parallèles en MPI et leurs accélérations

Nous présentons maintenant différents types de parallélisation suivant les mouvements de données et les calculs effectués, et discutons de l’accélération obtenue (*speed-up*) pour plusieurs problèmes. L’accélération est définie par le facteur de gain de temps :

$$s_p = \frac{t_1 = \text{temps pour un processus}}{t_p = \text{temps pour } p \text{ processus}}.$$

On cherche à obtenir une accélération linéaire, en  $O(P)$ . En pratique, il faut faire attention aux problèmes d’accès aux données (temps de communication, différents types de mémoire cache, etc.) On doit décomposer les données lorsque celles-ci sont trop volumineuses pour tenir dans l’espace mémoire local d’un processeur.

Souvent, on peut obtenir une bonne parallélisation lorsque le problème est *décomposable*. Par exemple, au jeu d’échec, il faut trouver le meilleur déplacement pour une configuration de l’échiquier donnée. L’espace des configurations est fini bien que combinatoirement très grand. Théoriquement,

on pourrait explorer tous les mouvements possibles : à chaque mouvement, on partitionne l'espace des configurations de l'échiquier. Les étapes de communication sont pour partager le problème et combiner les résultats (**reduce**). Ainsi, on s'attend à avoir une accélération linéaire. En pratique, la performance de la machine dépend de la profondeur de recherche lors de l'exploration de l'*espace des configurations* : c'est-à-dire, du nombre de coups dépliés à l'avance et de l'évaluation de l'échiquier après ces coups joués. Le calcul haute performance a permis de battre l'Homme aux échecs (l'ordinateur IBM Deep Blue avec à peu près 12 GFLOPS a battu au match revanche Kasparov en 1997), et maintenant on se tourne vers le défi du jeu de Go qui offre une plus grande richesse combinatoire.

Les problèmes ne se parallélisent pas forcément toujours bien. Par exemple, les problèmes avec des *domaines irréguliers et dynamiques* comme la simulation de la fonte de neige (qui demande des frontières changeantes et des remaillages locaux, etc.). Dans ce cas, afin d'obtenir une bonne parallélisation, on demande une gestion explicite dans le code de l'équilibrage (*load balancing*) entre les divers processus. La répartition dynamique des données entre les processus est coûteuse et l'accélération est difficile à prévoir, car elle dépend de la sémantique du problème sur les jeux de données, etc.

Regardons maintenant quelques exemples simples de programmes MPI.

### 2.10.1 Le produit matrice-vecteur

Ce programme calcule le produit  $M \times v$  où  $M$  est une matrice et  $v$  un vecteur colonne. Les matrices en C se comportent comme celles en Java. Nous verrons plus en détail l'utilisation des matrices dans le chapitre 5 portant sur l'algèbre linéaire.

L'algorithme pour calculer  $c = A \times b$  consiste à tout d'abord diffuser le vecteur  $b$  à tous les processus. C'est le processus  $P_0$  qui s'en charge mais tout le monde doit appeler `MPI_Bcast`. Puis  $P_0$  envoie à tous les processus une ligne de la matrice  $A$  ( $P_0$  envoie à  $P_j$  la  $j$ -ième ligne  $A_j$  de  $A$ ). Le processus  $P_j$  (qui a reçu au préalable  $b$  par diffusion) reçoit sa ligne  $A_j$ , et calcule localement le produit scalaire  $A_j \cdot b$  qui donne le résultat  $c_j$ . Enfin chaque processus  $j$ , renvoie son résultat  $c_j$  à  $P_0$  grâce à une opération collective de communication : un rassemblement, ou *gather*.

Code 13 – Produit matrice-vecteur en MPI `produitmatricevecteur442.cpp`

```
// Nom du programme : produitmatricevecteur442.cpp
// Compilez avec mpic++ produitmatricevecteur442.cpp -o produitmatricevecteur442.exe
// Exécutez avec mpirun -np 4 produitmatricevecteur442.exe

#include <stdio.h>
#include "mpi.h"

// taille des matrices et vecteurs (choisir np=dimension dans mpirun)
int dimension=4;

//
// Calcul matrice-vecteur: c = A x b
//
int main(int argc, char **argv)
{
    int A[dimension][dimension], b[dimension], c[dimension], tmpc, line[dimension],
        temp[dimension];
    int myid=-1, i, j;
```

```

MPI_Init(&argc, &argv);
MPI_Comm_rank(MPI_COMM_WORLD, &myid);
if (myid==0) {
    // Je suis le processeur  $P_0$ , j'initialise la matrice et le vecteur
    for ( i=0; i<dimension; ++i) {
        for ( j=0; j<dimension; ++j) {
            A[i][j]=i+j;
        }
    }

    for ( i=0; i<dimension; ++i) {
        b[i]=dimension-i;
    }

    // On affiche à la sortie (pour se rassurer :D)
    printf("Matrice A:\n");
    for (i=0; i<dimension; i++)
    {
        for (j=0; j<dimension; j++) {
            printf("%d\t", A[i][j]);
        }
        printf("\n");
    }

    printf("Vecteur b:\n");
    for (i=0; i<dimension; i++) {
        printf("%d\t", b[i]);
    }
    printf("\n");
}

// Diffuse le vecteur b à tous les autres processeurs : tout le monde appelle Bcast !
// Rappel de syntaxe :
// int MPI_Bcast(void *buffer, int count, MPI_Datatype datatype, int root, MPI_Comm comm)
MPI_Bcast(b, dimension, MPI_INT, 0, MPI_COMM_WORLD);
// code commun à tous les processus
printf("[proc %d] b:", myid);
for (i=0; i<dimension; i++)
    printf("%d\t", b[i]);
printf("\n");

if (myid==0) {
    // Maintenant,  $P_0$ =je, et j'envoie les lignes de A à tous les processus (en m'incluant aussi)
    for (i=0; i<dimension; ++i) { // pour toutes les lignes de la matrice A
        for (j=0; j<dimension; j++)
        {
            temp[j]=A[i][j];
        }
        // On utilise le tag 442+i pour envoyer  $A_i$  à  $P_i$ 
        MPI_Send(temp, dimension, MPI_INT, i, 442+i, MPI_COMM_WORLD);
    }
}

// Je reçois de  $P_0$  ma ligne avec mon tag unique myid
MPI_Recv(line, dimension, MPI_INT, 0, 442+myid, MPI_COMM_WORLD,
        MPI_STATUS_IGNORE);

printf("[proc %d] matrix line:", myid);

```

```

for (i=0; i<dimension; i++)
    {printf("%d\t", line[i]);}
printf("\n");

// Je fais mon calcul local de produit scalaire
tmpc=0;
for (i=0; i<dimension; i++) {
    tmpc+=line[i]*b[i];
}

printf("Proc. %d a fait son calcul produit scalaire local : %d\n", myid, tmpc);
/*
int MPI_Gather(void *sendbuf, int sendcount, MPI_Datatype sendtype,
void *recvbuf, int recvcnt, MPI_Datatype recvttype, int root,
MPI_Comm comm)
*/
// On assemble tous les resultats dans le tableau c sur P0
MPI_Gather(&tmpc, 1, MPI_INT, c, 1, MPI_INT, 0, MPI_COMM_WORLD);

if (myid==0) { // P0 affiche son resultat
    printf("[proc %d] c:", myid);
    for (i=0; i<dimension; i++)
        {printf("%d\t", c[i]);}
    printf("\n");
}

MPI_Finalize();
return 0;
}

```

À l'exécution de ce programme, nous obtenons en sortie :

```

[suede MPI]$ mpirun -np 4 produitmatricevecteur442.exe
Matrice A:
0      1      2      3
1      2      3      4
2      3      4      5
3      4      5      6
Vecteur b:
4      3      2      1
[proc 0] b:4      3      2      1
[proc 1] b:4      3      2      1
[proc 2] b:4      3      2      1
[proc 3] b:4      3      2      1
[proc 1] matrix line:1 2      3      4
Proc. 1 a fait son calcul produit scalaire local : 20
[proc 2] matrix line:2 3      4      5
[proc 0] matrix line:0 1      2      3
Proc. 0 a fait son calcul produit scalaire local : 10
[proc 3] matrix line:3 4      5      6
Proc. 3 a fait son calcul produit scalaire local : 40
Proc. 2 a fait son calcul produit scalaire local : 30
[proc 0] c:10    20    30    40

```

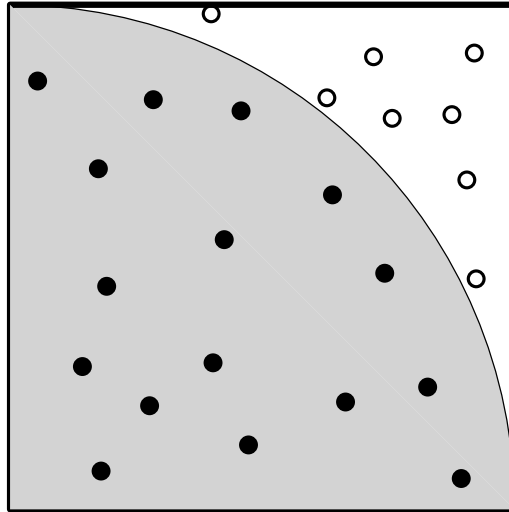


FIGURE 2.7 – Simulation de Monte-Carlo pour approximer  $\pi$  : on tire aléatoirement selon une loi uniforme dans le carré unité  $n$  points, et on compte ceux qui tombent à l'intérieur du cercle unité centré à l'origine ( $n_c$ ). On peut alors approcher  $\frac{\pi}{4}$  par le rapport  $\frac{n_c}{n}$ .

### 2.10.2 Calcul de $\pi$ approché par une simulation Monte-Carlo

On présente la méthode de Monte-Carlo qui consiste à approximer une intégrale par une somme discrète (en résumé :  $\int \approx \sum$ ). Pour le calcul approché de  $\pi$ , on tire  $n$  points aléatoirement selon la loi uniforme dans le carré unité. On calcule alors le rapport des points  $n_c$  tombant dans le quadrant du cercle unité sur le nombre de points  $n$  tirés. On en déduit :

$$\frac{\pi}{4} \approx \frac{n_c}{n}, \pi_n = \frac{4n_c}{n}$$

La figure 2.7 illustre le principe de cette d'approximation.

La valeur approchée de  $\pi$  converge très lentement mais cet estimateur de Monte-Carlo est statistiquement *consistent* puisque nous avons en théorie :

$$\lim_{n \rightarrow \infty} \pi_n = \pi.$$

Cette approche est facile à paralléliser et l'accélération est linéaire, comme attendue. L'idée consiste à tirer aléatoirement les points et calculer localement le nombre de points (variable **interieur**) tombant dans le quadrant du disque. Puis, nous agrégons la variable **interieur** avec une opération de réduction pour calculer la somme cumulée de tous les points ayant tombés dans le quadrant du disque.

Code 14 – Calcul de  $\pi$  approché par une simulation Monte-Carlo : `piMonteCarlo442.cpp`

```
// Nom du programme : piMonteCarlo442.cpp
// Compilez avec mpic++ piMonteCarlo442.cpp -o piMonteCarlo442.exe
```

```

// Exécutez avec mpirun -np 8 piMonteCarlo442

#include <stdio.h>
#include <stdlib.h>
#include <math.h> // pour M_PI et fabs
#include "mpi.h"

int main(int argc, char *argv[]) {
    int moi, nprocs;
    int n = 100000000; // nombre de points pour chaque processus
    // pour éviter les débordements, on doit avoir
    // n*nprocs < 2,147,483,647 (INT_MAX) pour le format 32-bit
    int i, interieur=0, totalInterieur;
    double x,y, approxpi;

    MPI_Init(&argc,&argv);

    MPI_Comm_size(MPI_COMM_WORLD,&nprocs);
    MPI_Comm_rank(MPI_COMM_WORLD,&moi);

    interieur = 0;

    // Il est important de choisir un générateur aléatoire différent
    // pour chaque processus sinon les processus vont tirer les mêmes nombres!
    srand(moi);

    for (i=0; i<n; i++) {
        x = (double) rand()/RAND_MAX;
        y = (double) rand()/RAND_MAX;
        // compte les points qui tombent dans le quadrant du disque
        if (x*x+y*y<1.0) interieur++;
    }

    approxpi = 4.0*interieur/(double)(n);
    // affiche la valeur avec la notation ingénieur
    printf("pi approche par le proc. %d avec %d points = %e erreur:%e\n",moi, n,
        approxpi, fabs(approxpi-M_PI));

    // Maintenant on accumule tous les résultats avec une réduction
    MPI_Reduce(&interieur,&totalInterieur,1,MPI_INT,MPI_SUM,0,MPI_COMM_WORLD);

    if (moi == 0) {
        approxpi = 4.0*totalInterieur/(double)(nprocs*n);
        printf("[accumulation] pi approche avec %d points = %e\n",n*nprocs,approxpi);
        printf("erreur d'approximation : %e\n",fabs(approxpi-M_PI));
    }

    MPI_Finalize();
}

```

Sur une machine à 8 cœurs (france), nous obtenons le résultat suivant :

```

pi approche par le proc. 5 avec 100000000 points = 3.142172e+00 erreur:5.797864e-04
pi approche par le proc. 2 avec 100000000 points = 3.141693e+00 erreur:1.001464e-04
pi approche par le proc. 1 avec 100000000 points = 3.141698e+00 erreur:1.052664e-04
pi approche par le proc. 7 avec 100000000 points = 3.141469e+00 erreur:1.235336e-04
pi approche par le proc. 0 avec 100000000 points = 3.141698e+00 erreur:1.052664e-04
pi approche par le proc. 3 avec 100000000 points = 3.141753e+00 erreur:1.604664e-04

```

```

pi approche par le proc. 6 avec 100000000 points = 3.141616e+00 erreur:2.342641e-05
pi approche par le proc. 4 avec 100000000 points = 3.141583e+00 erreur:9.933590e-06
[accumulation] pi approche avec 800000000 points = 3.141710e+00
erreur d'approximation : 1.176114e-04

```

Pour pouvoir utiliser n'importe quelle machine des salles informatiques, on commence par générer sa clef sécurité (en appuyant sur "RETURN" pour donner une phrase vide) :

```

[france ~]$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/users/profs/info/nielsen/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /users/profs/info/nielsen/.ssh/id_rsa.
Your public key has been saved in /users/profs/info/nielsen/.ssh/id_rsa.pub.
The key fingerprint is:

```

Puis on ajoute cette clef publique dans la liste des clefs autorisées :

```
cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys
```

Maintenant, on peut utiliser les autres machines aisément :

```

mpirun -np 12 -host thon,jura,simca piMonteCarlo442.exe

[essonne MPI]$ mpirun -np 12 -host thon,jura,simca piMonteCarlo442.exe
pi approche par le proc. 2 avec 100000000 points = 3.141693e+00 erreur:1.001464e-04
pi approche par le proc. 5 avec 100000000 points = 3.142172e+00 erreur:5.797864e-04
pi approche par le proc. 11 avec 100000000 points = 3.141732e+00 erreur:1.391064e-04
pi approche par le proc. 7 avec 100000000 points = 3.141469e+00 erreur:1.235336e-04
pi approche par le proc. 4 avec 100000000 points = 3.141583e+00 erreur:9.933590e-06
pi approche par le proc. 3 avec 100000000 points = 3.141753e+00 erreur:1.604664e-04
pi approche par le proc. 6 avec 100000000 points = 3.141616e+00 erreur:2.342641e-05
pi approche par le proc. 0 avec 100000000 points = 3.141698e+00 erreur:1.052664e-04
pi approche par le proc. 10 avec 100000000 points = 3.141671e+00 erreur:7.834641e-05
pi approche par le proc. 1 avec 100000000 points = 3.141698e+00 erreur:1.052664e-04
pi approche par le proc. 9 avec 100000000 points = 3.141427e+00 erreur:1.660936e-04
pi approche par le proc. 8 avec 100000000 points = 3.141587e+00 erreur:6.093590e-06
[accumulation] pi approche avec 1200000000 points = 3.141675e+00
erreur d'approximation : 8.217974e-05

```

Maintenant, on va passer à l'échelle grâce à SLURM. Pour connaître le cluster sur lequel on se trouve, on tape `sinfo` :

```

[essonne MPI]$ sinfo
PARTITION AVAIL  TIMELIMIT  NODES  STATE NODELIST
SN254*      up           15:00    50   idle ablette,ain,allier,anchois,
          anguille,ardennes,barbeau,barbue,baudroie,brochet,carmor,carrelet
          ,charente,cher,creuse,dordogne,doubs,essonne,finistere,gardon,
          gironde,gymnote,indre,jura,labre,landes,lieu,loire,lotte,manche,
          marne,mayenne,morbihan,moselle,mulet,murene,piranha,raie,requin,
          rouget,roussette,saone,saumon,silure,sole,somme,thon,truite,
          vendee,vosges

```

On demande à SLURM 32 processus avec au plus 8 processus par nœud (les ordinateurs sont des octo-cœurs).

```
salloc --ntasks=32 --ntasks-per-node=8 bash
```

On peut visualiser ce que SLURM nous a alloué en faisant :

```
set | grep SLURM
```

On trouve ici :

```
[essonne MPI]$ set | grep SLURM
SLURM_JOBID=93
SLURM_JOB_CPUS_PER_NODE='8(x4)'
SLURM_JOB_ID=93
SLURM_JOB_NODELIST=charente,cher,creuse,dordogne
SLURM_JOB_NUM_NODES=4
SLURM_JOB_PARTITION=SN254
SLURM_NNODES=4
SLURM_NODELIST=charente,cher,creuse,dordogne
SLURM_NODE_ALIASES='(null) '
SLURM_NPROCS=32
SLURM_NTASKS=32
SLURM_NTASKS_PER_NODE=8
SLURM_SUBMIT_DIR=/users/profs/info/nielsen/MPI
SLURM_SUBMIT_HOST=essonne.polytechnique.fr
SLURM_TASKS_PER_NODE='8(x4)'
```

Pour éviter les débordements numériques lorsqu'on utilise un grand nombre de processus, nous recompilons notre programme avec `int n = 10000000;` (un zéro en moins).

Il nous suffit alors de taper à la console :

```
[essonne MPI]$ mpirun piMonteCarlo442.exe
```

```
[essonne MPI]$ mpirun piMonteCarlo442.exe
pi approche par le proc. 3 avec 10000000 points = 3.141016e+00 erreur:5.766536e-04
pi approche par le proc. 7 avec 10000000 points = 3.141384e+00 erreur:2.082536e-04
pi approche par le proc. 0 avec 10000000 points = 3.141130e+00 erreur:4.626536e-04
pi approche par le proc. 5 avec 10000000 points = 3.142800e+00 erreur:1.207746e-03
pi approche par le proc. 9 avec 10000000 points = 3.140612e+00 erreur:9.802536e-04
pi approche par le proc. 12 avec 10000000 points = 3.142291e+00 erreur:6.985464e-04
pi approche par le proc. 2 avec 10000000 points = 3.141902e+00 erreur:3.089464e-04
pi approche par le proc. 1 avec 10000000 points = 3.141130e+00 erreur:4.626536e-04
pi approche par le proc. 8 avec 10000000 points = 3.141728e+00 erreur:1.357464e-04
pi approche par le proc. 25 avec 10000000 points = 3.142444e+00 erreur:8.509464e-04
pi approche par le proc. 15 avec 10000000 points = 3.141069e+00 erreur:5.238536e-04
pi approche par le proc. 13 avec 10000000 points = 3.141663e+00 erreur:7.014641e-05
pi approche par le proc. 24 avec 10000000 points = 3.141717e+00 erreur:1.241464e-04
pi approche par le proc. 10 avec 10000000 points = 3.141932e+00 erreur:3.397464e-04
pi approche par le proc. 27 avec 10000000 points = 3.141463e+00 erreur:1.298536e-04
```

```

pi approche par le proc. 29 avec 10000000 points = 3.141680e+00 erreur:8.694641e-05
pi approche par le proc. 30 avec 10000000 points = 3.141310e+00 erreur:2.826536e-04
pi approche par le proc. 28 avec 10000000 points = 3.141064e+00 erreur:5.290536e-04
pi approche par le proc. 11 avec 10000000 points = 3.141888e+00 erreur:2.949464e-04
pi approche par le proc. 31 avec 10000000 points = 3.141525e+00 erreur:6.785359e-05
pi approche par le proc. 26 avec 10000000 points = 3.141202e+00 erreur:3.906536e-04
pi approche par le proc. 14 avec 10000000 points = 3.141738e+00 erreur:1.449464e-04
pi approche par le proc. 4 avec 10000000 points = 3.142245e+00 erreur:6.525464e-04
pi approche par le proc. 6 avec 10000000 points = 3.141558e+00 erreur:3.505359e-05
pi approche par le proc. 22 avec 10000000 points = 3.142302e+00 erreur:7.097464e-04
pi approche par le proc. 19 avec 10000000 points = 3.141890e+00 erreur:2.977464e-04
pi approche par le proc. 20 avec 10000000 points = 3.141332e+00 erreur:2.606536e-04
pi approche par le proc. 17 avec 10000000 points = 3.140649e+00 erreur:9.438536e-04
pi approche par le proc. 18 avec 10000000 points = 3.142658e+00 erreur:1.065746e-03
pi approche par le proc. 16 avec 10000000 points = 3.140917e+00 erreur:6.758536e-04
pi approche par le proc. 21 avec 10000000 points = 3.141385e+00 erreur:2.078536e-04
pi approche par le proc. 23 avec 10000000 points = 3.141272e+00 erreur:3.202536e-04
[accumulation] pi approche avec 32000000 points = 3.141590e+00
erreur d'approximation : 2.166090e-06

```

On vérifie que nous avons obtenu une bien meilleure approximation en utilisant 32 processus.  
 Pour avoir le manuel des commandes SLURM, on tape :

```

man -M /usr/local/slurm/share/man/ sinfo
man -M /usr/local/slurm/share/man/ salloc
...

```

Pour libérer les ressources demandées à SLURM, on tape `exit` : cela permet de sortir de la commande `bash` demandée lors de la commande `salloc`. On obtient à la console un message de la forme :

```
salloc: Relinquishing job allocation xxx
```

Faites attention que vous ayez bien libérer toutes les ressources du cluster. Si vous avez tapé malencontreusement plusieurs fois des `salloc -ntasks=xx -ntasks-per-node=yy bash`, alors il faudra faire plusieurs `exit`.

### 2.10.3 Calcul approché du volume d'une molécule par intégration stochastique

Soit une molécule  $M$  modélisée par un jeu de  $n$  boules en 3D représentant géométriquement les atomes de  $M$ . Pour chaque boule, on connaît sa position et son rayon. On cherche à calculer le volume  $v(M)$  de cette molécule, c'est-à-dire de calculer l'union de  $n$  boules. On va approximer ce volume en faisant une intégration stochastique avec rejet : tout d'abord, on commence par calculer une boîte englobante BB, puis on va échantillonner uniformément dans cette boîte englobante, et rejeter tous les échantillons ne tombant pas dans l'union des boules. Soit  $e$  le nombre total d'échantillons tirés, et  $e'$  le nombre d'échantillons tombant dans l'union (c'est-à-dire, contenue dans au moins une boule). Alors on approxime  $v(M)$  comme suit :  $v(M) \simeq \frac{e'}{e}v(\text{BB})$ .

La figure 2.8 illustre ce procédé d'intégration stochastique par rejet pour le calcul d'une union de disques 2D. Le code séquentiel est donné ci-dessous :

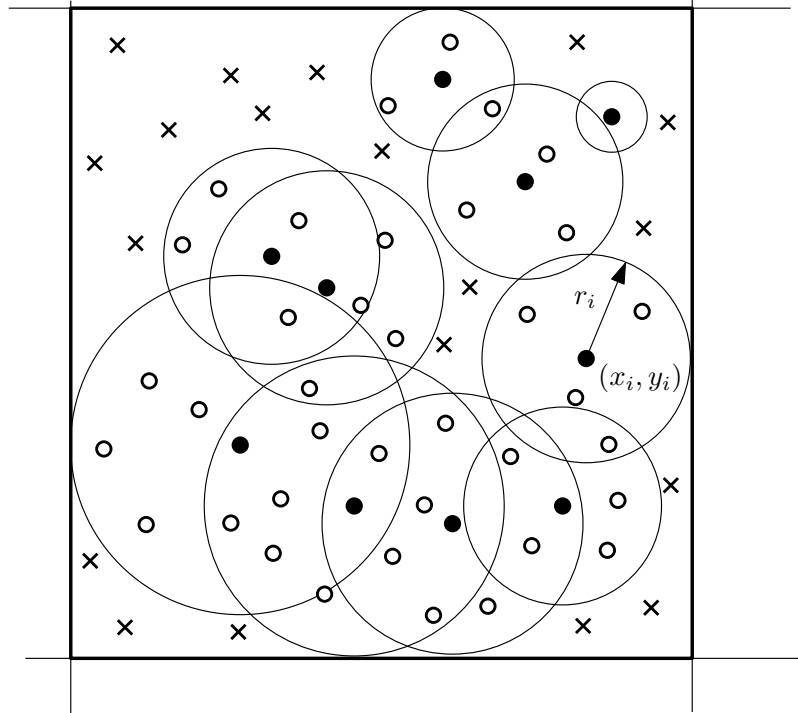


FIGURE 2.8 – Intégration stochastique à la Monte-Carlo avec rejet pour approximer l'union de boules : tout d'abord, on calcule une boîte englobante, puis on tire aléatoirement suivant une loi uniforme  $e$  échantillons, et on approxime le volume par  $\frac{e'}{e}v(\text{BB})$ , où  $e'$  est le nombre d'échantillons tombant dans au moins une des boules.

```

// filename: SequentialVolumeUnionSpheres.cpp
// Sequential implementation of the approximation of the volume of a set of spheres
#include <limits>
#include <math.h>
#include <iostream>
#include <stdlib.h>
#include <time.h>

#define n 8*2
#define d 3
#define e 8*1000

double get_rand(double min, double max) {
    double x = rand() / (double)RAND_MAX;
    return x * (max - min) + min;
}

double distance2(double p0[d], double p1[d]) {
    double x = 0;
    for (int i = 0; i < d; i++) {
        double diff = p0[i] - p1[i];
        x += diff * diff;
    }
    return x;
}

int main(int argc, char** argv) {
    srand(0);
    double radius[n];
    double C[n][d];
    // Generate data
    for (int i = 0; i < n; i++) {
        radius[i] = get_rand(1, 5);
        for (int j = 0; j < d; j++) C[i][j] = get_rand(-20, 20);
    }
    // Compute bounding box
    double bb[d][2];
    for (int i = 0; i < d; i++) {
        bb[i][0] = std::numeric_limits<double>::infinity();
        bb[i][1] = -std::numeric_limits<double>::infinity();
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < d; j++) {
            bb[j][0] = fmin(bb[j][0], C[i][j] - radius[i]);
            bb[j][1] = fmax(bb[j][1], C[i][j] + radius[i]);
        }
    }
    // Compute the volume of the bounding box
    double volBB = 1;
    for (int i = 0; i < d; i++) volBB *= bb[i][1] - bb[i][0];

    // Draw samples and perform rejection sampling
    int ePrime = 0;

```

```

    for (int i = 0; i < e; i++) {
        double pos[d];
        for (int j = 0; j < d; j++) pos[j] = get_rand(bb[j][0], bb[j][1]);
        for (int j = 0; j < n; j++) {

            if (distance2(pos, C[j]) < radius[j] * radius[j]) {
                ePrime++;
            }
        }
    }
    // Compute the volume
    double vol = volBB * (double)ePrime / double(e);
    std::cout << vol << std::endl;
    std::cout << ePrime << std::endl;
    return 0;
}

```

Parallélisons maintenant ce code sur un cluster de machines avec MPI. Initialement, on suppose que toutes les sphères se trouvent en mémoire dans le processus racine (de rang 0), et on doit distribuer ces données à tous les autres processus en utilisant une opération de diffusion personnalisée (*scattering*). Puis on calcule en parallèle la boîte englobante en prenant la boîte englobante des boîtes englobantes de chaque processus. Pour ce faire, on utilise un calcul collaboratif : une opération de réduction (avec les opérateurs binaires spécifiés en MPI par `MPI_MIN` et `MPI_MAX`). Alors tous les échantillons aléatoires sont tirés sur le processus racine, et distribués aux processus via une diffusion. Chaque processus teste si les échantillons tombent dans au moins une boule de son paquet de données, et on agrège les résultats avec une autre opération de réduction en utilisant l'opérateur binaire le OU logique : `MPI_LOR`.

Le programme parallèle en C++ utilisant MPI est donné ci-dessous :

```

// filename: MPIVolumeUnionSpheres.cpp
// Parallel implementation of the approximation of the volume of a set of spheres
#include <limits>
#include <math.h>
#include <iostream>
#include <stdlib.h>
#include <time.h>
#include "mpi.h"

#define n 8*2
#define d 3
#define e 8*1000

double get_rand(double min, double max) {
    double x = rand() / (double)RAND_MAX;
}

double distance2(double p0[d], double p1[d]) {
    double x = 0;
    for (int i = 0; i < d; i++) {
        double diff = p0[i] - p1[i];
        x += diff * diff;
    }
    return x;
}

```

```

int main(int argc, char** argv) {
    srand(0);
    MPI_Init(&argc, &argv);

    int n_proc, rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &n_proc);

    double radius0[n];
    double C0[n][d];
    // Generate data
    if (rank == 0) {
        for (int i = 0; i < n; i++) {
            radius0[i] = get_rand(1, 5);
            for (int j = 0; j < d; j++) C0[i][j] = get_rand(-20, 20);
        }
    }

    // Send data to processes
    double radius[n];
    double C[n][d];
    int begin = n / n_proc * rank;
    int loc_n = n / n_proc;
    MPI_Scatter(radius0, loc_n, MPI_DOUBLE, &(radius[begin]), loc_n, MPI_DOUBLE,
        0, MPI_COMM_WORLD);
    MPI_Scatter(C0, 3 * loc_n, MPI_DOUBLE, &(C[begin][0]), 3 * loc_n, MPI_DOUBLE,
        0, MPI_COMM_WORLD);
    double bb[d][2];

    // Compute the bounding box
    for (int i = 0; i < d; i++) {
        bb[i][0] = std::numeric_limits<double>::infinity();
        bb[i][1] = -std::numeric_limits<double>::infinity();

        for (int j = begin; j < begin + loc_n; j++) {
            bb[i][0] = fmin(bb[i][0], C[j][i] - radius[j]);
            bb[i][1] = fmax(bb[i][1], C[j][i] + radius[j]);
        }

        MPI_Reduce(rank ? &(bb[i][0]) : MPI_IN_PLACE, &(bb[i][0]), 1, MPI_DOUBLE,
            MPI_MIN, 0, MPI_COMM_WORLD);
        MPI_Reduce(rank ? &(bb[i][1]) : MPI_IN_PLACE, &(bb[i][1]), 1, MPI_DOUBLE,
            MPI_MAX, 0, MPI_COMM_WORLD);
    }

    // Compute the volume of the bounding box

    double volBB = 1;
    for (int i = 0; i < d; i++) volBB *= bb[i][1] - bb[i][0];
    // Draw variates and perform rejection sampling
    double samples[e][3];
    if (rank == 0) {
        for (int i = 0; i < e; i++) {
            for (int j = 0; j < d; j++) samples[i][j] = get_rand(bb[j][0], bb[j][1]);
        }
    }
}

```

```

    }
}
MPI_Bcast(samples, 3 * e, MPI_DOUBLE, 0, MPI_COMM_WORLD);
// Testing variates

bool hit[e];
for (int i = 0; i < e; i++) hit[i] = false;
for (int i = 0; i < e; i++) {
    for (int j = begin; j < begin + loc_n; j++) {
        if (distance2(samples[i], C[j]) < radius[j] * radius[j]) hit[i] =
            true;
    }
}

// Gather results and count the accepted variates

bool hit0[e];
for (int i = 0; i < e; i++) hit0[i] = false;

MPI_Reduce(hit, hit0, e, MPI_C_BOOL, MPI_LOR, 0, MPI_COMM_WORLD);

if (rank == 0) {
    int ePrime = 0;
    for (int i = 0; i < e; i++) {
        if (hit0[i]) ePrime++;
    }
    double vol = volBB * (double)ePrime / double(e);
    std::cout << vol << std::endl;
    std::cout << ePrime << std::endl;
}

MPI_Finalize();
return 0;
}

```

## 2.11 \* Pour en savoir plus : notes, références et discussion

Un précurseur du standard MPI était la bibliothèque PVM<sup>25</sup>, pour *Parallel Virtual Machine*, qui comprenait déjà des opérations de communication synchrones et asynchrones. Le standard MPI est souvent décrit dans les livres traitant du parallélisme [56, 21]. Nous n'en avons couvert que les principaux éléments. On recommande ces ouvrages [86, 41] qui couvrent respectivement les fonctionnalités de MPI-I et MPI-II. Il existe de nombreux autres mécanismes dans MPI : par exemple, on peut définir des types dérivés avec `MPI_type_struct`, etc. L'ouvrage [40] présente les toutes dernières fonctionnalités de MPI-3. Le livre [65] donne quelques exemples de programmes en MPI pour le calcul scientifique parallèle. Les procédures de préfixe parallèle (`scan`) en MPI sont finement décrites dans le papier [82]. Finalement, on pourra consulter l'ouvrage [77] qui montre comment implémenter les algorithmes parallèles en utilisant MPI. Il existe plusieurs *bindings* de l'API MPI. Dans ce chapitre, nous avons mentionné les interfaces en C, C++ et Boost. Il y a aussi l'interface de liaison du langage moderne Python<sup>26</sup> : `mpi4py`.<sup>27</sup>

25. <http://www.csm.ornl.gov/pvm/>

26. <https://www.python.org/>

27. <http://mpi4py.scipy.org/>

## 2.12 En résumé : ce qu'il faut retenir !

MPI est un standard qui permet de définir précisément l'interface et la sémantique des protocoles de communication et des calculs globaux (entres autres) pour des processus répartis sur un cluster lors de l'exécution d'un programme parallèle (processus avec leur propre mémoire distribuée). Les communications peuvent être bloquantes ou non-bloquantes, bufferisées ou non bufferisées, et on peut définir des barrières de synchronisation où tous les processus doivent s'attendre avant de pouvoir continuer. Il existe plusieurs implémentations de MPI (openmpi, mpich, etc.) et ces diverses implémentations peuvent être utilisées également dans plusieurs langages de programmation en utilisant les bindings adéquats (C/C++/C++-Boost/Fortran, etc.). Outre les communications de base (diffusion, diffusion personnalisée, rassemblement, échange total), le dernier standard MPI-3 offre plus de 200 fonctions et permet de gérer les entrées et les sorties en parallèle également.

## 2.13 Exercices

**Exercice 5 :** *Inférence statistique en parallèle avec MPI*

Soit  $X = \{x_1, \dots, x_n\} \subset \mathbb{R}$  un grand jeu de données supposés être des échantillons aléatoires indépendamment tirés d'une loi normale univariée  $N(\mu, \sigma^2) : x_1, \dots, x_n \sim N(\mu, \sigma^2)$ . On rappelle que les estimateurs non-biaisés du maximum de vraisemblance, pour, respectivement, la moyenne et la variance, sont :

$$\hat{\mu}_n = \frac{1}{n} \sum_{i=1}^n x_i \quad (2.1)$$

$$\hat{\sigma}_n^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \hat{\mu}_n)^2. \quad (2.2)$$

1. Montrez que  $\hat{\mu}_n$  et  $\hat{\sigma}_n^2$  peuvent se calculer uniquement avec  $n$ ,  $S_1 = \sum_{x \in X} x$  et  $S_2 = \sum_{x \in X} x^2$ . Donner les formules de  $\hat{\mu}_n$  et  $\hat{\sigma}_n^2$  explicitement en fonction de  $n$ ,  $S_1$  et  $S_2$  (et d'aucune autre variable).
2. On suppose les données  $X$  stockées sur un des ordinateurs dans un cluster distribué avec  $p$  ordinateurs. En utilisant les primitives MPI `scatter()` et `reduce()`, écrire le pseudo-code d'un algorithme distribué qui :
  - partage les données de manière équilibrée sur chacun des ordinateurs du cluster,
  - calcule  $\hat{\mu}_n$  et  $\hat{\sigma}_n^2$  de manière distribuée pour l'ensemble total  $X$ .

## Chapitre 3

# La topologie des réseaux d'interconnexion

Un résumé des points essentiels à retenir est donné dans §3.11.

### 3.1 Réseaux statiques/dynamiques et réseaux logiques/physiques

Dans ce cours, un *ordinateur parallèle* est une grappe de machines à mémoire distribuée reliées entre elles par un *réseau d'interconnexion*. On distingue deux types de réseaux : (i) les *réseaux statiques* qui sont fixés une fois pour toute et non modifiables, et (ii) les *réseaux dynamiques* qui peuvent être modifiables en cours d'exécution par un *gestionnaire de connexions* en fonction du trafic et de la congestion du réseau.

Lorsqu'on implémente un algorithme parallèle, on distingue également le *réseau physique* du *réseau logique*. Le réseau physique est le réseau où chaque nœud est un processeur (*Processing Element*, PE) et chaque lien relie deux processeurs pouvant communiquer *directement* entre eux. Le réseau logique, quant à lui, est une *abstraction* d'un réseau de communication *indépendante* de l'architecture matérielle qui facilite la mise en œuvre d'algorithmes parallèles. Ainsi le réseau logique dépend de l'algorithme (qui peut demander à organiser les processeurs par groupes) alors que le réseau physique dépend du matériel. En pratique, on obtient une performance optimale quand les réseaux physiques et logiques coïncident, sinon on cherche une *transposition*, encore appelée *plongement*, du réseau logique utilisé par les algorithmes parallèles dans le réseau physique matériel de la machine parallèle.

### 3.2 Réseaux d'interconnexions : modélisation par des graphes

Pour relier  $P$  processeurs (avec une mémoire propre attachée à chaque processeur) entre eux on a de très nombreuses façons de faire ! Aux deux extrêmes, on peut considérer soit (i) une architecture reliant tous les processeurs à un *bus commun* qui doit faire face cependant aux problèmes de collisions

des messages lors de son utilisation simultanée, ou soit (ii) un réseau point à point reliant chaque paire de processeurs entre eux par un lien dédié pour faire transiter les messages. Cette dernière solution ne passe pas à l'échelle car elle requiert une complexité quadratique de connexions.

La *topologie*<sup>1</sup> du réseau est décrite par un *graphe*  $G = (V, E)$  avec :

- $V$  : l'ensemble des *sommets* (*vertices*) qui représente les processeurs (*PEs*),
- $E$  : l'ensemble des *arêtes* (*edges*) qui représente les liens de communication entre les processeurs.

Les arêtes peuvent être orientées ou pas. Dans le premier cas, on préfère parler alors d'arcs de communication.

Pour construire une bonne topologie, on cherche à établir un bon équilibre (*trade-off*) entre deux critères opposés :

- minimiser le nombre de liens afin de diminuer le coût matériel, et
- maximiser le nombre des communications directes (arêtes) afin de diminuer le coût des communications des algorithmes parallèles implémentés.

Les liens peuvent être *unidirectionnels* ou *bidirectionnels* (revient à dédoubler les arêtes en des arcs de sens opposés). Dans le cas de *liens bidirectionnels*, lorsque la bande passante est *partagée* par deux messages sur le même lien allant dans des directions opposées, on parle de modèle *half-duplex*. Sinon, lorsque l'on garde la bande passante pour chaque direction, on parle de modèle *full-duplex*.

Lorsque nous avons des communications concurrentes multiples, on peut bénéficier de recouvrements : en effet, on peut supposer que les processeurs peuvent faire en concurrence des envois et des réceptions non-bloquants ainsi que des calculs locaux, le tout en même temps.

Sur un nœud du réseau logique à  $l$  liens, on peut définir le nombre maximum de communications concurrentes admissibles : on a un modèle *multi-port* lorsque l'envoi et la réception en parallèle est possible sur tous les liens. Sinon, on parle de *k-port* pour au plus  $k$  envois et  $k$  réceptions en parallèle (*1-port* en cas particulier).

On suppose dans la suite que le *routing* des messages dans le réseau se fait sans perte : aucun message n'est rejeté et on ne prend pas en compte les *contentions*. En pratique, cela demande un contrôle du flot des messages sur les liens/nœuds par un *gestionnaire de communications* qui doit arbitrer les messages. La communication avec contention signifie que les processeurs attachés au réseau peuvent émettre et recevoir des messages quand ils le désirent, et cela peut provoquer des collisions lorsqu'un lien de communication est déjà utilisé et de nouveau sollicité. La méthode de communication par contention entraîne une compétition pour l'utilisation des ressources.

Nous allons caractériser plus en détail les propriétés des réseaux d'interconnexion par l'étude de la topologie des graphes induits.

### 3.3 Caractéristiques des topologies

Un *chemin* dans un graphe  $G = (V, E)$  est une séquence de nœuds  $V_1, \dots, V_C$  telle que les arêtes  $(V_i, V_{i+1}) \in E$  pour  $1 \leq i \leq C - 1$ . La *longueur d'un chemin* dans un graphe est le nombre d'arêtes de ce chemin :  $L = C - 1$ . La *distance* entre deux nœuds dans un graphe est la longueur d'un plus court chemin reliant ces deux nœuds.

1. La topologie est un terme géométrique qui définit les propriétés globales des objets et des espaces comme le nombre de composantes connexes ou bien encore le nombre d'anses d'un objet, son genre, etc.

### 3.3.1 Degré et diamètre d'un graphe

On définit les principaux attributs structuraux d'un graphe  $G = (V, E)$  d'un réseau de communication comme suit :

- *dimension* : le nombre de nœuds du graphe (avec  $p = |V|$ ),
- *nombre de liens*,  $l = |E|$ ,
- *degré* d'un nœud : nombre de liens,  $d$ , partant ou arrivant à un nœud. Dans le cas de graphes orientés, on a pour un sommet  $s$   $d(s) = d_{\text{arrivant}}(s) + d_{\text{partant}}(s)$ . Lorsque le graphe est régulier, tous les nœuds ont le même degré, et on parle alors du degré d'un graphe,
- *diamètre*  $D$  : le maximum des distances entre deux nœuds du graphe.

### 3.3.2 Connexité et bissection

Il est fort utile de pouvoir faire grossir le nombre de nœuds de l'ordinateur parallèle en fonction de la taille des données du problème. Cela demande également de pouvoir construire des *topologies génériques* qui passent à l'échelle : c'est-à-dire, des topologies extensibles. On peut caractériser récursivement des topologies à partir de leurs sous-topologies en définissant :

- la *connexité* du réseau comme étant le nombre minimum de liens (arêtes) à enlever afin d'obtenir deux réseaux connexes,
- la *largeur de bissection*, notée  $b$ , comme étant le nombre minimum de liens nécessaires pour relier deux moitiés *semblables*.

### 3.3.3 Critères pour une bonne topologie du réseau

Quelles sont donc les bonnes topologies pour un réseau ?

On cherche à :

- minimiser le degré du réseau régulier afin d'avoir un coût matériel faible,
- minimiser le diamètre du réseau afin d'avoir des chemins courts pour les opérations de communication,
- maximiser la dimension du réseau : c'est-à-dire augmenter  $P$  (le nombre nœuds/processeurs) pour traiter de plus grands volumes de données (passage à l'échelle, *scalability*).

On peut aussi mentionner ces propriétés pour faciliter la comparaison des topologies :

- arguments en faveur d'une topologie :
  - uniformité ou symétrie,
  - capacité à partitionner en sous-réseaux de même topologie, ou à étendre le réseau en conservant la même topologie,
  - passage à l'échelle : augmenter la performance en proportion de sa dimension  $P$ ,
  - capacité à transformer le réseau en d'autres topologies,
  - facilité de routage.
- arguments négatifs d'une topologie :
  - augmentation du coût ou de la complexité du routage (degré élevé),
  - perte de robustesse face aux pannes (degré bas, connexité),
  - perte de performance en communication (degré bas et diamètre élevé),
  - perte de performance en calcul (petite dimension).

Nous allons maintenant lister les principales topologies usuelles.

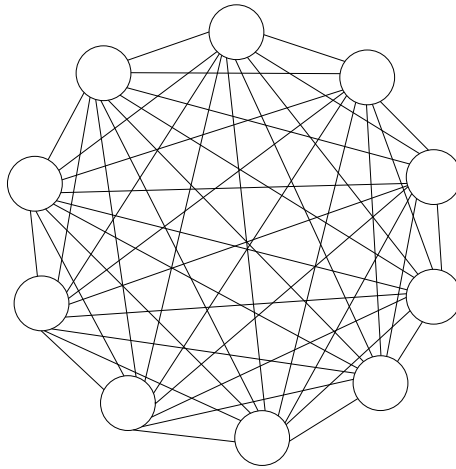


FIGURE 3.1 – Le réseau complet (ici,  $K_{10}$ ) minimise les coûts de communication mais a un coût matériel élevé ou des restrictions physiques qui empêchent son passage à l'échelle.

### 3.4 Les topologies usuelles : les réseaux statiques simples

#### 3.4.1 La clique : le graphe complet $K$

Le réseau complet à  $P$  nœuds est représenté par la clique (graphe complet), illustré à la figure 3.1. C'est le réseau idéal pour les communications car les processeurs sont tous à une distance unité les uns des autres (diamètre  $D = 1$ ). Par contre, le degré des nœuds est  $d = p - 1$  et le nombre de liens est quadratique :  $\binom{P}{2} = \frac{P(P-1)}{2}$ . Le réseau complet permet de simuler facilement *toutes les autres topologies* à  $P$  nœuds mais a malheureusement un coût élevé ou des limitations physiques<sup>2</sup> qui le limitent en pratique à de petites valeurs de  $P$ .

#### 3.4.2 L'étoile, l'anneau et l'anneau cordal

L'étoile ou *star graph* en anglais est illustrée à la figure 3.2 (a). Bien qu'efficace pour les communications, la topologie de l'étoile a une faible tolérance aux pannes lorsque le nœud central dysfonctionne. Le graphe de l'anneau est appelé *ring graph* en anglais et sa topologie est illustrée à la figure 3.2.

La topologie de l'anneau permet de faire des algorithmes *pipelinés* comme le produit matrice-vecteur, etc. Les liens de l'anneau peuvent être unidirectionnels (graphe orienté) ou bidirectionnels (graphe non-orienté).

Un inconvénient majeur est le temps de communication égal au diamètre :  $D = P - 1$  pour l'anneau orienté et  $D = \lfloor \frac{P}{2} \rfloor$  pour l'anneau non-orienté. La fonction  $\lfloor x \rfloor$  (partie entière) retourne l'entier le plus grand qui est plus petit ou égal à  $x$ .

Afin de limiter cet inconvénient des grands coûts de communication, on peut ajouter des *cordes* sur l'anneau et obtenir un anneau cordé (*chordal ring*) : les communications sont plus rapides car le diamètre  $D$  devient plus petit (dépend du nombre de cordes). Pour avoir une topologie régulière

2. En effet, il est difficile de connecter un grand nombre de cables à une machine en pratique !

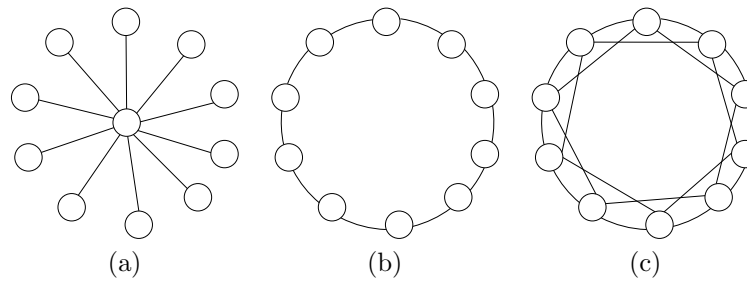


FIGURE 3.2 – La topologie de l'étoile (a) (avec  $P = 11$  nœuds) garantit un diamètre de 2 pour les communications mais est très vulnérable lorsqu'une panne apparaît sur le nœud central. Les topologies de (b) l'anneau et de (c) l'anneau cordal : rajouter des cordes fait baisser la valeur du diamètre.

(tous les nœuds jouant le même rôle dans le réseau) sur l'anneau cordal, on doit choisir la largeur des pas comme étant un diviseur du nombre total de nœuds. Par exemple, pour  $P = 10$  nœuds, on peut choisir des pas de largeur 2 et 5 (voir la figure 3.2).

### 3.4.3 Grilles et tores

La *grille* (*near-neighbor mesh* en anglais) est bien adaptée au domaine de l'image (2D pour des pixels et 3D pour des voxels) et aux algorithmes parallèles sur les matrices. Un inconvénient majeur de la grille est son degré irrégulier dû aux nœuds du bords qui nécessitent aux algorithmes parallèles de considérer ces cas particuliers.

La topologie du *tore* (*torus*) permet d'y remédier, et facilite ainsi un traitement uniforme des nœuds. La figure 3.3 illustre la différence entre la topologie de grille et celle du tore. Les topologies de grilles et de tores s'étendent à n'importe quelle dimension (important pour les calculs tensoriels modernes). D'ailleurs, nous remarquons que le tore 1D est l'anneau !

### 3.4.4 Le cube 3D et les cycles connectés en cube (CCC)

Le cube a un diamètre de 3 avec des sommets réguliers. On peut augmenter le nombre de nœuds en insérant un anneau à la place de chaque sommet du cube : on obtient ainsi la topologie du *Cycle Connecté en Cube*<sup>3</sup> (CCC). Ces topologies sont très souvent utilisées en pratique et se généralisent en toute dimension. La figure 3.4 illustre le cube et la topologie des cycles connectés en cube.

### 3.4.5 Arbres et arbres élargis (*fat trees*)

Beaucoup d'algorithmes parallèles utilisent des structures d'arbres avec requêtes. Les arbres permettent aussi d'implémenter aisément des *parcours en profondeur* (*depth-first search*) ou des *parcours en largeur* (*breadth-first search*). Il est donc important d'avoir la topologie du réseau physique qui correspond à la topologie logique (encore appelé *topologie virtuelle*) utilisée par l'algorithme. On note que plus on se rapproche de la racine plus la bande passante doit être grande car on remonte

3. L'avantage de la topologie CCC est que le degré  $d = 3$  au lieu de  $d = s$  en dimension  $s$ . Le nombre de nœuds  $p = 2^s s$ , diamètre  $D = 2s - 2 + \lfloor \frac{s}{2} \rfloor$  pour  $s > 3$  et  $D = 6$  quand  $s = 3$ .

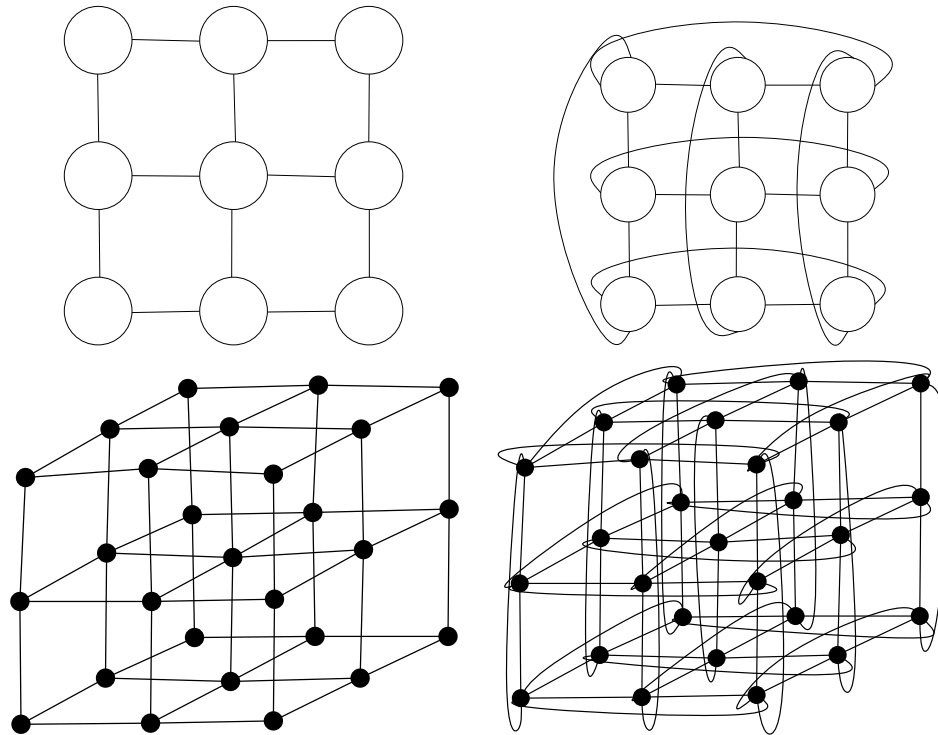


FIGURE 3.3 – Topologie irrégulière de la grille et topologie régulière du tore (en 2D et en 3D).

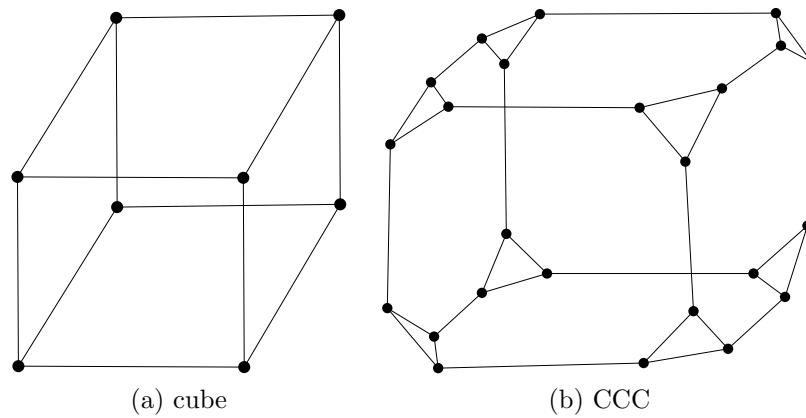


FIGURE 3.4 – Topologie du cube (a) et des cycles connectés en cube (b).

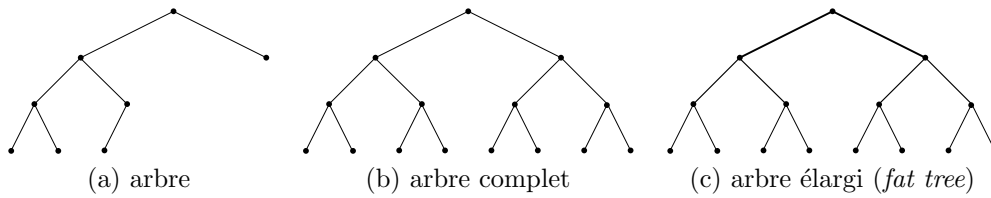


FIGURE 3.5 – Topologie des arbres (a), arbres complets (b) et des arbres élargis (c). Dans le cas des arbres élargis (*fat trees*), la bande passante des arêtes augmente plus on est proche de la racine.

| topologie      | processeurs $P$                | degré $k$      | diamètre $D$                          | #liens $l$              | $b$             |
|----------------|--------------------------------|----------------|---------------------------------------|-------------------------|-----------------|
| réseau complet | $P$                            | $P - 1$        | 1                                     | $\frac{P(P-1)}{2}$      | $\frac{P^2}{4}$ |
| anneau         | $P$                            | 2              | $\lfloor \frac{P}{2} \rfloor$         | $P$                     | 2               |
| grille 2D      | $P = \sqrt{P} \times \sqrt{P}$ | 2, 4           | $2(\sqrt{P} - 1)$                     | $2P - 2\sqrt{P}$        | $\sqrt{P}$      |
| tore 2D        | $P = \sqrt{P} \times \sqrt{P}$ | 4              | $2\lfloor \frac{\sqrt{P}}{2} \rfloor$ | $2P$                    | $2\sqrt{P}$     |
| hypercube      | $P = 2^d$                      | $d = \log_2 P$ | $d$                                   | $\frac{1}{2}P \log_2 P$ | $P/2$           |

TABLE 3.1 – Caractéristiques des topologies simples avec  $P$  le nombre de nœuds et  $b$  la largeur de bisection.

plus d'information des feuilles en général. Aussi, afin de tenir compte de cette bande passante qui doit décroître avec la hauteur des nœuds de l'arbre, on construit la topologie des arbres élargis (*fat trees*, voir la figure 3.5).

On résume les caractéristiques techniques des principales topologies dans la table 3.1.

## 3.5 La topologie de l'hypercube et le code de Gray

### 3.5.1 Construction récursive de l'hypercube

L'*hypercube* en dimension  $d$ , appelé  $d$ -cube, est une généralisation du carré en 2D et du cube en 3D. On construit récursivement un hypercube de dimension  $d$  à partir de deux hypercubes de dimension  $d - 1$  en reliant les copies des sommets correspondants par paires. La figure 3.6 illustre la construction de ces hypercubes. Il s'en suit qu'un hypercube en dimension  $d$  a  $2^d$  sommets et que chaque sommet a exactement  $d$  arêtes : le degré des nœuds de l'hypercube. L'hypercube a donc une *topologie régulière* car tous les sommets jouent le même rôle dans le réseau.

Comment étiqueter les nœuds de l'hypercube afin d'avoir des algorithmes de routage efficaces et faciles à implémenter ? On pourrait les numéroter arbitrairement mais on aurait alors besoin d'une table de routage pour connaître pour chaque nœud les numéros de ses  $d$  voisins. Cette méthode ne passe pas à l'échelle et demande de la mémoire supplémentaire. On cherche donc plutôt une

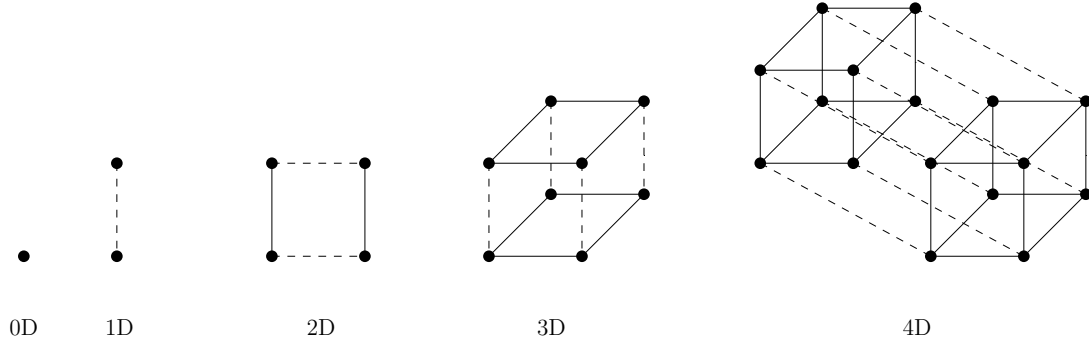


FIGURE 3.6 – Construction récursive de l'hypercube : hypercubes en dimension 0, 1, 2, 3 et 4. Un hypercube  $H_d$  de dimension  $d$  est construit récursivement à partir de deux hypercubes de dimension  $d - 1$  en joignant les sommets semblables (segments en pointillés).

représentation telle que deux nœuds voisins  $P$  et  $Q$  de l'hypercube diffèrent simplement par un bit dans la représentation binaire des numéros des nœuds. Il sera facile par exemple de vérifier que  $P = (0010)_2$  et  $Q = (1010)_2$  sont voisins puisque  $P \text{ XOR } Q = 1000$ . On rappelle la *table de vérité* du *ou exclusif* (*xor*) :

| XOR | 0 | 1 |
|-----|---|---|
| 0   | 0 | 1 |
| 1   | 1 | 0 |

De plus, on voudrait aussi que les  $d$  bits correspondent aux  $d$  axes de l'hypercube : ainsi si  $P$  et  $Q$  diffèrent par un bit (voisin) sur le  $d$ -ième bit, on envoie un message de  $P$  à  $Q$  en utilisant le lien de communication du  $d$ -ième axe. Cet étiquetage spécial des nœuds de l'hypercube est appelé le *code de Gray*.

### 3.5.2 Numérotage des nœuds avec le code de Gray

Le code de Gray  $G(i, x)$  est un *code binaire* qui a la particularité d'assurer que deux nœuds voisins de l'hypercube diffèrent par un bit dans leur représentation. On appelle le code de Gray un *code binaire réfléchi* car le code binaire de taille  $d$  se construit en copiant les mots du code binaire de taille  $d - 1$  précédés d'un 0, suivi des mêmes mots pris dans l'*ordre inverse* et précédés de 1. Historiquement, il a été breveté aux États-Unis en 1953 par Frank Gray (Bell Labs). La définition mathématique du code de Gray s'exprime récursivement par :

$$G_d = \{0G_{d-1}, 1G_{d-1}^{\text{réfléchi}}\}, \quad G_{-1} = \emptyset.$$

Soit  $i$  le rang du mot et  $x$  le nombre de bits du code. Alors nous avons :

$$\begin{aligned} G(0, 1) &= 0, \\ G(1, 1) &= 1, \\ G(i, x+1) &= G(i, x), i < 2^x, \\ G(i, x+1) &= 2^x + G(2^{x+1} - 1 - i, x), i \geq 2^x. \end{aligned}$$

Le code de Gray a de très nombreuses propriétés intéressantes outre le fait que les mots adjacents diffèrent par un bit : par exemple, c'est un code cyclique, et une séquence décroissante équivaut à une séquence croissante lorsque l'on flippe le bit de tête ( $0 \leftrightarrow 1$ ).

| Codage décimal | Codage binaire | Codage Gray (binaire réfléchi) |
|----------------|----------------|--------------------------------|
| 0              | 000            | 000                            |
| 1              | 001            | 001                            |
| 2              | 010            | 011                            |
| 3              | 011            | 010                            |
| 4              | 100            | 110                            |
| 5              | 101            | 111                            |
| 6              | 110            | 101                            |
| 7              | 111            | 100                            |

On décrit ci-dessous les mécanismes pour convertir les nombres de Gray en nombres binaires équivalents, et réciproquement :

- Pour convertir un code binaire  $n$  en le  $n$ -ième mot du code de la séquence de Gray, on procède comme suit : on considère les représentations binaires des nombres  $n$  et  $\lfloor \frac{n}{2} \rfloor$ , et on calcule leur *ou exclusif*, bit à bit. Notons que calculer  $\lfloor \frac{n}{2} \rfloor$  où  $n$  donné en représentation binaire revient simplement à décaler tous les bits à gauche d'un rang. Ceci s'exprime en langage Java/C++/C par l'opérateur<sup>4</sup> `>>` (en anglais, *left shift* pour les *bitwise Boolean operations*) comme l'illustre le code ci-dessous :

```
int n=3;
int ns;
// décale de 1 bit sur la gauche
ns = n>>1;
```

Par exemple, prenons le 4-ième nombre de la séquence binaire :  $n = (011)_2$  (codage décimal : 3). On a,  $\lfloor \frac{n}{2} \rfloor = ((011) \gg 1)_2 = (001)_2 = 1$ , et son équivalent dans la séquence du code de Gray est :  $(011)_2 \text{ XOR } (001)_2 = (010)_2$ . De même, si nous prenons le 7-ième nombre de la séquence binaire :  $n = (110)_2$  (codage décimal : 6). On a,  $(110)_2 \text{ XOR } ((110) \gg 1)_2 = (110)_2 \text{ XOR } (011)_2 = (101)_2$ .

- Pour convertir un code  $(g_{d-1}, \dots, g_0)_2$  de la séquence de Gray en code binaire équivalent, on utilise des opérations de somme préfixe comme suit : Pour le code binaire équivalent  $(b_{d-1}, \dots, b_0)_2$ , on calcule  $b_i$  de la façon suivante : on calcule modulo 2 la somme cumulée des bits de Gray se trouvant à gauche de  $g_i$  :  $\sum_i = (\sum_{j=i+1}^{d-1} g_j) \bmod 2$ . Puis nous faisons,  $b_i = \sum_i \text{ XOR } g_i$ . Par exemple, convertissons le code de Gray  $(101)_2$  en code binaire : On a  $\sum_0 = (\sum_{j=1}^2 g_j) \bmod 2 = (1 + 0) \bmod 2 = 1$  et  $1 \text{ XOR } 1 = 0$ , donc  $b_0 = 0$ . Pour  $b_1$ , on a  $\sum_1 = (\sum_{j=2}^2 g_j) \bmod 2 = 1$  et  $1 \text{ XOR } 0 = 1$ , donc  $b_1 = 1$ , et pour  $b_2$ , on a  $\sum_2 = 0$  et  $0 \text{ XOR } 1 = 1$  donc  $b_2 = 1$ . Ainsi, nous trouvons  $(110)_2$  qui correspond à 6 en codage décimal.

La *distance de Hamming* sur l'hypercube permet de calculer la distance entre deux nœuds du réseau. Soient  $P = (P_{d-1} \dots P_0)_2$  et  $Q = (Q_{d-1} \dots Q_0)_2$  deux sommets de l'hypercube de dimension  $d$ . La distance entre  $P$  et  $Q$  est la longueur du plus court chemin et équivaut à la distance de Hamming sur la représentation binaire de  $P$  et  $Q$  :

4. À ne pas confondre avec les opérations de stream en C++ (comme `cout<<"un message"<<endl;`).

$$\text{Hamming}(P, Q) = \sum_{i=0}^{d-1} 1_{P_i \neq Q_i}$$

Par exemple, nous avons  $\text{Hamming}(1011, 1101) = 2$ . On calcule cette distance de Hamming entre  $P$  et  $Q$  simplement en comptant le nombre de bits à 1 d'un XOR de  $P$  et  $Q$ .

### 3.5.3 Génération d'un code de Gray en C++

On donne ci-dessous une classe en C++ pour générer un code de Gray de dimension  $n$  :

```
class Gray {
public:
    vector<int> code(int n) {
        vector<int> v;
        v.push_back(0);

        for(int i = 0; i < n; i++) {
            int h = 1 << i;
            int len = v.size();

            for(int j = len - 1; j >= 0; j--)
                {v.push_back(h + v[j]);}
        }
        return v;
    }
};
```

On peut alors utiliser cette classe comme le montre cet exemple :

```
#include <iostream>
#include <vector>
#include <bitset>

using namespace std;

int main() {
    Gray g;
    vector<int> a = g.code(4);

    for(int i = 0; i < a.size(); i++)
        {cout << a[i] << "\t";}
    cout << endl;

    for(int i = 0; i < a.size(); i++)
        {cout << (bitset<8>) a[i] << "\t";}
    cout << endl;

    return 0;
}
```

Après compilation, l'exécution donne le résultat suivant à la console :

```
C:\INF442>graycode442
0       1       3       2       6       7       5       4       12      13
15      14      10      11      9       8
00000000      00000001      00000011      00000010      00000110
```

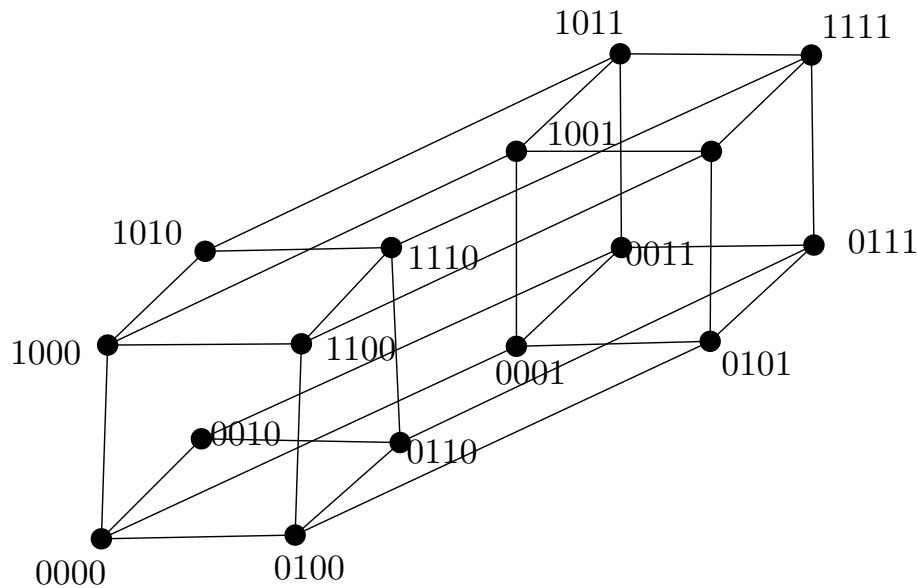


FIGURE 3.7 – Code de Gray étiquetant les nœuds d'un hypercube 4D.

|          |          |          |          |          |
|----------|----------|----------|----------|----------|
| 00000111 | 00000101 | 00000100 | 00001100 | 00001101 |
| 00001111 | 00001110 | 00001010 | 00001011 | 00001001 |
| 00001000 |          |          |          |          |

La figure 3.7 illustre le code de Gray sur un hypercube 4D.

### 3.5.4 Convertir les codes de Gray en codes binaires équivalents, et vice versa

La figure 3.8 illustre schématiquement le principe pour convertir un code de Gray en un code binaire. Tout d'abord, on ajoute un bit supplémentaire au code binaire que l'on fixe à zéro (le *bit de poids fort* ou *Most Significant Bit, MSB*). Le  $i$ -ème bit  $g_i$  du code de Gray est mis à zéro si et seulement si les bits voisins de code binaire correspondant coïncident. Ce test peut être implémenté dans le matériel avec une porte logique : un OU exclusif, ou XOR en anglais. Ainsi quand on converti un code binaire en code de Gray, tous les bits  $g_i$  peuvent être calculés en parallèle (par exemple, sur le modèle CREW PRAM).

Pour convertir un code de Gray en son code équivalent binaire, on commence par le bit de poids fort, et on enchaîne une cascade de conversion de bits jusqu'à aboutir au bit de poids faible (*Least Significant Bit, LSB*)

La figure 3.8 et la figure 3.9 montrent quelques exemples de conversion.

### 3.5.5 \* Produit cartésien de graphes (opérateur noté $\otimes$ )

Le produit cartésien de graphes est noté par l'opérateur binaire  $\otimes$ . Soient  $G_1 = (V_1, E_1)$  et  $G_2 = (V_2, E_2)$  deux graphes connectés. Le produit cartésien  $G = G_1 \otimes G_2 = (V, E)$  est défini

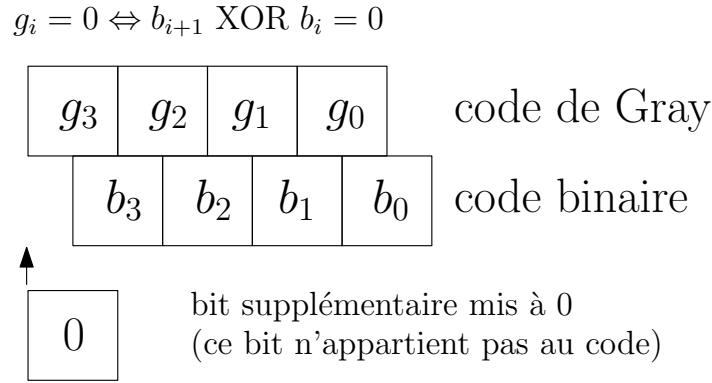


FIGURE 3.8 – Procédure pour convertir un code de Gray en son code binaire équivalent, et vice versa.

comme suit :

- les sommets  $V : V = V_1 \times V_2 = \{(u_1, u_2), u_1 \in V_1, u_2 \in V_2\}$ ,
- les arêtes  $E$  :

$$((u_1, u_2), (v_1, v_2)) \in E \Leftrightarrow \begin{cases} u_1 = v_1 & (u_2, v_2) \in E_2 \\ u_2 = v_2 & (u_1, v_1) \in E_1 \end{cases}$$

La figure 3.10 montre un exemple de produit cartésien de graphes (son opération inverse peut être interprétée comme une factorisation de graphes).

Le produit de deux arêtes est un cycle à 4 sommets :  $K_2 \otimes K_2 = C_4$  ( $K_2$  est la clique à deux nœuds qui est une arête). Le produit de  $K_2$  et d'un chemin d'un graphe est un graphe d'*échelle* (*ladder graph*). Le produit de deux chemins donne une grille, etc.

L'hypercube de dimension  $d$  peut s'obtenir comme  $d$  produits d'une arête :

$$\underbrace{K_2 \otimes \dots \otimes K_2}_{d \text{ fois}} = \text{Hypercube}_d$$

On en déduit facilement aussi la *propriété de clôture* des hypercubes par produit cartésien des graphes induits :

$$\text{Hypercube}_{d_1} \otimes \text{Hypercube}_{d_2} = \text{Hypercube}_{d_1+d_2}.$$

## 3.6 Algorithmes de communication sur les topologies

### 3.6.1 Les communications sur l'anneau

On considère la topologie de l'anneau avec  $P$  nœuds :  $P_0, \dots, P_{P-1}$ . On suppose les liens directionnels et l'anneau orienté<sup>5</sup>. On rappelle en MPI les deux fonctions de base : `Comm_size()` qui retourne la valeur de  $P$  et `Comm_rank()` qui donne le numéro d'ordre des nœuds, indexés entre 0 et  $P-1$ .

On suppose le mode de calcul SPMD (*Single Program Multiple Data*) sur l'anneau : tous les processeurs exécutent le même code et les calculs se font dans l'espace mémoire local. On se donne

5. Lorsque l'on dessine l'anneau, on peut supposer la direction dans le sens des aiguilles d'une montre.

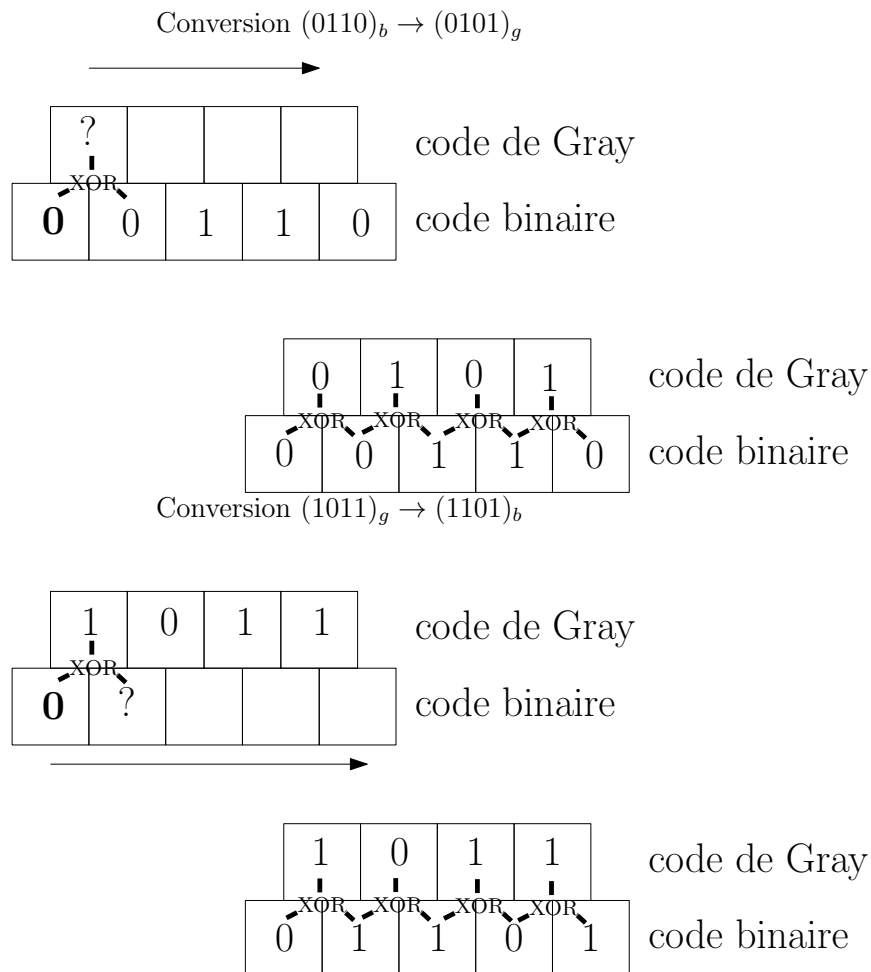


FIGURE 3.9 – Deux exemples de conversion de code de Gray  $\leftrightarrow$  code binaire : conversion du code binaire  $(0110)_b$  en son code équivalent de Gray  $(0101)_g$ , et conversion du code de Gray  $(1011)_g$  en son code binaire équivalent  $(1101)_b$ .

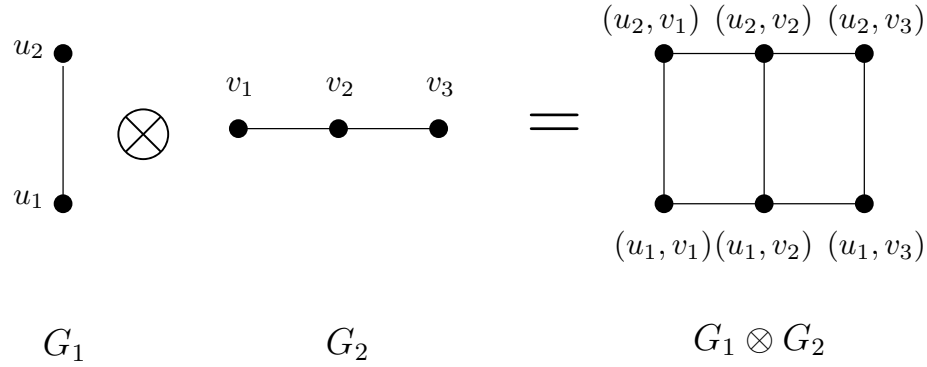


FIGURE 3.10 – Exemple d'un produit cartésien de graphes.

les deux opérations de communication élémentaires **send** et **receive** qui permettent aux processus  $P_i$  de communiquer comme suit :

- **send(adresse, longueur)** : envoi d'un message à  $\boxed{P_{(i+1) \bmod P}}$  commençant à l'espace mémoire local de  $P_i$  **adresse** et de longueur **longueur** (en octets)
- **receive (adresse, longueur)** : réception d'un message de  $\boxed{P_{(i-1) \bmod P}}$  et rangement dans l'espace mémoire locale de  $P_i$  à l'adresse **adresse**

Par rapport à l'interface MPI<sup>6</sup>, on ne considère pas d'attributs de *tags* ici dans les messages (tous les nœuds appartiennent au même groupe de communication). Une condition nécessaire pour qu'un programme soit correct est qu'à chaque opération **send** corresponde une opération **receive**.

Les opérations de communication **send** et **receive** sont bloquantes et on peut donc arriver à des situations de *blocage* (*deadlock*). On pourra également considérer le scénario d'un **Isend()** non-bloquant et d'un **receive()** bloquant, ou encore le scénario d'un **Isend()** non-bloquant et d'un **Ireceive()** non-bloquant.

Afin de mesurer le coût des communications, on note  $l$  la longueur du message. Le coût d'un envoi ou d'une réception est modélisé par la fonction linéaire  $\alpha + \tau l$ , avec

- $\alpha$  : le coût d'initialisation incompressible qui donne lieu à la latence, et
- $\tau$  : le *débit* qui caractérise la *vitesse de transmission* en régime stable.

Envoyer ou recevoir un message de taille  $l$  à une distance  $d \leq P - 1$  d'un processus courant coûte donc naïvement  $d(\alpha + \tau l)$ .

Les quatres opérations basiques de communications globales sur l'anneau sont :

- la *diffusion* (*broadcast*),
- la *diffusion personnalisée* (*scatter*),
- le *rassemblement* (*gather*),
- le *comméragé* ou *échange total* (*all-to-all, total exchange*).

Dans le standard MPI, notons qu'il existe encore de nombreux autres modes de communications entre les processus comme l'*échange total personnalisé*, etc.

6. La syntaxe en MPI est bien plus riche. Voir <http://www.mcs.anl.gov/research/projects/mpi/sendmode.html>

### La diffusion sur l’anneau

Sans perte de généralité, on suppose que  $P_0$  envoie un message de longueur  $l$  à tous les autres processus  $P_1, \dots, P_{P-1}$  de l’anneau orienté, étape après étape, comme l’illustre la figure 3.11.

Cette diffusion demande donc  $P - 1$  étapes, avec le message acheminé séquentiellement à  $P_1, \dots, P_{P-1}$ . En effet, pour que le message parcoure les nœuds de  $P_a$  à  $P_b$ , on a besoin de  $b - a + 1$  étapes. Le coût de la diffusion est donc  $(P - 1)(\alpha + \tau l)$ . On remarque ici l’importance relative des paramètres  $\alpha$  et  $\tau$  du réseau vis-à-vis de la longueur du message.

Une implémentation en communications bloquantes de cet algorithme est donné dans l’algorithme 1.

**Données :** adresseM : adresse mémoire du message.  $l$  : longueur du message  
**Résultat :** Diffusion d’un message à partir du processus  $P_k$  à  $P_0, \dots, P_{P-1}$

```
// Nombre de processus
P = Comm_size();
// Rang du processus courant
r = Comm_rank();
si  $r = k$  alors
| // Je suis  $P_k$ , j’envoie le message
| send(adresseM,l);
sinon
| si  $r = (k - 1 \bmod P)$  alors
| | // Dernier processus : on reçoit
| | receive(adresseM,l);
| sinon
| | // Proc. au milieu, on reçoit puis on renvoie
| | receive(adresseM,l);
| | send(adresseM,l);
| fin
fin
```

**Algorithme 1 :** Algorithme de diffusion (*broadcast*) sur l’anneau orienté par communications bloquantes.

On a utilisé la fameuse technique de *round robin* (étymologiquement provenant du français “ruban rond”!).

Voici le code équivalent complet en MPI pour transmettre un message sur l’anneau entre deux nœuds :

Code 15 – Communication sur l’anneau orienté en MPI : `communicationanneau442.cpp`

```
// Nom du programme : communicationanneau442.cpp
// Compilez avec mpic++ communicationanneau442.cpp -o communicationanneau.exe
// Exécutez avec mpirun -np 8 communicationanneau.exe

#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, nprocs;
    int message=-1;
```

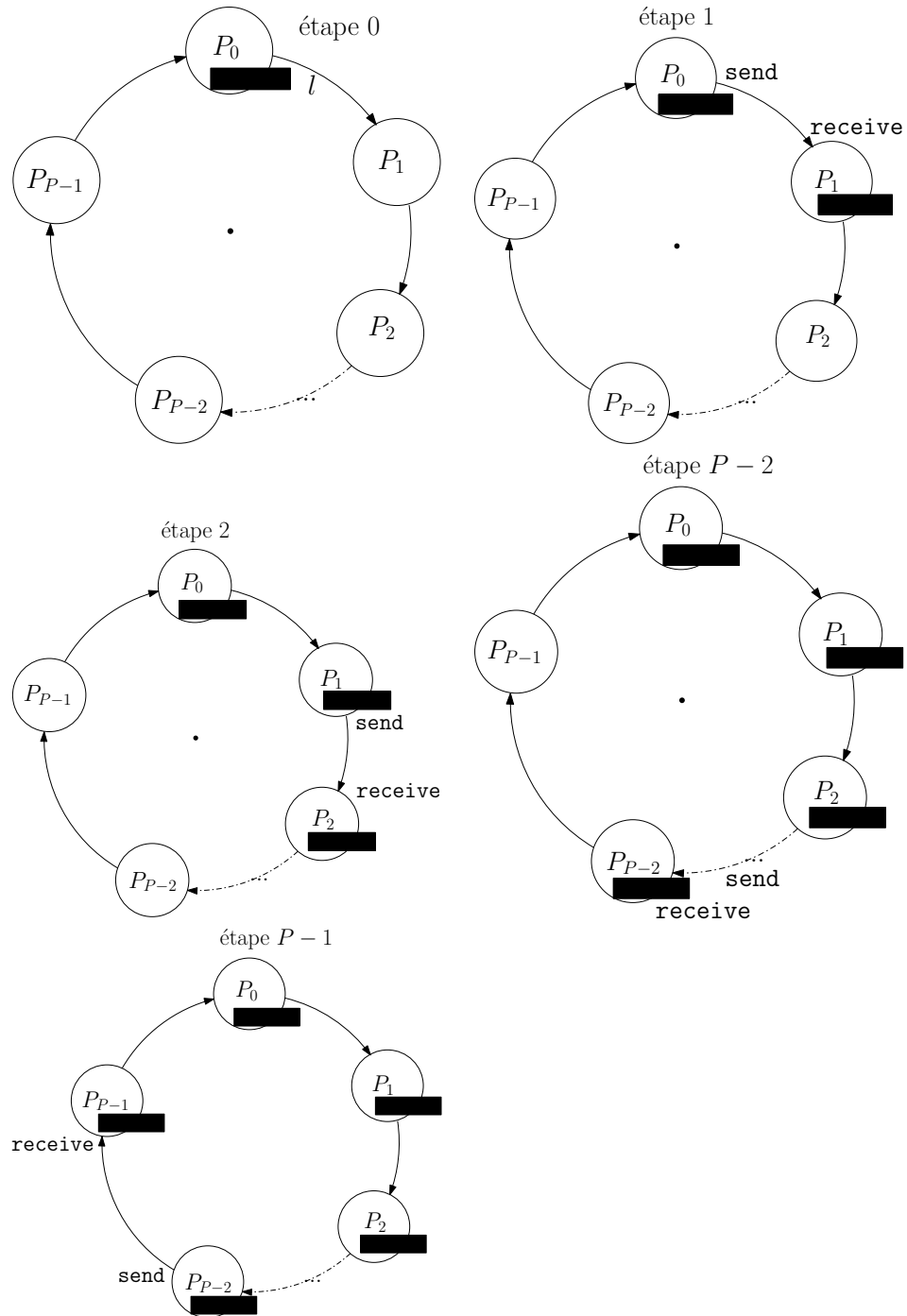


FIGURE 3.11 – Illustration de la diffusion sur l’anneau.

```

int Pstart=2, Pend=6;

MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &nprocs);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);

if (rank==0)
    {printf("Envoi sur l'anneau d'un message de P%d a P%d",Pstart,Pend);}

// On simule avec des communications bloquantes par lien,
// l'envoi d'une message de Pstart a Pend sur l'anneau orienté
if (rank==Pstart) {
    // je suis l'envoyeur, je prépare mon message et je l'envoie
    message=442;
    MPI_Send(&message, 1, MPI_INT, Pstart+1, 0, MPI_COMM_WORLD);
} else
{
    if (rank==Pend)
    { // Je suis le destinataire, et je reçois seulement
        MPI_Recv(&message, 1, MPI_INT, Pend-1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE
        );
    }
    else if ((rank>Pstart)&&(rank<Pend))
    { // Je suis un nœud intermédiaire de la topologie de l'anneau
        MPI_Recv(&message, 1, MPI_INT, rank-1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE
        );
        printf("Je suis P%d. Recu le message de %d et le renvoie a %d\n", rank,
            rank-1,rank+1);
        MPI_Send(&message, 1, MPI_INT, rank+1, 0, MPI_COMM_WORLD);
    }
    else
    {printf("%d: Je ne fais rien moi !\n",rank);}
}

printf("Proc. %d message=%d\n", rank, message);

MPI_Finalize();
return 0;
}

```

```

mpirun -np 8 communicationanneau.exe
Envoi sur l'anneau d'un message de P2 a P60: Je ne fais rien moi !
Proc. 0 message=-1
Proc. 2 message=442
Je suis P3. Recu le message de 2 et le renvoie a 4
Proc. 3 message=442
1: Je ne fais rien moi !
Proc. 1 message=-1
Je suis P4. Recu le message de 3 et le renvoie a 5
Proc. 4 message=442
Je suis P5. Recu le message de 4 et le renvoie a 6
Proc. 5 message=442
7: Je ne fais rien moi !

```

```
Proc. 7 message=-1
Proc. 6 message=442
```

### La diffusion personnalisée (*scatter*)

La *diffusion personnalisée* ou *scatter* en anglais consiste à envoyer depuis un nœud un message personnalisé à chaque autre nœud de l'anneau. On considère que c'est  $P_0$  le nœud qui fait la demande d'une diffusion personnalisée. Le message  $M_i$  pour le nœud  $P_i$  se trouve à l'adresse mémoire locale `adresse[i]` de  $P_0$ . On utilise les procédures `Isend()` non-bloquantes et `receive()` bloquantes. On a choisi le `receive()` bloquant afin de s'assurer que l'on reçoive les messages dans le bon ordre, sans croisement. Nous allons procéder par une technique de *recouvrement* des différentes communications ! La figure 3.12 illustre la diffusion personnalisée.

Une implémentation de la primitive de communication `scatter` sur l'anneau est donnée ci-dessous :

```
// Diffusion personnalisée sur l'anneau :
// - émetteur initial : processus  $P_k$ 
// - longueur du message  $l$ 
// - messages stockés dans le tableau 'adresse'

scatter(k, adresse, l)
{
    q = Comm_rank();
    p = Comm_size();

    if (q == k)
    {
        // je suis  $P_k$ , l'initiateur !
        // j'envoie en communication non-bloquante
        // les messages personnalisés

        for (i=1; i<p; i=i+1)
            {Isend(adresse[k-i mod p], i);}
    }
    else
    {
        receive(adresse, l);

        for (i=1; i<k-q mod p; i = i+1)
            {Isend(adresse, l);
             receive(temp, l);
             adresse = temp; }
    }
}
```

On voit que le coût d'une diffusion personnalisée a une complexité en  $(P-1)(\alpha + \tau l)$ , et est donc identique au coût d'une diffusion non personnalisée grâce à la technique de recouvrement.

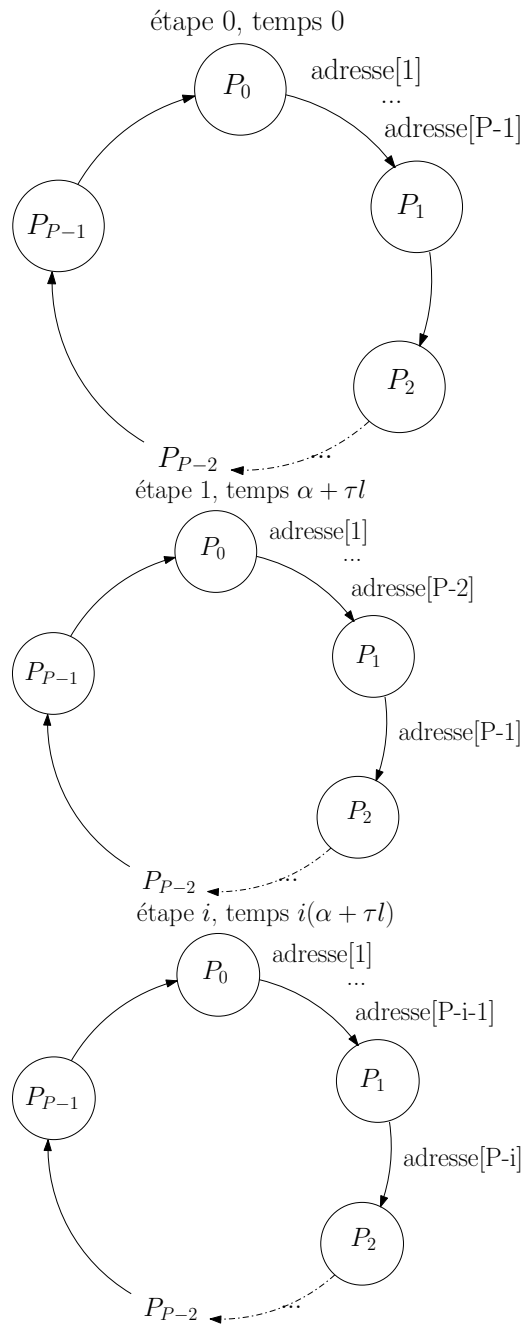


FIGURE 3.12 – Illustration de la diffusion personnalisée sur l’anneau (étapes successives de haut en bas, et de gauche à droite).

### L'échange total : le commérage

L'échange total encore appelé le commérage consiste à ce que chaque processus  $P_i$  envoie un message à tous les autres processus  $P_j$ . Initialement, chaque  $P_i$  a son message  $M_{i,j}$  à l'adresse `monAdresse` à envoyer à tous les autres  $P_j$ . À la fin de l'algorithme du commérage, tous les processus ont un tableau `adresse []` tel que `adresse[j]` contient le message envoyé par le processus  $P_j$ .

Une implémentation de `all-to-all` sur l'anneau est décrite ci-dessous :

```
// Le commérage ou échange total
all-to-all(monAdresse, adr, l)
{
    q = Comm_rank();
    p = Comm_size();

    adresse[q] = monAdresse;

    for (i=1; i<p; i++) {
        send(adresse[q-i+1 mod p], l);
        Ireceive(adresse[q-i mod p], l);
    }
}
```

Le coût d'une communication de commérage sur l'anneau est donc  $(P-1)(\alpha + \tau l)$ . Ça a la même complexité que pour l'échange total personnalisé.

### La diffusion pipelinée pour optimiser les coûts de communication

Puisque le temps de nos algorithmes pour une diffusion simple et pour une diffusion personnalisée (en utilisant la stratégie du pipeline) est le même en  $(P-1)(\alpha + \tau l)$ , on peut améliorer le temps de la diffusion simple comme suit :

- on découpe le message  $M$  en  $r$  morceaux (on suppose  $l \bmod r = 0$ , c'est-à-dire que  $r$  divise  $l$ ),
- le processus émetteur envoie successivement les  $r$  morceaux afin de pouvoir recouvrir partiellement les temps de communication.

Soient `adresse[0], ... adresse[r-1]` les adresses des  $r$  morceaux du message. Une implémentation de la *diffusion pipelinée* sur l'anneau est donnée par l'algorithme ci-dessous :

```
broadcast(k, adresse, l)
{
    q = Comm_rank();
    p = Comm_size();

    if (q == k)
    {for (i=0; i<r; i++) send(adresse[i], l/r);}
      else
    {if (q == k-1 mod p)
      {
        for (i=0; i<r; i++) {Ireceive(adresse[i], l/r);}
      }
    }
}
```

```

    }
    else {
Ireceive(adresse[0], 1/r);

for (i=0; i<r-1; i++) {
    send(adresse[i], 1/r);
    receive(adresse[i+1], 1/r);
}
}
}

```

Considérons le coût de cette diffusion pipelinée : Le premier morceau du message,  $M_0$ , arrive au dernier processus de la chaîne  $P_{P-1}$  en temps  $(P-1)(\alpha + \tau \frac{l}{r})$ , puis les  $r-1$  morceaux suivants arrivent les uns après les autres. On rajoute donc un temps de  $(r-1)(\alpha + \tau \frac{l}{r})$ , et on obtient ainsi un coût global de  $(P-2+r)(\alpha + \tau \frac{l}{r})$ .

Afin d'obtenir le meilleurs temps, il faut choisir la taille  $r$  des morceaux. Un petit calcul de  $\min_r f(r) = (P-2+r)(\alpha + \tau \frac{l}{r})$  donne la solution optimale  $r^*$  des longueurs des morceaux :

$$r^* = \sqrt{\frac{l(P-2)\tau}{\alpha}}.$$

On aboutit ainsi au temps total de la diffusion pipelinée :

$$\left(\sqrt{(P-2)\alpha} + \sqrt{\tau l}\right)^2.$$

On note que quand la longueur  $l$  des messages devient grande (asymptotiquement quand  $l \rightarrow \infty$ ), le coût de cette diffusion pipelinée devient  $\tau l$ , et est donc indépendant de  $P$  car les termes en  $P$  deviennent négligeables (pour  $P$  fixé).

Nous avons présenté les algorithmes fondamentaux sur l'anneau à titre d'exemple. Il est intéressant de considérer les autres topologies et de voir comment celles-ci influent sur les algorithmes de communication. Par exemple, que se passe-t-il si on choisit la topologie de l'*étoile* au lieu de l'anneau pour les opérations de communication ? La distance maximale est de 2 et on doit considérer des *opérations bufferisées* ou alors gérer les *débordements (overflow)* des buffers mémoires par un gestionnaire de routage. On peut aussi rajouter des cordes sur l'anneau bidirectionnel (non-orienté) et considérer un débit différent sur les liens  $(\tau, \tau')$  suivant les directions de communication, etc.

Nous allons voir maintenant succinctement un algorithme de diffusion simple sur l'hypercube qui montre les communications arborescentes sur cette topologie.

### 3.6.2 Un algorithme de diffusion sur l'hypercube : communications arborescentes

Entre deux nœuds  $P$  et  $Q$ , il existe exactement  $\text{Hamming}(P, Q)!$  chemins deux à deux disjoints. Par exemple, on a  $\text{Hamming}(00, 11) = 2! = 2$  et il existe deux chemins distincts,  $00 \rightarrow 10 \rightarrow 11$  et  $00 \rightarrow 01 \rightarrow 11$  :

```

00  ↔  01
↑↓      ↑↓
10  ↔  11

```

Considérons la diffusion : l'envoi d'un message d'un processeur racine à tous les autres processeurs. On pourrait diffuser sur tous les liens du nœud racine au niveau 0, puis au niveau 1, tous les processeurs diffuseraient sur leurs liens, etc. Cela donnerait un algorithme inefficace car on aurait une redondance des messages transitant sur les liens !

Un meilleur algorithme de routage sur l'hypercube consiste à partir des *poids faibles* (souvent la convention prise<sup>7</sup>), à acheminer le message en transformant la représentation binaire de  $P$  en  $Q$  en flipant le bit (c'est-à-dire, en faisant une communication sur le lien) aux positions des 1 du  $P \text{ XOR } Q$ .

Par exemple, pour communiquer de  $P = (1011)_2$  à  $Q = (1101)_2$ , on calcule d'abord  $P \text{ XOR } Q = (0110)_2$ . On en déduit que  $P$  envoie le message à  $P' = (1001)_2$  sur le lien 1 puis  $P'$  envoie le message à  $P'' = (1101)_2$  sur le lien 2.

Considérons maintenant un *algorithme de diffusion* sur l'hypercube. À partir du nœud émetteur  $P_0 = (0...0)_2$  que l'on renomme en  $(10...0)_2$  en rajoutant un bit supplémentaire à 1 en tête, on procède comme suit : les processeurs reçoivent le message sur le lien correspondant à leur premier 1 et propagent le message sur les liens qui précèdent ce premier 1. Cela demande  $d = \log_2 P$  étapes, où  $P$  est le nombre de processeurs. On peut aussi suivre l'ordre partant du bit de *poids faible* et allant jusqu'au bit de *poids fort*. À l'étape  $i$ ,  $2^{i-1}$  nœuds reçoivent le message (pour  $i \in \{1, \dots, d\}$  où  $d$  est la dimension de l'hypercube).

La figure 3.13 illustre ces différentes étapes de communication sur l'hypercube de dimension 4. Il existe de meilleurs algorithmes que nous ne présentons pas dans ce cours par manque de temps.

L'arbre de diffusion obtenu est appelé *arbre binomial* recouvrant de l'hypercube. La figure 3.14 illustre un tel arbre binomial.

L'hypercube est une topologie très populaire car c'est une topologie régulière qui passe à l'échelle (extensible), et permet de plus de réaliser la topologie de l'anneau et la topologie d'un réseau torique (de taille  $2^r \times 2^s$  dans un  $(d = r + s)$ -cube, en utilisant la numérotation  $(\text{Gray}_r, \text{Gray}_s)$ ).

### 3.7 Plongements de topologies dans d'autres topologies

En introduction, nous avons expliqué la différence entre le réseau physique qui repose sur le matériel et le réseau logique qui est considéré par les algorithmes parallèles. Lorsque le réseau logique coïncide avec le réseau physique, nous obtenons une bonne performance, sinon on doit émuler ce réseau logique sur le réseau physique par un mécanisme de *transposition* qui met en correspondance les nœuds physiques/logiques. La transposition est encore appelée un *plongement*. Les paramètres à optimiser sont la *dilatation* qui se définit comme la distance maximale dans le réseau physique entre deux nœuds voisins du réseau logique, et l'*expansion* qui s'exprime par :

$$\text{expansion} = \frac{\#\text{nœuds du réseau physique}}{\#\text{nœuds du réseau logique}}.$$

Une bonne transposition cherche à avoir une dilatation de 1 et une expansion de 1 car on veut éviter la perte de performance des communications lorsque la dilatation est supérieure à 1 et la perte de performance en calcul lorsque l'expansion est inférieure à 1 (le cas de plusieurs nœuds logiques mappés sur un même nœud physique).

7. On peut aussi considérer l'ordre des bits par les *poids forts*. En anglais, poids faible set dit *Least Significant Bit* (LSB) et poids fort *Most Significant Bit* (MSB)

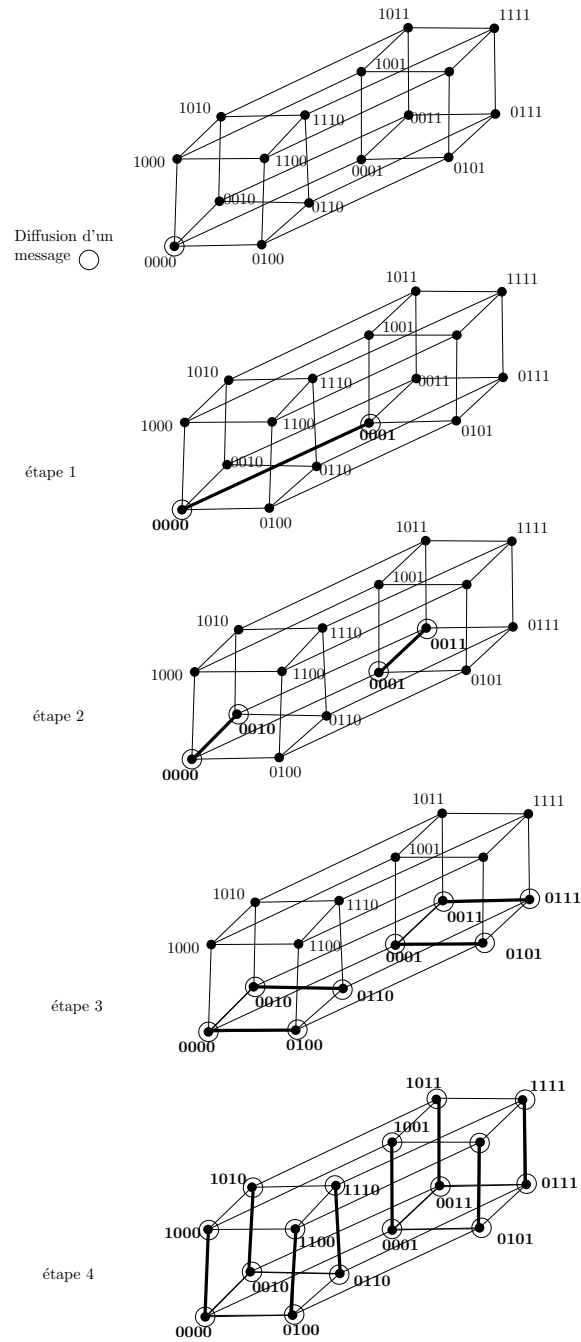


FIGURE 3.13 – Les étapes de l’algorithme de diffusion sur l’hypercube 4D : message partant du nœud (0000) et se propageant du bit de poids faible au bit de poids fort. À l’étape  $i$ ,  $2^{i-1}$  nœud(s) reçoivent le message, pour  $i \in \{1, \dots, 4\}$ .

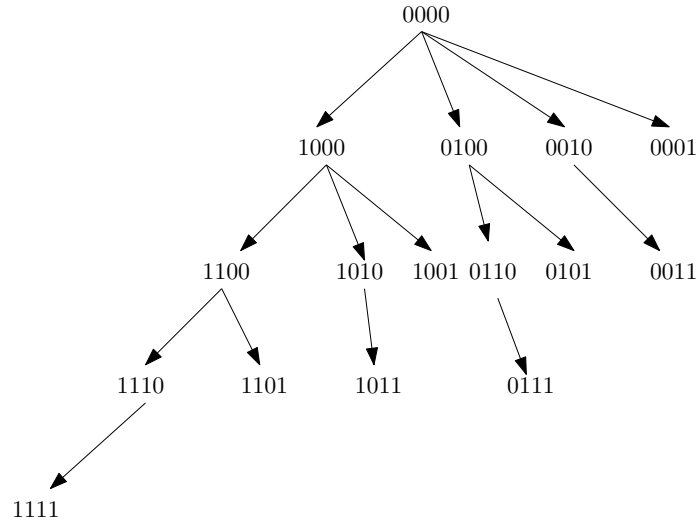


FIGURE 3.14 – Arbre binomial recouvrant de l’hypercube (ici, de dimension 4 à 16 nœuds). Le nombre de bits à 1 est constant par niveau.

Il est facile de transposer l’anneau  $n = 2^d$  sur l’hypercube  $H_d = \{0, 1\}^d$  comme le démontre la figure 3.15. Toutefois, ce schéma n’est pas optimal car on note une dilatation importante pour relier deux nœuds voisins particuliers de l’anneau qui correspond au degré de l’hypercube (ici,  $d = 3$ ).

Par contre, on peut obtenir une transposition optimale avec une dilatation et une expansion de 1 en mappant le nœud  $A_i$  de l’anneau au processeur  $H_{G(i,d)}$  de l’hypercube, où  $G(i,d)$  est le  $i$ -ème mot du code de Gray à  $d$  bits. On forme ainsi un cycle réalisant l’anneau sur l’hypercube en utilisant la propriété cyclique du code de Gray. La figure 3.16 illustre cette transposition optimale.

On peut également transposer de façon optimale la grille  $2D$  (degré 2 ou 4) et les arbres binaires sur l’hypercube.

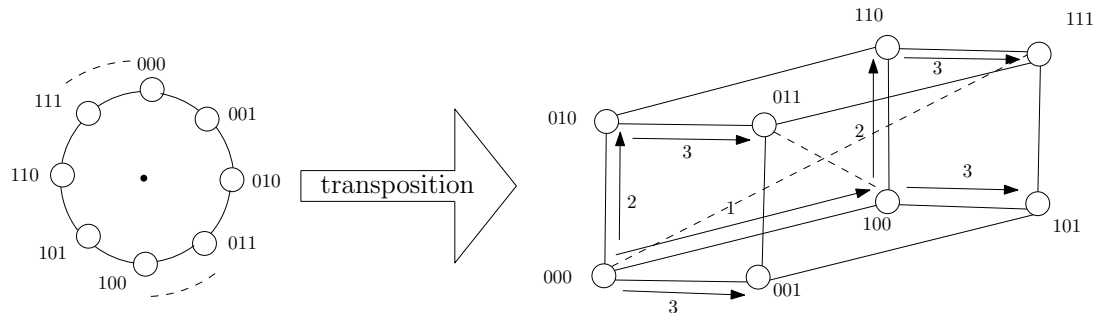


FIGURE 3.15 – Un exemple d’une transposition de l’anneau à 8 nœuds sur l’hypercube (le cube en dimension 3). Les arêtes logiques de l’anneau en pointillées requièrent trois liens physiques sur l’hypercube (diamètre, la dilatation est égale à trois pour une expansion de un).

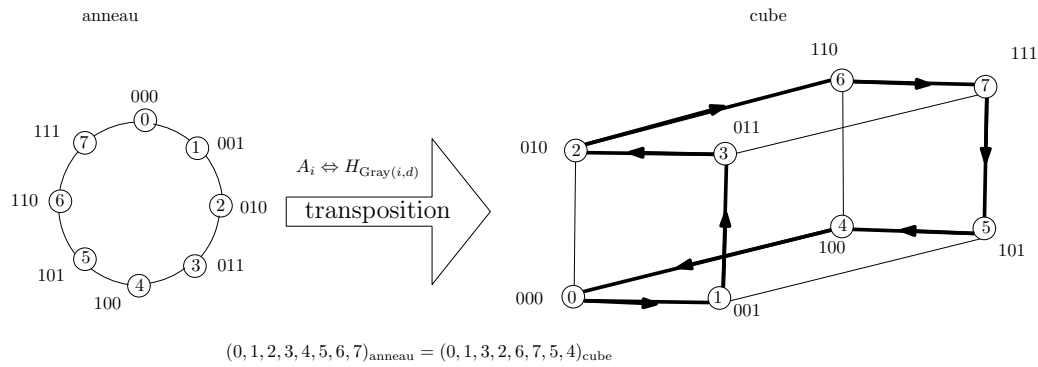


FIGURE 3.16 – Transposition optimale du réseau logique de l’anneau sur le réseau physique de l’hypercube en associant au nœud  $A_i$  de l’anneau le nœud  $H_{G(i,d)}$  de l’hypercube.

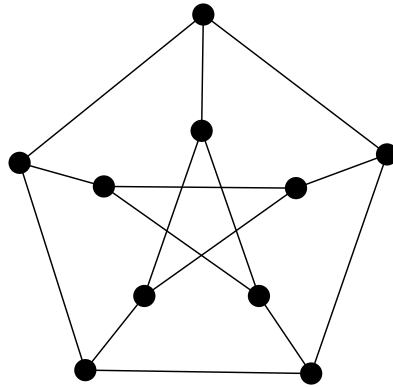


FIGURE 3.17 – Un exemple d’une topologie régulière complexe : Le graphe de Petersen (avec  $d = 3$ ,  $D = 2$  et  $n = 10$ ).

### 3.8 \* Topologies régulières complexes

Une *topologie régulière*<sup>8</sup> est une topologie où chaque sommet joue indifféremment le même rôle. Soit  $N(d, D) = p$  le nombre maximum de nœuds dans un graphe régulier de degré  $d$  et de diamètre  $D$ . On a  $N(d, D) = p$  pour :

- l’anneau avec  $d = 2$  et  $D = \lfloor \frac{p}{2} \rfloor$ ,
- le graphe complet avec  $d = p - 1$  et  $D = 1$ ,
- l’hypercube avec  $d = \log_2 p$  et  $D = \log_2 p$ .

La figure 3.17 montre une topologie régulière plus complexe : le graphe de Petersen avec  $d = 3$  et  $D = 2$ . On peut aussi obtenir des topologies régulières complexes en utilisant le produit cartésien de graphes réguliers : par exemple,  $K_3 \otimes C_5$  (où  $C_5$  est l’anneau à 5 nœuds, un cycle), etc.

D’une manière générale, on a les inégalités de Moore qui donnent des bornes supérieures sur

8. Par analogie aux cinq solides platoniques.

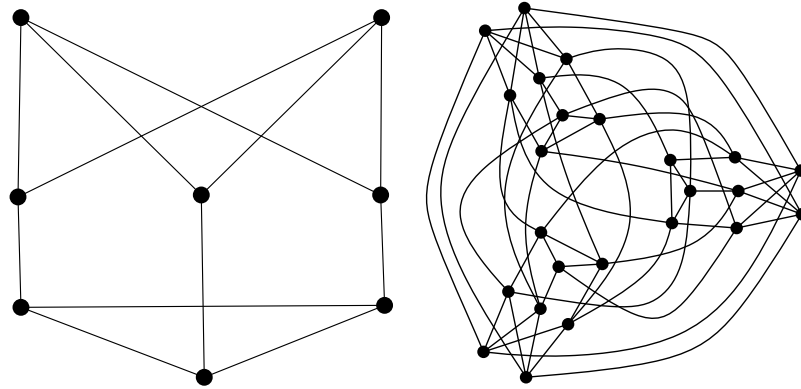


FIGURE 3.18 – Topologie régulière de  $K3 \otimes X8$  ( $X8$  est le graphe dessiné à gauche) :  $p = 24$  sommets de degré  $d = 5$  réalisant un diamètre  $D = 2$ .

le nombre maximum de sommets d'une topologie régulière dont les degrés des sommets sont tous égaux à  $d$  pour un diamètre borné par  $D$  :

- $N(2, D) \leq 2D + 1$
- $N(d, D) \leq \frac{d(d-1)^D - 2}{d-2}, d > 2$
- $N(16, 10) = 12, 951, 451, 931$

Pour le graphe de Petersen, on voit que la borne supérieure de Moore  $N(3, 2)$  donne  $\frac{3 \times 2^2 - 2}{1} = 10$ . Elle est donc optimale puisque le graphe de Petersen a 10 nœuds. En général, les bornes de Moore ne sont pas toutes optimales. Par exemple, le produit cartésien du graphe  $X8$  et de la clique  $K3$  donne lieu à une topologie régulière à  $p = 24$  nœuds avec un degré  $d = 5$  et un diamètre  $D = 2$  (voir la figure 3.18). La borne supérieure de Moore obtenue est égale à 26, et n'est donc pas optimale. Trouver de meilleures bornes est un problème actuel de recherche en soi : pour les curieux, voir cet état de l'art [67].

### 3.9 \* Réseaux d'interconnexions sur la puce

Les processeurs modernes sont multi-cœurs (souvent de 4 à 8 cœurs), voire massivement multi-cœurs pour des architectures dédiées. Par exemple, le Intel Xeon® Phi<sup>9</sup> est un processeur  $x86$  qui réalise une performance de 3 TFlops avec 72 cœurs (*cores*). La figure 3.19 montre la complexité de cette prouesse technologique. Cette puce (*chip*) fabriquée avec un gravage lithographique à 14 nm permet de construire des super-calculateurs en les assemblant modulairement. Actuellement, on vise à obtenir des super-ordinateurs qui calculent de l'ordre de l'exaflops.

Afin de minimiser la *latence* lors des calculs qui doivent accéder à la mémoire, on crée une hiérarchie de différents types de mémoire : registres, caches<sup>10</sup>  $L_1, L_2, L_3$ , *mémoire vive dynamique* (*DRAMs*), etc. Un accès à la DRAM coûte  $\times 100$  *cycles d'horloge* (*clock cycle*)! On a aussi besoin

9. [http://en.wikipedia.org/wiki/Xeon\\_Phi](http://en.wikipedia.org/wiki/Xeon_Phi)

10. Les CPUs utilisent des caches pour réduire le temps moyen d'accès des données de la mémoire principale. Un cache est une petite mémoire rapide proche du CPU qui recopie les données fréquemment utilisées de la mémoire principale. Les caches sont eux-mêmes organisés en une hiérarchie de caches :  $L_1, L_2, L_3$ , etc ( $L$  pour *Layer*). Voir [http://en.wikipedia.org/wiki/CPU\\_cache](http://en.wikipedia.org/wiki/CPU_cache)

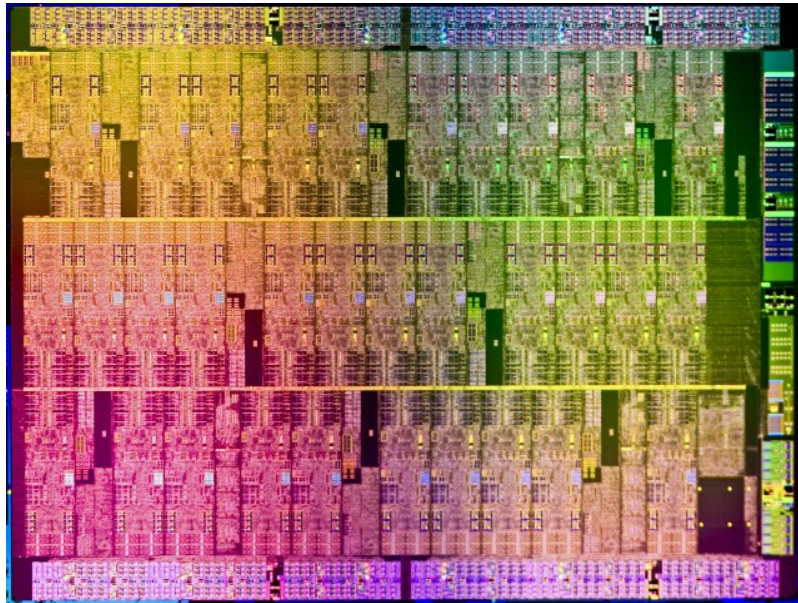


FIGURE 3.19 – Le processeur Intel Xeon® Phi à 72 cœurs. Image provenant du site Web <http://www.extremetech.com/>

de réseaux d'interconnexions pour communiquer entre les cœurs de la puce (*on-chip interconnection networks*). On est passé du design *fonction centrique* au design *communication centrique*.

Dans le cas d'une architecture d'interconnexion par *bus partagé*, illustrée à la figure 3.20, un seul nœud à la fois peut utiliser le bus pour envoyer un message sinon on a une *collision* lorsqu'au moins deux processeurs veulent envoyer en même temps un message. Par contre, lorsque les messages sont émis, ils sont diffusés partout car écoute qui veut (*concurrent read*, CR)! La diffusion est donc très efficace sur le bus. Pour contre-carrer ces contentions, on peut se servir d'un *jeton* qui garantit l'absence de collisions car on respecte le protocole qu'il faut avoir l'unique jeton pour pouvoir avoir le droit d'envoyer un message.

Le *commutateur* ou *switch* requiert un temps initial pour sa création mais après garantit de ne plus avoir de collisions (ni donc d'arbitrage à faire). Plusieurs transferts en cours sont possibles si on utilise des liens du switch mutuellement exclusifs.

Le routage peut se faire par *circuit switching* ou bien par *packet switching*. Dans le cas du circuit switching, on réserve d'abord les liens pour une connexion entre la source et la destination (comme le réseau téléphonique). Pour le routage par packet switching, chaque paquet est acheminé (*routed*) séparément. Les liens sont utilisés seulement quand les données sont transférées (comme le protocole Internet).

Un *commutateur* (*crossbar*) permet de faire communiquer toute paire de nœuds en une passe avec peu de délai (figure 3.21). L'inconvénient du crossbar est sa complexité en  $O(P^2)$  switches. La figure 3.22 montre un réseau crossbar.

Afin de passer à l'échelle, on peut utiliser le *réseau oméga* qui est constitué de  $\frac{P}{2} \log P$  switches, des petits crossbars de taille  $2 \times 2$ , organisé en  $\log P$  étages. Cela demande moins de complexité par lien qu'un réseau crossbar mais on obtient un délai plus long en acheminement, en  $O(\log P)$ . La

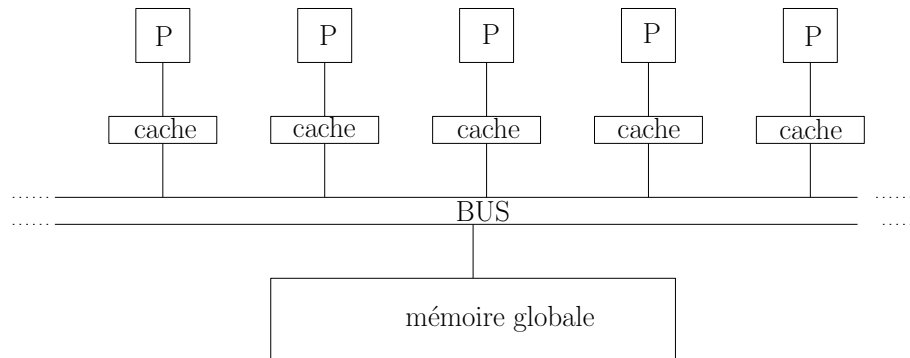


FIGURE 3.20 – Communication avec un bus partagé et contention : collisions possibles quand deux processeurs veulent envoyer en même temps un message.

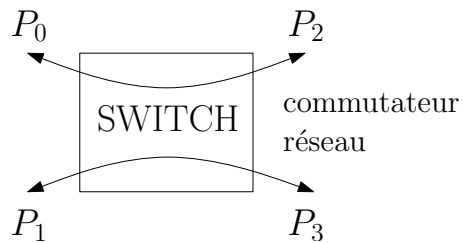


FIGURE 3.21 – Illustration d'un commutateur (*crossbar*  $4 \times 4$  initialisé pour des communications entre les processeurs  $P_1$  et  $P_3$ , et  $P_2$  et  $P_4$ ).

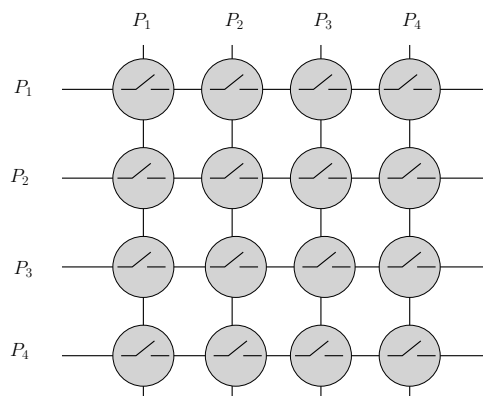


FIGURE 3.22 – Réseau crossbar.

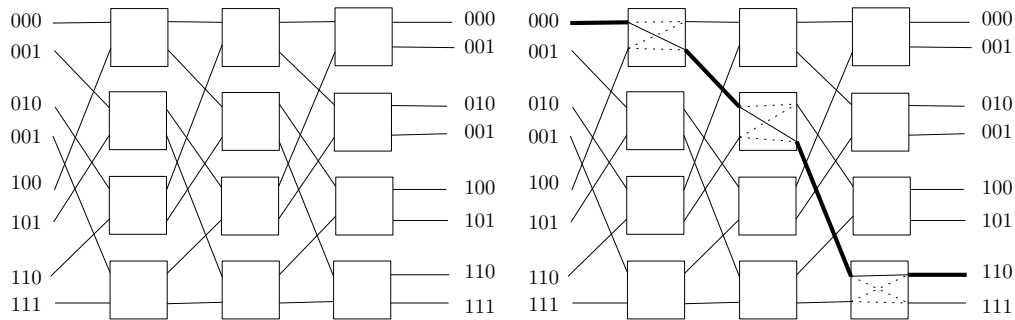


FIGURE 3.23 – Réseau dynamique multi-étage oméga : pour communiquer du processeur 000 au processeur 110, les messages commutent à travers des switchs  $2 \times 2$ .

figure 3.23 illustre le *réseau oméga* et montre un exemple de routage pour communiquer un message entre le processeur 000 et le processeur 110. L'algorithme de routage est simple mais le réseau est bloquant (car on ne peut pas acheminer plusieurs messages en même temps sans collision).

### 3.10 \* Pour en savoir plus : notes, références et discussion

L'ouvrage [60] donne une présentation des différentes topologies des réseaux d'interconnexion ainsi que de leurs applications dans certains algorithmes parallèles. Borner le nombre maximal de nœuds d'une topologie régulière de degré  $d$  et diamètre  $D$  est un problème de recherche encore ouvert [67].

### 3.11 En résumé : ce qu'il faut retenir !

On modélise le réseau d'interconnexion des processeurs à mémoire distribuée par un graphe dont les sommets sont les nœuds des machines et les arêtes les liens de communication entre ces nœuds. On distingue le réseau physique du réseau logique qui lui est utilisé par l'algorithme parallèle pour ses opérations de communication. Lorsque le réseau logique diffère du réseau physique, on effectue une transposition du réseau logique sur le réseau physique qui cherche à minimiser la dilatation (la distance maximale sur le réseau physique de nœuds qui sont voisins dans le réseau logique) et l'expansion (le rapport du nombre de nœuds du réseau physique sur le nombre de nœuds du réseau logique). La topologie des réseaux étudie les propriétés des familles de graphes génériques qui dépendent du nombre de nœuds. Les topologies usuelles de base sont l'anneau, l'étoile, la grille, le tore, les arbres, et l'hypercube. Dans une topologie régulière, tous les sommets jouent le même rôle et cela facilite en pratique l'implémentation des algorithmes parallèles. L'hypercube est une topologie souvent utilisée car elle est régulière de degré  $d$  avec  $2^d$  nœuds, et elle permet d'implémenter les primitives de communication comme la diffusion aisément en utilisant le code de Gray. L'hypercube permet aussi de simuler d'autres topologies par transposition comme les anneaux, les arbres, les grilles, etc.

### 3.12 Exercices

#### Exercice 6 : Communication sur l'hypercube

Dans ce problème, vous pouvez faire référence aux fonctions suivantes :

- les opérateurs binaires Booléens AND, OR, XOR, NOT ;
- `bin`, qui transforme un entier dans son encodage en base 2 ;
- `Gray`, qui transforme un entier dans son code de Gray ;
- $2^i$ , la  $i$ -ème puissance de 2 ;
- `dec`, qui transforme un vecteur binaire dans l'entier correspondant ;
- `leftmost`, qui donne la composante la plus à gauche d'un vecteur quelconque : par exemple, `leftmost(( $x_4, x_5, x_6, x_7$ )) =  $x_4$  ;`
- `significant( $g$ )`, définie pour tout argument  $g \neq 0$ , qui donne l'index  $i$  de la composante non-zéro la plus à gauche dans le vecteur binaire  $g = (g_{d-1}, \dots, g_0)$  : par exemple, en prenant  $g = (0, 1, 0, 1)$  et  $d = 4$ , on a `significant((0, 1, 0, 1)) =  $d - 2 = 2$  ;`
- `rightfill`, qui donne le vecteur binaire obtenu en changeant en 1 tous les zéros les plus à droite d'un vecteur binaire  $(g_{d-1}, \dots, g_0)$  donné : par exemple, `rightfill((0, 1, 0, 0)) = (0, 1, 1, 1)`.

1. Donnez un exemple de topologie irrégulière, et une de topologie régulière de degré 3, avec 4 sommets. Est-ce que l'hypercube a une topologie régulière ? Combien de voisins à chaque nœud de l'hypercube de dimension  $d$  ?
2. Donnez deux exemples de topologie régulière de degré 3 : une avec 12 sommets et l'autre avec 16 sommets. Généralisez à une famille de topologies régulières de degré 3 avec  $4k$  sommets pour tout  $k \geq 3$ .
3. Calculez la distance de Hamming entre  $(0, 1, 0, 1)_2$  et  $(1, 1, 1, 0)_2$ .
4. Quels sont les voisins du nœud  $(0, 1, 1, 0)_2$  dans la topologie de l'hypercube avec le code de Gray ? Plus généralement, étant donné un nœud étiqueté  $g = (g_{d-1}, \dots, g_0) \in \mathbb{F}_2^d$  dans le code de Gray, donnez la formule d'une fonction `neighbor $_i$`  :  $\mathbb{F}_2^d \rightarrow \mathbb{F}_2^d$  pour obtenir l'étiquetage de son  $i$ -ème voisin (pour tout  $i \in \{0, \dots, d-1\}$ ).
5. Dessinez un carré en 2D ( $d = 2$ ), et étiquetez les nœuds avec leur code de Gray. Considérez la fonction  $f$ , qui associe un vecteur binaire à un intervalle à bornes entières, comme suit :

$$\begin{array}{ll}
 (0, 0) & \rightarrow [0, 3] \\
 (0, 1) & \rightarrow [1, 1] \\
 (1, 0) & \rightarrow [2, 3] \\
 (1, 1) & \rightarrow [3, 3]
 \end{array}$$

Maintenant dessinez un cube en 3D ( $d = 3$ ), et étiquetez les nœuds avec leur code de Gray.

Considérez l'extension de la fonction  $f$  au cas  $d = 3$  :

|             |               |          |
|-------------|---------------|----------|
| $(0, 0, 0)$ | $\rightarrow$ | $[0, 7]$ |
| $(0, 0, 1)$ | $\rightarrow$ | $[1, 1]$ |
| $(0, 1, 0)$ | $\rightarrow$ | $[2, 3]$ |
| $(0, 1, 1)$ | $\rightarrow$ | $[3, 3]$ |
| $(1, 0, 0)$ | $\rightarrow$ | $[4, 7]$ |
| $(1, 0, 1)$ | $\rightarrow$ | $[5, 5]$ |
| $(1, 1, 0)$ | $\rightarrow$ | $[6, 7]$ |
| $(1, 1, 1)$ | $\rightarrow$ | $[7, 7]$ |

Généralisez la fonction  $f$  au cas d'un hypercube quelconque en  $d$  dimensions : c'est-à-dire, donnez une formule, valide pour tout entier  $d$ , pour associer les vecteurs binaires en  $\mathbb{F}_2^d$  à des intervalles appropriés, cohérents avec les intervalles donnés dans les tableaux en ci-dessus.

[C'est évident qu'il y a plusieurs solutions possibles à cet exercice ; on veut évaluer ici votre capacité de *généralisation* et d'*abstraction* dans l'identification d'une extension de  $f$  qui soit cohérente et utile pour le reste de l'exercice.]

6. Le processus maître sur un cluster de  $2^d$  processeurs en topologie d'hypercube en dimension  $d$  contient le tableau  $x = (x_0, \dots, x_{p-1})$ , où  $p = 2^d$ . Donnez le pseudo-code d'un algorithme **scatter** (diffusion personnalisée) sur l'hypercube (avec nœud racine 0) grâce auquel l' $i$ -ème processus reçoit la donnée  $x_i$ . Vous pouvez utiliser la fonction  $f$  définie au point 5.



# Chapitre 4

## Le tri parallèle

Un résumé des points essentiels à retenir est donné dans §4.12.

### 4.1 Le tri en séquentiel : quelques rappels élémentaires

Soit  $X = \{x_1, \dots, x_n\}$  un ensemble de  $n$  réels stockés en mémoire dans un tableau contigu à  $n$  cases  $X[0], \dots, X[n-1]$  (avec l'indice  $i$  décalé,  $X[i] = x_{i+1}$ , commençant à 0 et finissant à  $n-1$ ). On cherche à trier par ordre croissant, c'est-à-dire à donner la liste triée  $(x_{(1)}, \dots, x_{(n)})$  avec  $x_{(1)} \leq \dots \leq x_{(n)}$ . Cela revient donc à chercher une permutation  $\sigma$  sur les indices de 1 à  $n$  telle que  $x_{\sigma(1)} \leq \dots \leq x_{\sigma(n)}$  (souvent en statistique, on note  $\sigma(i) = (i)$  pour les statistiques d'ordre). Il existe  $n!$  permutations distinctes, et on peut donc “dé-trier” de  $n!$  façons différentes une liste triée d'éléments distincts.

Trier en parallèle sur  $P$  processeurs à mémoire distribuée suppose que les données soient déjà réparties en  $P$  tableaux  $X_0, \dots, X_{P-1}$  sur les  $P$  processus  $P_0, \dots, P_{P-1}$  à mémoire distribuée, et qu'à la fin de l'algorithme de tri parallèle, on ait tous les éléments de  $X_0$  triés et inférieurs à tous les éléments de  $X_1$ , etc.

#### 4.1.1 Quelques rappels succincts sur les principaux algorithmes de tri

**tri à bulles : BubbleSort.** C'est le *tri par propagation* qui consiste à faire remonter les plus grands éléments du tableau. Il est appelé ainsi par analogie aux bulles d'air qui remontent à la surface de l'eau. La figure 4.1 illustre un exemple de tri à bulles, ou *BubbleSort* en anglais. Le *tri à bulles* est un algorithme naïf mais très facile à programmer qui demande un temps quadratique, en  $O(n^2)$ .

**le tri rapide : QuickSort.** Il s'agit d'un algorithme *récuratif* randomisé avec le choix d'un élément *pivot* qui demande un temps amorti en  $\tilde{O}(n \log n)$ . Quicksort choisit le premier élément  $X[0]$  du tableau comme pivot, puis partitionne  $X$  en trois parties : le tableau  $X_{<}$  des éléments strictement inférieurs au pivot,  $X_{>}$  le tableau des éléments strictement supérieurs au pivot, et  $X_{=}$  le tableau des éléments égaux au pivot (de taille 1 si tous les éléments sont distincts). Ensuite, Quicksort appelle récursivement le tri sur les tableaux  $X_{<}$  et  $X_{>}$  et retourne ainsi la

liste triée comme suit :

$$\text{QuickSort}(X) = (\text{QuickSort}(X_{<}), X_{=}, \text{QuickSort}(X_{>})).$$

Attention, n'oubliez pas de faire une permutation aléatoire  $\sigma_{\text{random}}$  avant d'appeler **QuickSort** sinon vous risquez le pire temps quadratique ! Une analyse plus fine, montre que **Quicksort** demande un temps  $\tilde{O}(n \log p)$  si on a  $p$  éléments distincts.

**Le tri par fusion de listes.** **MergeSort** est un algorithme récursif qui partage d'abord les données en coupant les listes en deux jusqu'à obtenir des listes élémentaires d'un seul élément (et donc triées) puis fusionne ces sous-listes deux à deux en remontant jusqu'à la liste triée pour le tableau complet des données (figure 4.3). L'opération élémentaire consiste à fusionner deux *sous-listes triées* en une liste triée : cela peut se faire en temps linéaire, et **MergeSort** coûte au final  $O(n \log n)$ .

**Le tri par base.** Le tri **RadixSort** est basé sur la représentation des éléments en nombres binaires sur  $b$  bits :  $x_i = \sum_{j=0}^{b-1} x_i^{(j)} 2^j$ . On trie d'abord les éléments en deux paquets en fonction de la valeur de leur bit à 1 ou 0, en commençant par le bit de poids faible jusqu'au bit de poids fort. La complexité du tri par base est  $O(bn)$ . Notez cependant qu'on ne peut avoir qu'au plus  $n = 2^b$  nombres distincts. C'est-à-dire, qu'on doit avoir  $b \geq \log_2 n$  pour garantir d'avoir des éléments bien distincts, et que la complexité de **RadixSort** dans ce cas est  $O(nb) = O(n \log n)$ .

Cette liste des principaux tris séquentiels est loin d'être exhaustive. On verra notamment au § 4.3 le tri par rang, **RankSort**.

### 4.1.2 Complexité des algorithmes de tri

Pour connaître la *borne inférieure* du problème qui consiste à trier  $n$  éléments supposés distincts, on remarque qu'une comparaison<sup>1</sup>  $\leq$  permet de couper en deux l'ensemble des permutations possibles (avec le prédicat  $\leq$  qui retourne soit 0 pour faux, soit 1 pour vrai). Pour trouver la permutation qui trie les  $n$  éléments, on part donc de la permutation identité jusqu'à la permutation finale qui donne la liste triée. Analysons la complexité du tri en introduisant le formalisme des arbres de décision. Un *arbre de décision* est un arbre binaire plein qui liste tous les choix possibles pour l'évaluation des prédicats dans les nœuds internes, et tous les résultats possibles comme sortie dans ses feuilles. La figure 4.2 illustre un tel arbre de décision pour trier la séquence de trois nombres  $(a_1, a_2, a_3)$ .

On cherche donc la profondeur de l'*arbre de décision*<sup>2</sup> dans l'ensemble des permutations. Pour un arbre binaire à  $m$  nœuds, la profondeur de l'arbre est au moins  $\lfloor \log_2 m \rfloor$ . Puisqu'on a  $n!$  permutations possibles pour trier, on en déduit que la profondeur minimale de l'arbre de décision est  $\lfloor \log_2 n! \rfloor$ . On utilise la formule de Stirling qui approxime la factorielle  $n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$  pour en déduire que  $\log_2 n! = O(n \log n)$ . Trier demande donc  $\Omega(n \log n)$  opérations de comparaison. Notons que le modèle de calcul est important pour établir des bornes inférieures. On considère le modèle *real-RAM* qui suppose que les opérations élémentaires d'arithmétique se font en temps constant sur des nombres réels. Pour d'autres modèles de calcul, on peut trier en temps déterministe en temps  $O(n \log \log n)$  en utilisant un espace mémoire linéaire [43] (tri entier, *integer sorting*). Il existe aussi des *algorithmes adaptatifs* qui tirent partie du fait que les listes peuvent déjà être partiellement triées [14].

1. Ici, on choisit l'opérateur de comparaison  $\leq$  qui est la négation de  $>$ .

2. [http://en.wikipedia.org/wiki/Decision\\_tree](http://en.wikipedia.org/wiki/Decision_tree)

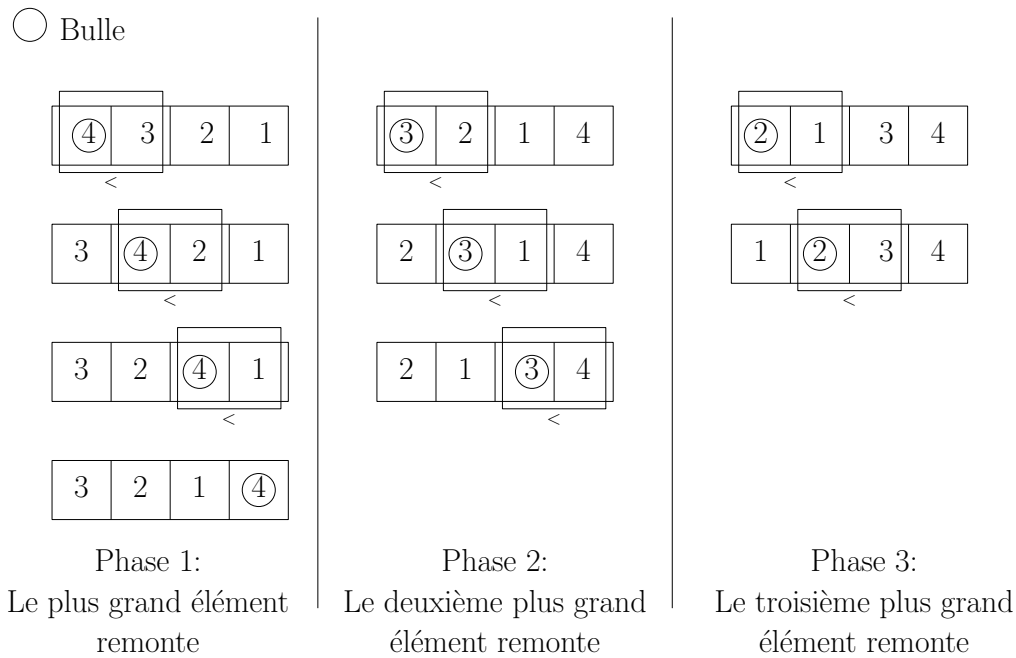


FIGURE 4.1 – Exemple du tri à bulles qui demande un temps quadratique : on compare les paires d'éléments adjacents en faisant glisser la fenêtre de comparaison. À la fin de la première phase, l'élément le plus grand se trouve en dernière position du tableau. Puis on recommence et la deuxième phase place le second plus grand élément à la fin du tableau, etc.

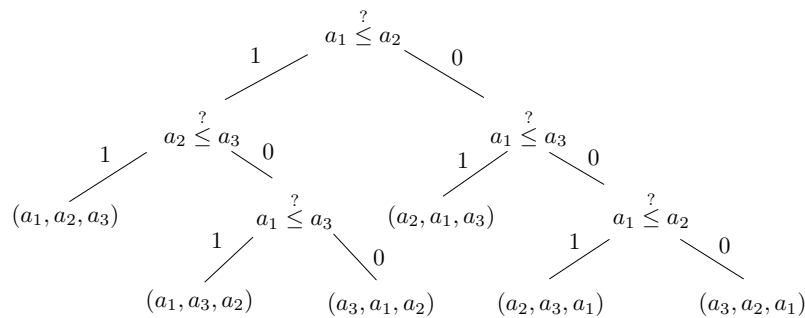


FIGURE 4.2 – Illustration d'un arbre de décision pour trier la liste  $(a_1, a_2, a_3)$  de 3 nombres : les nœuds internes évaluent des prédicats de comparaison  $x \leq y$  entre deux éléments  $x$  et  $y$  qui peuvent être soit vrais (valeurs booléennes à 1) soit faux (valeurs booléennes à 0). Dans le pire des cas, il faut parcourir un chemin le plus long entre la racine et une feuille de cet arbre binaire plein (avec  $3! = 6$  feuilles, chacune étant associée à une permutation des entrées).

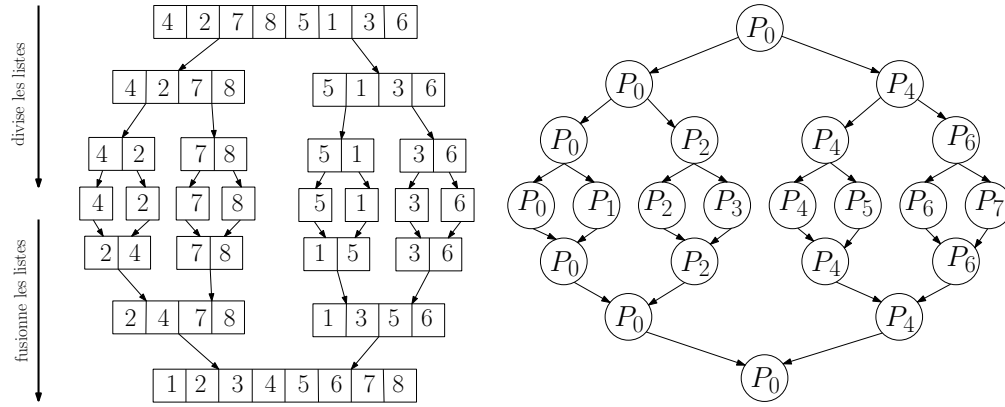


FIGURE 4.3 – Illustration de la parallélisation du tri par fusion de listes : MergeSort parallèle.

## 4.2 Le tri parallèle par fusion de listes : MergeSort parallèle

La figure 4.3 montre comment on peut paralléliser (à petits grains) l'algorithme de tri par fusion de listes. On utilise  $P = n$  processeurs pour diviser les données et fusionner les sous-listes triées. Supposons que  $n$  est une puissance de 2. Une analyse de la complexité montre un temps séquentiel  $t_{\text{seq}}$  :

$$t_{\text{seq}} = O\left(\sum_{i=0}^{\log n} 2^i \frac{n}{2^i}\right) = O(n \log n),$$

puisque comme le montre la figure 4.3 nous avons lors des étapes de fusion  $2^i$  listes triées de taille  $\frac{n}{2^i}$  au niveau  $(\log n) - i$ . À chaque niveau, nous avons  $n$  éléments, et au niveau  $\log n$ , nous avons bien  $n$  listes singletons qui forment les feuilles de l'arbre de fusion.

Regardons maintenant le temps parallèle :

$$t_{\text{par}} = O\left(2 \sum_{i=0}^{\log n} \frac{n}{2^i}\right) = O(n),$$

puisque  $\sum_{k=0}^n q^k = \frac{1-q^{n+1}}{1-q}$ , et donc  $\sum_{i=0}^{\log n} \frac{1}{2^i} = \frac{1-(\frac{1}{2})^{1+\log n}}{1-\frac{1}{2}} \leq 2$ . Cette méthode est donc inefficace puisque l'accélération<sup>3</sup> est  $\frac{t_{\text{seq}}}{t_{\text{par}}} = O(\log n)$  alors qu'on aurait aimé idéalement un facteur d'accélération  $O(P) = O(n)$ . Comme l'indique la figure 4.3, lorsque l'on fusionne les sous-listes triées, des processus se retrouvent sans travail.

## 4.3 Le tri parallèle par rang : RankSort

Peut-on trier en parallèle en temps  $O(\log n)$  ? Nous allons voir que la réponse est affirmative avec l'algorithme trivialement parallèle basé sur le rang, RankSort en anglais, mais que cet algorithme

3. On rappelle que l'accélération est définie comme le rapport du temps d'un algorithme séquentiel sur le temps d'un algorithme parallèle. On peut aussi majorer le temps de l'exécution séquentielle par celui de l'algorithme parallèle en prenant  $P = 1$ . En d'autres termes,  $\text{speedup}(P) = \frac{t_{\text{seq}}}{t_{\text{par}}(P)} \leq \frac{t_{\text{par}}(1)}{t_{\text{par}}(P)}$ .

est loin de donner une accélération optimale.

Pour chaque donnée  $X[i]$ , on calcule son *rang* dans un tableau :  $R[i] = |\{X[j] \in X \mid X[j] < X[i]\}|$ . C'est-à-dire que le rang  $R[i]$  de  $X[i]$  est le nombre d'éléments du tableau inférieurs à lui-même. Le plus petit élément a comme rang 0 et le plus grand élément a le rang  $n - 1$ . Puis on range les éléments dans un nouveau *tableau auxiliaire* qui sera trié  $Y : Y[R[i]] = X[i]$ . Notez qu'on a supposé tous les éléments distincts pour les calculs de rang.

Le calcul du rang est lui-même facilement *parallélisable* sur  $P = n$  nœuds : Pour un  $X[i]$  donné, on évalue le prédicat  $X[j] < X[i], \forall j$ , et on agrège toutes les réponses en comptant 0 lorsque le prédicat est faux et 1 lorsque le prédicat est vrai. Ce qui donne :

$$R[i] = \sum_{j=0}^{n-1} \underbrace{1_{[X[j] < X[i]]}}_{\text{Prédicat booléen converti en 0 ou 1}}$$

L'algorithme RankSort en séquentiel est ainsi composé d'une double boucle :

```

for (i = 0; i < n; i++)
{ // pour chaque nombre
  rang = 0;
  for (j = 0; j < n; j++)
  { // on compte les nombres plus petits que lui
    if (a[i] > a[j])
      {rang++;}
  }
  // puis on le recopie à la place de son rang dans le nouveau tableau b[]
  b[rang] = a[i];
}

```

Le temps séquentiel est quadratique,  $t_{\text{seq}} = O(n^2)$ , et le temps en parallèle est  $t_{\text{par}} = O(P) = O(n)$  quand  $P = n$  (temps linéaire). Considérons RankSort // avec  $P = n^2$ , un nombre quadratique de processeurs. Par exemple, pour de petites valeurs de  $n$ , on peut utiliser l'unité de calcul graphique (GPU, *Graphical Processing Unit*) constituée de plusieurs milliers de noyaux de calcul (*cores*).

Pour calculer le rang d'un seul élément, on utilise désormais  $n$  processus en effectuant une opération de réduction qui calcule une somme globale (à la `MPI_Reduce/MPI_Sum`). On utilise au total  $P = n^2$  processus pour calculer tous les rangs. Le processeur  $P_{i,j}$  évalue le prédicat  $1_{[X[j] < X[i]]}$ , et on calcule le rang de  $X[i]$  en agréant les valeurs des prédicats des processeurs  $P_{i,*}$ . La notation  $P_{i,*}$  signifie le groupe des processus  $P_{i,j}$  pour  $1 \leq j \leq P$ .

Le temps de calcul en parallèle est donc le temps de calcul d'une opération collective de réduction. On peut supposer que cette réduction qui dépend de la *topologie* du réseau d'interconnexion se fait en temps logarithmique pour l'hypercube de dimension  $\log n$  mais en temps linéaire pour une topologie d'anneau (voir le chapitre 3 sur la topologie). Ainsi, en utilisant  $P = n^2$  processeurs, sur une topologie d'hypercube, nous avons :

$$t_{\text{par}} = O(\log n).$$

Si l'on choisit la topologie du graphe complet d'interconnexion, alors une opération de réduction se fait en temps constant (en supposant que l'on peut recevoir en même temps les  $P - 1$  données de ses voisins), et l'algorithme de tri RankSort avec  $P = n^2$  processus demande alors un temps constant, en  $O(1)$ .

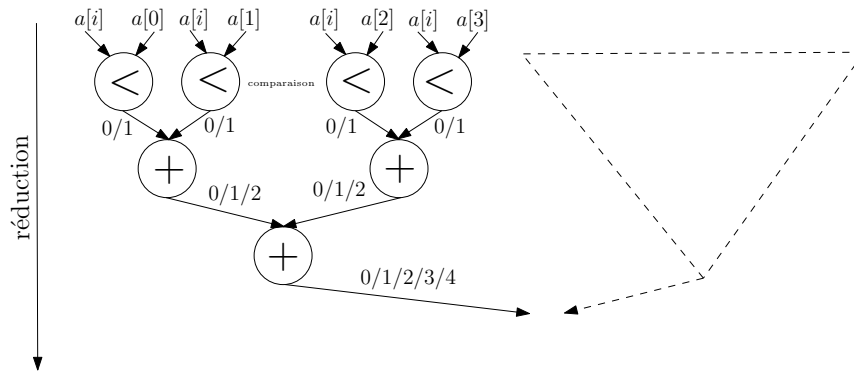


FIGURE 4.4 – Calcul du rang dans RankSort par agrégation des prédicats  $1_{[X[j] < X[i]]}$  : une opération collective de réduction.

## 4.4 QuickSort en parallèle : ParallelQuickSort

On rappelle que pour une valeur pivot  $x$ , on partitionne les données (*split*) en deux<sup>4</sup> sous-tableaux  $X_{\leq x}$  et  $X_{> x}$ , puis qu'on trie récursivement les sous-ensembles  $X_{\leq x} \leftarrow \text{QuickSort}(X_{\leq x})$  et  $X_{> x} \leftarrow \text{QuickSort}(X_{> x})$ , et qu'enfin on assemble les sous-tableaux triés :  $\text{QuickSort}(X) = (\text{QuickSort}(X_{\leq x}), \text{QuickSort}(X_{> x}))$ .

Si l'on choisit *aléatoirement* le pivot  $x$  dans  $X$ , on obtient un *algorithme randomisé* en temps  $\tilde{O}(n \log n)$  en moyenne sur toutes les permutations des données. Sinon lorsque le pivot est choisi de façon déterministe, on calcule un quantile comme la *médiane* (en temps linéaire) pour équilibrer la taille des sous-tableaux, et on obtient un *algorithme déterministe* en  $O(n \log n)$ .

L'algorithme QuickSort en C++ en utilisant la STL<sup>5</sup> (pour *Standard Template Library* en anglais) peut se programmer comme suit :

Code 16 – L'algorithme QuickSort implémenté en C++ avec la Standard Template Library (STL) : quicksort442.cpp

```
// Nom du programme : quicksort442.cpp
// Compilez avec g++ quicksort442.cpp -o quicksort442.exe
// Exécutez avec ./quicksort442.exe

# include <vector>
# include <iostream>
using namespace std;

// choix du pivot : élément se trouvant à l'index médian
template <class T> void quickSort(vector<T>&v, unsigned int low, unsigned int
    high)
{
    if (low >= high) return;
    // sélectionne la valeur du pivot
    unsigned int pivotIndex = (low + high) / 2;
```

4. Dans cette partie, on considère deux sous-tableaux  $X_{\leq}$  et  $X_{>}$  au lieu de trois sous-tableaux  $X_{<}$ ,  $X_{=}$  et  $X_{>}$ . Cela simplifie la description du tri rapide sans pour autant en changer la théorie.

5. [http://en.wikipedia.org/wiki/Standard\\_Template\\_Library](http://en.wikipedia.org/wiki/Standard_Template_Library)

```

    // partitionne le vecteur
    pivotIndex = pivot (v, low, high, pivotIndex);
    // tri les deux sous-vecteurs récursivement
    if (low < pivotIndex) quickSort(v, low, pivotIndex);
    if (pivotIndex < high) quickSort(v, pivotIndex + 1, high);
}

template <class T> void quickSort(vector<T> & v)
{
    unsigned int numberElements = v.size ();
    if (numberElements > 1)
        quickSort(v, 0, numberElements - 1);
}

template <class T> unsigned int pivot (vector<T> & v, unsigned int start,
    unsigned int stop, unsigned int position)
{
    // on échange le pivot avec la position initiale
    swap (v[start], v[position]);
    // partitionne les valeurs
    unsigned int low = start + 1;
    unsigned int high = stop + 1;
    while (low < high)
        if (v[low] < v[start])
            low++;
        else if (v[--high] < v[start])
            swap (v[low], v[high]);
    // et on re-échange le pivot à sa place initiale
    swap (v[start], v[--low]);
    return low;
}

// Petit exemple de démonstration
int main() {
    vector<int> v(100);
    for (int i = 0; i < 100; i++)
        v[i] = rand()%442;
    quickSort(v);
    vector<int>::iterator itr = v.begin();
    while (itr != v.end ()) {
        cout << *itr << " ";
        itr++;
    }
    cout << "\n";
    return 0;
}

```

On compile en C++ avec le compilateur g++ :

```
g++ quicksort442.cpp -o quicksort442.exe
```

Et on exécute dans la console en tapant :

```
./quicksort442.exe
```

On obtient la sortie suivante :

```
./quicksort442.exe
```

```

4 8 15 17 22 37 38 44 53 55 57 57 65 65 69 79 84 95 100 102 105 105
    107 121 122 133 134 137 140 141 147 156 158 159 163 166 168 168
    187 189 191 197 204 214 218 218 222 224 226 231 232 233 234 235
    244 247 248 250 254 261 262 274 282 286 287 293 293 296 299 308
    309 314 316 325 334 340 342 349 349 353 354 354 366 378 380 388
    388 393 393 402 405 407 408 413 422 425 427 428 430 439

```

On définit l'*élément médian* pour  $n = 2m + 1$  comme étant l'élément dans un tableau trié en position  $m = \frac{n-1}{2}$ . En général, on choisit l'indice  $\lfloor \frac{n}{2} \rfloor$  pour définir la *médiane* : l'élément au "milieu". L'algorithme 2 rappelle l'algorithme classique pour calculer en temps linéaire déterministe le  $k$ -ième élément (ou la médiane) d'un ensemble de valeurs (*median and order statistics*).

```

Data :  $S$  un ensemble de  $n = |S|$  nombres,  $k \in \mathbb{N}$ 
Result : Retourne le  $k$ -ième élément de  $S$ 
if  $n \leq 5$  then
    // cas terminal de la récursivité
    trier  $S$  et retourner le  $k$ -ième élément;
else
    Diviser  $S$  en  $\lceil \frac{n}{5} \rceil$  groupes;
    // Le dernier groupe a 5 (complet) ou  $n \bmod 5$  éléments
    Calculer les médianes des groupes  $M = \{m_1, \dots, m_{\lceil \frac{n}{5} \rceil}\}$ ;
    // Calcul du pivot  $x$ , la médiane
     $x \leftarrow \text{SELECT}(M, \lceil \frac{n}{5} \rceil, \lfloor \frac{\lceil \frac{n}{5} \rceil + 1}{2} \rfloor)$ ;
    Partitionner  $S$  en deux sous-ensembles  $L = \{y \in S : y \leq x\}$  et  $R = \{y \in S : y > x\}$ ;
    if  $k \leq |L|$  then
        return  $\text{SELECT}(L, |L|, k)$ ;
    else
        return  $\text{SELECT}(R, n - |L|, k - |L|)$ ;
    end
end

```

**Algorithme 2 :** Calcul du  $k$ -ième élément (médiane quand  $k = \lfloor \frac{n}{2} \rfloor$ ) par un algorithme récursif SELECT (déterministe) en temps linéaire.

Maintenant, parallélisons Quicksort : soit  $P$  machines (chacune s'occupant d'un processus), on cherche à trier les données déjà distribuées sur les machines  $P_0, \dots, P_{P-1}$  en  $P$  sous-ensembles  $X_0, \dots, X_{P-1}$  de taille<sup>6</sup>  $\frac{n}{P}$ . On note  $X_i \leq X_j$  si et seulement si  $\forall x_i \in X_i, \forall x_j \in X_j, x_i \leq x_j$ . Initialement, les paquets  $X_0, \dots, X_{P-1}$  ne sont pas ordonnés. L'idée maîtresse de cette première parallélisation de Quicksort consiste à partitionner les données sur les processus en échangeant des messages de telle façon qu'à la fin de la partition, on ait  $X_0 \leq \dots \leq X_{P-1}$ .

Une implémentation directe de QuickSort en parallèle consiste à choisir de façon aléatoire le pivot  $x$  et à le diffuser (*broadcast*) à tous les autres processus. Chaque processus  $P_p$  partitionne alors son tableau en deux sous-tableaux  $X_{\leq}^p$  et  $X_{>}^p$  en utilisant le pivot  $x$ . Puis chaque *processus supérieur*  $p \geq P/2$  envoie sa *liste inférieure*  $X_{\leq}^p$  à un *processus partenaire*  $p' = p - P/2 \leq P/2$ , et reçoit en retour une *liste supérieure*  $X_{>}^{p'}$ , et vice-versa. Pour le tri par ordre décroissant, il suffit d'inverser les listes supérieures/inférieures envoyées. Les processus se séparent alors en deux groupes et on

6. On suppose que  $P$  divise  $n$  :  $n \bmod P = 0$  et que  $P$  est une puissance de 2 (donc  $n$  est aussi une puissance de 2).

applique *récurivement* l'algorithme. La figure 4.5 illustre cet algorithme **ParallelQuicksort** (*Parallel QuickSort*) pour le tri par ordre décroissant :  $X_0 \geq \dots \geq X_{P-1}$ .

Remarque : l'algorithme **Quicksort** séquentiel avec  $\log P$  niveaux récursifs donne un "arbre" d'appels de fonctions lorsqu'on visualise la pile qui partitionne les données :  $X_0 \leq X_1 \leq \dots \leq X_{P-1}$  en temps moyen  $\tilde{O}(n \log P)$  (algorithme randomisé) tel que les paquets de données  $X_i$  ne sont pas encore triés mais qu'on a bien  $X_i \leq X_j$  pour tout  $i \leq j$ . Il reste alors à trier les données sur chaque processus indépendamment les uns des autres par un algorithme adéquat.

On résume **ParallelQuicksort** comme suit :

- les processus supérieurs à  $P/2$  ont des valeurs supérieures au pivot, et les processus inférieurs à  $P/2$  ont des valeurs inférieures au pivot (pour le tri par ordre croissant),
- après  $\log P$  appels récursifs, chaque processus a une liste de valeurs complètement disjointe des autres, et
- la plus grande valeur des données de  $P_i$  est inférieure à la plus petite valeur des données de  $P_{i+1}$  pour tout  $i$ ,
- chaque processus trie alors son paquet (par exemple, avec **Quicksort** séquentiel) dans le cas terminal de la récursivité sur les processus.

Un inconvénient majeur de **ParallelQuicksort** est que les processus sur les machines effectuent des travaux qui peuvent être déséquilibrés car la taille de leurs listes *dépend* des pivots choisis et diffusés. Ce phénomène est illustré à la figure 4.5 (illustré ici pour le tri par ordre décroissant) et montre un déséquilibre de charge.

Nous allons maintenant voir deux algorithmes qui rectifient les charges de travail de chaque processeur : l'algorithme **HyperQuickSort** et l'algorithme **PSRS** pour *Parallel Sorting by Regular Sampling*.

## 4.5 L'algorithme amélioré HyperQuickSort

Les  $P$  processus commencent par un **Quicksort** séquentiel sur  $\frac{n}{P}$  données en temps  $\tilde{O}(\frac{n}{P} \log \frac{n}{P})$ . Puis le processus responsable du choix du pivot choisit la *médiane* de son tableau trié (donc à la position  $\frac{n}{2P}$ ). Ce "processus pivot" diffuse (*broadcast*) le pivot à tous les autres processus de son groupe. Les processus partitionnent alors leurs données en deux sous-listes  $X_{\leq}$  et  $X_{>}$  en fonction du pivot. Il s'ensuit les échanges des listes entre les processus partenaires (comme pour **QuickSort** parallèle), et sur chaque processus, on fusionne ses deux sous-listes triées en une liste triée en temps linéaire. Finalement, on appelle récursivement **HyperQuickSort** sur les processus de son groupe. La figure 4.6 illustre cet algorithme récursif.

Faisons l'analyse de la complexité en temps amorti moyen de **HyperQuickSort** avec les hypothèses suivantes : les listes sont supposées à peu près équilibrées et les temps de communication sont dominés par les temps de transmission (les temps de latence sont donc ignorés car supposés négligeables). Le **Quicksort** initial requiert  $\tilde{O}(\frac{n}{P} \log \frac{n}{P})$ , les comparaisons pour les  $\log P$  étapes de fusion coûtent  $\tilde{O}(\frac{n}{P} \log P)$ , et le coût pour les communications pour les  $\log P$  échanges de sous-listes est :  $\tilde{O}(\frac{n}{P} \log P)$ . Ainsi, le temps parallèle global est en  $\tilde{O}(\frac{n}{P} \log \frac{n}{P}) + \tilde{O}(\frac{n}{P} \log P) = \tilde{O}(\frac{n}{P} \log(\frac{n}{P} + P)) = \tilde{O}(\frac{n}{P} \log(n))$  car  $\frac{n}{P} \leq n$  et  $P \leq n$ . On obtient donc un facteur d'accélération optimal en  $\tilde{O}(P)$  sous les conditions de nos hypothèses.

En pratique, les listes que nous avons supposées à peu près équilibrées ne le sont pas vraiment dans les applications ! On va donc proposer un tri alternatif qui choisit de meilleurs pivots pour effectuer les partitions : c'est le tri parallèle par échantillonnage régulier (**PSRS**).

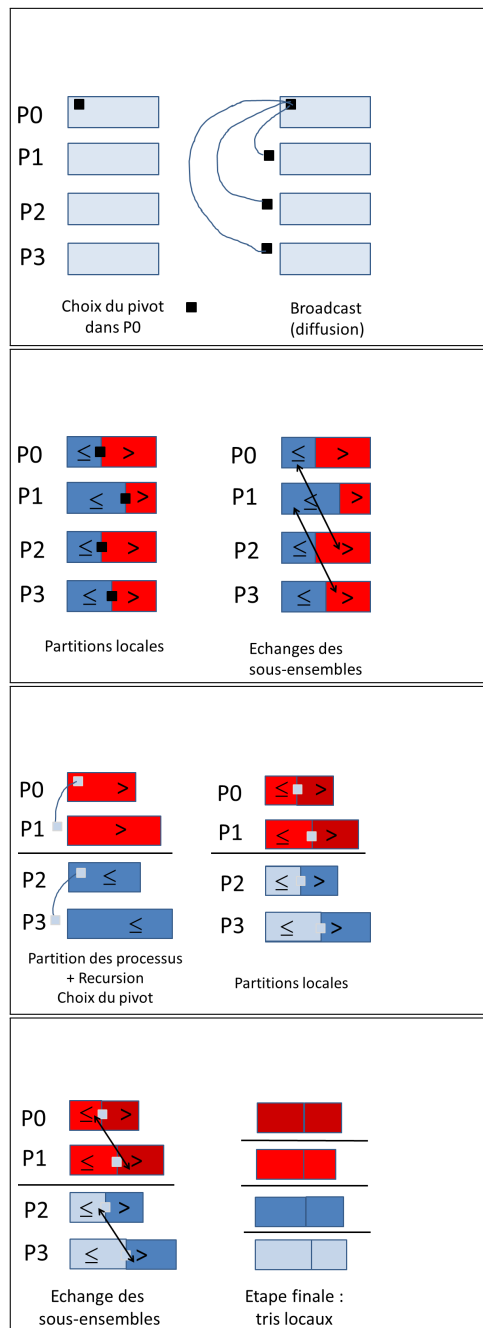


FIGURE 4.5 – Illustration de l'algorithme **ParallelQuicksort** sur 4 processeurs pour trier une suite de nombres par ordre *décroissant*  $X_0 \geq X_1 \geq X_2 \geq X_3$  : choix du pivot, diffusion du pivot, partitionnement avec le pivot, échange de sous-tableaux entre processeurs partenaires, et récursion.

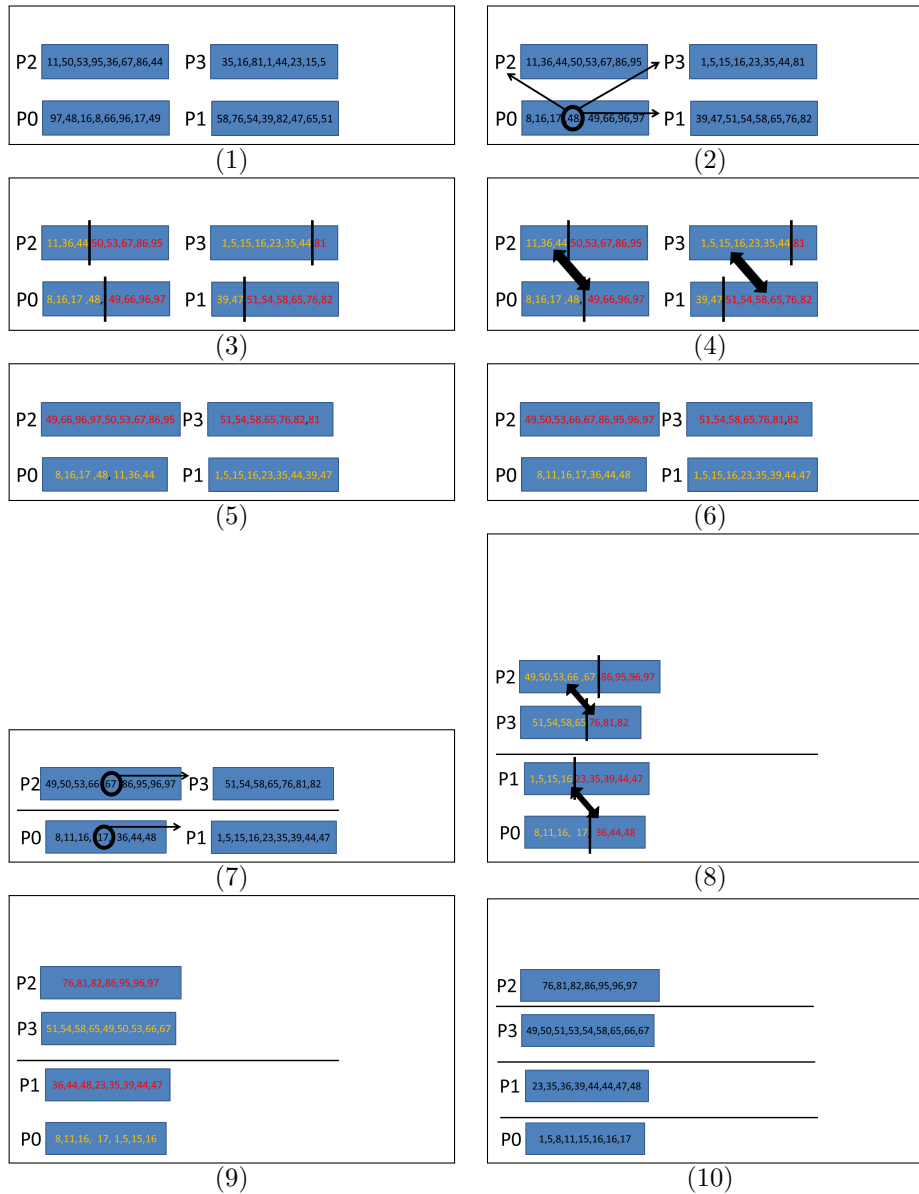


FIGURE 4.6 – Illustration de l'algorithme **HyperQuickSort** pour trier une liste de nombres dans l'ordre croissant sur 4 processeurs  $P_0, P_1, P_2$  et  $P_3$  : (1) initialisation, (2) choix du pivot 48, (3) partition des données avec 48, (4) échange des listes entre processus partenaires, (5) listes échangées, (6) fusion des listes, (7) récursivité sur les groupes  $\rightarrow$  Pivot 67||17, (8) partition et échange, (9) listes échangées, (10) fusion des sous-listes triées.

## 4.6 \* Le tri parallèle PSRS par échantillonnage régulier (*PSRS*)

Le tri parallèle par échantillonnage régulier ou *Parallel Sorting by Regular Sampling* (PSRS) est un algorithme de tri en *quatre phases*. Ici, le nombre de machines/processus  $P$  ne doit pas nécessairement être une puissance de 2. Nous décrivons l'algorithme PSRS comme suit :

1. chaque processus  $P_i$  trie avec un algorithme séquentiel (comme QuickSort) ses données locales et choisit ses  $P$  éléments aux *positions régulières* suivantes :

$$0, \frac{n}{P^2}, \frac{2n}{P^2}, \dots, \frac{(P-1)n}{P^2}$$

On obtient ainsi un échantillonnage régulier des données triées. Notons que nous devons avoir nécessairement  $n \geq P^2$ .

2. un processus dédié rassemble (*gather*) et trie tous ces échantillons réguliers, puis *sélectionne*  $P-1$  pivots parmi les  $P^2$  échantillons (la différence entre les indexes de deux pivots successifs est au plus  $\lceil \frac{P^2}{P-1} \rceil$ ). Ce processus *diffuse* alors ces  $P-1$  pivots, et chaque processus partitionne sa liste triée en  $P$  morceaux. Notons que certains sous-tableaux des partitions peuvent être vides ( $\emptyset$ ) comme l'illustre l'exemple de la figure 4.7.
3. chaque processus  $P_i$  garde sa  $i$ -ème partition et envoie sa  $j$ -ème partition au processus  $j$ ,  $\forall j \neq i$ . C'est une opération de diffusion totale (*all-to-all* ou *total exchange*).
4. chaque processus fusionne ses  $P$  partitions en une liste finale triée.

La figure 4.7 montre le principe de fonctionnement de cet algorithme PSRS sur un jeu de données.

On analyse la complexité de PSRS comme suit : chaque processus fusionne environ  $\frac{n}{P}$  éléments et ceci est maintenant *expérimentalement vérifié* en pratique ! On suppose que le réseau d'interconnexion permet  $P$  communications simultanées. On résume les coûts des étapes de PSRS.

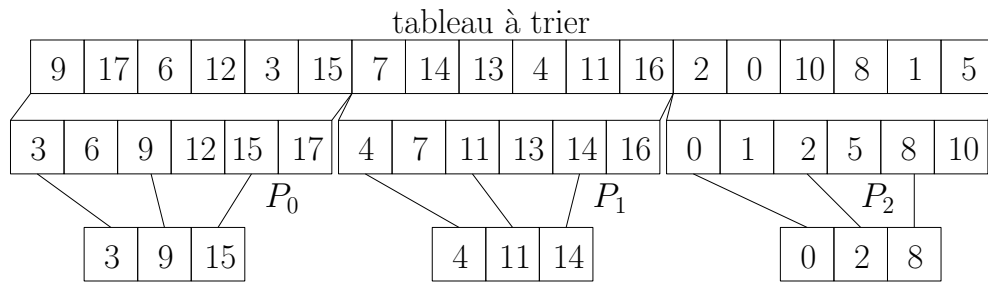
- Coût des calculs locaux :
  - QuickSort en temps  $\tilde{O}(\frac{n}{P} \log \frac{n}{P})$ ,
  - tri des échantillons réguliers :  $O(P^2 \log P)$ ,
  - fusion des sous-listes :  $O(\frac{n}{P} \log P)$ .
- Coût des communications :
  - rassemblement (*gather*), diffusion des pivots (*broadcast*),
  - diffusion totale (*all-to-all*) :  $O(\frac{n}{P})$ .

## 4.7 \* Le tri sur la grille 2D : ShearSort

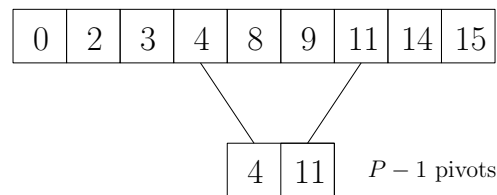
On présente ici un algorithme bien adapté à la topologie de la *grille* : l'algorithme *ShearSort*. À l'étape finale, la séquence triée peut être rangée soit ligne par ligne sur la grille, soit en forme de *serpentin* (motif en zigzag) comme le montre la figure 4.8.

Pour trier, on alterne le tri sur les lignes avec le tri sur les colonnes jusqu'à obtenir la séquence triée après  $\log n$  étapes. Si l'on désire le motif du serpent, on doit aussi alterner les directions (c'est-à-dire, les tris ascendants avec les tris descendants) sur les lignes.

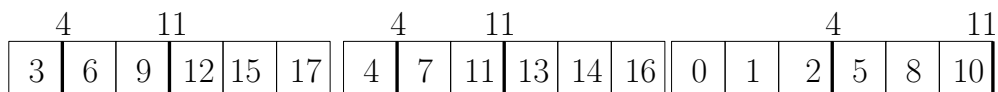
Analysons la complexité du tri *ShearSort* sur une grille 2D de taille  $P = \sqrt{n} \times \sqrt{n} = n$ . En triant en parallèle  $\sqrt{n}$  nombres en  $O(\sqrt{n})$ , le temps parallèle est  $t_{\text{par}} = O((\log n) \times \sqrt{n}) = O(\sqrt{n} \log n)$ . Le



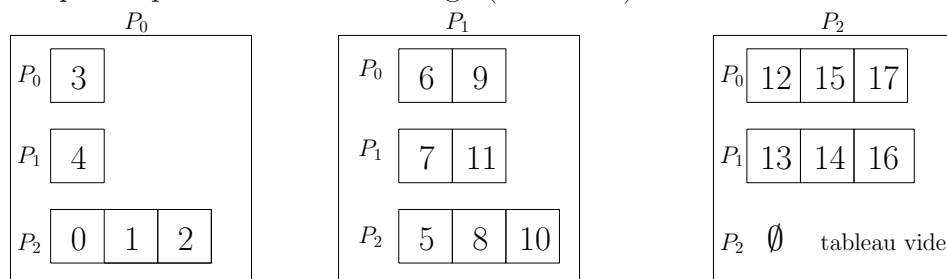
étape 1 : tri et échantillonnage régulier



étape 2 : rassemblement (*gather*) choix de  $P - 1$  pivots



étape 3 : partition et commérage (*all-to-all*)



étape 4 : fusion des  $P$  sous-listes sur chaque processus

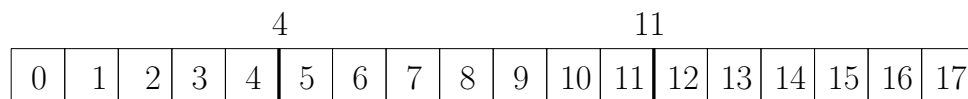


tableau trié

FIGURE 4.7 – Déroulement de l'algorithme de tri parallèle par échantillonnage régulier, dit algorithme PSRS.

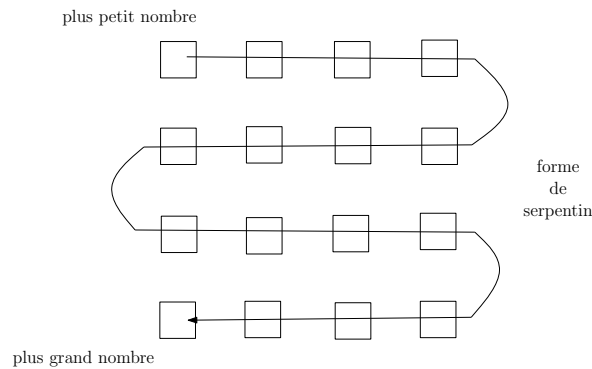


FIGURE 4.8 – À la fin du tri **ShearSort**, les éléments sont rangés par ordre croissant en forme de *serpentin* (motif en zigzag).

tri en séquentiel coûte quant à lui :  $t_{\text{seq}} = O(n \log n)$ . On obtient donc une accélération (*speed-up*) en  $\frac{t_{\text{seq}}}{t_{\text{par}}} = O(\sqrt{n}) = O(\sqrt{P})$ , ce qui n'est pas optimal!

La figure 4.9 illustre les différentes étapes du tri **ShearSort** pour une séquence donnée.

## 4.8 Tri par réseaux de comparaisons

On considère le tri par comparaison de *paires d'éléments* en présentant le *réseau de tri* dit des comparaisons paires et impaires (*odd-even transposition*) dont le principe repose sur le tri à bulles (*BubbleSort*). Ce tri comporte deux phases par cycle :

- la *phase paire* où on effectue la comparaison et échange (*swap*) des paires d'éléments dites paires :

$$(X[0], X[1]), (X[2], X[3]), \dots$$

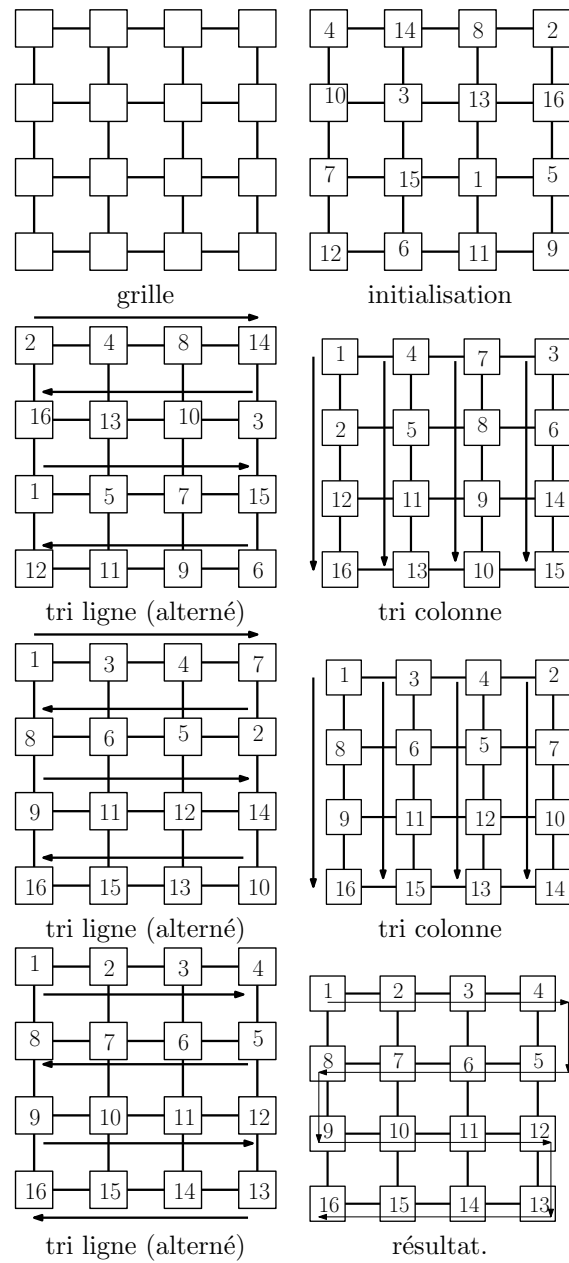
- la *phase impaire* où on effectue la comparaison et échange des paires d'éléments dites impaires :

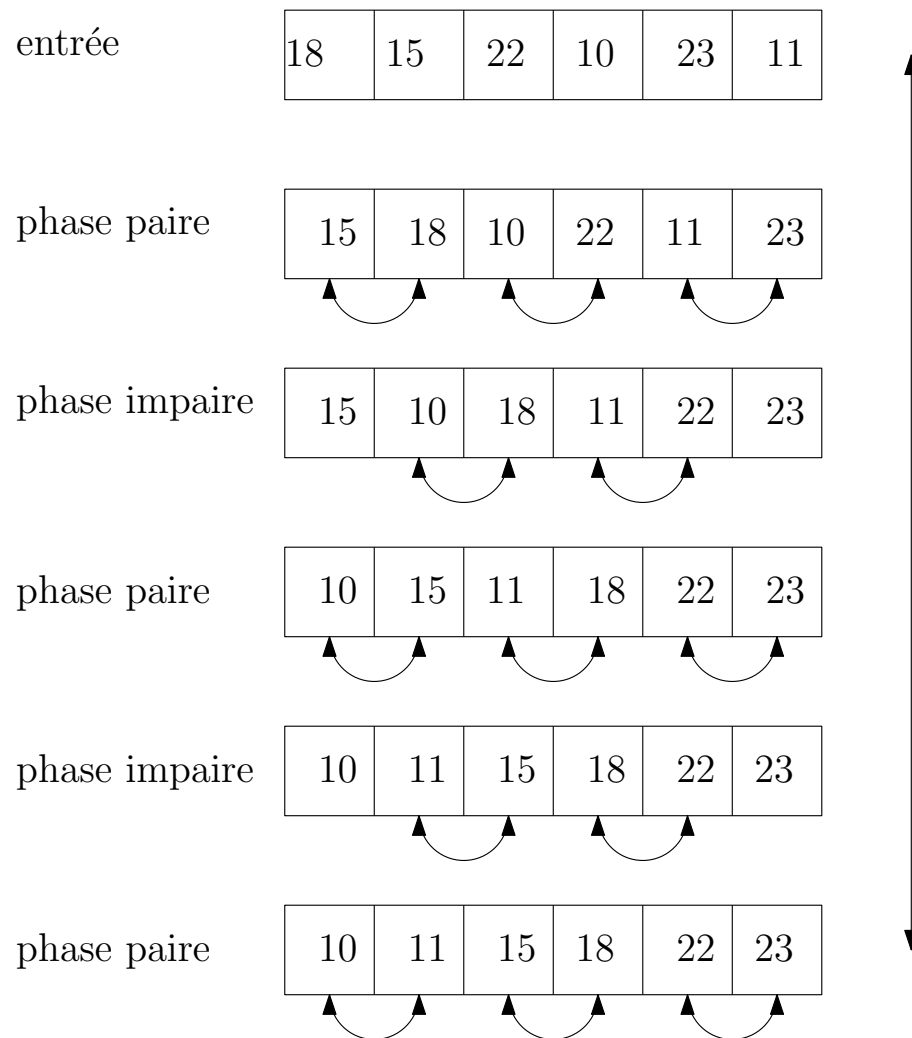
$$(X[1], X[2]), (X[3], X[4]), \dots$$

Pour trier un tableau de  $n$  éléments, cet algorithme demande  $n$  cycles de phases paires/phases impaires. La figure 4.10 montre un exemple d'utilisation de cet algorithme.

En C/C++, cet algorithme s'écrit simplement comme suit :

```
void Odd_even_sort(int a[], int n)
{
    int phase, i;
    for (phase = 0; phase < n; phase++)
        if (phase % 2 == 0)
            // phase paire
            for (i = 1; i < n; i += 2)
                if (a[i-1] > a[i])
                    swap(&a[i], &a[i-1]);
        else
            // phase impaire
            for (i = 1; i < n-1; i += 2)
```

FIGURE 4.9 – Déroulement des étapes du tri **ShearSort**. Trier demande  $\log n$  étapes.

FIGURE 4.10 – Le tri par comparaisons de paires d’éléments paires et impaires demandent  $n$  cycles.

```

    {if (a[i] > a[i+1])
      swap(&a[i], &a[i+1]);}
  }
}

```

On peut généraliser le tri pair/impair en considérant non plus des paires d'éléments mais des paires de *groupes d'éléments*. On trie alors les  $n/P$  éléments de chaque groupe sur son processus (avec disons **QuickSort** en séquentiel), puis on envoie et reçoit les éléments des paires de processus. Si le rang du processus est inférieur à celui du rang du processus de sa paire, alors on garde les valeurs les plus petites, sinon on garde les valeurs les plus grandes. On répète  $P$  fois ce cycle. On peut ainsi obtenir une *granularité de parallélisme* allant de  $P = n$  à  $P = 2$ . L'algorithme peut être très facilement implémenté. La figure 4.11 illustre le fonctionnement de cet algorithme.

Analysons maintenant la complexité de cet algorithme : le tri initial demande un temps de calcul en  $O(\frac{n}{P} \log \frac{n}{P})$  pour des paquets de taille  $\frac{n}{P}$ , puis on répète  $P$  cycles en triant les plus petites valeurs des plus grandes valeurs dans chaque phase en temps  $O(\frac{n}{P})$  (en fusionnant les listes et en gardant la moitié concernée de chaque côté du  $<$ ), et on communique à chaque processeur  $O(\frac{n}{P})$  éléments. En ignorant les temps de latence des communications, nous obtenons ainsi une complexité globale en temps  $O(\frac{n}{P} \log \frac{n}{P} + n)$ . Cet algorithme est très intéressant sur un réseau de communication qui utilise la topologie de l'*anneau bidirectionnel*.

## 4.9 \* Fusion de listes triées par un circuit de comparateurs

On peut construire des circuits qui implémentent des algorithmes pour trier des séquences de nombres. La figure 4.12 montre l'élément de base de ces circuits : la boîte comparateur-échangeur. On peut trier par fusion de deux *sous-listes triées* en utilisant un *circuit de comparateurs* (un algorithme implémenté en hardware) comme le montre la figure 4.13. On peut ainsi construire le *circuit de comparaisons* récursivement comme le montre le schéma de la figure 4.13.

## 4.10 \* Tri récursif de séquences bitoniques

Nous présentons l'algorithme de tri par *fusion récursive de listes bitoniques*, historiquement proposé par Ken Batchier.<sup>7</sup> Une *séquence bitonique* est une *séquence unimodale* avec un seul point extremum (soit un plus haut, un maximum, soit un plus bas, un minimum) en considérant la séquence *cycliquement* (topologie du tore 1D). La recherche d'un élément dans une séquence bitonique peut se faire en temps logarithmique par dichotomie.

On peut faire une *partition bitonique* comme suit :

- on associe à chaque élément de la première moitié de la liste un élément de la deuxième moitié :  $x_i \leftrightarrow x_{i+\frac{n}{2}}$ ,
- on compare ces paires et on les range avec l'ordre (min, max),
- chaque élément de la première moitié est ainsi plus petit que *tous* les éléments de la seconde moitié,
- les deux moitiés sont des listes bitoniques de longueur  $\frac{n}{2}$  (invariant),
- cette séquence de comparaisons *ne dépend pas* des données. Cette propriété essentielle contraste donc avec **MergeSort** dont le comportement dépend des valeurs des données.

7. [http://en.wikipedia.org/wiki/Ken\\_Batcher](http://en.wikipedia.org/wiki/Ken_Batcher)

Configuration initiale



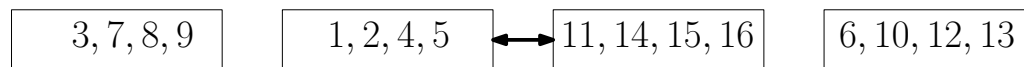
Configuration après les tris locaux



Phase 1(pair)



Phase 2 (impair)



Phase 3 (pair)



Phase 4 (impair)

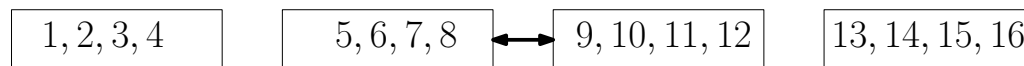


FIGURE 4.11 – Généralisation du tri paires paires/impaires au tri groupes pairs/impairs.

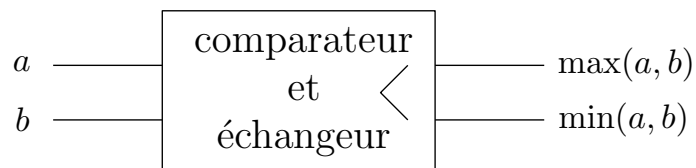


FIGURE 4.12 – Une boîte comparateur-échangeur qui prend deux entrées et retourne en sortie le minimum et le maximum.

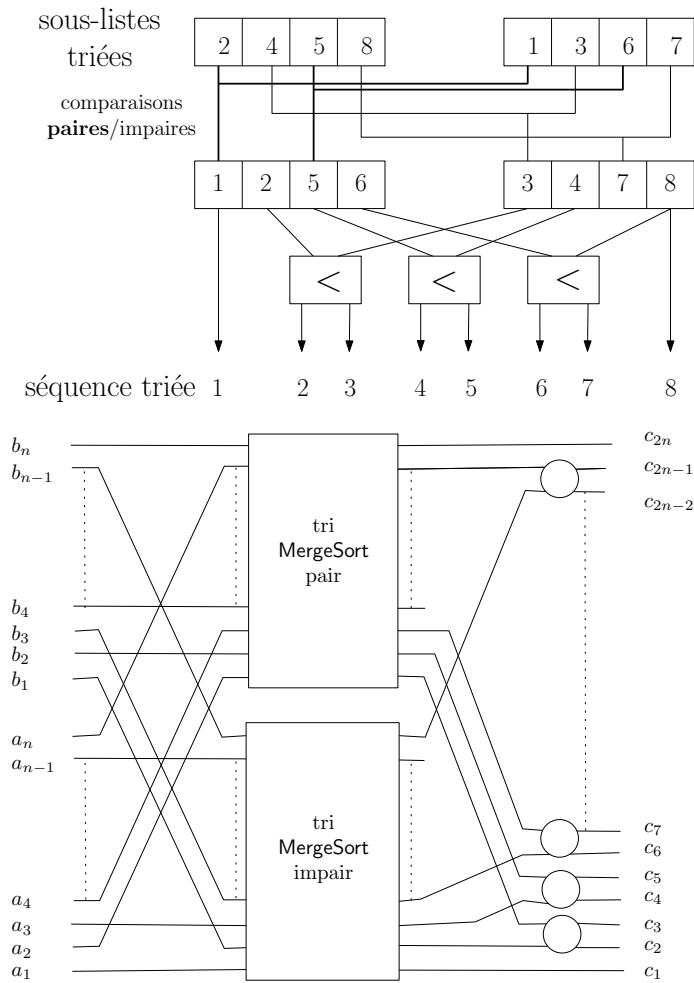


FIGURE 4.13 – Circuit de comparateurs pour la fusion de sous-listes triées.

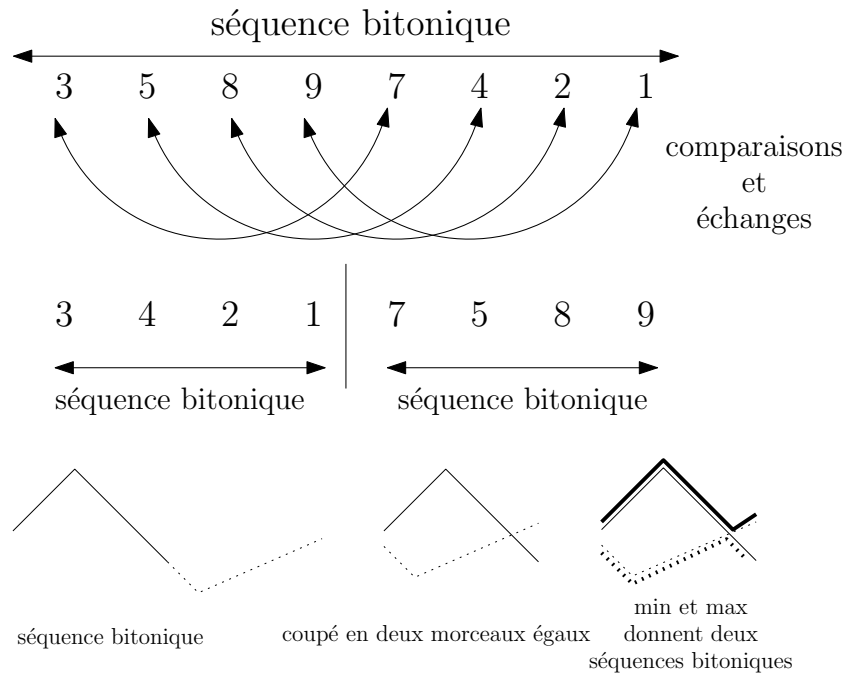


FIGURE 4.14 – Haut : découpage équilibré d’une séquence bitonique en deux séquences bitoniques par comparaison-échange de  $x_i$  avec  $x_{i+\frac{n}{2}}$ . Bas : une preuve visuelle que l’on obtient bien deux séquences bitoniques en prenant le min et le max sur les deux sous-séquences de la séquence bitonique.

On réalise ainsi un découpage binaire, *binary split*, et obtient en sortie deux séquences bitoniques  $B_1$  et  $B_2$  avec les éléments de  $B_1$  tous inférieurs aux éléments de  $B_2$ . Le cas terminal de la récursivité est lorsque nous avons une séquence à un seul élément (qui est donc bitonique et triée!). La figure 4.14 montre le principe de fonctionnement de cet algorithme. Un exemple de tri est illustré à la figure 4.15.

Étudions la complexité du tri *BitonicMergeSort* : (1) chaque partition bitonique coûte  $\frac{n}{2}$  comparaisons, (2) nous avons  $\log n$  niveaux récursifs de partitions bitoniques, et (3)  $\log n$  niveaux de fusions bitoniques (*bitonic merge*). Le nombre de comparaisons-échanges s’élève donc au total à  $O(n \log^2 n)$ . La figure 4.16 montre le tri bitonique implémenté avec un réseau de comparateurs.

## 4.11 \* Pour en savoir plus : notes, références et discussion

Les algorithmes classiques de tri parallèle sont décrits dans de nombreux ouvrages [60, 21]. Trier en  $O(\log n (\log \log n)^2)$  sur l’hypercube a été étudiés dans ce papier [25]. Même si trier a une complexité bien connue en  $\Omega(n \log n)$  sur le modèle real-RAM, cela peut être plus ou moins facile de trier si la séquence a des morceaux déjà partiellement triés : en pratique, on recherche donc des algorithmes adaptatifs [14] pour le tri qui prennent en compte d’autres paramètres des entrées afin d’être compétitifs et dans le pire des cas retrouver la complexité  $O(n \log n)$ .

L’opération fondamentale dans le tri est la comparaison qui à une paire d’éléments retourne sa

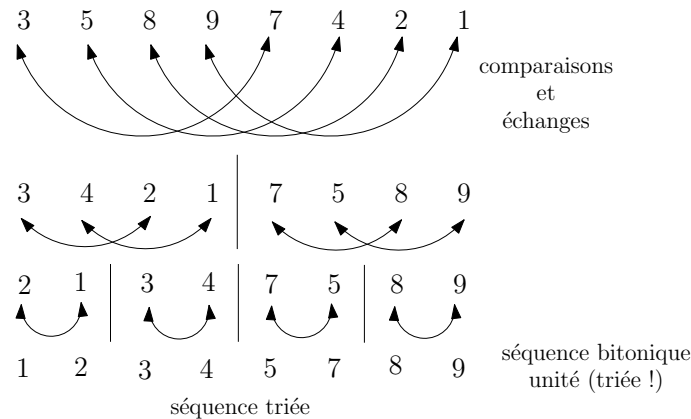


FIGURE 4.15 – Appels récurrents des découpages binaires en séquences bitoniques.

paire triée :

$$(a, b) \xrightarrow{\text{comparaison} <} (\min(a, b), \max(a, b))$$

On peut choisir la granularité en considérant  $A$  et  $B$  comme étant des paquets de données, et non plus de simples éléments. L'opérateur  $A < B$  signifie alors de trier l'ensemble  $A \cup B$  et de retourner en sortie la paire  $(A', B')$  avec  $A'$  la première moitié des éléments et  $B'$  la seconde moitié. Ainsi, on peut bâtir à partir de réseaux de tri des algorithmes parallèles en contrôlant la granularité des paquets de données pour l'opération  $<$ .

## 4.12 En résumé : ce qu'il faut retenir !

Il existe de très nombreux algorithmes pour trier  $n$  nombres en séquentiel avec une complexité optimale en  $\Theta(n \log n)$ . On peut trier sur des machines ou architectures parallèles à mémoire distribuée en considérant la granularité des tris locaux. L'algorithme QuickSort avec le choix du pivot aléatoire se parallélise tel quel mais fait apparaître un déséquilibre de charge. Pour contrecarrer ce phénomène, l'algorithme HyperQuickSort commence d'abord par trier les données avant de choisir le pivot. L'algorithme *Parallel Sort Regular Sampling* (PSRS), quant à lui, possède deux phases pour choisir plusieurs pivots en même temps dans un souci de pouvoir équilibrer les charges des processus. Le tri peut se faire aussi sur des réseaux matériels de comparateurs que l'on peut simuler sur des machines parallèles en prenant des groupes de données à la place de données élémentaires.

## 4.13 Exercices

**Exercice 7 :** Le tri ShearSort sur des groupes de données

Généraliser le tri ShearSort sur la topologie de la grille 2D en considérant des groupes de  $\frac{n}{P}$  éléments pour chaque nœud. Quelle est la complexité de cet algorithme et l'accélération obtenue ?

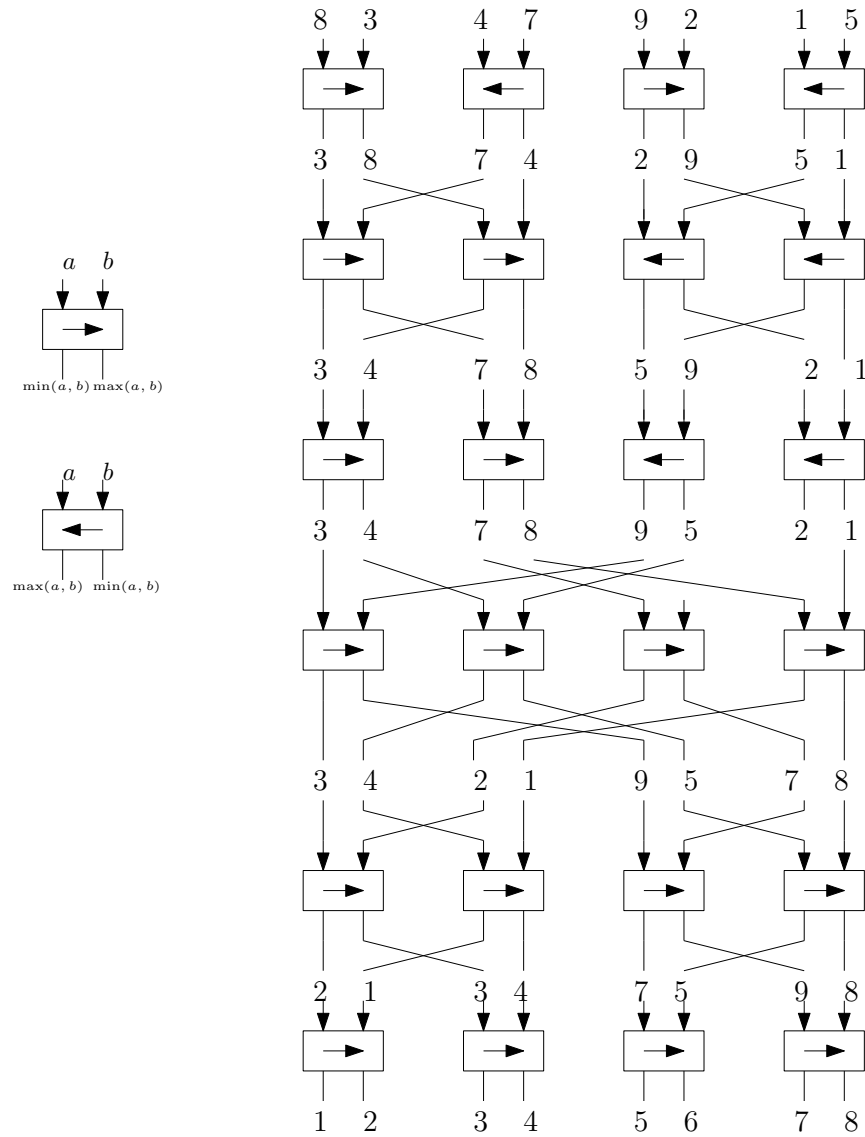


FIGURE 4.16 – Réseau de comparateurs pour le tri bitonique : le réseau est statique et ne dépend pas des données en entrée.

**Exercice 8 :** *L'algorithme HyperQuickSort en MPI*

Écrire en pseudo-code l'algorithme HyperQuickSort. Que se passe-t-il si nous supposons que seulement  $l$  éléments sont distincts parmi les  $n$  données, avec  $l \ll n$  ?



# Chapitre 5

## Algèbre linéaire en parallèle

Un résumé des points essentiels à retenir est donné dans §5.6.

### 5.1 Algèbre linéaire distribuée

#### 5.1.1 Algèbre linéaire pour la science des données

L'algorithmique de l'*algèbre linéaire* est riche et variée. En informatique, elle s'utilise de façon *omniprésente* dans de nombreux algorithmes, souvent en utilisant des bibliothèques logicielles qui cachent aux utilisateurs l'implémentation (fastidieuse puisque optimisée) des algorithmes fondamentaux. Ces bibliothèques logicielles contiennent essentiellement des opérations de produit, diverses factorisations matricielles (SVDs, etc.), et des décompositions de matrice (comme la décomposition LU ou celle de Cholesky, etc). On retrouve les techniques d'algèbre linéaire dans tous les domaines des sciences numériques. Plus particulièrement, dans les *sciences des données* (*Data Science*), on retrouve aussi cette algèbre linéaire dans les trois grandes classes de problèmes de l'apprentissage qui sont :

**Le regroupement.** On cherche parmi les données des groupes homogènes : c'est la découverte de classes (parfois encore appelé classification non-supervisée).

**La classification.** Étant donné un jeu de données d'entraînement préalablement étiqueté en classes, on cherche à classer de nouvelles données pas encore étiquetées grâce à un classifieur (prédiction d'une variable discrète).

**La régression.** Étant donné un jeu de données et une fonction sur ces données, on cherche le meilleur modèle de la fonction qui explique les données. Cela permet de pouvoir interpoler et extrapoler les valeurs de cette fonction pour de nouvelles données. En général, la régression permet d'analyser la relation d'une variable par rapport à une autre.

Ces trois problèmes fondamentaux sont décrits visuellement à la figure 5.1.

Introduisons concisément les principes de base de la régression linéaire : on cherche à prédire la valeur  $\hat{y} = f(x)$  d'une fonction à une position requête  $x$  comme  $f(x) = \hat{\beta}_0 + \sum_{i=1}^d \hat{\beta}_i x_i$ , une fonction affine représentée géométriquement par un hyperplan.

En augmentant la dimensionalité des données en rajoutant une coordonnée supplémentaire  $x_0 = 1$ , on unifie l'évaluation de la fonction par un produit scalaire : en effet, considérons  $x \leftarrow (x, 1)$

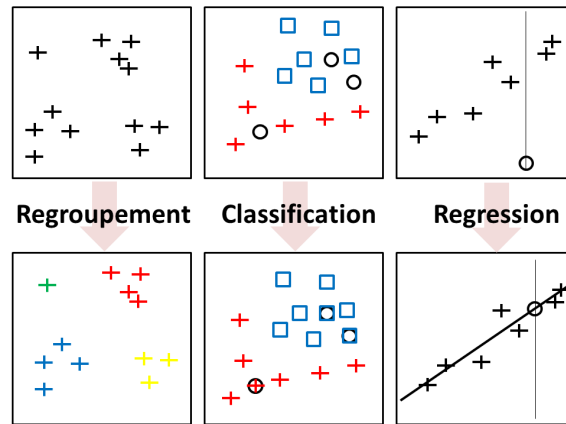


FIGURE 5.1 – Les trois piliers de l'apprentissage dans les sciences des données : le regroupement (plat ou hiérarchique), la classification et la régression.

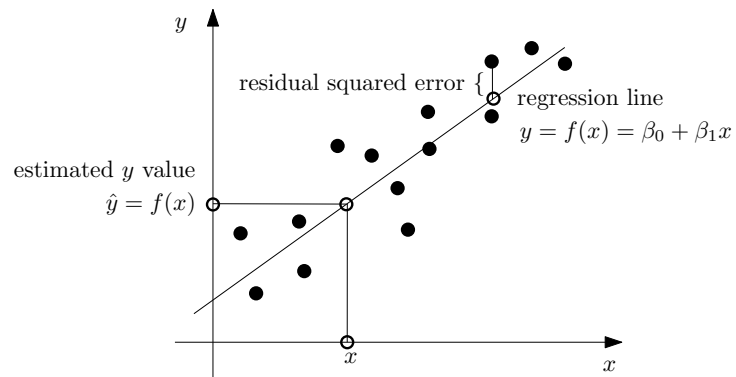


FIGURE 5.2 – La régression linéaire cherche le meilleur hyperplan (modèle linéaire) qui soutient les données (une droite en 2D) en minimisant la somme des carrés des résidus.

et  $f(x) = \sum_{i=0}^d \hat{\beta}_i x_i = x_i^\top \beta$  ( $d+1$  paramètres à évaluer). Soit un ensemble d'observations  $\{(x_1, y_1), \dots, (x_n, y_n)\} \in \mathbb{R}^d$  pour lequel on recherche le meilleur modèle linéaire en minimisant la somme des résidus au carré (*Residual Sum of Squares, RSS*) :

$$\hat{\beta} = \min_{\beta} \sum_{i=1}^n (y_i - x_i^\top \beta)^2$$

Pour la régression linéaire ordinaire, on considère la matrice des données  $X$  de dimension  $n \times (d+1)$  avec  $y$  un vecteur colonne de dimension  $n$  et  $\beta$  les  $d+1$  coefficients de l'hyperplan à estimer en minimisant la somme des résidus au carré :

$$\text{RSS}(\beta) = \sum_{i=1}^n (y_i - x_i^\top \beta)^2 = (y - X\beta)^\top (y - X\beta)$$

En prenant le gradient  $\nabla_{\beta} \text{RSS}(\beta)$  (vecteur des dérivées partielles), on obtient ce qui est communément appelée l'équation normale :

$$X^\top (y - X\beta) = 0$$

Quand la matrice  $X^\top X$  n'est pas singulière (déterminant différent de zéro), on peut estimer  $\hat{\beta}$  minimisant les moindres carrés des erreurs résiduelles en utilisant la *matrice pseudo-inverse de Penrose-Moore* :

$$\hat{\beta} = (X^\top X)^{-1} X^\top y = X^\dagger y$$

avec  $X^\dagger = (X^\top X)^{-1} X^\top$ .

Comment peut-on ou doit-on mesurer l'erreur du modèle aux données? Dans une régression linéaire ordinaire, on considère les erreurs de projections verticales, et on a vu que l'on obtenait une formule simple. Une approche différente consiste à mesurer les carrés des erreurs de projections orthogonales : c'est la régression linéaire totale. La figure 5.4 montre la différence entre la régression linéaire ordinaire et la régression linéaire totale. Toutefois, la régression linéaire totale est plus compliquée à calculer, et il n'existe pas de formule simple pour la calculer, comme c'est le cas de la régression linéaire totale.

La regression peut être utilisée pour des tâches de classification.

### 5.1.2 Algèbre linéaire classique

L'algèbre linéaire classique considère des *vecteurs colonnes* :  $v = \begin{bmatrix} v_1 \\ \vdots \\ v_l \end{bmatrix}$  et des matrices  $M =$

$\begin{bmatrix} m_{1,1} & \dots & m_{1,c} \\ \vdots & \ddots & \vdots \\ m_{l,1} & \dots & m_{l,c} \end{bmatrix}$ , à  $l$  lignes et  $c$  colonnes, carrées ou non. Il existe plusieurs types de matrices

comme les matrices denses de taille  $l \times c$  qui demandent  $O(lc)$  espace mémoire, les matrices diagonales qui requièrent  $O(l)$  mémoire, les matrices tridiagonales<sup>1</sup>, les matrices symétriques, les matrices

1. [http://fr.wikipedia.org/wiki/Matrice\\_tridiagonale](http://fr.wikipedia.org/wiki/Matrice_tridiagonale)

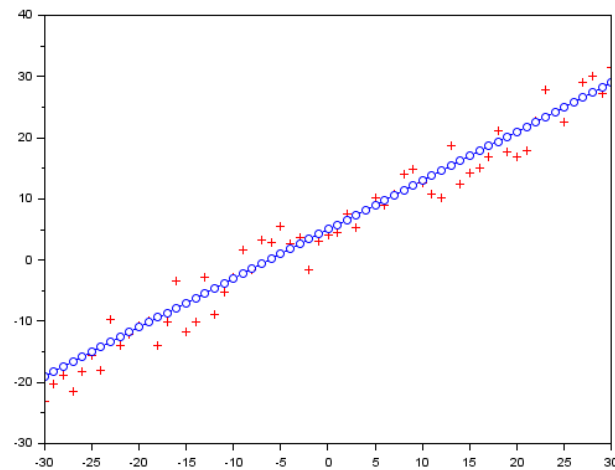


FIGURE 5.3 – Exemple d’une régression linéaire ordinaire calculée à partir d’un modèle linéaire bruité.

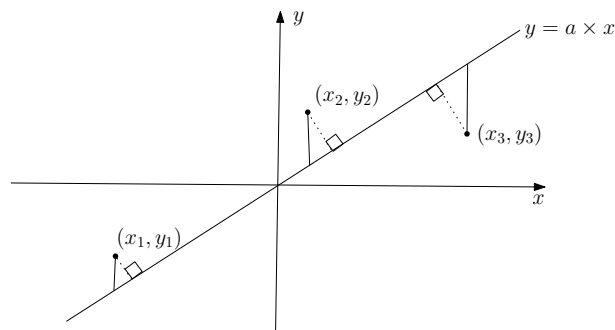


FIGURE 5.4 – Différence entre régression linéaire ordinaire et la régression linéaire totale : la régression linéaire ordinaire minimise les erreurs de projections verticales au carré tandis que la régression linéaire totale minimise les erreurs de projections orthogonales.

symétriques définies positives<sup>2</sup> (*positive definite matrices*) que l'on retrouve dans les matrices de covariance en analyse des données, les matrices triangulaires (supérieures ou inférieures), les matrices de Toeplitz<sup>3</sup>, les matrices creuses (*sparse matrix*) qui ont  $o(lc)$  entrées différentes de zéro, etc. Les vecteurs et les matrices sont des cas particuliers d'une définition plus générale que sont les *tenseurs* (avec leur algèbre multi-linéaire). Les opérations les plus basiques en algèbre linéaire sont les additions et les multiplications. Prenons  $l = c = d$  pour les dimensions des matrices carrées et des vecteurs. Le *produit scalaire* :

$$\langle u, v \rangle = \sum_{i=1}^d u^{(i)} v^{(i)} = u^\top \times v,$$

se calcule en temps linéaire  $O(d)$ , le produit matrice-vecteur  $y = Ax$  en temps quadratique ( $O(d^2)$ ), et le produit matrice-matrice  $M = M_1 \times M_2$  se calcule naïvement en temps cubique,  $O(d^3)$ . Notons que la complexité optimale du produit matrice-matrice n'est toujours pas connue. Un des premiers algorithmes battant la méthode naïve en  $O(d^3)$  est l'algorithme de Strassen<sup>4</sup> qui requiert  $O(d^{\log_2 7}) = O(n^{2,8073549221})$  multiplications. Cet algorithme utilise la décomposition de la matrice en matrices blocs et privilégie l'optimisation du nombre de multiplications (car les opérations d'addition peuvent se faire arithmétiquement plus rapidement).

De très nombreux algorithmes dont la décomposition LU (*lower upper*) sont implémentés dans la bibliothèque standardisée BLAS<sup>5</sup> (*Basic Linear Algebra Subroutines*) qui regroupe ses primitives en plusieurs niveaux hiérarchisés suivant la complexité de ses opérations. En C++, on utilise la bibliothèque `boost ublas`<sup>6</sup> pour manipuler les matrices.

Nous allons voir quelques algorithmes classiques de multiplication sur les topologies de l'anneau et du tore. Notons que même en séquentiel, on ne connaît pas d'algorithme optimal pour le calcul du produit de deux matrices puisque le meilleur algorithme connu à ce jour à une complexité en temps  $O(n^{2.3728639})$  [36] grâce à une analyse fine de l'algorithme de Coppersmith et Winograd [38]. Aussi surprenant que cela puisse paraître, la multiplication matricielle est donc un défi actuel aux enjeux cruciaux !

### 5.1.3 Le produit matrice-vecteur : $y = Ax$

Le produit matrice-vecteur calcule  $y = A \times x$  pour  $A$  une matrice carrée de taille  $d \times d$  et  $x$  un  $d$ -vecteur colonne. Les éléments du vecteur colonne  $c$  se calculent de la manière suivante :

$$y_i = \sum_{k=1}^d a_{i,k} x_k.$$

Chaque élément  $y_i$  ne dépend que de  $x$  et d'une seule ligne de  $A$ . Toutes les entrées  $y_i$  peuvent être donc calculées de façons *indépendantes*. C'est cette observation qui est à la source de la parallélisation<sup>7</sup> du produit matriciel en produits scalaires indépendants :

2. Une matrice  $M$  est définie positive ssi.  $\forall x \neq 0, x^\top M x > 0$ .

3. Matrices à diagonales constantes. Voir [http://fr.wikipedia.org/wiki/Matrice\\_de\\_Toeplitz](http://fr.wikipedia.org/wiki/Matrice_de_Toeplitz)

4. [http://fr.wikipedia.org/wiki/Algorithme\\_de\\_Strassen](http://fr.wikipedia.org/wiki/Algorithme_de_Strassen)

5. <http://www.netlib.org/blas/>

6. [http://www.boost.org/doc/libs/1\\_57\\_0/libs/numeric/ublas/doc/](http://www.boost.org/doc/libs/1_57_0/libs/numeric/ublas/doc/)

7. Idéalement, on aimerait que le compilateur se charge automatique de paralléliser du code séquentiel. Beaucoup de travaux sont effectués dans ce domaine comme la détection et traitement de nids de boucle. Toutefois, les algorithmes parallèles doivent être conçus à la base pour pouvoir tirer pleinement profit de l'architecture sous-jacente.

$$y_i = \langle a_i, x \rangle,$$

où  $a_i$ , indique la  $i$ -ème ligne de la matrice  $A$ .

Pour paralléliser le produit matrice-vecteur sur  $P$  processus à mémoire distribuée, il suffit donc d'allouer à chaque processus,  $\frac{n}{P}$  lignes de la matrice  $A$ . Ainsi, on partitionne le problème (**scatter**), puis on calcule les  $\frac{n}{P}$  produits scalaires localement sur chaque processus (qui contient les données de  $x$ ), et ensuite on combine les résultats (**reduce**) pour obtenir le vecteur  $y$ . On verra une autre technique sur l'anneau très prochainement où les données du vecteur  $x$  circulent par blocs.

Ce type de calcul produit vecteur-matrice est très bien adapté aux GPUs (avec le GPGPU). On peut faire du HPC avec le GPU mais il faut alors faire attention au format **double** de IEEE 754 qui n'est pas toujours supporté dans les cartes graphiques et pose un problème de reproductibilité des calculs.

Avant de regarder plus en détail des algorithmes de produits matriciels sur différentes topologies, regardons comment nous pouvons distribuer les données sur les processus.

#### 5.1.4 Motifs pour le parallélisme des données matricielles

Un des grands avantages du HPC est aussi de pouvoir traiter de plus grandes données en les partitionnant sur les différentes mémoires locales des processus. On cherche alors dans les algorithmes à calculer localement avec les données présentes sur le nœud et à limiter le volume de communications pour échanger les données entre les processeurs. On distingue plusieurs *motifs* de distribution des données. Par exemple, citons le *motif bloc-colonne* et le motif *bloc-colonne cyclique* où  $b$  est la largeur du bloc élémentaire, souvent prise à  $\frac{n}{P}$ . Cette répartition des données est illustrée à la figure 5.5 et est très utilisée pour les calculs sur l'anneau. On a aussi de façon similaire, le *motif bloc-ligne* et le motif *bloc-ligne cyclique* qui deviennent les motifs *bloc-colonne* et *bloc-colonne cyclique* si l'on considère la matrice transposée en entrée. Pour les topologies de la grille ou du tore, on préfère les motifs en *damier* : le *motif 2D bloc ligne-colonne* ou le motif *2D bloc ligne-colonne cyclique* (cf. la figure 5.5).

Reconsidérons le produit matrice vecteur pour des matrices denses avec le motif bloc colonne 1D. En BLAS, une opération de base est le produit matrice-vecteur avec accumulation :

$$y \leftarrow y + Ax.$$

Soit  $A(i)$ , la matrice bloc ligne de taille  $\frac{n}{P} \times n$  qui se trouve initialement sur le processeur  $P_i$ . Pour faire une opération de produit  $y = Ax$ , on effectue tout d'abord une diffusion personnalisée (*broadcast*) de  $x$ , et chaque processeur reçoit alors son sous-vecteur  $x(i)$  et peut calculer en local  $y(i) = A(i) \times x(i)$ . Une opération de communication de type rassemblement peut alors être faite pour récupérer le résultat du vecteur  $y$ .

Pour le produit matriciel parallèle, les algorithmes vont donc dépendre des motifs des données choisies, de la topologie du réseau d'interconnexion des machines et des types d'opération de communication utilisés.

Nous commençons par le produit matrice-vecteur sur l'anneau avant de regarder plusieurs algorithmes classiques pour le produit de matrices sur la grille.

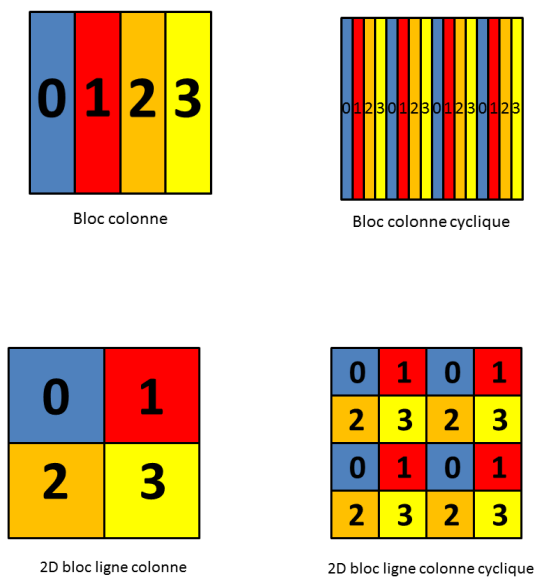


FIGURE 5.5 – Motifs pour la répartition des données sur les nœuds. Le motif peut être choisi en fonction de la topologie : bloc-colonne pour l’anneau et damier pour la grille.

## 5.2 Produit matrice-vecteur sur la topologie de l'anneau

Soit  $A$  une matrice de dimension  $(n, n)$  et  $x$  un vecteur colonne à  $n$  composantes, dont les coordonnées sont indexées de 0 à  $n - 1$  :

$$x = \begin{bmatrix} x_0 \\ \vdots \\ x_{n-1} \end{bmatrix}.$$

On désire calculer le produit matrice-vecteur  $y = A \times x$  sur un anneau constitué de  $P$  processeurs avec  $\frac{n}{P} = r \in \mathbb{N}$ . Bien que cela puisse paraître un problème assez naïf, rappelons que même en temps séquentiel, on ne connaît pas la complexité du produit de deux matrices<sup>8</sup> !

Comme nous l'avons déjà signalé, calculer le produit  $Ax$  en séquentiel revient à calculer  $n$  produits scalaires. Cela peut donc se faire en temps quadratique à l'aide de deux boucles imbriquées :

```
for (i=0; i<n; i++) {
  for (j=0; j<n; j++) {
    y[i] = y[i]+a[i][j]*x[j];
    // ou bien encore
    // y[i] += a[i][j]*x[j]
  }
}
```

On obtient ainsi une complexité quadratique  $O(n^2)$ . En *calcul vectoriel*, on fait en parallèle cette opération (SIMD) :

$$y = a[i, ]^\top x.$$

Cette opération de base est bien optimisée sur les processeurs modernes (par exemple, en utilisant le jeu d'instructions SSE d'Intel).

On peut distribuer le calcul  $Ax$  en répartissant le calcul des produits scalaires sur les  $P$  processeurs : chaque processeur  $P_i$  a en mémoire  $r = n/P$  lignes de  $A$  rangées dans une matrice  $AP$  de dimension  $r \times n$ . Le processeur  $P_i$  contient les lignes  $ir$  à  $(i+1)r - 1$  et les composantes de même rang des vecteurs  $x$  et  $y$ . Ainsi toutes les données et le résultat sont équitablement répartis sur la mémoire des nœuds. On a utilisé la partition par *bloc de lignes* de la matrice sur les mémoires locales des processus.

Illustrons maintenant le principe du calcul en prenant l'anneau simple à  $P = 2$  nœuds : en choisissant  $r = 1$ , on fait donc des calculs sur des matrices/vecteurs de dimension  $n = rP = 2$ . Le calcul  $y = Ax$  s'explicite dans ce cas comme suit :

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix} \times \begin{bmatrix} x_1 \\ x_2 \end{bmatrix},$$

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} a_{1,1}x_1 + a_{1,2}x_2 \\ a_{2,1}x_1 + a_{2,2}x_2 \end{bmatrix}.$$

Dans ce cas, on voit comment faire *tourner les données* sur l'anneau afin de faire le calcul en deux étapes comme suit :

---

8. En effet, cette complexité du produit matriciel a une borne inférieure quadratique  $\Omega(n^2)$  (le nombre d'entrées de la matrice) et on connaît "seulement" un algorithme de complexité  $O(n^{2.37\dots})$ , qui bat l'algorithme trivial en temps cubique  $O(n^3)$ .

— étape 1 :  $x_i$  se trouve sur  $P_i$  et on calcule :

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} \boxed{a_{1,1}x_1} + a_{1,2}x_2 \\ a_{2,1}x_1 + \boxed{a_{2,2}x_2} \end{bmatrix}$$

— étape 2 :  $x_i$  se trouve sur  $P_{(i+1) \bmod P}$  et on calcule :

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} a_{1,1}x_1 + \boxed{a_{1,2}x_2} \\ \boxed{a_{2,1}x_1} + a_{2,2}x_2 \end{bmatrix}$$

Dans le cas général, il s'agit donc de faire tourner les sous-vecteurs de  $x$  de taille  $\frac{n}{P} = r$  sur l'anneau, et de calculer les produits localement en accumulant les résultats sur le vecteur  $y$ .

Dans le cas général d'un anneau à  $P$  nœuds, en regroupant par paquets de taille  $\frac{n}{P} = r$ , le produit se décompose par bloc comme :

$$\begin{bmatrix} y_1 \\ \vdots \\ y_P \end{bmatrix} = \begin{bmatrix} A_1 \\ \vdots \\ A_P \end{bmatrix} \times \begin{bmatrix} x_1 \\ \vdots \\ x_P \end{bmatrix}.$$

On note  $X$  le vecteur des vecteurs blocs :  $X = \begin{bmatrix} x_1 \\ \vdots \\ x_P \end{bmatrix}$ .

A l'étape 0, on commence par initialiser  $y \leftarrow 0$ , puis on répète  $P$  fois le calcul du produit d'une sous-partie des sous-matrices de taille  $r \times r$ , et on additionne au fur et à mesure ces résultats dans  $y$ , qui joue le rôle d'*accumulateur*. La figure 5.6 illustre le procédé. Notons que les déplacements des blocs de la matrice  $A$  se font sur la topologie du tore 2D et que les déplacements des blocs du vecteur  $X$  se font sur l'anneau qui est un tore 1D.

Pour illustrer cet algorithme, prenons le cas  $n = 8$ ,  $P = 4$ , et  $r = \frac{n}{P} = 2$ . Au départ, on initialise  $y$  au vecteur zéro, et les données de la matrice  $A$  et du vecteur  $x$  sont réparties comme suit sur les processeurs :

|       |                                                                                                                                                                                                |                                            |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------|
| $P_0$ | $\begin{bmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} & a_{0,4} & a_{0,5} & a_{0,6} & a_{0,7} \end{bmatrix}$                                                                                  | $\begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$ |
| $P_1$ | $\begin{bmatrix} a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} & a_{2,5} & a_{2,6} & a_{2,7} \\ a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} & a_{3,5} & a_{3,6} & a_{3,7} \end{bmatrix}$ | $\begin{bmatrix} x_2 \\ x_3 \end{bmatrix}$ |
| $P_2$ | $\begin{bmatrix} a_{4,0} & a_{4,1} & a_{4,2} & a_{4,3} & a_{4,4} & a_{4,5} & a_{4,6} & a_{4,7} \\ a_{5,0} & a_{5,1} & a_{5,2} & a_{5,3} & a_{5,4} & a_{5,5} & a_{5,6} & a_{5,7} \end{bmatrix}$ | $\begin{bmatrix} x_4 \\ x_5 \end{bmatrix}$ |
| $P_3$ | $\begin{bmatrix} a_{6,0} & a_{6,1} & a_{6,2} & a_{6,3} & a_{6,4} & a_{6,5} & a_{6,6} & a_{6,7} \\ a_{7,0} & a_{7,1} & a_{7,2} & a_{7,3} & a_{7,4} & a_{7,5} & a_{7,6} & a_{7,7} \end{bmatrix}$ | $\begin{bmatrix} x_6 \\ x_7 \end{bmatrix}$ |

À chaque étape, on fait tourner une partie du vecteur  $x$  sur l'anneau, et les processeurs calculent leur produit matrice-vecteur bloc et additionne ce résultat à leur sous-vecteur  $y$  correspondant :

— étape 1 : calcul local matrice  $\times$  vecteur bloc :

$$\begin{array}{c}
 P_0 \\
 P_1 \\
 P_2 \\
 P_3
 \end{array}
 \begin{bmatrix}
 \underline{a}_{0,0} & \underline{a}_{0,1} & a_{0,2} & a_{0,3} & a_{0,4} & a_{0,5} & a_{0,6} & a_{0,7} \\
 \underline{a}_{1,0} & \underline{a}_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} & a_{1,5} & a_{1,6} & a_{1,7} \\
 a_{2,0} & a_{2,1} & \underline{a}_{2,2} & \underline{a}_{2,3} & a_{2,4} & a_{2,5} & a_{2,6} & a_{2,7} \\
 a_{3,0} & a_{3,1} & \underline{a}_{3,2} & \underline{a}_{3,3} & a_{3,4} & a_{3,5} & a_{3,6} & a_{3,7} \\
 a_{4,0} & a_{4,1} & a_{4,2} & a_{4,3} & \underline{a}_{4,4} & \underline{a}_{4,5} & a_{4,6} & a_{4,7} \\
 a_{5,0} & a_{5,1} & a_{5,2} & a_{5,3} & \underline{a}_{5,4} & \underline{a}_{5,5} & a_{5,6} & a_{5,7} \\
 a_{6,0} & a_{6,1} & a_{6,2} & a_{6,3} & a_{6,4} & a_{6,5} & \underline{a}_{6,6} & \underline{a}_{6,7} \\
 a_{7,0} & a_{7,1} & a_{7,2} & a_{7,3} & a_{7,4} & a_{7,5} & \underline{a}_{7,6} & \underline{a}_{7,7}
 \end{bmatrix}
 \begin{bmatrix}
 x_0 \\
 x_1 \\
 x_2 \\
 x_3 \\
 x_4 \\
 x_5 \\
 x_6 \\
 x_7
 \end{bmatrix}$$

— étape 1' : on fait tourner les sous-vecteurs  $x$  sur l'anneau dans la direction  $\downarrow$  :

$$\begin{array}{c}
 P_0 \\
 P_1 \\
 P_2 \\
 P_3
 \end{array}
 \begin{bmatrix}
 a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} & a_{0,4} & a_{0,5} & a_{0,6} & a_{0,7} \\
 a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} & a_{1,5} & a_{1,6} & a_{1,7} \\
 a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} & a_{2,5} & a_{2,6} & a_{2,7} \\
 a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} & a_{3,5} & a_{3,6} & a_{3,7} \\
 a_{4,0} & a_{4,1} & a_{4,2} & a_{4,3} & a_{4,4} & a_{4,5} & a_{4,6} & a_{4,7} \\
 a_{5,0} & a_{5,1} & a_{5,2} & a_{5,3} & a_{5,4} & a_{5,5} & a_{5,6} & a_{5,7} \\
 a_{6,0} & a_{6,1} & a_{6,2} & a_{6,3} & a_{6,4} & a_{6,5} & a_{6,6} & a_{6,7} \\
 a_{7,0} & a_{7,1} & a_{7,2} & a_{7,3} & a_{7,4} & a_{7,5} & a_{7,6} & a_{7,7}
 \end{bmatrix}
 \begin{bmatrix}
 x_6 \\
 x_7 \\
 x_0 \\
 x_1 \\
 x_2 \\
 x_3 \\
 x_4 \\
 x_5
 \end{bmatrix}$$

— étape 2 : calcul local sous-matrice  $\times$  vecteur :

$$\begin{array}{c}
 P_0 \\
 P_1 \\
 P_2 \\
 P_3
 \end{array}
 \begin{bmatrix}
 a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} & a_{0,4} & a_{0,5} & \underline{a}_{0,6} & \underline{a}_{0,7} \\
 a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} & a_{1,5} & \underline{a}_{1,6} & \underline{a}_{1,7} \\
 \underline{a}_{2,0} & \underline{a}_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} & a_{2,5} & a_{2,6} & a_{2,7} \\
 \underline{a}_{3,0} & \underline{a}_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} & a_{3,5} & a_{3,6} & a_{3,7} \\
 a_{4,0} & a_{4,1} & \underline{a}_{4,2} & \underline{a}_{4,3} & a_{4,4} & a_{4,5} & a_{4,6} & a_{4,7} \\
 a_{5,0} & a_{5,1} & \underline{a}_{5,2} & \underline{a}_{5,3} & a_{5,4} & a_{5,5} & a_{5,6} & a_{5,7} \\
 a_{6,0} & a_{6,1} & a_{6,2} & a_{6,3} & \underline{a}_{6,4} & \underline{a}_{6,5} & a_{6,6} & a_{6,7} \\
 a_{7,0} & a_{7,1} & a_{7,2} & a_{7,3} & \underline{a}_{7,4} & \underline{a}_{7,5} & a_{7,6} & a_{7,7}
 \end{bmatrix}
 \begin{bmatrix}
 x_6 \\
 x_7 \\
 x_0 \\
 x_1 \\
 x_2 \\
 x_3 \\
 x_4 \\
 x_5
 \end{bmatrix}$$

— étape 2' : on fait tourner les sous- $x$  sur l'anneau :

$$\begin{array}{c}
 P_0 \\
 P_1 \\
 P_2 \\
 P_3
 \end{array}
 \begin{bmatrix}
 a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} & a_{0,4} & a_{0,5} & a_{0,6} & a_{0,7} \\
 a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} & a_{1,5} & a_{1,6} & a_{1,7} \\
 a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} & a_{2,5} & a_{2,6} & a_{2,7} \\
 a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} & a_{3,5} & a_{3,6} & a_{3,7} \\
 a_{4,0} & a_{4,1} & a_{4,2} & a_{4,3} & a_{4,4} & a_{4,5} & a_{4,6} & a_{4,7} \\
 a_{5,0} & a_{5,1} & a_{5,2} & a_{5,3} & a_{5,4} & a_{5,5} & a_{5,6} & a_{5,7} \\
 a_{6,0} & a_{6,1} & a_{6,2} & a_{6,3} & a_{6,4} & a_{6,5} & a_{6,6} & a_{6,7} \\
 a_{7,0} & a_{7,1} & a_{7,2} & a_{7,3} & a_{7,4} & a_{7,5} & a_{7,6} & a_{7,7}
 \end{bmatrix}
 \begin{bmatrix}
 x_4 \\
 x_5 \\
 x_6 \\
 x_7 \\
 x_0 \\
 x_1 \\
 x_2 \\
 x_3
 \end{bmatrix}$$

— étape 3 : calcul local sous-matrice  $\times$  vecteur :

$$\begin{array}{c}
 P_0 \\
 P_1 \\
 P_2 \\
 P_3
 \end{array}
 \begin{array}{c}
 \left[ \begin{array}{cccccccc} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} & \underline{a}_{0,4} & \underline{a}_{0,5} & a_{0,6} & a_{0,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{1,0} & a_{1,1} & a_{1,2} & a_{1,2} & \underline{a}_{1,4} & \underline{a}_{1,5} & a_{1,6} & a_{1,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} & a_{2,5} & \underline{a}_{2,6} & \underline{a}_{2,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} & a_{3,5} & \underline{a}_{3,6} & \underline{a}_{3,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} \underline{a}_{4,0} & \underline{a}_{4,1} & a_{4,2} & a_{4,3} & a_{4,4} & a_{4,5} & a_{4,6} & a_{4,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} \underline{a}_{5,0} & \underline{a}_{5,1} & a_{5,2} & a_{5,3} & a_{5,4} & a_{5,5} & a_{5,6} & a_{5,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{6,0} & a_{6,1} & \underline{a}_{6,2} & \underline{a}_{6,3} & a_{6,4} & a_{6,5} & a_{6,6} & a_{6,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{7,0} & a_{7,1} & \underline{a}_{7,2} & \underline{a}_{7,3} & a_{7,4} & a_{7,5} & a_{7,6} & a_{7,7} \end{array} \right]
 \end{array}
 \begin{array}{c}
 \left[ \begin{array}{c} x_4 \\ x_5 \end{array} \right] \\
 \left[ \begin{array}{c} x_6 \\ x_7 \end{array} \right] \\
 \left[ \begin{array}{c} x_0 \\ x_1 \end{array} \right] \\
 \left[ \begin{array}{c} x_2 \\ x_3 \end{array} \right]
 \end{array}$$

— étape 3' : on fait tourner les sous- $x$  sur l'anneau :

$$\begin{array}{c}
 P_0 \\
 P_1 \\
 P_2 \\
 P_3
 \end{array}
 \begin{array}{c}
 \left[ \begin{array}{cccccccc} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} & a_{0,4} & a_{0,5} & a_{0,6} & a_{0,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} & a_{1,4} & a_{1,5} & a_{1,6} & a_{1,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} & a_{2,4} & a_{2,5} & a_{2,6} & a_{2,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} & a_{3,4} & a_{3,5} & a_{3,6} & a_{3,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{4,0} & a_{4,1} & a_{4,2} & a_{4,3} & a_{4,4} & a_{4,5} & a_{4,6} & a_{4,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{5,0} & a_{5,1} & a_{5,2} & a_{5,3} & a_{5,4} & a_{5,5} & a_{5,6} & a_{5,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{6,0} & a_{6,1} & a_{6,2} & a_{6,3} & a_{6,4} & a_{6,5} & a_{6,6} & a_{6,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{7,0} & a_{7,1} & a_{7,2} & a_{7,3} & a_{7,4} & a_{7,5} & a_{7,6} & a_{7,7} \end{array} \right]
 \end{array}
 \begin{array}{c}
 \left[ \begin{array}{c} x_2 \\ x_3 \end{array} \right] \\
 \left[ \begin{array}{c} x_4 \\ x_5 \end{array} \right] \\
 \left[ \begin{array}{c} x_6 \\ x_7 \end{array} \right] \\
 \left[ \begin{array}{c} x_0 \\ x_1 \end{array} \right]
 \end{array}$$

— étape 4 : calcul local sous-matrice  $\times$  vecteur :

$$\begin{array}{c}
 P_0 \\
 P_1 \\
 P_2 \\
 P_3
 \end{array}
 \begin{array}{c}
 \left[ \begin{array}{cccccccc} a_{0,0} & a_{0,1} & \underline{a}_{0,2} & \underline{a}_{0,3} & a_{0,4} & a_{0,5} & a_{0,6} & a_{0,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{1,0} & a_{1,1} & \underline{a}_{1,2} & \underline{a}_{1,3} & a_{1,4} & a_{1,5} & a_{1,6} & a_{1,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} & \underline{a}_{2,4} & \underline{a}_{2,5} & a_{2,6} & a_{2,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} & \underline{a}_{3,4} & \underline{a}_{3,5} & a_{3,6} & a_{3,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{4,0} & a_{4,1} & a_{4,2} & a_{4,3} & a_{4,4} & a_{4,5} & \underline{a}_{4,6} & \underline{a}_{4,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} a_{5,0} & a_{5,1} & a_{5,2} & a_{5,3} & a_{5,4} & a_{5,5} & \underline{a}_{5,6} & \underline{a}_{5,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} \underline{a}_{6,0} & \underline{a}_{6,1} & a_{6,2} & a_{6,3} & a_{6,4} & a_{6,5} & a_{6,6} & a_{6,7} \end{array} \right] \\
 \left[ \begin{array}{cccccccc} \underline{a}_{7,0} & \underline{a}_{7,1} & a_{7,2} & a_{7,3} & a_{7,4} & a_{7,5} & a_{7,6} & a_{7,7} \end{array} \right]
 \end{array}
 \begin{array}{c}
 \left[ \begin{array}{c} x_2 \\ x_3 \end{array} \right] \\
 \left[ \begin{array}{c} x_4 \\ x_5 \end{array} \right] \\
 \left[ \begin{array}{c} x_6 \\ x_7 \end{array} \right] \\
 \left[ \begin{array}{c} x_0 \\ x_1 \end{array} \right]
 \end{array}$$

L'algorithme pour le produit matrice-vecteur s'écrit donc simplement en pseudo-code comme suit :

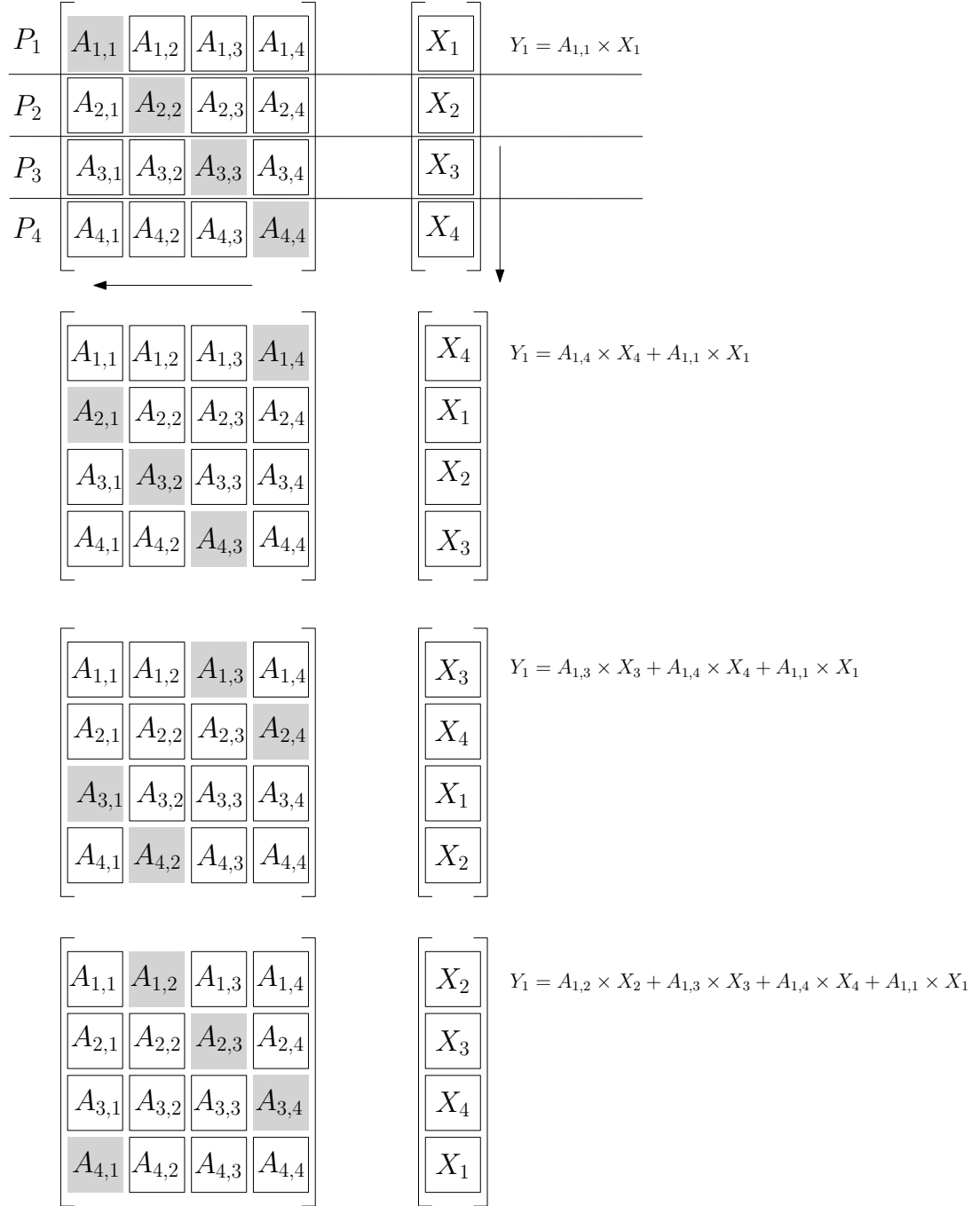
```

produitMatriceVecteur(A, x, y) {
    q = Comm_rank(); // rang du processus
    p = Comm_size(); // nombre de processus
    r = n/p; // taille des blocs

    for(step=0; step<p; step++) {
        // on envoie le bloc de x sur le prochain nœud de l'anneau
        send(x, r); // communication non-bloquante

        // calcul local : produit matrice-vecteur bloc
        for (i=0; i<r; i++) {
            for (j=0; j<r; j++) {
                y[i] = y[i] + a[i, (q-step mod p)r + j] * x[j];
            }
        }
    }
}

```

FIGURE 5.6 – Illustration du produit matrice-vecteur  $Y = A \times X$  par blocs sur l'anneau.

```

// on reçoit le bloc de x du processus précédent de l'anneau
receive(temp, r);
x = temp;
}
}

```

Analysons maintenant la complexité de cet algorithme. Soient  $u$  le temps de calcul élémentaire,  $\alpha$  le temps de latence, et  $\tau$  le débit du réseau des liens de l'anneau. On répète  $P$  étapes identiques et chaque étape dure le temps le plus long entre (1) faire le *calcul local* en  $r^2u$  et (2) transmettre/recevoir  $x$  avec un temps de communication en  $\alpha + \tau r : \max(r^2u, \alpha + \tau r)$ . Pour des grandes matrices ( $n$  grand), on a  $r^2u \gg \alpha + \tau r$  et on obtient une complexité globale en  $\frac{n^2}{P}u$ . L'efficacité de la parallélisation tend vers 1 puisque l'accélération tend vers  $P$ .

En résumé, utiliser simultanément  $P$  processeurs a permis de partitionner la matrice  $A$  en  $P$  morceaux : on voit donc que le HPC ne sert pas uniquement à aller plus vite, mais aussi à permettre de résoudre de plus gros problèmes, c'est-à-dire des volumes de données bien plus importants en les répartissant équitablement sur les différentes mémoires locales des processus (dans le cas idéal, un processus est alloué sur un processeur).

### 5.3 Produit matriciel sur la grille (*outer product algorithm*)

On présente un algorithme simple pour calculer le produit matriciel  $C = A \times B$  sur une grille de processeurs. Les matrices sont toutes carrées de dimension  $n \times n$ . Supposons que  $P = n \times n$  et que les éléments scalaires  $a_{i,j}$ ,  $b_{i,j}$  et  $c_{i,j}$  des matrices  $A$ ,  $B$  et  $C$  se trouvent dans la mémoire locale du processeur  $P_{i,j}$ . On désire calculer  $c_{i,j} = \sum_{k=1}^n a_{i,k} \times b_{k,j}$ . Le calcul du coefficient  $c_{i,j}$  va se faire en  $n$  étapes en initialisant au préalable tous les  $c_{i,j}$  à zéro.

À l'étape  $k$  (pour  $k \in \{1, \dots, n\}$ ), on utilise un mécanisme de double diffusion horizontale et verticale comme suit :

- diffusions horizontales :  $\forall i \in \{1, \dots, P\}$ , le processeur  $P_{i,k}$  diffuse *horizontalement*  $a_{i,k}$  sur la ligne  $i$ , c'est-à-dire aux processeurs  $P_{i,*}$  (tous les processeurs  $P_{i,j}$  avec  $j \in \{1, \dots, n\}$ ),
- diffusions verticales :  $\forall j \in \{1, \dots, P\}$ , le processeur  $P_{k,j}$  diffuse *verticalement*  $b_{k,j}$  sur la colonne  $k$ , c'est-à-dire aux processeurs  $P_{*,j}$  (tous les processeurs  $P_{i,j}$  avec  $i \in \{1, \dots, n\}$ ),
- calculs locaux indépendants : chaque processeur  $P_{i,j}$  peut alors calculer et mettre à jour la valeur de  $c_{i,j}$  comme suit :  $c_{i,j} \leftarrow c_{i,j} + a_{i,k} \times b_{k,j}$ .

Bien entendu, on peut faire les calculs locaux sur des matrices blocs à la place des coefficients des matrices (et obtenir une version pavée du produit matriciel). Cet algorithme est implémenté dans la bibliothèque ScaLAPACK<sup>9</sup> sous le nom de *outer product algorithm*.

### 5.4 Produit matriciel sur la topologie du tore

On considère maintenant la topologie du *tore*  $2D$  et le produit matriciel de deux matrices  $M = M_1 \times M_2$ . Soit  $\sqrt{P} \in \mathbb{N}$  le côté de la grille torique à  $\sqrt{P} \times \sqrt{P} = P$  processeurs. Chaque processeur  $P_i$  peut communiquer avec ses 4 voisins directs comme le montre la figure 5.7 : nœuds situés au Nord, au Sud, à l'Est, et à l'Ouest.

9. <http://www.netlib.org/scalapack/>

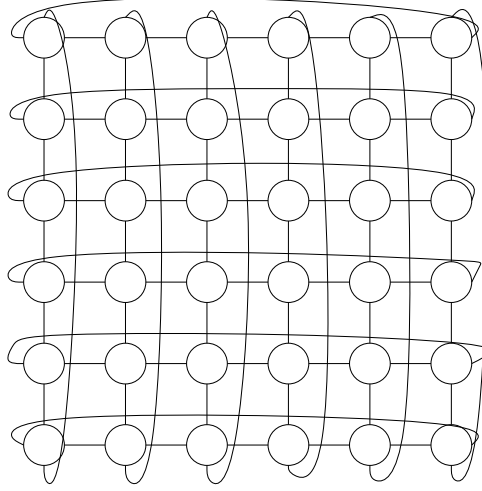


FIGURE 5.7 – Dans la topologie du tore en 2D, chaque processeur peut communiquer directement avec ses quatre voisins, notés Nord, Sud, Est, Ouest.

Sur les matrices, il est souvent utile d'introduire le *produit d'Hadamard* et le *produit de Kröneckers* qui se définissent comme suit :

— produit d'Hadamard (scalaire-scalaire) :

$$A \circ B = [A \circ B]_{i,j} = [a_{i,j} \times b_{i,j}]_{i,j},$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \circ \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} & a_{13}b_{13} \\ a_{21}b_{21} & a_{22}b_{22} & a_{23}b_{23} \\ a_{31}b_{31} & a_{32}b_{32} & a_{33}b_{33} \end{bmatrix}.$$

— Produit de Kröneckers (scalaire-bloc) :

$$A \otimes B = \begin{bmatrix} a_{11}B & \cdots & a_{1n}B \\ \vdots & \ddots & \vdots \\ a_{m1}B & \cdots & a_{mn}B \end{bmatrix}$$

Nous allons voir trois algorithmes pour le calcul du produit de matrices sur le tore : (1) l'algorithme de Cannon, (2) l'algorithme de Fox, et (3) l'algorithme de Snyder.

Mathématiquement, le produit matriciel  $C = A \times B = [c_{i,j}]_{i,j}$  doit calculer les coefficients de la matrice  $C$  comme :

$$c_{i,j} = \sum_{k=1}^n a_{i,k} \times b_{k,j} \quad \forall 1 \leq i, j \leq n.$$

On peut encore réécrire ce calcul à l'aide d'un produit scalaire :

$$c_{i,j} = \langle a_{i,\cdot}, b_{\cdot,j} \rangle,$$

avec  $a_{i,\cdot}$  le vecteur constitué de la  $i$ -ème ligne de la matrice  $A$  et  $b_{\cdot,j}$  le vecteur constitué de la  $j$ -ème colonne de la matrice  $B$ .

Seulement, afin de pouvoir faire ce calcul localement sur les processus, il faut que les données  $a_{i,k}$  et  $b_{k,j}$  soient présentes sur le processus. Initialement, les données des matrices  $A$  et  $B$  sont distribuées sur la grille par *blocs* de taille  $\sqrt{\frac{n}{P}} \times \sqrt{\frac{n}{P}}$ . Les différents algorithmes de multiplication matricielle sur le tore dépendent donc de la façon de faire ces calculs locaux en faisant *circuler les blocs* de matrices. Dans ces trois algorithmes (Cannon/Fox/Snyder), le processus  $P_{i,j}$  est responsable du calcul de  $C_{i,j} = \sum_{k=1}^{\sqrt{P}} A_{i,k} \times B_{k,j}$  où  $P$  est le nombre total de processus (côté de la grille) et  $A_{\cdot,\cdot}$  et  $B_{\cdot,\cdot}$  les matrices blocs de  $A$  et  $B$ , respectivement

### 5.4.1 L'algorithme de Cannon

Afin d'illustrer cet algorithme de multiplication matricielle, considérons cette situation de départ sur le tore 2D  $4 \times 4$  :

$$\begin{bmatrix} c_{0,0} & c_{0,1} & c_{0,2} & c_{0,3} \\ c_{1,0} & c_{1,1} & c_{1,2} & c_{1,3} \\ c_{2,0} & c_{2,1} & c_{2,2} & c_{2,3} \\ c_{3,0} & c_{3,1} & c_{3,2} & c_{3,3} \end{bmatrix} \leftarrow \begin{bmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix} \times \begin{bmatrix} b_{0,0} & b_{0,1} & b_{0,2} & b_{0,3} \\ b_{1,0} & b_{1,1} & b_{1,2} & b_{1,3} \\ b_{2,0} & b_{2,1} & b_{2,2} & b_{2,3} \\ b_{3,0} & b_{3,1} & b_{3,2} & b_{3,3} \end{bmatrix}$$

L'algorithme de Cannon nécessite des opérations de *pre-skewing*<sup>10</sup> des matrices avant les calculs locaux et des opérations de *post-skewing* (opérations inverses des opérations de pre-skewing) après ces calculs locaux. Les communications des sous-matrices blocs de  $A$  et de  $B$  correspondent à des *rotations horizontales* et à des *rotations verticales* qui sont respectivement des décalages de lignes ou de colonnes. On réalise ces communications uniquement entre voisins directs sur le tore.

Dans l'algorithme de Cannon, on commence par préparer les matrices des données  $A$  et  $B$  en faisant une étape de *preskewing* :

- matrice  $A$  : on *décale horizontalement* (*preskew*),  $A \xleftarrow{\text{skew}}$

$$\begin{bmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,1} & a_{1,2} & a_{1,3} & a_{1,0} \\ a_{2,2} & a_{2,3} & a_{2,0} & a_{2,1} \\ a_{3,3} & a_{3,0} & a_{3,1} & a_{3,2} \end{bmatrix}$$

- matrice  $B$  : on *décale verticalement* (*preskew*),  $B \uparrow \text{skew}$

$$\begin{bmatrix} b_{0,0} & b_{1,1} & b_{2,2} & b_{3,3} \\ b_{1,0} & b_{2,1} & b_{3,2} & b_{0,3} \\ b_{2,0} & b_{3,1} & b_{0,2} & b_{1,3} \\ b_{3,0} & b_{0,1} & b_{1,2} & b_{2,3} \end{bmatrix}$$

La configuration initiale sur le tore est donc la suivante :

$$\begin{bmatrix} c_{0,0} & c_{0,1} & c_{0,2} & c_{0,3} \\ c_{1,0} & c_{1,1} & c_{1,2} & c_{1,3} \\ c_{2,0} & c_{2,1} & c_{2,2} & c_{2,3} \\ c_{3,0} & c_{3,1} & c_{3,2} & c_{3,3} \end{bmatrix} = \begin{bmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,1} & a_{1,2} & a_{1,3} & a_{1,0} \\ a_{2,2} & a_{2,3} & a_{2,0} & a_{2,1} \\ a_{3,3} & a_{3,0} & a_{3,1} & a_{3,2} \end{bmatrix} \times \begin{bmatrix} b_{0,0} & b_{1,1} & b_{2,2} & b_{3,3} \\ b_{1,0} & b_{2,1} & b_{3,2} & b_{0,3} \\ b_{2,0} & b_{3,1} & b_{0,2} & b_{1,3} \\ b_{3,0} & b_{0,1} & b_{1,2} & b_{2,3} \end{bmatrix}$$

10. Difficile à traduire en français. Disons pré-déformations par décalages.

On peut donc faire les calculs locaux puisque les indices correspondent sur chaque processus :  $c_{i,j} \leftarrow c_{i,j} + a_{i,l} \times b_{l,j}$ . Puis on opère une rotation 1D sur  $A$  (on fait tourner les lignes) et une rotation 1D sur  $B$  (on fait tourner les colonnes) pour obtenir cette configuration :

$$\begin{bmatrix} c_{0,0} & c_{0,1} & c_{0,2} & c_{0,3} \\ c_{1,0} & c_{1,1} & c_{1,2} & c_{1,3} \\ c_{2,0} & c_{2,1} & c_{2,2} & c_{2,3} \\ c_{3,0} & c_{3,1} & c_{3,2} & c_{3,3} \end{bmatrix} = \begin{bmatrix} a_{0,1} & a_{0,2} & a_{0,3} & a_{0,0} \\ a_{1,2} & a_{1,3} & a_{1,0} & a_{1,1} \\ a_{2,3} & a_{2,0} & a_{2,1} & a_{2,2} \\ a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix} \times \begin{bmatrix} b_{1,0} & b_{2,1} & b_{3,2} & b_{0,3} \\ b_{2,0} & b_{3,1} & b_{0,2} & b_{1,3} \\ b_{3,0} & b_{0,1} & b_{1,2} & b_{2,3} \\ b_{0,0} & b_{1,1} & b_{2,2} & b_{3,3} \end{bmatrix}$$

Les indices correspondent une nouvelle fois et on effectue les calculs locaux, et on répète ainsi  $\sqrt{P}$  fois. À la fin des itérations, on effectue les opérations inverses de preskewing afin de retrouver la décomposition initiale par blocs des matrices  $A$  et  $B$  sur les processus du tore. L'algorithme de Cannon décrit en *pseudo-code* est donné dans l'algorithme 3.

```

Données :  $P$  processus avec la topologie du tore :  $P = \sqrt{P} \times \sqrt{P}$ . Matrices  $A, B$  stockées
par blocs sur les processus.
Résultat : Retourne le produit matriciel  $C = A \times B$ 
// Pré-traitement des matrices  $A$  et  $B$ 
// Preskew  $\leftarrow$  : éléments diagonaux de  $A$  alignés verticalement sur la première
colonne
PreskewHorizontal( $A$ );
// Preskew  $\uparrow$  : éléments diagonaux de  $B$  alignés horizontalement sur la
première ligne
PreskewVertical( $B$ );
// Initialise les blocs de  $C$  à 0
 $C = 0$ ;
pour  $k = 1$  à  $\sqrt{P}$  faire
     $C \leftarrow C + \text{ProduitsLocaux}(A, B)$ ;
    // décalage vers la gauche  $\leftarrow$ 
    RotationHorizontale( $A$ );
    // décalage vers le haut  $\uparrow$ 
    RotationVerticale( $B$ );
fin
// Post-traitement des matrices  $A$  et  $B$  : opérations inverses du
pré-traitement
// Preskew  $\rightarrow$ 
PostskewHorizontal( $A$ );
// Preskew  $\downarrow$ 
PostskewVertical( $B$ );

```

**Algorithme 3 :** Algorithme parallèle de Cannon pour le calcul du produit matriciel  $C = A \times B$  par matrices blocs.

La figure 5.8 illustre le déroulement de cet algorithme. L'algorithme de Cannon ne demande que des communications point à point entre les processeurs voisins. Les blocs de la matrice  $C$  restent, quant à eux, toujours à la même position : le processus. Les calculs locaux peuvent éventuellement être recouverts avec les opérations de communication si celles-ci sont bufferisées.

Pour vérifier que l'algorithme est bien correct, il suffit de vérifier (et se convaincre en regardant la figure 5.8) que tous les calculs locaux de matrices blocs ont eu lieu :

$$C_{i,j} = \sum_{k=1}^{\sqrt{P}} A_{i,k} \times B_{k,j} \quad \forall 1 \leq i, j \leq \sqrt{P}.$$

### 5.4.2 L'algorithme de Fox

Dans l'algorithme du produit matriciel de Fox, initialement, les données des matrices blocs  $A$  et  $B$  ne bougent pas : c'est-à-dire, qu'il n'y a pas d'opérations de pré-traitement requises. On va effectuer des *diffusions* horizontales des diagonales de  $A$  (décalées vers la droite) et des rotations verticales de  $B$ , de bas en haut. La figure 5.9 illustre les 3 diagonales d'une matrice carrée  $A$  de taille  $3 \times 3$ .

Prenons le cas de matrices  $4 \times 4$ . La configuration de départ est donc :

$$\begin{bmatrix} c_{0,0} & c_{0,1} & c_{0,2} & c_{0,3} \\ c_{1,0} & c_{1,1} & c_{1,2} & c_{1,3} \\ c_{2,0} & c_{2,1} & c_{2,2} & c_{2,3} \\ c_{3,0} & c_{3,1} & c_{3,2} & c_{3,3} \end{bmatrix} \leftarrow \begin{bmatrix} a_{0,0} & a_{0,1} & a_{0,2} & a_{0,3} \\ a_{1,0} & a_{1,1} & a_{1,2} & a_{1,3} \\ a_{2,0} & a_{2,1} & a_{2,2} & a_{2,3} \\ a_{3,0} & a_{3,1} & a_{3,2} & a_{3,3} \end{bmatrix} \times \begin{bmatrix} b_{0,0} & b_{0,1} & b_{0,2} & b_{0,3} \\ b_{1,0} & b_{1,1} & b_{1,2} & b_{1,3} \\ b_{2,0} & b_{2,1} & b_{2,2} & b_{2,3} \\ b_{3,0} & b_{3,1} & b_{3,2} & b_{3,3} \end{bmatrix}$$

On commence par diffuser la première diagonale de  $A$  (les blocs de travail pour la matrice  $A$  sont stockés dans des *mémoires tampons différentes* de celles des blocs de  $A$ ) :

$$\begin{bmatrix} c_{0,0} & c_{0,1} & c_{0,2} & c_{0,3} \\ c_{1,0} & c_{1,1} & c_{1,2} & c_{1,3} \\ c_{2,0} & c_{2,1} & c_{2,2} & c_{2,3} \\ c_{3,0} & c_{3,1} & c_{3,2} & c_{3,3} \end{bmatrix} \leftarrow \begin{bmatrix} a_{0,0} & a_{0,0} & a_{0,0} & a_{0,0} \\ a_{1,1} & a_{1,1} & a_{1,1} & a_{1,1} \\ a_{2,2} & a_{2,2} & a_{2,2} & a_{2,2} \\ a_{3,3} & a_{3,3} & a_{3,3} & a_{3,3} \end{bmatrix} \times \begin{bmatrix} b_{0,0} & b_{0,1} & b_{0,2} & b_{0,3} \\ b_{1,0} & b_{1,1} & b_{1,2} & b_{1,3} \\ b_{2,0} & b_{2,1} & b_{2,2} & b_{2,3} \\ b_{3,0} & b_{3,1} & b_{3,2} & b_{3,3} \end{bmatrix}$$

On peut donc faire tous les calculs locaux car les indices correspondent : c'est-à-dire que le deuxième indice de  $a$  correspond au premier indice de  $b$ . Puis on effectue une rotation verticale de  $B$  (décalage vers le haut  $\uparrow$ ), et on diffuse la deuxième diagonale de  $A$  pour obtenir cette configuration :

$$\begin{bmatrix} c_{0,0} & c_{0,1} & c_{0,2} & c_{0,3} \\ c_{1,0} & c_{1,1} & c_{1,2} & c_{1,3} \\ c_{2,0} & c_{2,1} & c_{2,2} & c_{2,3} \\ c_{3,0} & c_{3,1} & c_{3,2} & c_{3,3} \end{bmatrix} \xleftarrow{+} \begin{bmatrix} a_{0,1} & a_{0,1} & a_{0,1} & a_{0,1} \\ a_{1,2} & a_{1,2} & a_{1,2} & a_{1,2} \\ a_{2,3} & a_{2,3} & a_{2,3} & a_{2,3} \\ a_{3,0} & a_{3,0} & a_{3,0} & a_{3,0} \end{bmatrix} \times \begin{bmatrix} b_{1,0} & b_{1,1} & b_{1,2} & b_{1,3} \\ b_{2,0} & b_{2,1} & b_{2,2} & b_{2,3} \\ b_{3,0} & b_{3,1} & b_{3,2} & b_{3,3} \\ b_{0,0} & b_{0,1} & b_{0,2} & b_{0,3} \end{bmatrix}$$

Les indices des blocs locaux de  $A$  et de  $B$  correspondent, on peut donc faire les calculs matriciels locaux en accumulant sur les matrices blocs locaux de  $C$ , et on répète autant de fois que nous avons de lignes dans les matrices  $A$  et  $B$ . En pseudo-code, l'algorithme de Fox est décrit dans l'algorithme 4.

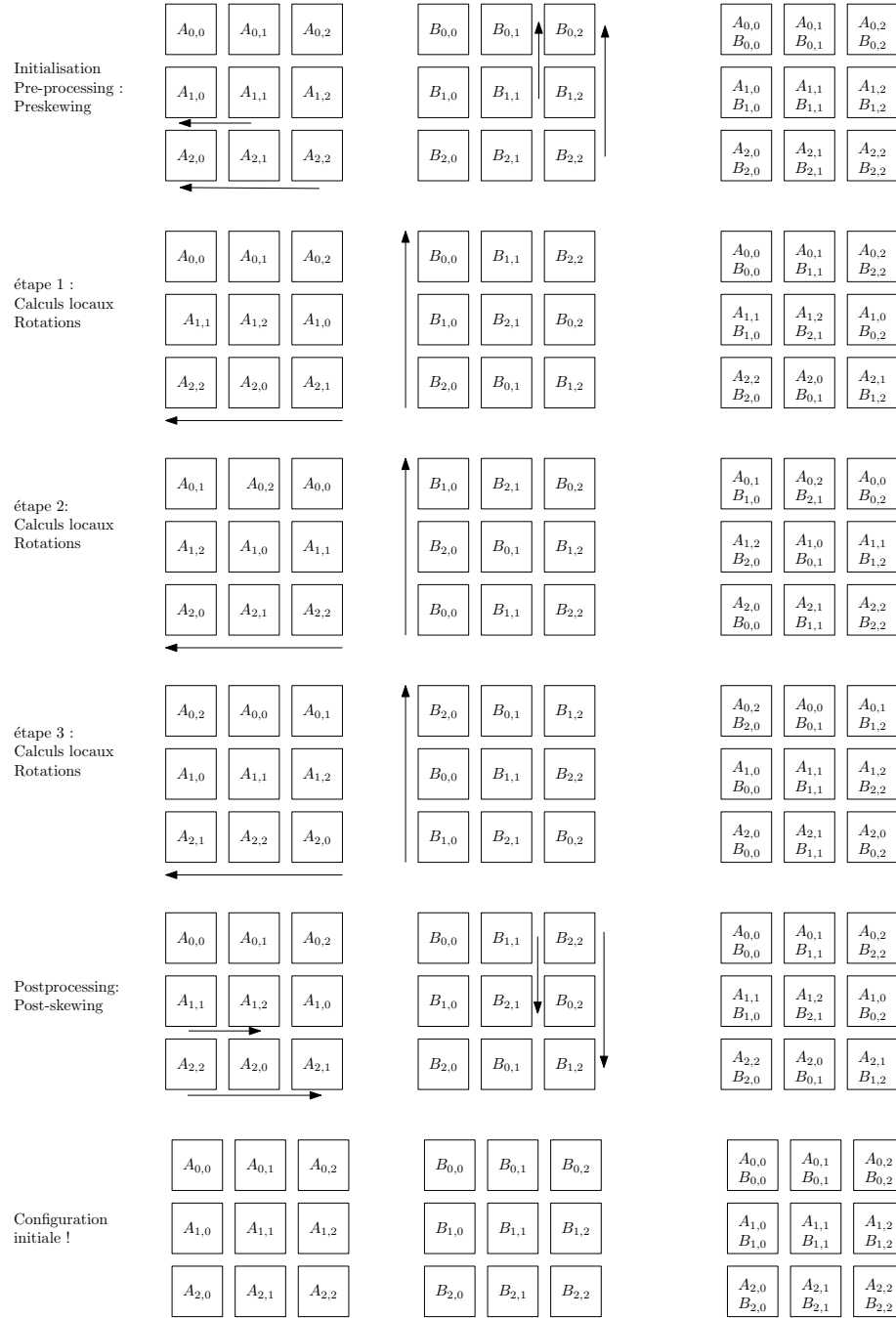
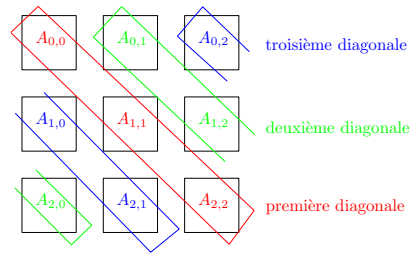


FIGURE 5.8 – Illustration de l’algorithme de Cannon : preskewing, boucle de calculs locaux et rotations, et postskewing.

FIGURE 5.9 – Les diagonales d’une matrice carrée (ici, de taille  $3 \times 3$ ).

**Données :**  $P$  processus avec la topologie du tore :  $P = \sqrt{P} \times \sqrt{P}$ . Matrices  $A, B$  stockées par blocs sur les processus.

**Résultat :** Retourne le produit matriciel  $C = A \times B$

// Initialise les blocs de  $C$  à 0

$C = 0$ ;

**pour**  $i = 1$  à  $\sqrt{P}$  **faire**

    // Broadcast

    Diffusion de la  $i$ -ième diagonale de  $A$  sur les lignes de processus du tore;

    // Multiply

$C \leftarrow C + \text{ProduitsLocaux}(A, B)$ ;

    // Roll

    // Rotation verticale : décalage vers le haut  $\uparrow$

    RotationVerticale( $B$ );

**fin**

**Algorithme 4 :** Algorithme parallèle de Fox pour le calcul du produit matriciel  $C = A \times B$  par matrices blocs.

La figure 5.10 illustre le déroulement des étapes de l’algorithme de Fox. Originellement, cet algorithme a été conçu pour l’hypercube de Caltech (US) bien que la topologie logique (ou topologie virtuelle) choisie soit le tore 2D. Cet algorithme est souvent retenu en anglais comme l’algorithme *broadcast-multiply-roll*.

### 5.4.3 L’algorithme de Snyder

Dans l’algorithme de Snyder, initialement, on commence par transposer  $B$  :  $B \leftarrow B^\top$ . On calcule les *sommes globales* sur les lignes de processeurs (cela revient à faire un produit scalaire sur les matrices blocs de  $A$  et  $B$ ), et l’accumulation des résultats se fait sur les diagonales de  $C$ , décalées à chaque étape vers la droite. On fait communiquer les données par des rotations 1D verticales, de bas en haut. L’algorithme de Snyder est résumé en pseudo-code dans l’Algorithme 5 et son déroulement à la figure 5.11.

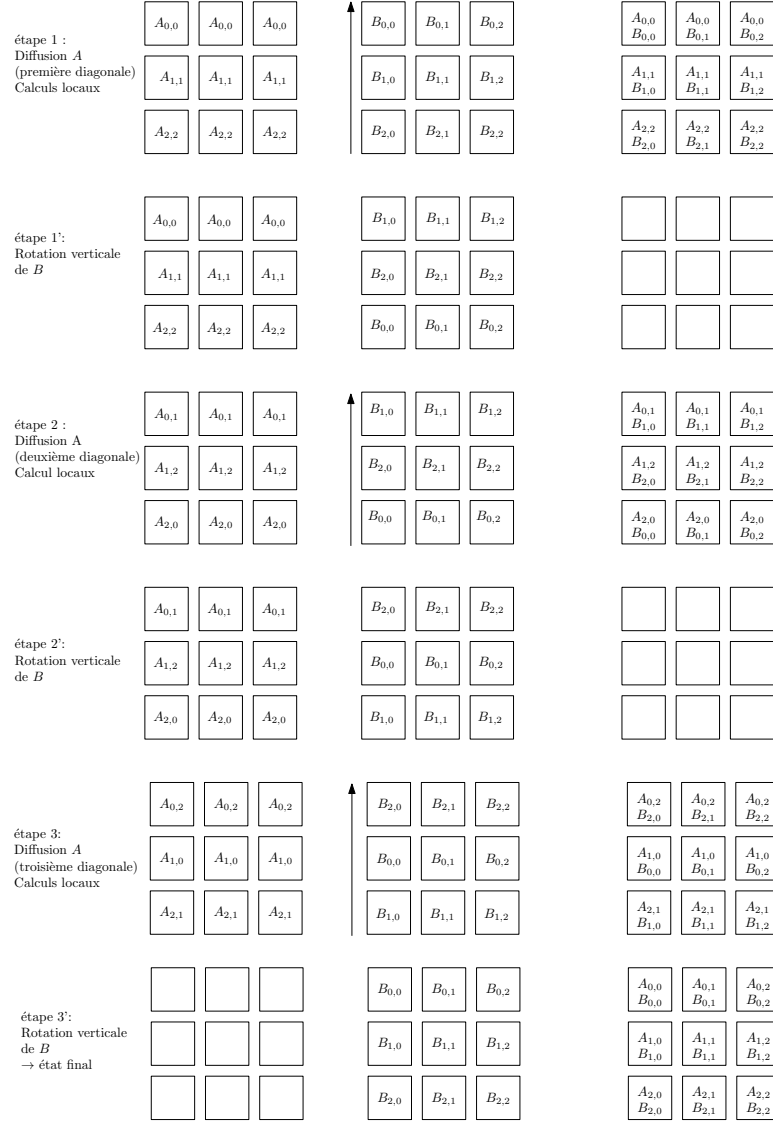


FIGURE 5.10 – Déroulement de l’algorithme de Fox (*broadcast-multiply-roll*) : pas de pré-traitement ni de post-traitement. À l’étape  $i$ , on diffuse la  $i$ -ème diagonale de  $A$ , calcule les produits locaux, et opère une rotation verticale sur  $B$ .

| Algorithme          | Cannon                      | Fox                    | Snyder                              |
|---------------------|-----------------------------|------------------------|-------------------------------------|
| pré-traitement      | preskewing de $A$ et $B$    | rien                   | transposition $B \leftarrow B^\top$ |
| produits matriciels | en place                    | en place               | $\sum$ sur les lignes PEs           |
| mouvements $A$      | gauche $\rightarrow$ droite | diffusion horizontale  | rien                                |
| mouvements $B$      | bas $\rightarrow$ haut      | bas $\rightarrow$ haut | bas $\rightarrow$ haut              |

TABLE 5.1 – Tableau comparatif des algorithmes de produit matriciel sur la topologie du tore.

```

Data :  $A, B, C$  trois matrices array[0..d-1, 0..d-1]
Result : Produit matriciel  $C = A \times B$ 
// Preskewing
Transpose  $B$ ;
// Phase de calcul
for  $k = 1$  to  $\sqrt{P}$  do
    // Produit scalaire ligne par ligne sur  $A$  et  $B$ 
    Calcule localement par bloc :  $C = A \times B$ ;
    // On calcule les matrices blocs définitives de  $C$  pour la  $k$ -ième diagonale
    // Somme globale équivaut au produit scalaire d'une ligne de  $A$  avec une
    // ligne de  $B$ 
    Somme globale  $\sum$  de  $C$  sur les processeurs lignes pour la  $k$ -ième diagonale de  $C$ ;
    Décalage vertical de  $B$ ;
end
// On transpose  $B$  afin de retrouver la matrice initiale
Transpose  $B$ ;

```

**Algorithme 5 :** Pseudo-code pour l'algorithme de Snyder (produit matrice-matrice).

#### 5.4.4 Comparatif des trois algorithmes de produit matriciel sur le tore

Nous avons vu les trois algorithmes principaux pour le produit matriciel sur le tore : l'algorithme de Cannon, l'algorithme de Fox et l'algorithme de Snyder. On donne un comparatif de ces méthodes à la Table 5.1.

### 5.5 \* Pour en savoir plus : notes, références et discussion

On recommande cet ouvrage [38] de référence pour le traitement des matrices. Le calcul matriciel peut être étendu au calcul tensoriel qui généralise les notions de vecteur et matrice. L'algorithme de Cannon [19] date de 1969, celui de Fox [34] de 1987, et celui de Snyder [61] de 1992. Le livre [21] est un ouvrage de référence pour les algorithmes parallèles et décrit les trois algorithmes de Cannon,

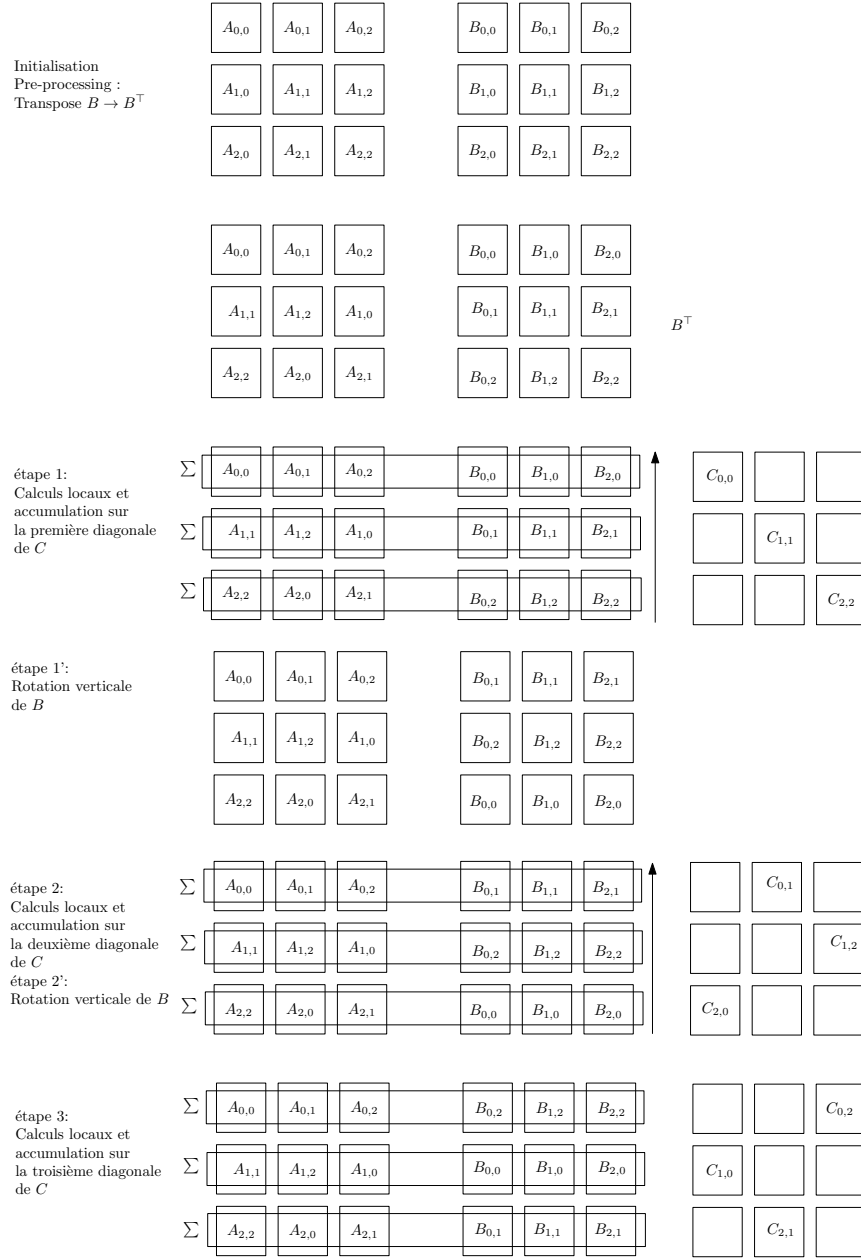


FIGURE 5.11 – Illustration de l'algorithme de Snyder pour le calcul matriciel  $C = A \times B$  : on commence par transposer  $B$  puis à la  $i$ -ième étape, on calcule tous les produits locaux et on les accumule sur chaque ligne de processus pour obtenir les coefficients de la  $i$ -ème diagonale de  $C$ , et on fait une rotation verticale sur  $B^\top$ .

Fox et Snyder pour le produit matriciel. Le calcul scientifique parallèle des matrices avec OpenMP et MPI est couvert dans le livre [65].

## 5.6 En résumé : ce qu'il faut retenir !

Les primitives de base en algèbre linéaire sont les opérations de multiplication de type matrice-vecteur ou de type matrice-matrice. Les produits matrice-vecteur et matrice-matrice peuvent se réinterpréter à l'aide de produits scalaires. En algorithmique parallèle, on suppose les données matricielles réparties initialement par bloc sur les différents nœuds et on cherche à minimiser les étapes de communication. Suivant la topologie choisie, on distribue les données matricielles soit par bloc-colonne pour l'anneau soit par damier pour le tore. Bien que le produit matriciel de matrices soit une opération fondamentale, on ne connaît toujours pas la complexité de cet algorithme, et souvent l'algorithme naïf cubique est utilisé ou parallélisé.

## 5.7 Exercices

### **Exercice 9 :** *Produits de matrices symétriques*

Comment optimiser les produits de matrices sur le tore lorsque les entrées sont des matrices symétriques ?

### **Exercice 10 :** *Produit de matrices sur l'anneau*

Proposez des algorithmes de produit matriciel sur l'anneau.



## Chapitre 6

# Le modèle de calcul MapReduce

Un résumé des points essentiels à retenir est donné dans §6.8.

### 6.1 Le défi de pouvoir traiter les BigData rapidement

Le formalisme MapReduce (ou Hadoop pour un équivalent *open source* en Java) offre un cadre simple pour paralléliser et exécuter des algorithmes. Ce paradigme de programmation parallèle pour traiter les données volumineuses (*data-intensive parallelism*) a été initialement développé par Google en 2003. En deux mots, MapReduce est un modèle *abstrait* de programmation parallèle pour les grandes données sur un cluster de machines, simple à utiliser, facilement extensible, et qui résiste aux pannes matérielles<sup>1</sup> et aux pannes de réseaux, etc. Déjà en 2007, l'entreprise Google traitait un volume de données d'environ 20 PB<sup>2</sup> par jour. Dans le domaine scientifique, la taille des données provenant des divers capteurs des instruments à haut débit (*high-throughput scientific instruments*) augmente à un rythme plus soutenu que la loi de Moore (astronomie, séquençage de génomes, etc.). Aussi, les données des fichiers logs des applications webs sur Internet sont finement analysées<sup>3</sup> (*app logs, click (stream) analysis*). Les moteurs de recherche doivent pouvoir indexer le plus rapidement possible les documents sur Internet (dont les contenus sont obtenus par un moteur de *crawling*).

Un des défis est de pouvoir traiter en *batched processing* ces BigData. De nos jours, on relève même le défi de pouvoir faire ces analyses en temps “réel” (et non plus en mode *batch*) pour les flots de données (*feeds*, comme le flot continu des *tweets* ou des réseaux sociaux). Il est donc courant que les principaux industriels du marché participent au développement de plateformes afin de répondre à leurs besoins (voir par exemple, Apache storm<sup>4</sup> pour un système de calcul temps réel distribué

---

1. Lorsqu'on utilise un cluster de quelques centaines d'ordinateurs bon marché, il n'est pas rare (voire fréquent !) d'avoir des disques durs qui lâchent ou alors des cartes réseaux sur les machines qui dysfonctionnent, etc. Il faut donc incorporer des mécanismes de redondance pour se prémunir de ces pannes.

2. 1 Peta-octet (PB) équivaut à 1024 Tera-octet (TB), et un TB vaut 1024 Giga-octet (GB). Si votre disque dur est de 1 Go, alors 20 Po c'est 20 millions de fois sa capacité...

3. Par exemple, on aime segmenter les données clicks des utilisateurs d'un site en sessions afin de savoir quel lot de pages a été visité pendant une session pour pouvoir optimiser le site (et les publicités !), détecter les communautés d'utilisateurs, etc.

4. <https://storm.apache.org/>

ou encore Apache Spark<sup>5</sup> pour un système de traitement sur les flots de données, Hive<sup>6</sup> pour un langage à la SQL pour faire des requêtes dans de grandes bases de données, etc.).

Pour pouvoir se représenter un ordre de grandeur, signalons qu'un péta-octet (Po, ou PB en anglais) permet de stocker environ 10 milliards de photos (Facebook, Flickr, Instagram, etc.) ou encore 13 années de vidéo HD (YouTube, DailyMotion, etc.). Sur un PC standard, il faudrait plus de sept années pour traiter ces 20 Po de données en temps linéaire. Il faut donc développer des algorithmes parallèles avec des temps de calculs parallèles sous-linéaires! (*sub-linear time*). MapReduce permet aussi facilement de pouvoir traiter des téra-octets (To) sur de petits clusters d'une dizaine de machines (cadre d'une PME).

## 6.2 Le principe de base de MapReduce

### 6.2.1 Processus *mappers* et processus *reducers*

Bien que MapReduce soit un modèle de programmation parallèle extrêmement simple, il offre néanmoins une grande richesse et souplesse en termes d'applications. En effet, beaucoup de problèmes peuvent être résolus en deux étapes fondamentales :

1. étape 1 (MAP) : on mappe une fonction sur une séquence de données et on produit de nouveaux éléments associés à des clefs,
2. étape 2 (REDUCE) : on réduit les nouveaux éléments qui ont la même clef ensemble.

Par exemple, pour calculer les occurrences des mots dans un grand corpus de documents, à chaque mot  $w_i$  du corpus de textes, on associe la paire  $w_i \mapsto (k_i, v_i)$  où  $k_i = k(w_i) = w_i$  est la clef pour la donnée  $w_i$  et  $v_i = v(w_i) = 1$  la valeur de la fonction évaluée (une occurrence par mot), puis on réduit par groupe de mots  $G(k) = \{w_i \mid k(w_i) = k\}$  ayant la même clef  $k$  en calculant la somme cumulée :  $r(k) = \sum_{w \in G(k)} v(w)$ . On remarque que la fonction **map** peut se calculer en parallèle par des *processus indépendants* sur les données préalablement distribuées sur les machines d'un cluster. Par contre, les processus **reduce** qui prennent en entrée le résultat des valeurs calculées par **map** ne sont pas indépendants. La procédure **map** transforme les données initiales en données intermédiaires : des paires (clef;valeur).

Puisque nous manipulons de grands jeux de données, les fonctions **map** et **reduce** vont être implémentées par des processus sur le cluster de machines. Le parallélisme de MapReduce est donc un parallélisme à *granularité fine* (les calculs locaux sont élémentaires), et les processus qui implémentent la fonction **map** sont appelés *mappers* tandis que ceux qui implémentent la fonction **reduce** sont appelés *reducers*.

### 6.2.2 \* Les fonctions *map* et *reduce* dans les langages fonctionnels

Ces deux notions de **map** et **reduce** existaient déjà en *programmation fonctionnelle*. Par exemple en Lisp (Common Lisp/Scheme) ou en OCaml<sup>7</sup>, on peut programmer simplement ces deux opérations comme suit :

— **map** :

$$(x_1, \dots, x_n) \xrightarrow{f} (f(x_1), \dots, f(x_n)),$$

5. <https://spark.apache.org/streaming/>

6. <https://hive.apache.org/>

7. <http://caml.inria.fr/ocaml/>

— **reduce** :

$$(x_1, \dots, x_n) \xrightarrow{r(\sum)} \sum_{i=1}^n x_i.$$

Pour le **reduce**, on requiert un *opérateur binaire* qui peut être commutatif (comme la somme) ou non (comme le produit de matrices).

En Lisp<sup>8</sup>, ces deux fonctions sont appelées comme suit :

```
— map
CL-USER > (mapcar #'sqrt '(3 4 5 6 7))
(1.7320508 2.0 2.236068 2.4494899 2.6457513)

— reduce
CL-USER > (reduce #'+ '(1 2 3 4 5))
15
```

En OCaml, on implémente le **map** comme un opérateur unaire :

```
# let square x=x*x;;
val square : int -> int = <fun>
# let mapliste = List.map square;;
val mapliste : int list -> int list = <fun>
# mapliste [4;4;2];;
- : int list = [16; 16; 4]
#
```

De même, le **reduce** en OCaml à la syntaxe suivante pour un opérateur binaire  $f$  :

```
fold_right f [e1;e2; ... ;en] a = (f e1 (f e2 (f en a)))
fold_left f a [e1;e2; ... ;en]=(f .. (f (f a e1) e2) ... en)
```

Exemple :

```
List.fold_left ( + ) 0 [1;2;3;4] ;;
List.fold_right ( + ) [1;2;3;4] 0 ;;
```

Puisqu'en programmation fonctionnelle on n'utilise pas les variables des langages impératifs (C/C++/Java), on peut donc facilement paralléliser ces opérations. MapReduce fournit en plus des outils pour le contrôle et le monitoring des tâches MapReduce

## 6.3 Typage et le méta-algorithme MapReduce

Dans le formalisme de MapReduce, les primitives **map** et **reduce** sont typées<sup>9</sup> comme suit :

— Mapper :

$$\boxed{\text{map}(k_1, v_1) \rightarrow \text{list}(k_2, v_2)}$$

8. On peut télécharger le Common Lisp à <http://www.lispworks.com/>

9. Le typage dans les langages informatiques a été introduit dans les cours du Tronc Commun, en INF311 [72] et INF321. Un type définit un domaine de valeurs que peut prendre une donnée. Par exemple, on parle de type booléen (**boolean**), de type entier stocké sur 32 bits (**int**), de type réel au format IEEE 754 simple précision (**float**), de type réel au format IEEE 754 double précision (**double**), etc.

— Reducer :

$$\text{reduce}(k_2, \text{list}(v_2)) \rightarrow \text{list}(v_2)$$

Une étape de MapReduce comporte trois phases :

1. *Mapper* : émet à partir des entrées des paires (clef,valeur),
2. *Sorter* : regroupe les paires intermédiaires par la valeur de leur clef,
3. *Reducer* : sur les clefs intermédiaires, on applique un *calcul de préfixe* (réduction, accumulation, agrégation) sur toutes les valeurs associées à une même clef, et ceci pour toutes les clefs.

Seules les étapes *mapper* et *reducer* dépendent des fonctions définies par l'utilisateur (*User-defined functions*, UDFs). La phase *sorter* est gérée automatiquement par le système MapReduce.

L'algorithme MapReduce est donc simple puisqu'il applique le mapper sur les données en entrées, regroupe les listes de résultats dans une liste de paires (clef2,valeur2), ré-arrange la liste en une liste de paires (clef2, liste de valeur2) pour chaque valeur distincte de clef2, puis appelle le reducer sur chaque élément de la nouvelle liste, et agrège les résultats.

La figure 6.1 illustre ces trois phases fondamentales de MapReduce.

On donne deux autres exemples de parallélisation par MapReduce :

- le **grep** distribué<sup>10</sup>. En Unix, on compte en ordre décroissant toutes les lignes qui matchent une expression régulière dans les fichiers d'un répertoire avec la syntaxe suivante :  
`grep -Eh regularexpression repertoire/* | sort | uniq -c | sort -nr`  
 Par exemple, c'est utile pour analyser les fichiers de logs d'un site web pour connaître les pages les plus vues qui matchent une expression régulière donnée. Avec le formalisme, MapReduce, on peut faire ce grep comme ceci :
  - **map** : émet la valeur 1 si une ligne correspond au motif de l'expression régulière,
  - **reduce** : fonction somme cumulée (opérateur binaire associatif "+").
- liste inversée des références du web :
  - **map** : émet des paires (target,source) pour chaque lien sur une URL target trouvé dans une page source,
  - **reduce** : concatène toutes les URLs associées à une URL donnée.

Afin d'être général, MapReduce considère que les valeurs sont des chaînes de caractères.

## 6.4 Un exemple complet de programme MapReduce en C++

On présente le programme complet pour compter les mots dans les documents. Tout d'abord, on écrit les deux fonctions **map** et **reduce** en pseudo-code comme suit en notant que les entrées, valeurs et sorties sont toutes encodées dans des chaînes de caractères :

```
Fonction utilisateur map. map(String input_key, String input_value)

  for each word w in input_value
    EmitIntermediate(w, "1");
```

10. <http://wiki.apache.org/hadoop/Grep>

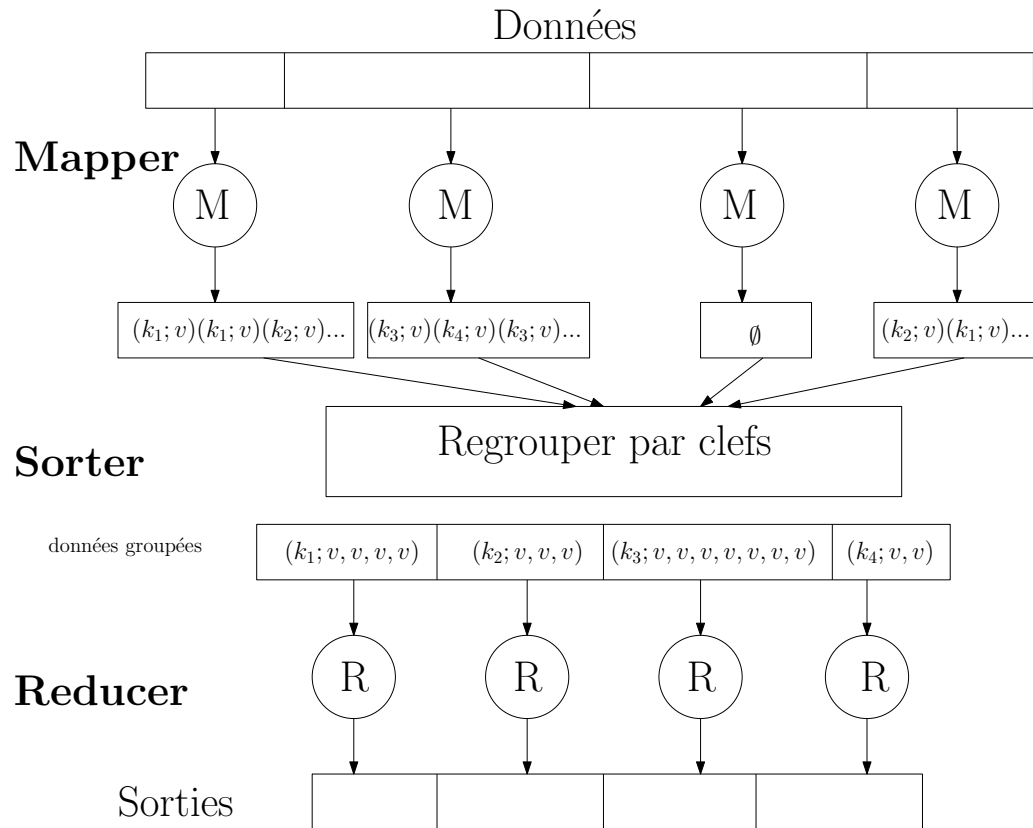


FIGURE 6.1 – Modèle d'exécution de MapReduce en trois phases : (1) mapper, (2) sorter et (3) reducer.

**Fonction utilisateur reduce.**

```

    reduce(String output_key, Iterator intermediate_values)

    int result = 0;

    for each v in intermediate_values
        {result += ParseInt(v);}

    Emit(AsString(result));

```

Pour information, le code complet correspondant en MapReduce C++ est donné ci-dessous. La fonction  $f$  pour mapper les données est la suivante (processus `map`) :

```

#include "mapreduce/mapreduce.h"

// fonction utilisateur pour mapper
// User Defined Function (UDF)

class WordCounter :
public Mapper {
public:
    virtual void Map(const MapInput& input) {
        const string& text = input.value();
        const int n = text.size();

        for (int i = 0; i < n; ) {
            // Skip past leading whitespace
            while ( (i < n) && isspace(text[i]))
                i++;

            // Find word end
            int start = i;
            while ( (i < n) && !isspace(text[i]))
                i++;
            if (start < i)
                EmitIntermediate(text.substr(start, i-start), "1");
        }
    }
};

REGISTER_MAPPER(WordCounter);

```

Le code pour la partie *Reducer* est :

```

// fonction utilisateur de réduction

class Adder :
public Reducer {
    virtual void Reduce(ReduceInput* input) {
        // Iterate over all entries with the
        // same key and add the values
        int64 value = 0;
        while (!input->done ()) {
            value += StringToInt(input->value());
            input->NextValue();
        }
    }
};

```

```

        // Emit sum for input->key()
        Emit(IntToString(value));
    }
};

```

```
REGISTER_REDUCER(Adder);
```

On peut alors utiliser ces fonctions dans un programme MapReduce comme suit :

```

int main(int argc, char** argv) {
    ParseCommandLineFlags(argc, argv);
    MapReduceSpecification spec;

    // On indique les listes de fichiers en entrée dans "spec"
    for (int i = 1; i < argc; i++) {
        MapReduceInput* input = spec.add_input();
        input->set_format("text");
        input->set_filepattern(argv[i]);
        input->set_mapper_class("WordCounter");
    }

    // On définit les sorties :
    MapReduceOutput* out = spec.output();
    out->set_filebase("/gfs/test/freq");
    out->set_num_tasks(100);
    out->set_format("text");
    out->set_reducer_class("Adder");

    // Optionnel : on optimise localement en faisant
    // les sommes partielles (combine) sur chaque processus
    out->set_combiner_class("Adder");

    // On fixe divers paramètres
    spec.set_machines(2000);
    spec.set_map_megabytes(100);
    spec.set_reduce_megabytes(100);

    // Now run it
    MapReduceResult result;

    if (!MapReduce(spec, &result)) abort();
    return 0;
}

```

## 6.5 Modèle d'exécution de MapReduce et architecture

Le système MapReduce fonctionne sur un cluster de machines avec une très grande tolérance aux différents types de panne<sup>11</sup> qui peuvent survenir. En effet, lorsqu'on utilise plusieurs milliers voire plusieurs centaines de milliers<sup>13</sup> de machines, il est *fréquent* de rencontrer des pannes comme une machine ou un disque dur (*HDD*) qui tombe subitement en panne, ou alors le réseau qui devient très lent sur une partie des machines (congestion du trafic, pannes de cartes réseaux), etc.

11. La philosophie de Google est d'utiliser beaucoup d'ordinateurs bons marchés et de concevoir dès le départ des systèmes robustes aux pannes en implémentant des systèmes de redondance (par exemple, le système de fichiers GFS inspiré du célèbre système de fichiers en Unix, le NFS<sup>12</sup>).

13. Ces chiffres sont bien gardés secrètement dans l'industrie

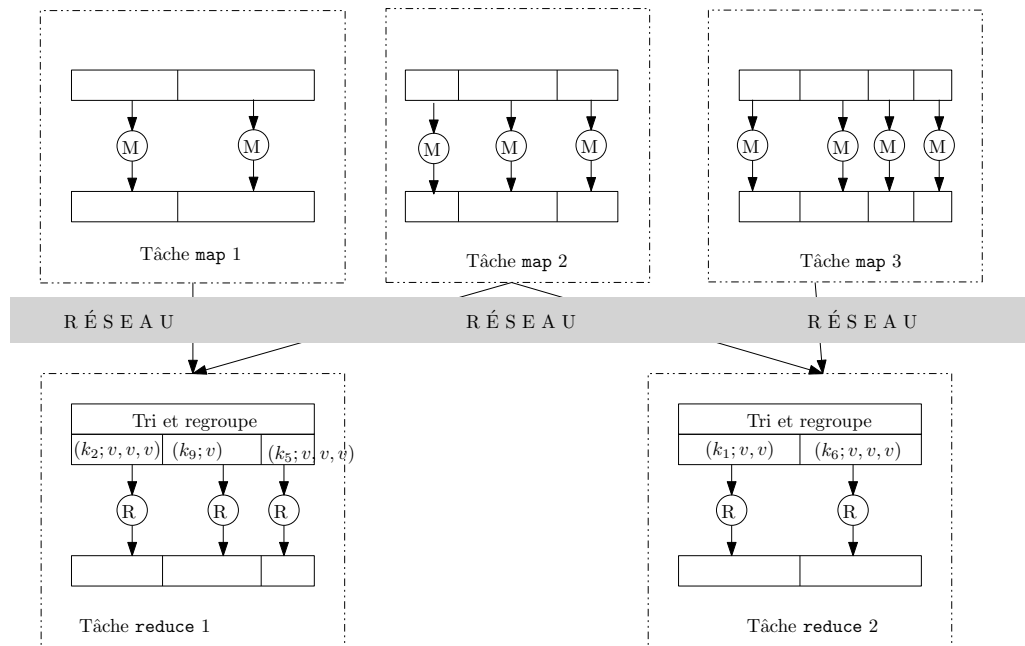


FIGURE 6.2 – Modèle d'exécution de MapReduce : Les données et les processus **map** sont indépendamment alloués par MapReduce sur des processus qui utilisent localement les données. Les processus **reduce** récupèrent les paires (clef,valeur) pour calculer l'opération de réduction.

Certaines machines peuvent aussi être trop lentes dues à leurs surcharges : on les appelle en anglais des *stragglers*. MapReduce est tolérant aux pannes et implémente une architecture de type maître-esclave (*master/worker architecture*). MapReduce relancera automatiquement les tâches qui n'ont pas pu aboutir avant un certain temps (en implémentant un mécanisme de *time-out*). La figure 6.2 illustre l'exécution des processus **map** et **reduce**. Afin d'être plus rapide, l'ordonnanceur de tâches de MapReduce peut aussi décider de lancer une même tâche sur plusieurs machines (*redondance par réplication*) afin d'obtenir le résultat dès qu'une des tâches aura fini.

Le transfert de données sur le réseau étant coûteux (latence incompressible et débit borné), l'ordonnanceur de MapReduce alloue les tâches de préférence sur les machines où résident les données. MapReduce offre aussi une interface web qui permet de visualiser diverses statistiques sur les tâches exécutées, prévoit la durée d'exécution des tâches en cours, etc. C'est le côté surveillance et contrôle (*monitoring*) des activités MapReduce.

Puisqu'on a une architecture maître-esclave, on est vulnérable si la machine maître tombe en panne. Aussi, le nœud Master écrit périodiquement les structures de données du Master afin de pouvoir reprendre le calcul en cours si celui-ci devait tomber lui-même en panne. Les données sont stockées sur le *Google File System* (GFS) qui divise les fichiers en blocs de 64MB et sauvegarde plusieurs copies de ces morceaux de fichiers sur des machines différentes afin d'être robuste et tolérant aux pannes. MapReduce bénéficie aussi de plusieurs optimisations. Par exemple, le **combiner** combine localement les clefs intermédiaires identiques sur une machine afin de diminuer le trafic sur le réseau. Notons que quelques adaptations sont parfois nécessaires au niveau du combiner : par

exemple, si on calcule la moyenne, le **combiner** ne doit pas renvoyer seulement la moyenne, mais aussi le nombre d'éléments.

Le succès de MapReduce est principalement dû à une parallélisation automatique et à une distribution des tâches de calcul. L'utilisateur a seulement besoin de définir deux fonctions : **map** et **reduce**. MapReduce distribue les données, se charge de l'équilibrage des tâches (*load balancing*), et offre un cadre de calcul parallèle très tolérant aux pannes. Son abstraction simple et propre permet d'implémenter de nombreuses applications comme le tri, des algorithmes parallèles en apprentissage et en *fouille de données* (*data mining*), etc. De plus, le système est fourni comme une solution complète avec des outils de contrôle pour le *monitoring* qui permettent de visualiser et d'ajuster les ressources allouées aux différentes tâches si nécessaire en cours d'exécution.

## 6.6 \* Utiliser le paradigme MapReduce en MPI avec MR-MPI

On a expliqué le modèle de calcul parallèle MapReduce. On peut aussi implémenter un algorithme MapReduce avec MPI. Mieux encore, il existe déjà une bibliothèque qui nous mûche le travail : MR-MPI.<sup>14</sup> La documentation en ligne se trouve ici.<sup>15</sup> Par exemple, la méthode **collate** agrège sur tous les processus un objet de type **KeyValue** et le convertit en objet **KeyMultiValue**. Cette méthode renvoie le nombre total de paires uniques (key,value).

Le programme ci-dessous montre comment calculer le nombre d'occurrences des mots d'un ensemble de fichiers. Il affiche en sortie les 2015 mots les plus fréquents.

```
#include "stdio.h"
#include "stdlib.h"
#include "string.h"
#include "sys/stat.h"

#include "mpi.h"

#include "mapreduce.h"
#include "keyvalue.h"

using namespace MAPREDUCE_NS;

void fileread(int, KeyValue *, void *);
void sum(char *, int, char *, int, int *, KeyValue *, void *);
int ncompare(char *, int, char *, int);
void output(int, char *, int, char *, int, KeyValue *, void *);

struct Count {int n, limit, flag;};

/* Syntaxe : wordfreq file1 file2 */

int main(int narg, char **args)
{
    MPI_Init(&narg, &args);
    int me, nprocs;

    MPI_Comm_rank(MPI_COMM_WORLD, &me);
```

14. <http://mapreduce.sandia.gov/>

15. <http://mapreduce.sandia.gov/doc/Manual.html>

```

    MPI_Comm_size(MPI_COMM_WORLD, &nprocs);

    MapReduce *mr = new MapReduce(MPI_COMM_WORLD);

    int nwords = mr->map(narg-1, &fileread, &args[1]);
    mr->collate(NULL);
    int nunique = mr->reduce(&sum, NULL);
    mr->sort_values(&ncompare);

    Count count;
    count.n = 0;
    count.limit = 2015;
    count.flag = 0;
    mr->map(mr->kv, &output, &count);
    mr->gather(1);
    mr->sort_values(&ncompare);

    count.n = 0;
    count.limit = 10;
    count.flag = 1;
    mr->map(mr->kv, &output, &count);

    delete mr;
    MPI_Finalize();
}

/* Pour chaque mot, emission (key=word,valeur=NULL) */
void fileread(int itask, KeyValue *kv, void *ptr)
{
    char **files = (char **) ptr;

    struct stat stbuf;
    int flag = stat(files[itask], &stbuf);
    int filesize = stbuf.st_size;

    FILE *fp = fopen(files[itask], "r");
    char *text = new char[filesize+1];
    int nchar = fread(text, 1, filesize, fp);
    text[nchar] = '\0';
    fclose(fp);

    char *whitespace = " \t\n\f\r\0";
    char *word = strtok(text, whitespace);

    while (word) {
        kv->add(word, strlen(word)+1, NULL, 0);
        word = strtok(NULL, whitespace);
    }

    delete [] text;
}

/* emet des paires (key=mot, valeur=nombre d'occurences) */
void sum(char *key, int keybytes, char *multivalue,
int nvalues, int *valuebytes, KeyValue *kv, void *ptr)
{
    kv->add(key, keybytes, (char *) &nvalues, sizeof(int));
}

```

```

}

/* fonction de comparaison pour le tri */

int ncompare(char *p1, int len1, char *p2, int len2)
{
    int i1 = *(int *) p1;
    int i2 = *(int *) p2;
    if (i1 > i2) return -1;
    else if (i1 < i2) return 1;
    else return 0;
}

/* affiche en sortie les mots selectionnes */

void output(int itask, char *key, int keybytes, char *value,
int valuebytes, KeyValue *kv, void *ptr)
{
    Count *count = (Count *) ptr;
    count->n++;
    if (count->n > count->limit) return;

    int n = *(int *) value;
    if (count->flag) printf("%d %s\n", n, key);
    else kv->add(key, keybytes, (char *) &n, sizeof(int));
}

```

## 6.7 \* Pour en savoir plus : notes, références et discussion

On peut installer MapReduce sur sa propre machine en installant une *machine virtuelle*<sup>16</sup>. L'utilisation d'Hadoop en Java est décrite dans le cours INF431 [29] (avec le système de fichiers, HDFS). L'implémentation de MapReduce dépend aussi de l'environnement comme les machines multi-cœurs à mémoire partagée, les multiprocesseurs NUMA (*Non-uniform memory access*), les architectures à mémoire distribuée sur un réseau d'interconnexion, etc. Par exemple, une implémentation de MapReduce<sup>17</sup> est disponible en MPI, et son implémentation efficace est discutée dans ce papier [48]. Des exemples d'applications de MapReduce en MPI sur les graphes sont discutés dans ce papier [80]. On peut utiliser Hadoop [29] sur sa propre machine (pas besoin d'avoir un cluster) en mode *single node* pour compiler et déboguer ses programmes avant de les déployer en production sur des clusters. On peut définir et louer des configurations de clusters sur les différentes plateformes en nuages (*cloud computing*) académiques (Grid 5000, etc.) ou industrielles comme celles d'Amazon (EC2), de Microsoft Azure, de Google, etc. Lorsque les algorithmes nécessitent d'accéder aux données qui ne peuvent pas toutes tenir dans la mémoire vive principale, on parle de traitement avec des données externes (*out-of-core processing*, comme par exemple la visualisation de données massives externes, *out-of-core visualization*). Dans ce cadre, on s'intéresse notamment aux algorithmes qui considèrent les données arrivant en flots continus. Les données sont alors lues et traitées en une ou plusieurs passes (*streaming algorithms* [53]).

16. <http://www.thecloudavenue.com/2013/01/virtual-machine-for-learning-hadoop.html>

17. <http://mapreduce.sandia.gov/>

## 6.8 En résumé : ce qu'il faut retenir !

MapReduce est un paradigme de calcul en parallèle pour traiter les BigData sur des grands clusters. Un programme MapReduce inclut deux fonctions définies par l'utilisateur **map** et **reduce** qui opèrent sur des paires (clef,valeur). Le système MapReduce exécute un programme en parallèle en trois étapes : (1) une étape dite *mapper* qui part des données pour émettre des paires (clef,valeur) en utilisant la fonction utilisateur **map**, (2) une étape du système appelée *sorter* qui a pour but de regrouper les paires (clef,valeur) en groupes de clefs, et (3) une étape dite *reducer* qui effectue une réduction en agrégeant toutes les valeurs pour une même clef par une deuxième fonction **reduce** définie par l'utilisateur. Le système MapReduce repose sur une architecture maître-esclave, et s'occupe d'allouer les différentes ressources en gérant la localité des données, et distribuer les données, d'exécuter les processus **map** (tous indépendants) et **reduce** en fonction de l'état des ressources (charge CPU, trafic réseau, etc.). MapReduce fournit également des outils pour le contrôle des tâches et sauvegarde périodiquement l'état de la machine maître en cas de panne de cette dernière machine. Afin de diminuer le trafic réseau, une étape d'optimisation optionnelle qui porte le nom de *combiner* permet d'effectuer les réductions localement sur les machines avant d'envoyer les données résultats du mapper. Le paradigme MapReduce a été originellement développé en C++ par Google en s'appuyant sur le système de fichiers parallèle GFS. Une implémentation en logiciel libre est connue sous le nom Hadoop (en Java<sup>TM</sup>) qui repose sur son propre système de fichiers parallèle HDFS.

## Deuxième partie

# Introduction aux sciences des données avec MPI



## Chapitre 7

# Les $k$ -moyennes : le regroupement par partitions

Un résumé des points essentiels à retenir est donné dans §7.11.

### 7.1 La recherche exploratoire par le regroupement

Les grands jeux de données communément appelés *BigData* sont devenus omniprésents, et il est important d'établir des traitements informatiques efficaces pour découvrir des *structures pertinentes* dans ces mers de données gigantesques. La *recherche exploratoire* consiste à trouver de telles informations structurelles sans connaissance au préalable : on parle encore d'apprentissage non-supervisé dans ce cas précis.

L'ensemble des données  $X = \{x_1, \dots, x_n\}$  est souvent statique (fixé au départ de l'analyse) comme disons une collection d'images, et on recherche des structures comme des amas compacts de données parmi celles-ci. Chaque donnée  $x_i \in \mathbb{X}$  est un vecteur de  $d$  *attributs* :  $x_i = (x_i^1, \dots, x_i^d)$ , appelé encore *feature* en anglais. On préfère la notation  $x_i^j$  à celle  $x_i^{(j)}$  pour décrire la  $j$ -ième coordonnée du vecteur  $x_i$ .

Les attributs  $x_i^j$  peuvent être *quantitatifs* (c'est-à-dire numériques), *catégorielles* (qualitatives, comme des mots d'un dictionnaire), *catégorielles ordonnées* (par exemple petit < moyen < grand ou bien encore les notes A < B < C < D < E) ou bien encore un mélange de ces types. La recherche exploratoire se distingue de la *classification supervisée* qui consiste à apprendre un classifieur  $C(\cdot)$  à partir de données étiquetées appartenant au jeu d'entraînement  $Z = \{(x_1, y_1), \dots, (x_n, y_n)\}$  où les  $x_i$  sont les attributs et les  $y_i$  les classes, afin de pouvoir dans un deuxième temps classer de *nouvelles observations*  $x_j$  pas encore étiquetées du *jeu de test* en  $\hat{y}_j = C(x_j)$ . La notation  $\hat{y}_j$  indique que l'on a *estimé* la classe d'appartenance (un processus d'*inférence* à partir du jeu d'entraînement).

Le *regroupement* est un ensemble de techniques (appelé *clustering* en anglais) qui consiste à identifier des ensembles *homogènes* de données formant les *groupes* (*clusters*). Ces groupes peuvent représenter des catégories sémantiques des données : par exemple, des fleurs regroupées par espèce dans une base de données d'images de fleurs. Un jeu classique de données disponible sur le répertoire

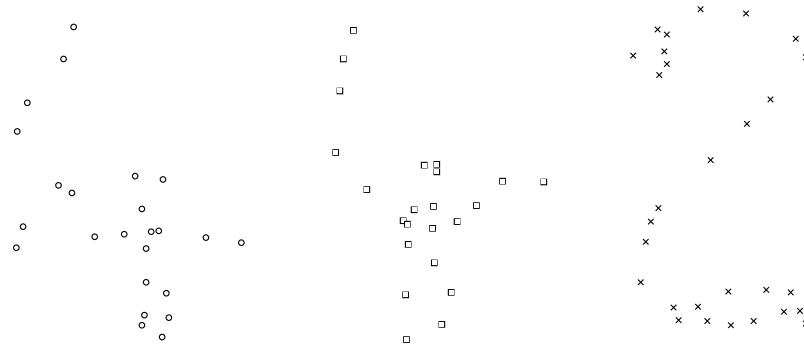


FIGURE 7.1 – La recherche exploratoire consiste à trouver des structures inhérentes au jeu de données comme des amas de données : les clusters. Le regroupement est un ensemble de techniques qui recherche des clusters homogènes dans les données. Ici en 2D, l’œil humain perçoit trois clusters car ils forment des groupes bien séparés visuellement : ‘4’ (disques), ‘4’ (carrés), et ‘2’ (croix). En pratique, les jeux de données sont de grandes dimensions et ne peuvent être inspectés visuellement facilement : on requiert des algorithmes de clustering pour trouver automatiquement ces groupes de données.

publique UCI<sup>1</sup> est le fichier *Iris*<sup>2</sup> qui comporte  $n = 150$  données numériques, de dimension  $d = 4$  (attributs définis par la longueur et largeur des sépales et la longueur et largeur des pétales), classés en  $k = 3$  groupes : les *Iris setosa*, les *Iris virginica* et les *Iris versicolor*.

En résumé, si la classification étiquette les observations en classes préalablement définies, le regroupement permet quant à lui la découverte de ces classes (*class discovery*).

### 7.1.1 Le regroupement par partitions

Le regroupement par partitions consiste à diviser  $X = \{x_1, \dots, x_n\}$  en  $k$  groupes homogènes  $G_1 \subset X, \dots, G_k \subset X$  tels que :

$$\begin{aligned} X &= \bigcup_{i=1}^k G_i, \quad \forall i \neq j, G_i \cap G_j = \emptyset, \\ X &:= \bigsqcup_{i=1}^k G_i \end{aligned}$$

La notation  $a := b$  permet d’indiquer que c’est une égalité par définition (et non pas une égalité provenant d’une dérivation par un calcul). Une donnée  $x_i$  se trouve ainsi dans un seul et unique groupe  $G_{l(x_i)}$  : on parle de *clustering dur* (*hard clustering*, par opposition au clustering doux, *soft clustering*, qui lui donne un poids positif  $l_{i,j} > 0$  d’appartenance des  $x_i$  à tous les groupes  $G_j$ , avec  $\sum_{j=1}^k l_{i,j} = 1$ ). La fonction  $l(\cdot) : x \rightarrow \{1, \dots, k\}$  est une *fonction d’étiquetage* qui indique l’appartenance de  $x_i$  au groupe  $G_{l(x_i)}$  :  $l_{i,j} = 1$  si et seulement si (ssi)  $j = l(x_i)$ . On note  $L = [l_{i,j}]$  la matrice d’appartenance de taille  $n \times k$  (*membership matrix*).

1. <https://archive.ics.uci.edu/ml/datasets.html>

2. [http://en.wikipedia.org/wiki/Iris\\_flower\\_data\\_set](http://en.wikipedia.org/wiki/Iris_flower_data_set)

### 7.1.2 Coût d'un regroupement et regroupement par modèles

Trouver un bon regroupement par partition  $X = \uplus_{i=1}^k G_i$  dans les données demande de savoir évaluer les différents regroupements possibles. Souvent, on procède dans la direction inverse! À partir d'une fonction d'évaluation donnée, on cherche un algorithme qui partitionne  $X$  en minimisant cette *fonction de coût*. Une fonction de coût  $e_k(\cdot; \cdot)$  générique (encore appelée *fonction d'énergie* ou *fonction objective*) est écrite comme la somme des coûts pour chaque groupe :

$$e_k(X; G_1, \dots, G_k) = \sum_{i=1}^k e_1(G_i),$$

avec  $e_1(G)$  la fonction de coût pour un simple groupe.

On peut aussi associer à chaque groupe  $G_i$  un *modèle*  $c_i$  qui définit le "centre"  $c_i = c(G_i)$  de ce cluster. Les centres  $c_i$  sont appelés *prototypes*, et ils permettent de définir une distance entre une donnée  $x \in X$  et un cluster  $G$  :

$$D_M(x, G) = D(x, c(G)).$$

La fonction  $D_M(x, G)$  indique la distance entre une donnée  $x$  et un cluster  $G$  induite par son modèle : le prototype du cluster. La fonction  $D(p, q)$  quant à elle est une distance appropriée entre deux données à définir. En d'autres termes, on a  $D_M(x, G) = D(x, c)$  où  $c = c(G)$  est le prototype de  $G$ .

Ainsi, étant donné l'ensemble  $C = \{c_1, \dots, c_k\}$  des  $k$  prototypes, on peut définir le coût d'un regroupement comme :

$$e_k(X; C) = \sum_{i=1}^n \min_{j=1}^k D(x_i, c_j),$$

et le coût d'un groupe est défini par  $e_1(G, c) = \sum_{x \in G} D(x, c)$ . On utilise la notation  $\min_{j=1}^k x_j$  pour indiquer l'élément minimum dans l'ensemble  $\{x_1, \dots, x_k\}$  :  $\min_{j=1}^k x_j = \min\{x_1, \dots, x_k\}$ . Le regroupement par modèles avec un centre par groupe permet de définir la partition de  $X$  induite par les  $k$  prototypes  $C = (c_1, \dots, c_k)$  :  $G(C) = \uplus_{j=1}^k G_j$ , avec  $G_j = \{x_i \in X : D(x_i, c_j) \leq D(x_i, c_l), \forall l \in \{1, \dots, k\}\}$ . On a donc aussi  $e_k(X; C) = \sum_{i=1}^k \sum_{x \in G_i} D(x, c_i)$ .

Il existe de nombreuses fonctions de coût qui donnent lieu à des partitions différentes. Nous allons présenter celle la plus utilisée en pratique, les  $k$ -moyennes<sup>3</sup> (*k-means* en anglais), et expliquer pourquoi la minimisation de celle-ci donne des bonnes partitions.

## 7.2 La fonction de coût des $k$ -moyennes

La fonction de coût des  $k$ -moyennes cherche à minimiser la *distance au carré* des données à leur centre le plus proche :

$$e_k(X; C) = \sum_{i=1}^n \min_{j=1}^k \|x_i - c_j\|^2.$$

3. L'appellation " $k$ -moyennes" veut à la fois signifier la fonction de coût et l'algorithme de Lloyd que nous allons présenter.

Notons que la distance au carré  $D(x, c) = \|x - c\|^2$  (bien que symétrique ( $D(x, c) = D(c, x)$ ) et égale à zéro ssi  $x = c$ ) n'est *pas une métrique* car elle ne satisfait pas l'inégalité triangulaire de la distance Euclidienne :  $\|x - c\|_2 = \sqrt{\sum_{j=1}^d (x^j - c^j)^2}$ .

En fait, il y a de bonnes raisons de choisir cette distance Euclidienne au carré plutôt que la distance Euclidienne : En effet, le coût d'un cluster  $e_1(G) = e_1(G, c)$  est minimisé en prenant pour prototype du cluster le centre de masse  $c$  appelé *centroïde* :

$$c(G) := \arg \min_c \sum_{x \in G} \|x - c\|^2 = \frac{1}{|G|} \sum_{x \in G} x,$$

où  $|G|$  désigne la cardinalité de  $G$ , c'est-à-dire son nombre d'éléments. La notation  $\arg \min_x f(x)$  signifie l'argument qui donne lieu au minimum dans le cas où celui-ci est unique.<sup>4</sup>

Le coût minimal est donc  $e_1(G, c) = \sum_{x \in G} \|x - c(G)\|^2 := v(G)$ , la *variance non-normalisée* de  $G$ .

En effet, la *variance normalisée* de  $X$  se définit classiquement en statistique comme :

$$v(X) = \frac{1}{n} \sum_{i=1}^n \|x_i - \bar{x}\|^2,$$

avec  $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$  le centre de masse. On peut récrire cette variance pour un nuage de points  $X$   $d$ -dimensionnel comme :

$$v(X) = \left( \frac{1}{n} \sum_{i=1}^n x_i^2 \right) - \bar{x}^\top \bar{x}.$$

Cette formule est identique de celle de la variance d'une variable aléatoire  $X$  :  $\mathbb{V}[X] = \mathbb{E}[(X - \mu(X))^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$  où  $\mu(X) = \mathbb{E}[X]$  est l'espérance.

On peut définir un attribut de poids  $w_i = w(x_i) > 0$  pour chaque donnée  $x_i$  de  $X$  tel que  $\sum_{i=1}^n w_i = 1$ . Le théorème ci-dessous caractérise le prototype  $c_1$  pour un regroupement basique en  $k = 1$  groupe ( $X = G_1$ ) :

**Théorème 3.** Soit  $X = \{(w_1, x_1), \dots, (w_n, x_n)\} \subset \mathbb{R}^d$  un jeu de  $n$  données pondérées par  $w_i > 0$  tel que  $\sum_{i=1}^n w_i = 1$ . Le centre  $c$  qui minimise la variance  $v(X) = \sum_{i=1}^n w_i \|x_i - c\|^2$  est l'unique barycentre :  $c = \bar{x} = \sum_{i=1}^n w_i x_i$ .

*Démonstration.* On note  $\langle x, y \rangle$  le produit scalaire :  $\langle x, y \rangle = x^\top y = \sum_{j=1}^d x^j y^j = \langle y, x \rangle$ . Le produit scalaire est une forme bilinéaire symétrique :  $\langle \lambda x + b, y \rangle = \lambda \langle x, y \rangle + \langle b, y \rangle$  pour  $\lambda \in \mathbb{R}$ . La distance Euclidienne au carré  $D(x, y) = \|x - y\|^2$  s'écrit comme :  $D(x, y) = \langle x - y, x - y \rangle = \langle x, x \rangle - 2\langle x, y \rangle + \langle y, y \rangle$ .

On cherche à minimiser :  $\min_{c \in \mathbb{R}^d} \sum_{i=1}^n w_i \langle x_i - c, x_i - c \rangle$ . On récrit cette minimisation comme

4. Sinon, on peut choisir le plus petit élément  $x$  qui donne lieu au minimum suivant un ordre lexicographique sur  $\mathbb{X}$ . On peut également écrire de façon équivalente  $\arg \min$  au lieu de  $\arg \min$ .

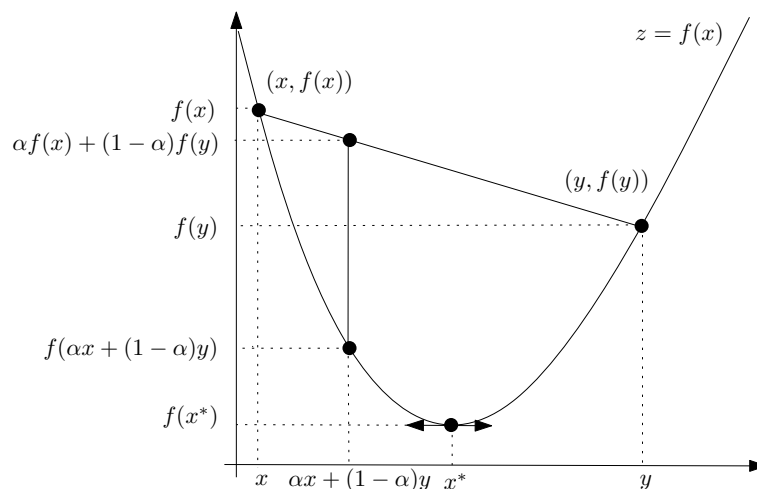


FIGURE 7.2 – Une fonction  $C^2$  est strictement convexe ssi.  $f(\alpha x + (1 - \alpha)y) < \alpha f(x) + (1 - \alpha)f(y)$  pour  $x \neq y$ , et pour tout  $\alpha \in (0, 1)$ . Lorsqu'un minimum existe, ce minimum est global et vérifie  $f'(x^*) = 0$ .

suit :

$$\begin{aligned} \min_{c \in \mathbb{R}^d} \quad & \sum_{i=1}^n w_i \langle x_i - c, x_i - c \rangle \\ & \sum_{i=1}^n w_i (\langle x_i, x_i \rangle - 2\langle x_i, c \rangle + \langle c, c \rangle) \\ & \left( \sum_{i=1}^n w_i \langle x_i, x_i \rangle \right) - 2 \left\langle \sum_{i=1}^n w_i x_i, c \right\rangle + \langle c, c \rangle \end{aligned}$$

On peut enlever le terme  $\sum_{i=1}^n w_i \langle x_i, x_i \rangle$  de la minimisation car il est indépendant de  $c$ . Ainsi, on cherche à minimiser :  $\min_{c \in \mathbb{R}^d} E(c) := -2 \langle \sum_{i=1}^n w_i x_i, c \rangle + \langle c, c \rangle$ .

On rappelle qu'une fonction  $f(x)$  de classe  $C^2$  est *strictement convexe* ssi. pour  $x \neq y$ , et pour tout  $\alpha \in (0, 1)$ , on a :

$$f(\alpha x + (1 - \alpha)y) < \alpha f(x) + (1 - \alpha)f(y).$$

La figure 7.2 illustre cette notion de convexité. Une condition équivalente pour la stricte convexité d'une fonction  $f(x)$  est que sa dérivée second soit strictement positive ( $f''(x) > 0$ ,  $x \in \mathbb{R}$ ). Si une fonction strictement convexe admet un minimum (peut ne pas exister comme dans le case de  $f(x) = \exp(x)$ ) alors ce minimum  $x^*$  est unique, et il vérifie  $f'(x^*) = 0$ . En analyse multivariée, on étend ces résultats avec le vecteur Jacobien  $\nabla_x f(x) = (\frac{\partial f(x)}{\partial x_i})_i$  et la matrice Hessienne  $\nabla_x^2 f(x) = (\frac{\partial^2 f(x)}{\partial x_i \partial x_j})_{i,j}$  des dérivées secondes. Une fonction multivariée est strictement convexe si sa matrice Hessienne est strictement positive définie. Dans le cas d'une fonction multivariée  $f(x_1, \dots, x_d)$  séparable ( $F(x^1, \dots, x^d) = \sum_{j=1}^d f(x^j)$ ), cela revient à vérifier que la fonction univariée  $f(x)$  est strictement convexe.



FIGURE 7.3 – La variance quantifie la dispersion des points à leur centroïde. Illustration avec deux ensembles de points, l'un a une petite variance (à gauche) et l'autre a une plus grande variance (à droite).

On cherche à minimiser  $\min_{c \in \mathbb{R}^d} E(c) := -2 \langle \sum_{i=1}^n w_i x_i, c \rangle + \langle c, c \rangle$  qui est une fonction séparable pour  $c = (c^1, \dots, c^d)$ . On a les  $d$  dérivées partielles :

$$\frac{\partial}{\partial c^j} E(c) = -2 \sum_{i=1}^n w_i x_i^j + 2c^j, \quad \forall j \in \{1, \dots, d\}$$

et les  $d^2$  dérivées secondes :

$$\frac{\partial^2}{\partial c^j \partial c^l} E(c) = 2, \quad \text{pour } l = j, \forall j \in \{1, \dots, d\}$$

La fonction  $E(c)$  est donc strictement convexe et admet un *unique minimum global*, obtenu en annulant toutes les dérivées partielles<sup>5</sup> :

$$\frac{\partial}{\partial c^j} E(c) = 0 \Leftrightarrow c^j = \sum_{i=1}^n w_i x_i^j.$$

On a ainsi montré que le minimiseur des distances euclidiennes au carré pondérées est l'unique barycentre :  $c = \bar{x} = \sum_{i=1}^n w_i x_i$ . Le centroïde est encore appelé *isobarycentre*.  $\square$

La variance du cluster quantifie la grandeur de dispersion des points à leur centroïde. La figure fig :variancedispersion illustre deux ensembles de points, l'un avec une petite variance et l'autre avec une plus grande variance.

Si au lieu de la distance Euclidienne au carré, on avait choisi la distance Euclidienne alors on aurait obtenu le point de Fermat-Weber qui généralise la notion de médiane. On l'appelle donc encore médiane géométrique.<sup>6</sup> Le point de Fermat-Weber bien qu'unique n'admet pas de formule

5. Pour une fonction réelle multivariée, on note  $\nabla_x F(x)$  son gradient, le vecteur des dérivées partielles, et  $\nabla_x^2 F(x)$  son Hessien, la matrice des dérivées secondes. Une fonction lisse  $F$  est strictement convexe ssi  $\nabla^2 F \succ 0$  où  $M \succ 0$  indique que la matrice  $M$  est définie positive :  $\forall x \neq 0, x^\top M x > 0$ . Une fonction strictement convexe admet un unique minimum  $x^*$  tel que  $\nabla F(x^*) = 0$ .

6. [http://en.wikipedia.org/wiki/Geometric\\_median](http://en.wikipedia.org/wiki/Geometric_median)

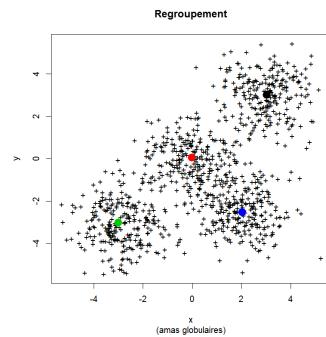


FIGURE 7.4 – La fonction de coût des  $k$ -moyennes recherche des amas globulaires de faibles variances dans les données en minimisant la fonction de coût associée. Les  $k$ -moyennes sont un algorithme de regroupement par modèles où à chaque cluster on associe un prototype : son centre. Ici, on a choisi  $k = 4$  groupes avec les  $k$ -moyennes (centroïdes) indiquées visuellement par les gros disques.

close connue, mais peut être approximé itérativement arbitrairement finement. Regrouper en minimisant la fonction objective  $\min_C \sum_{i=1}^n \min_{j=1}^k \|x_i - c_j\|$  est appelé les  $k$ -médianes ( $k$ -medians). Le regroupement optimal qui minimise la fonction de coût des  $k$ -moyennes peut être bien différent de celui qui minimise la fonction de coût des  $k$ -médianes. En effet, la position du centroïde peut être très différente de celle de la médiane pour un seul cluster.

De plus, la position du centroïde peut être facilement corrompue par un seul point aberrant (dit *outlier*) alors que la médiane est quant à elle beaucoup plus robuste. On dit que le *breakdown point* du centroïde est 0 car l'ensemble  $X$  augmenté d'un seul point aberrant  $p_0$  peut faire dévier le centroïde de  $X$  aussi loin que l'on veut : Par exemple, en un 1D, en prenant  $p_0 \rightarrow \infty$ . Par contre la médiane a un breakdown point de  $\lfloor \frac{n}{2} \rfloor$  : il faut 50% de points outliers se dirigeant vers  $+\infty$  pour tirer la médiane de  $X$  aussi vers l'infini.

Notons que trouver le centre d'un cluster est un cas spécial de regroupement en  $k = 1$  cluster. Avec la fonction de coût de la distance Euclidienne au carré, on trouve donc la “moyenne” des attributs (d'où le nom de cette technique de clustering : les  $k$ -moyennes). La figure 7.4 illustre sur un jeu de données un regroupement par les  $k$ -moyennes.

### 7.2.1 Réécriture de la fonction de coût : regrouper ou séparer les données

La fonction de coût des  $k$ -moyennes cherche à trouver  $k$  amas de points globulaires, c'est-à-dire de faibles variances. En effet, la fonction de coût se réinterprète comme la minimisation de la *somme pondérée des variances* des clusters :

$$\begin{aligned}
& \min_{C=\{c_1, \dots, c_k\}} \sum_{i=1}^n \min_{j=1}^k w_i \|x_i - c_j\|^2, \\
& \min_{C=\{c_1, \dots, c_k\}} \sum_{j=1}^k \sum_{x \in G_j} w(x) \|x - c_j\|^2 \\
& \min_{C=\{c_1, \dots, c_k\}} \sum_{j=1}^k W_j v(G_j),
\end{aligned}$$

avec  $W_j := \sum_{x \in G_j} w(x)$  le poids cumulé des éléments du groupe  $G_j$  (cf. l'exercice 7.12).

On montre aussi que regrouper les données en clusters homogènes correspond de façon équivalente à *séparer* les données de  $X$  en groupes : en effet, notons  $A := \sum_{i=1}^n \sum_{j=i+1}^n \|x_i - x_j\|^2$ , la constante de la somme des distances au carré pour toutes les paires de points de  $X$ . Pour une partition donnée, nous pouvons toujours décomposer  $A$  en deux paquets : le paquet des intra-distances d'un même cluster et le paquet des inter-distances entre deux clusters distincts :

$$A = \left( \sum_{l=1}^k \sum_{x_i, x_j \in G_l} \|x_i - x_j\|^2 \right) + \left( \sum_{l=1}^k \sum_{x_i \in G_l, x_j \notin G_l} \|x_i - x_j\|^2 \right).$$

Ainsi, minimiser la somme des distances au carré intra-clusters  $\sum_{l=1}^k \sum_{x_i, x_j \in G_l} \|x_i - x_j\|^2$  revient à maximiser la somme des distances au carré inter-clusters puisque  $A$  est une constante (pour un ensemble fixé de points) :

$$\begin{aligned}
& \min_C \sum_{l=1}^k \sum_{x_i, x_j \in G_l} \|x_i - x_j\|^2, \\
& = \min_C \left( A - \sum_{l=1}^k \sum_{x_i \in G_l, x_j \notin G_l} \|x_i - x_j\|^2 \right), \\
& \equiv \max_C \sum_{l=1}^k \sum_{x_i \in G_l, x_j \notin G_l} \|x_i - x_j\|^2
\end{aligned}$$

La notation  $\equiv$  indique l'équivalence. On a  $\arg \min_x (f(x) + g(y)) = \arg \min_x f(x)$ , et donc  $\min_x (f(x) + g(y)) \equiv \min_x f(x)$  (les termes incluant uniquement le paramètre  $y$  peuvent être ignorés dans l'optimisation).

On aboutit donc à une *vision duale* intuitive pour définir un bon regroupement :

- regrouper en groupes homogènes afin de minimiser la somme des variances pondérées, ou
- séparer les données afin de maximiser les distances au carré interclusters

### 7.2.2 Calculabilité : complexité du calcul des $k$ -moyennes

Optimiser les  $k$ -moyennes est un problème **NP-dur** dès que la dimension  $d > 1$  et que le nombre de clusters  $k > 1$ . Ces notions de classes de complexité ont été introduites en INF412 [17] (aux chapitres 11 et 12).

On rappelle que  $P$  est la classe des problèmes décisionnels (Oui/Non) que l'on peut résoudre en temps polynomial, et que  $NP$  est la classe des problèmes pour lesquels on peut vérifier la solution d'un problème décisionnel en temps polynomial (comme par exemple, 3-SAT<sup>7</sup>). On définit la classe  $NP$ -complète comme la classe des problèmes de  $NP$  qui peuvent se résoudre entre eux par réduction en temps polynomial :  $X \propto_{\text{polynomial}} Y, \forall Y \in NP$ . C'est-à-dire qu'un problème  $NP$ -complet est équivalent à tous les autres problèmes  $NP$ -complets après une réduction en temps polynomial. La classe  $NP$ -dure quant à elle, est la classe des problèmes  $X$ , pas nécessairement dans  $NP$ , tels que  $\exists Y \in NP - \text{Complet} \propto_{\text{polynomial}} X$ .

Quand  $k = 1$ , on a vu qu'on peut calculer la solution optimale pour le prototype d'un groupe, le barycentre, en temps linéaire. Pour  $d = 1$  (la dimension ou nombre d'attributs des données), on peut calculer les  $k$ -moyennes de façon optimale en utilisant la programmation dynamique : en utilisant un espace  $O(nk)$ , on peut résoudre les  $k$ -moyennes pour  $n$  scalaires en temps  $O(n^2k)$  avec un espace mémoire  $O(nk)$  (cf. les exercices).

**Théorème 4** (Complexité des  $k$ -moyennes). *Le regroupement qui maximise la fonction de coût des  $k$ -moyennes est un problème  $NP$ -dur quand  $k > 1$  et  $d > 1$ . Quand  $d = 1$ , on peut trouver par programmation dynamique en temps  $O(n^2k)$  le coût minimum des  $k$ -moyennes en utilisant un espace mémoire  $O(nk)$ .*

Puisqu'en général, on ne connaît pas d'algorithmes polynomiaux ("rapides") pour les  $k$ -moyennes, on va utiliser des heuristiques. On distingue deux grandes classes de telles heuristiques :

1. les *heuristiques globales* qui ne dépendent pas d'une initialisation, et
2. les *heuristiques locales* qui itérativement améliorent une solution en partant d'une configuration (partition) donnée.

Bien entendu, on peut initialiser les heuristiques locales avec celles des méthodes globales.

### 7.3 L'heuristique locale de Lloyd pour les $k$ -moyennes

On présente la méthode de Lloyd (1957) qui consiste à partir d'une initialisation donnée, à répéter les deux étapes suivantes :

**Allocation en groupes.** Pour tout  $x_i \in X$ , soit  $l_i = \arg \min_l \|x_i - c_l\|^2$ , et formons les  $k$  groupes  $G_j = \{x_i : l_i = j\}$  de cardinalité  $n_j = |G_j|$ .

**Mise à jour des centres.** Pour tout  $j \in \{1, \dots, k\}$ , calculer les centres de masse :  $c_j = \frac{1}{n_j} \sum_{x \in G_j} x$  (ou les barycentres  $c_j = \frac{1}{\sum_{x \in G_j} w(x)} \sum_{x \in G_j} w(x)x$ ).

La figure 7.5 illustre quelques étapes de l'algorithme des  $k$ -moyennes de Lloyd. Puisque les centres bougent au fur et à mesure des itérations, on l'appelle aussi méthode des centres mobiles (ou encore nuée dynamique).

**Théorème 5.** *L'heuristique de Lloyd converge de façon monotone vers un minimum local en un nombre fini d'itérations borné par  $\binom{n}{k}$ .*

7. Le problème 3-SAT qui consiste à dire si une formule booléenne à  $n$  clauses de 3 littéraux est satisfaisable ou pas est un problème  $NP$ -complet (théorème de Cook).

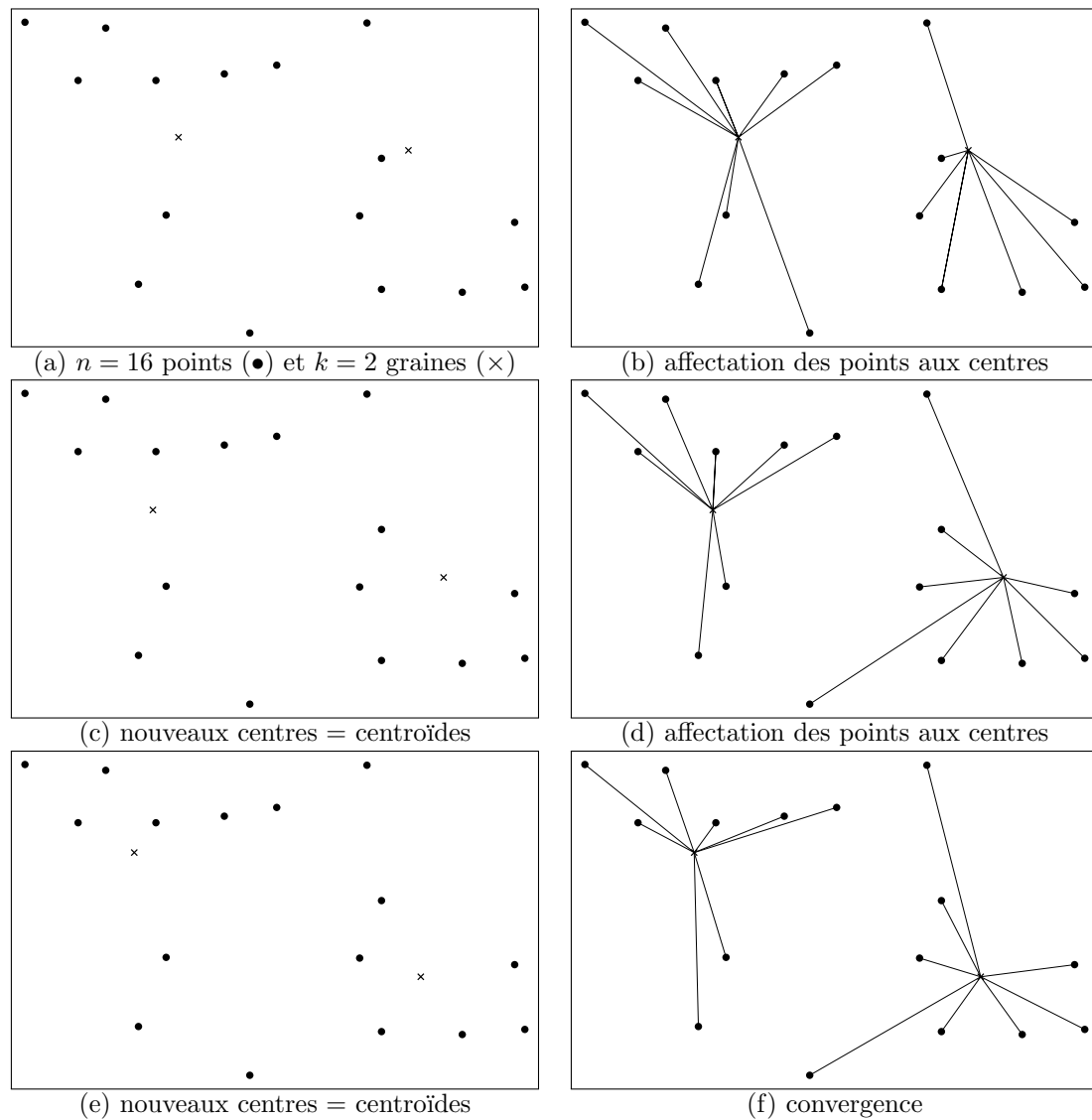


FIGURE 7.5 – Quelques étapes de l'algorithme des  $k$ -moyennes de Lloyd : (a)  $n = 16$  points ( $\bullet$ ) et initialisation aléatoire des  $k = 2$  centres ( $\times$ ), (b) affectation des données aux centres, (c) mise à jour des centres en prenant les centroïdes des groupes, (d) affectation des données aux centres, (e) mise à jour des centres en prenant les centroïdes des groupes, jusqu'à la convergence (f) vers un minimum local de la fonction de coût (notez que (f) et (d) donne le même appariement).

*Démonstration.* Soit  $G_1^{(t)}, \dots, G_k^{(t)}$  la partition de  $X$  à l'étape  $t$  de coût  $e_k(X, C_t)$  et  $G(C_t) = \biguplus_{j=1}^k G_j^{(t)}$  les groupes induits par les  $k$  centres  $C_t$ . À l'étape  $t + 1$ , puisqu'on alloue les points aux clusters dont les centres sont les plus proches pour former les groupes  $G^{(t+1)}$ , on minimise donc :

$$e_k(X; G(C^{(t+1)})) \leq e_k(X, G(C_t)).$$

Rappelons que nous avons la fonction de coût des  $k$ -moyennes qui est égale à la somme (pondérée) des variances des clusters :  $e_k(X; G(C^{(t+1)})) = \sum_{j=1}^k v(G_j^{(t+1)}, c_j)$ . Lors de la remise à jour des centres par les centroïdes des groupes (les points qui minimisent la somme des distances au carré de ces groupes), pour chaque groupe, nous avons  $v(G_j^{(t+1)}, c(G_j^{(t+1)})) \leq v(G_j^{(t+1)}, c_j)$ , et donc il s'en suit que :

$$e_k(X; C_{t+1}) \leq e_k(G(C^{(t+1)}); C_t) \leq e_k(X; C_t)$$

Puisque  $e_k(X; C) \geq 0$  et que l'on ne repète pas deux fois la même partition parmi les  $O(\binom{n}{k})$  partitions possibles<sup>8</sup>, on converge nécessairement vers un minimum local après un nombre fini d'itérations.  $\square$

Nous présentons quelques observations sur les  $k$ -moyennes :

**Observation 1.** *Bien que cette heuristique de Lloyd fonctionne très bien en pratique, il a été démontré que dans le pire des cas nous pouvons néanmoins avoir un nombre exponentiel d'itérations, et ceci même dans le cas planaire  $d = 2$  [94, 45]. Dans le cas 1D et pour  $k = 2$ , les  $k$ -moyennes de Lloyd peuvent demander  $\Omega(n)$  itérations avant la convergence [45].*

**Observation 2.** *Notons que même si la fonction de coût est minimale pour une valeur donnée, il peut y avoir un nombre exponentiel de solutions équivalentes : par exemple, considérons un ensemble de 4 points constituant les sommets d'un carré. Pour  $k = 2$ , nous avons deux solutions possibles (deux arêtes parallèles). Maintenant, faisons  $\frac{n}{4}$  copies de ces 4 points très éloignés les uns des autres, et prenons  $k = \frac{n}{4}$  : il existe  $2^k$  regroupements optimaux.*

**Observation 3.** *Il se peut que lors d'une étape d'allocation des points aux centres des clusters dans l'algorithme de Lloyd, certains clusters se retrouvent vides : ce cas est relativement rare en pratique mais ce phénomène augmente avec la dimension. Prendre garde donc lors de l'implémentation de ces exceptions. Cette situation de groupe qui devient vide est illustrée dans la figure 7.6. Notons que ce n'est pas un problème en soi, car dans ce cas-là, on peut choisir autant de nouveaux points pour les nouveaux prototypes afin de réinitialiser ces clusters vides (réinitialisation partielle, hot restart), ce qui fait nécessairement baisser la somme des variances. C'est même une aubaine !*

Les  $k$ -moyennes de Lloyd sont une heuristique locale qui garantit la convergence monotone à partir d'une configuration donnée (soit par les  $k$  prototypes initialisés, soit par une partition qui induit les prototypes centroïdes). Nous présentons maintenant quelques méthodes d'initialisation : des heuristiques des  $k$ -moyennes globales.

---

8. Le nombre de partitions d'un ensemble à  $n$  éléments en  $k$  sous-ensembles non-vides est le deuxième nombre de Stirling :  $\left\{ \begin{matrix} n \\ k \end{matrix} \right\} = \frac{1}{k!} \sum_{j=0}^k (-1)^{k-j} \binom{k}{j} j^n$ .

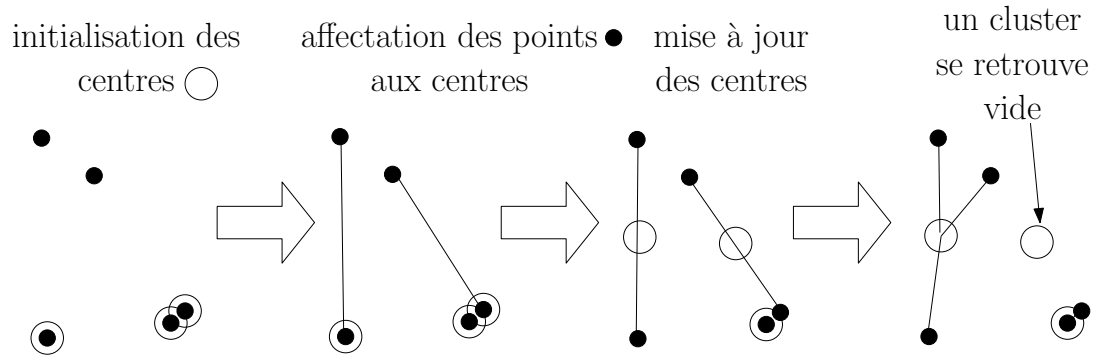


FIGURE 7.6 – Heuristique de Lloyd et apparition de clusters vides : Les centres des clusters sont dessinés avec des gros cercles. Initialisation suivie d’une allocation puis d’une mise à jour des prototypes avec une nouvelle phase d’allocation. On voit qu’un cluster contenant un prototype devient vide.

## 7.4 Initialisation des $k$ -moyennes

### 7.4.1 Initialisation aléatoire (dite de Forgy)

On choisit les  $k$  graines distinctes (*seeds*) aléatoirement dans  $X$  (on tire aléatoirement  $k$  indexes distincts entre 1 et  $n$ ). Il existe  $\binom{n}{k}$  possibilités. Puis on forme la partition des groupes  $G(C) = \{G_1, \dots, G_k\}$  à partir de ces graines tirées  $C = \{c_1, \dots, c_k\}$ . Nous n’avons aucune garantie théorique que  $e_k(X, G)$  soit proche du minimum global  $e_k^*(X, G) = \min_C e_k(X, G)$ . Ainsi, afin d’augmenter nos chances, on peut initialiser  $l$  fois en tirant les graines aléatoirement  $C_1, \dots, C_l$  puis garder seulement les graines qui ont donné le meilleur coût : c’est-à-dire  $C_{l^*}$  avec  $l^* = \arg \min_l e_k(X; G(C_l))$ . C’est la politique en anglais dite de *restart*.

### 7.4.2 Initialisation avec les $k$ -moyennes globales

Choisissons d’abord la première graine  $c_1$  aléatoirement, puis les graines  $c_2$  jusqu’à  $c_k$  itérativement de façon gloutonne. Soit  $C_{\leq i} = \{c_1, \dots, c_i\}$  l’ensemble des  $i$  premières graines. On choisit  $c_i$  parmi  $X$  de façon à minimiser  $e_i(X, C_{\leq i})$ . Puis, on considère alors les  $n$  possibilités  $c_1 = x_1, \dots, c_1 = x_n$  pour la graine  $c_1$ , et on garde la meilleure initialisation parmi ces  $n$  initialisations gloutonnes.

### 7.4.3 Initialisation probabiliste garantie avec les $k$ -moyennes++

On considère une initialisation aléatoire qui garantit avec une *grande probabilité* une bonne initialisation. Notons  $e_k^*(X) = \min_C e_k(X; C) = e_k(X, C^*)$  la valeur du minimum global de la fonction de coût des  $k$ -moyennes, avec  $C^* = \arg \min_C e_k(X; C)$ . Une  $(1 + \epsilon)$ -approximation des  $k$ -moyennes est définie par un ensemble de prototypes  $C$  tel que :

$$e_k^*(X) \leq e_k(X, C) \leq (1 + \epsilon)e_k^*(X).$$

Autrement dit, le rapport  $\frac{e_k(X, C)}{e_k^*(X)}$  est au plus  $1 + \epsilon$ .

Les  $k$ -moyennes++ choisissent itérativement aléatoirement les graines en pondérant la probabilité de tirer le point  $x_i$  en fonction de la distance au carré du point  $x_i$  aux centres déjà choisis. Notons  $D^2(x, C)$  le minimum de la distance Euclidienne au carré de  $x$  à un élément de  $C$  :  $D^2(x, C) = \min_{c \in C} \|x - c\|^2$ .

Pour un ensemble d'éléments pondérés  $X$ , l'algorithme des  $k$ -moyennes++ s'écrit comme suit :

— Choisir  $c_1$  aléatoirement uniformément dans  $X$  :  $C_{++} = \{c_1\}$ .

— Pour  $i = 2$  à  $k$

Tirer  $c_i = x \in X$  avec une probabilité :

$$p(x) = \frac{w(x)D^2(x, C_{++})}{\sum_y w(y)D^2(y, C_{++})}$$

$$C_{++} \leftarrow C_{++} \cup \{c_i\}.$$

**Théorème 6** ( $k$ -means++ [3]). *L'initialisation probabiliste des  $k$ -moyennes++ garantit en espérance :  $\mathbb{E}[e_k(X, C_{++})] \leq 8(2 + \ln k)e_k^*(X)$ .*

C'est-à-dire que les  $k$ -moyennes++ sont  $\tilde{O}(\log k)$  compétitives (la notation tilde dans  $\tilde{O}(\cdot)$  indique le caractère probabiliste en moyenne). La preuve technique se trouve dans le papier [3]. Nous donnons ici la preuve élémentaire dans le cas simple d'un seul cluster ( $k = 1$ ) où l'on choisit aléatoirement un point  $x_0 \in X$ . On note  $c^*$  le centre de masse de  $X$ . On a :

$$\mathbb{E}[e_1(X)] = \frac{1}{|X|} \sum_{x_0 \in X} \sum_{x \in X} \|x - x_0\|^2.$$

On utilise maintenant la décomposition suivante (provient de la décomposition communément appelée *décomposition biais-variance*<sup>9</sup>) pour tout  $z$  :

$$\sum_{x \in X} \|x - z\|^2 - \sum_{x \in X} \|x - c^*\|^2 = |X| \times \|c^* - z\|^2.$$

On en déduit donc :

$$\begin{aligned} \mathbb{E}[e_1(X)] &= \frac{1}{|X|} \sum_{x_0 \in X} \left( \sum_{x \in X} \|x - c^*\|^2 + |X| \|x_0 - c^*\|^2 \right), \\ &= 2 \sum_{x \in X} \|x - c^*\|^2, \\ &= 2e_1^*(X). \end{aligned}$$

Ainsi, dans le cas  $k = 1$  cluster, en tirant aléatoirement la graine  $c_1$  uniformément dans  $X$ , on garantit en espérance un facteur d'approximation de 2.

9. Soit  $Z \sim p(z)$  une variable aléatoire de densité  $p(z)$ , et  $\mu(Z) = \mathbb{E}[Z]$  son espérance. On a  $\mathbb{E}[(Z - \mu(Z))^2] = \mathbb{E}[Z^2 - 2\mu(Z)\mathbb{E}[Z] + \mu^2(Z)] = \mathbb{E}[Z^2] - 2\mu(Z)\mathbb{E}[Z] + \mu^2(Z) = \mathbb{E}[Z^2] - \mu^2(Z)$ . On en déduit le corollaire  $\mathbb{E}[Z^2] = \mathbb{E}[(Z - \mu(Z))^2] + \mu^2(Z)$ . On peut interpréter un jeu de données avec poids comme une variable aléatoire discrète.

## 7.5 La quantification de vecteurs et les $k$ -moyennes

### 7.5.1 La quantification

La quantification est un procédé qui permet de compresser (avec perte contrôlée) les données.

En quantification de vecteurs (*vector quantization*), on a l'ensemble  $X = \{x_1, \dots, x_n\}$  que l'on cherche à coder avec des mots  $c_1, \dots, c_k$  qui forme le *livre de codes*  $C = \{c_1, \dots, c_k\}$  (*codebook*). On dispose d'une fonction de codage (ou de quantification) et de décodage :

- fonction de codage (quantification)  $i(\cdot) : x \in \mathbb{R}^d \rightarrow \{1, \dots, k\}$
- fonction de décodage :  $c(\cdot)$

Pour coder avec compression un message  $(t_1, \dots, t_m)$  de  $m$  éléments sur un alphabet  $X$  de  $n$  caractères ( $t_i \in X$ ), on associe à  $t_i$  son code  $i(t_i)$  qui figure parmi  $k$  mots. Par exemple, on cherche à partir d'une image couleur codée sur 24-bits ( $X$ ), à trouver la meilleure palette en  $k$  couleurs (les prototypes  $C$  des  $k$ -moyennes). Au lieu de coder l'image de  $m = w \times h$  pixels sur  $24m$  bits, on code la table des couleurs sur  $24k$  bits, puis on transmet  $m \times \log k$  bits pour le contenu de l'image. Ainsi on gagne de l'espace mémoire pour la représentation mais le codage se fait avec une perte d'information. L'erreur de *distorsion* occasionnée par la quantification d'un alphabet à  $n$  lettres en un alphabet en  $k$  lettres est  $E = \frac{1}{n} \sum_{l=1}^n \|x_l - c(i(x_l))\|^2$ , l'*erreur moyenne quadratique* ou *Mean Square Error* (MSE).

Les  $k$ -moyennes permettent de trouver un codebook minimisant cette erreur :  $e_k(X, C) = v(X) - v(C)$ , la variance représente l'information et on cherche à minimiser la perte d'information. On peut montrer que la fonction de coût se réécrit comme :

$$\begin{aligned} e_k(X, C) &= \sum_{i=1}^n \min_{j=1}^k \|x_i - c_j\|^2, \\ &= \mathbb{V}(X) - \mathbb{V}(C), \text{ avec } C = \{(n_j, c_j)\}_{j=1}^k \end{aligned}$$

Les  $k$ -moyennes se réinterprètent comme minimiser la différence des variances entre deux variables aléatoires discrètes : La variable aléatoire discrète  $X$  du jeu de données et la variable aléatoire discrète de sa quantification par  $k$  centres  $C$ . Quantifier c'est donc minimiser la différence de variance entre la variable aléatoire discrète  $X$  sur  $n$  états et la variable aléatoire  $K$  sur  $k$  états. La variance joue le rôle d'"information" (la *dispersion*, en anglais *scatter* ou encode *spread*), et on cherche à minimiser la différence d'"information" entre les deux variables aléatoires.

### 7.5.2 \* Les minima locaux de Lloyd engendrent des partitions de Voronoï

Pour tout  $x_i \in X$ , on associe l'*étiquette* dans  $\mathbb{N}$  :

$$l_C(x) = \arg \min_{j=1}^k \|x - c_j\|^2$$

On peut étendre cette fonction d'étiquetage à tout l'espace  $\mathbb{X}$ , et on obtient une partition de  $\mathbb{X}$  appelé *diagramme de Voronoï* illustré par la figure 7.7 (a). Une cellule  $V_j$  du diagramme Voronoï se définit comme :

$$V_j = \{x \in \mathbb{R}^d : \|x - c_j\| \leq \|x - c_l\| \forall l \in \{1, \dots, n\}\}.$$

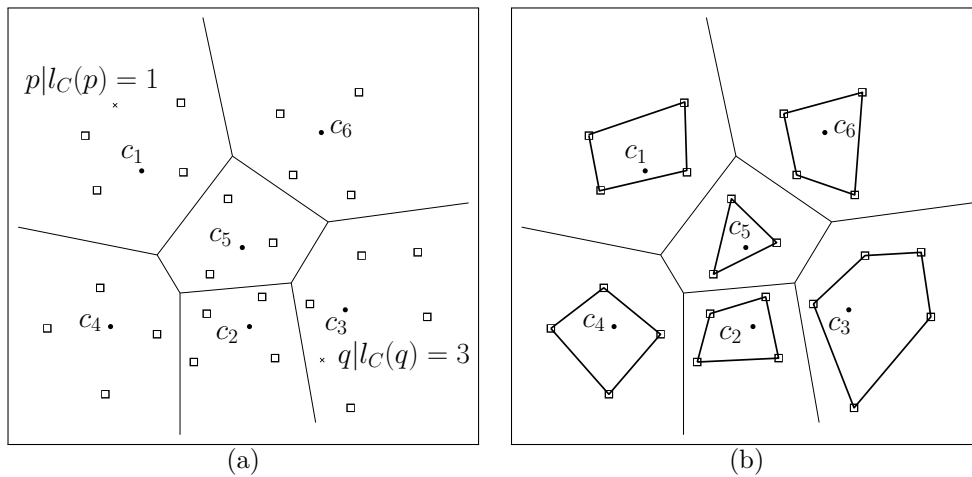


FIGURE 7.7 – (a) Diagramme de Voronoï induit par les  $k$  centres  $C$ , et partition de Voronoï de  $X$  induite par  $C$ . (b) Les enveloppes convexes des groupes (clusters) ne s'intersectent pas deux à deux.

Notons que la distance Euclidienne ou la distance Euclidienne au carré donne lieu au même diagramme de Voronoï. En fait, les cellules de Voronoï ne changent pas si on applique une fonction strictement monotone à la fonction distance de base. La fonction carré est une telle fonction monotone sur  $\mathbb{R}_+$ .

On remarque qu'à la convergence de l'heuristique de Lloyd, les groupes  $G_i$  forment une partition de Voronoï, et ont leur enveloppe convexe (*convex hull*) deux à deux disjointes (comme le montre la figure 7.7 (b)) :  $\forall i \neq j, \text{co}(G_i) \cap \text{co}(G_j) = \emptyset$  où  $\text{co}(X) = \{x : x = \sum_{x_i \in X} \lambda_i x_i, \sum_{i=1}^n \lambda_i = 1, \lambda_i \geq 0\}$ . Rappelons que dans l'heuristique itératif de Lloyd, la prochaine étape serait de prendre comme centres des groupes les centroïdes des groupes.

Les *diagrammes de Voronoï* sont une structure de données très utile en géométrie algorithmique : elle permet de résoudre de nombreux problèmes comme la planification de trajectoires pour les robots en évitant au mieux les points générateurs des cellules, le maillage d'un ensemble de points par une structure duale que l'on appelle la *triangulation de Delaunay*.

## 7.6 Interprétation physique des $k$ -moyennes : décomposition de l'inertie

Considérons maintenant l'ensemble  $X = \{(x_i, w_i)\}_i$  comme  $n$  masses positionnées aux positions  $x_i$  avec des poids respectifs  $w_i$ . En physique, le concept d'*inertie* mesure la résistance d'un corps à changer son mouvement de rotation autour d'un point donné. On mesure l'inertie totale  $I(X)$  du nuage de points  $X$  comme la somme des carrés des distances des points par rapport au *centre de gravité*  $c = \sum_{i=1}^k w_i x_i$  :

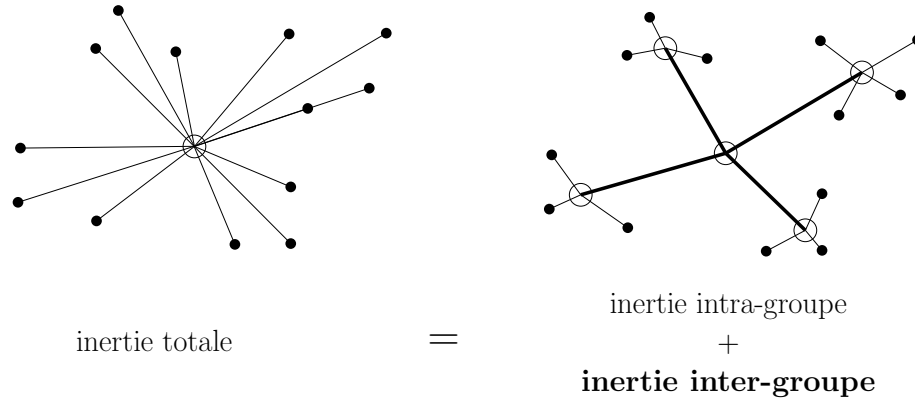


FIGURE 7.8 – L’inertie totale est invariante par découpage en groupes. Les  $k$ -moyennes cherchent à trouver la décomposition qui minimise l’inertie intra-groupe.

$$I(X) := \sum_{i=1}^n w_i \|x_i - c\|^2.$$

On remarque ainsi que si l’on augmente les poids des points, l’inertie augmente. De même, si on éloigne les poids du centre de gravité, il est plus “dur” à faire tourner ce nuage de points autour de  $c$ . Les  $k$ -moyennes peuvent donc se réinterpréter physiquement comme à identifier  $k$  groupes tels que la somme des inerties des groupes par rapport à leurs barycentres est minimale. La formule de Huygens donne un invariant ou identité entre l’inertie totale et sa décomposition en somme de l’inertie intra-groupes plus l’inertie inter-groupes.

**Théorème 7** (Formule de Huygens : décomposition de l’inertie). *L’inertie totale  $I(X) = \sum_{i=1}^n w_i \|x_i - \bar{x}\|^2 = I_{\text{intra}}(G) + I_{\text{inter}}(C)$  où l’inertie intra-groupes  $I_{\text{intra}}(G) = \sum_{i=1}^k I(G_i) = \sum_{i=1}^k \sum_{x_j \in G_i} w_j \|x_j - c_i\|^2$  et l’inertie inter-groupes  $I_{\text{inter}}(C) = \sum_{i=1}^k W_i \|c_i - c\|^2$  (un seul centre  $c$ ) avec les  $W_i = \sum_{x \in G_i} w(x)$ .*

La figure 7.8 illustre deux décompositions en groupes qui ont la même inertie totale. Notons que puisque l’inertie totale est invariante, minimiser l’inertie intra-groupes revient à maximiser l’inertie inter-groupes.

## 7.7 Choix du nombre $k$ de groupes : sélection de modèle

Jusqu’à présent nous avons considéré  $k$  comme donné au préalable. Un problème important en pratique est de déterminer le nombre  $k$  de clusters : c’est le problème de la *sélection de modèle*. Pour n’importe quelle valeur de  $k$ , on a la fonction de coût optimale des  $k$ -moyennes  $e_k^*(X)$  (que l’on peut estimer empiriquement avec l’heuristique de Lloyd sur plusieurs initialisations). On remarque que  $e_k(X)$  diminue monotonement jusqu’à  $e_n(X) = 0$  (chaque point constitue son cluster).

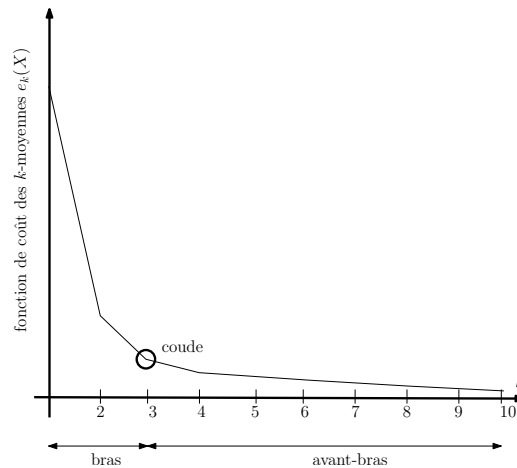


FIGURE 7.9 – Choisir  $k$  en utilisant la méthode du coude : le coude est la valeur de  $k$  qui délimite la zone de forte décroissance (le bras) à la zone du plateau (l’avant-bras).

### 7.7.1 Méthode du coude

Pour choisir correctement la valeur de  $k$ , on peut utiliser la méthode du coude. C’est une méthode de décision visuelle : on dessine la fonction  $(k, e_k(X))$  pour  $k \in \{1, \dots, n\}$ , et on choisit  $k$  au niveau du coude (*elbow*) comme illustré à la figure 7.9. La raison est qu’au début le coût décroît rapidement quand on augmente  $k$  puis on atteint un plateau où la décroissance est bien moins prononcée. La valeur optimale de  $k$  est à cette transition entre forte et faible décroissances. On l’appelle méthode du coude car le graphe de la fonction  $f(k) = e_k(X)$  ressemble à un bras (avec le plateau l’avant-bras) : le coude donne le nombre optimal de clusters. Cette méthode est coûteuse à mettre en œuvre, et parfois la jonction entre bras et avant-bras est peu visible.

### 7.7.2 Proportion de la variance expliquée par $k$

On calcule la proportion de la variance expliquée par  $k$  classes :

$$R^2(k) = \frac{I_{\text{inter}}(k)}{I_{\text{totale}}}$$

On a  $0 < R^2(k) \leq 1$ . On choisit  $k^*$  qui minimise  $\frac{R^2(k)}{R^2(k+1)}$  :

$$k^* = \arg \min_k \frac{R^2(k)}{R^2(k+1)}.$$

Nous avons présenté deux méthodes pour sélectionner le nombre de groupes<sup>10</sup> du modèle de regroupement. Il existe de nombreuses autres façons de choisir  $k$ . Certains algorithmes regroupent

10. En apprentissage, on parle de la complexité du modèle. Ici, chaque groupe est représenté par un prototype. Le modèle est donc l’ensemble de ces  $k$  centres, et la complexité du modèle est  $k$ .

les données sans nécessiter au préalable une connaissance sur le nombre de clusters,  $k$ . C'est le cas par exemple de l'algorithme de la propagation par affinité [35].

## 7.8 Les $k$ -moyennes sur une grappe de machines pour les grandes données

Il existe de nombreuses façons de paralléliser l'algorithme des  $k$ -moyennes sur une grappe de  $p$  Unités de Calcul (UCs, ou *Processing Units/Elements*, PUs/PEs en anglais) à mémoire distribuée reliées entre-elles par un réseau d'interconnexion. Les unités de calcul communiquent entre-elles par des messages via l'interface MPI : Message Passing Interface. Envoyer un message requiert un temps de latence ainsi qu'un coût proportionnel à la longueur du message.

La taille du problème pour les  $k$ -moyennes peut être caractérisée par : le nombre d'attributs  $d$  (la dimension du nuage de points), le nombre de données  $n$ , et le nombre de clusters  $k$ . On considère  $k \ll n$  si bien que les  $k$  centres occupant un espace mémoire  $O(dk)$  tiennent dans les mémoires vives de chaque UC. Puisqu'en pratique la mémoire de chaque UC est fixe (en  $O(1)$ ), cela veut dire que l'on considère ici  $k = O(1)$ . Par contre  $n$  est considéré très grand devant  $k$ , et requiert l'ensemble  $X$  des données à être distribué entre les différentes UCs. Afin de paralléliser efficacement les  $k$ -moyennes, on utilise la propriété suivante des barycentres Euclidiens :

**Théorème 8** (Décomposition en paquets et règle de composabilité des barycentres). *Soit  $X_1$  et  $X_2$  deux jeux de données pondérés avec leurs sommes respectives des poids totaux  $W_1$  et  $W_2$ . On a la propriété de composition suivante :*

$$\bar{x}(X_1 \cup X_2) = \frac{W_1}{W_1 + W_2} \bar{x}(X_1) + \frac{W_2}{W_1 + W_2} \bar{x}(X_2),$$

où  $\bar{x}(X_i)$  est le barycentre de  $X_i$ , pour  $i \in \{1, 2\}$ .

On se base sur cette propriété pour distribuer les calculs des centroïdes sur une partition de  $X$  en  $p$  sous-ensembles  $X_1, \dots, X_p$  de données où  $p$  est le nombre de processeurs. L'algorithme parallèle est décrit dans l'algorithme 6.

À l'exécution du programme, chaque UC connaît le nombre d'UCs,  $p$ , par la primitive `MPI_Comm_size()` et son numéro (ou rang) entre 0 et  $p - 1$  par la fonction `MPI_Comm_rank()`. Le processeur  $P_0$  initialise les  $k$  prototypes et diffuse (*broadcast*) à tous les autres processeurs ces  $k$  prototypes avec la syntaxe : `MPI_Bcast(C, processeur_racine)`. Ensuite, on entre dans une boucle `while` jusqu'à la convergence : chaque UC  $P_l$  calcule la fonction d'étiquetage de son paquet de données  $X_l$ , et la somme cumulée des vecteurs des  $k$  groupes concernant  $P_l$  ainsi que le nombre d'éléments locaux dans chaque groupe. Puis, on agrège et redistribue globalement toutes les sommes cumulées des éléments et les cardinalités des groupes en utilisant `MPI_Allreduce`. La fonction d'agrégation (associative et commutative) peut se choisir parmi un ensemble d'opérateurs binaires comme `+` ou `min`, etc. On indique donc cette opération binaire dans les arguments de la fonction `MPI_Allreduce` : ici, `MPI_SUM`.

Le vrai code MPI avec l'API en C d'OpenMPI<sup>11</sup> diffère syntaxiquement de l'algorithme 6 dans les arguments des fonctions MPI qui demandent la longueur des messages et le type de données envoyé, etc. Rappelons que la fonction de diffusion (*broadcast*) doit être appelé par tous les processus, et que les communications par messages sont toutes bloquantes.

11. <http://www.open-mpi.org/>

Si l'on utilise l'API C++ de MPI, alors on envoie les  $k$  centroïdes à tous les autres processus en faisant :

```
for(int i = 0 ; i < k; i++)
    {MPI::COMM_WORLD.Bcast(centroid[i], dimension , MPI::DOUBLE, 0);}
```

Si l'on utilise l'API C de MPI, alors le code serait :

```
for(int i = 0 ; i < k; i++)
    {MPI_Bcast(centroid[i], dimension , MPI_DOUBLE, 0, MPI_COMM_WORLD);}
```

On gagne un facteur  $p$  (optimal) en rapidité par rapport à l'algorithme séquentiel.

## 7.9 Évaluation et comparaisons des partitions obtenues par les méthodes de regroupement

Il est important d'avoir une *vérité terrain* (c'est-à-dire un jeu de données pour lequel on connaît un bon regroupement) pour quantifier l'efficacité des différentes techniques de regroupement. Sans vérité terrain, nous n'aurions uniquement qu'une analyse subjective des clusters qui permettrait de mettre en valeur telle ou telle technique de clustering. Mais alors se poserait le problème de visualiser les regroupements quand  $d > 3$  ?

Lorsque l'on dispose de vérités terrains par des jeux de données dont on connaît déjà le nombre de clusters et l'appartenance aux clusters (c'est-à-dire la classe pour chaque donnée), on peut calculer divers indexes qui montrent la concordance du résultat obtenu en sortie de l'algorithme de regroupement avec celui étiqueté préalablement donné (et supposé optimal!).

```

/* Les  $k$ -moyennes sous MPI */
p = MPLComm_size();
r = MPLComm_rank();
avantMSE = 0;
/* Mean Square Error, la fonction de perte pour les  $k$ -moyennes */
MSE =  $\infty$ ;
if r = 0 then
    /* On initialise les centroïdes (choix aléatoire) */
    Initialiser  $C = (c_1, \dots, c_k)$ ;
    MPLBcast(C, 0);
end
while MSE  $\neq$  avantMSE do
    avantMSE = MSE;
    MSE' = 0;
    for j = 1 to k do
        m'_j = 0;
        n'_j = 0;
    end
    for i = r(n/p) to (r + 1)(n/p) - 1 do
        for j = 1 to k do
            calculer  $d_{i,j} = d^2(x_i, m_j)$ ;
        end
        Trouver le centroïde  $m_l$  le plus proche de  $x_i : l = \arg \min_j d_{i,j}$ ;
        /* Mettre à jour les quantités */
        m'_l = m'_l +  $x_i$ ;
        n'_l = n'_l + 1;
        MSE' = MSE' +  $d^2(x_i, m_j)$ ;
    end
    /* Agrégation : propriété de composition des centroïdes */
    for j = 1 to k do
        MPLAllreduce(n'_j, n_j, MPLSUM);
        MPLAllreduce(m'_j, m_j, MPLSUM);
        /* Pour ne pas diviser par zéro */
        n_j = max(n_j, 1);
        m_j = m_j / n_j;
    end
    /* Met à jour la valeur de la fonction de coût */
    MPLAllreduce(MSE', MSE, MPLSUM);
end

```

**Algorithme 6 :** Les  $k$ -moyennes parallélisées sur  $p$  processus à mémoire distribuée.

### 7.9.1 L'index de Rand

L'index de Rand (du nom de son “inventeur”) calcule la similarité entre deux partitions :  $G = \uplus G_i$  et  $G' = \uplus G'_i$  (disons, celui obtenu par les  $k$ -moyennes et un jeu de données correctement étiqueté).

On compare toutes les  $\binom{n}{2}$  paires  $(x_i, x_j)$  de points et on compte ceux qui se trouvent dans les mêmes clusters ( $a$ ) de ceux qui se trouvent dans des clusters différents ( $b$ ). On obtient ainsi l'index de Rand compris entre 0 et 1 :

$$\text{Rand}(G, G') = \frac{a + b}{\binom{n}{2}},$$

avec

- $a : \#\{(i, j) : l(x_i) = l(x_j) \wedge l'(x_i) = l'(x_j)\}$
- $b : \#\{(i, j) : l(x_i) \neq l(x_j) \wedge l'(x_i) \neq l'(x_j)\}$

La notation  $\text{condition}_1 \wedge \text{condition}_2$  veut dire que les deux conditions doivent être vraies (le ET logique). Notons que l'index de Rand évite de renuméroter les  $k$  groupes pour rendre les partitions compatibles : il y aurait sinon  $k!$  permutations à prendre en compte, pas réalisable en pratique !

### 7.9.2 L'information mutuelle normalisée (NMI)

L'information mutuelle normalisée ou *Normalized Mutual Information* (NMI) tire son principe de la théorie de l'information. Notons  $n_{j,j'} = \{x \in G_j \wedge x \in G'_{j'}\}$ .

$$\text{NMI}(G, G') = \frac{\sum_j^k \sum_{j'}^{k'} n_{j,j'} \log \frac{n \times n_{j,j'}}{n_j n_{j'}}}{\sqrt{\left(\sum_j^k n_j \log \frac{n_j}{n}\right) \left(\sum_{j'}^{k'} n'_j \log \frac{n'_j}{n}\right)}}$$

La NMI est une estimation de  $\frac{I(X;Y)}{\sqrt{H(X)H(Y)}}$  où  $I(X;Y)$  est l'information mutuelle entre deux variables aléatoires et  $H(\cdot)$  désigne l'entropie de Shannon.

## 7.10 \* Pour en savoir plus : notes, références et discussion

Nous avons vu comment regrouper des données en minimisant une fonction de coût : la somme pondérée des variances des  $k$  groupes. L'algorithme des  $k$ -moyennes a été introduit pour la première fois par Hugo Steinhaus [87] en 1956 (sous une forme différente toutefois, basée sur l'inertie d'un corps), et puis redécouvert à maintes reprises sous différentes formes (quantification, etc.). Nous avons décrit les techniques traditionnelles majeures des  $k$ -moyennes. Suivant la fonction de coût choisie les heuristiques pour regrouper peuvent être plus ou moins efficaces, et la solution du regroupement plus ou moins pertinente. Si l'on décrit axiomatiquement les propriétés d'un bon regroupement, on peut montrer qu'il n'existe pas de fonction de coût à minimiser qui respecte ces propriétés axiomatiques [55] (voir aussi les travaux [97, 20]).

L'heuristique de Lloyd a été étudiée dans [62] en 1957. En pratique, l'heuristique d'Hartigan décrite dans l'exercice 7.12 est revenue à la mode [85] pour son efficacité et ses minima globaux meilleurs que Lloyd.

L'initialisation probabiliste par les  $k$ -moyennes++ date de 2007 [3]. Une initialisation déterministe garantie des  $k$ -moyennes est détaillée dans [66] (2000) : on obtient une  $(1 + \epsilon)$ -approximation en temps  $O(\epsilon^{-2k^2 d} n \log^k n)$ . Parmi les problèmes NP-durs,  $k$ -means est plutôt "facile" à approximer puisque le problème admet un schéma d'approximation en temps polynomial (*Polynomial Time Approximation Scheme*, PTAS) [5] : c'est-à-dire que pour tout  $\epsilon > 0$ , on obtient une  $(1 + \epsilon)$ -approximation des  $k$ -moyennes en temps polynomial.

L'algorithme des  $k$ -moyennes parallélisé sur mémoire distribuée par MPI a été étudié en détail dans [28]. L'algorithme de Lloyd met à jour l'attribution des points aux groupes à chaque étape (*batched k-means*) : plus finement, on pourrait mettre à jour après chaque nouveau changement d'étiquette des points aux groupes en considérant les points un à un, et on obtiendrait ainsi l'heuristique des  $k$ -moyennes de MacQueen [64] (1967), présenté dans l'exercice 7.12.  $k$ -Means++ est intrinsèquement séquentiel et une généralisation pour architecture parallèle  $k$ -means|| a été proposée [11]. La mise à jour peut se faire sur architecture parallèle aussi par morceaux (*mini-batched k-means*) comme décrit dans [83]. Aujourd'hui avec l'avènement des *BigData*, on cherche à regrouper des jeux gigantesques avec des *milliards* ( $k$ ) de groupes [8]. Les techniques d'approximation par sous-ensembles cœurs (*core-sets*) permettent de réduire des données gigantesques en tailles minuscules [31] en contre partie d'une approximation. Une technique de construction de tels sous-ensembles cœurs en parallèle est détaillée dans [12] (2013). Un autre sujet d'actualité dans les sciences des données est le traitement des données qui respecte la *confidentialité* des données. Sur ce point, citons cet article [93] qui détaille une méthode pour le regroupement sécurisé de données confidentielles partitionnées verticalement <sup>12</sup> avec l'algorithme des  $k$ -moyennes de Lloyd.

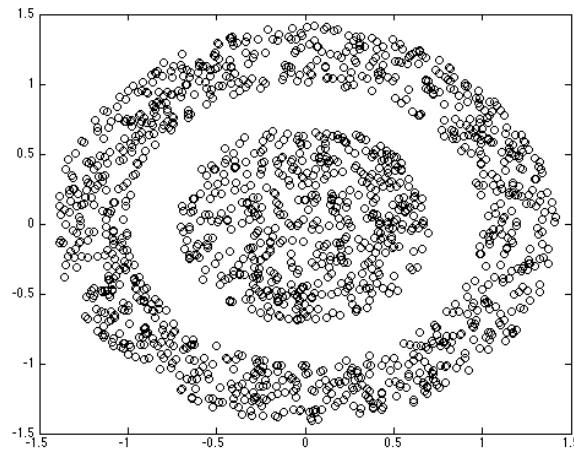
On a vu que les  $k$ -moyennes cherchaient des amas globulaires dans les données. Bien que cette technique de regroupement soit très utilisée en pratique, elle ne permet pas de résoudre tous les types de clustering, loin de là ! Par exemple, la figure 7.10 montre un jeu de données que l'œil humain classe aisément en deux groupes et pour lequel les  $k$ -moyennes n'obtiendront pas la solution car la partition de Voronoï ne permet pas d'isoler les deux classes en deux cellules de Voronoï convexes. Ce type de regroupement est résolu par les  $k$ -moyennes à noyau <sup>13</sup> [27], ou bien encore par le clustering spectral [63], etc. Un des problèmes rencontrés dans les algorithmes de regroupement est que l'on doit souvent fixer  $k$ , le nombre de clusters. Le récent et très populaire algorithme appelé *propagation par affinité* (*affinity propagation*, publié dans la prestigieuse revue américaine *Science* [35]) permet de regrouper les données en échangeant des messages entre ces données, et permet de découvrir automatiquement la valeur de  $k$ .

Pour conclure, signalons que le problème du regroupement en général n'est pas bien défini (sauf lorsqu'on précise une fonction de coût adéquate) et les mécanismes de perception humain de clusters sont étudiés finement (théorie de la Gestalt). Le regroupement est un domaine de recherche intarissable et en permanente expansion comme l'atteste les nombreux nouveaux algorithmes !

---

12. C'est-à-dire que chaque entité possède seulement un bloc de dimensions des données et toutes les entités partagent l'identifiant des données.

13. Un noyau est une transformation des données permettant de les rendre linéairement séparables. Il est toujours possible de séparer les données en augmentant la dimension des données transformées.

FIGURE 7.10 – Une limitation des  $k$ -moyennes.

## 7.11 En résumé : ce qu’il faut retenir !

La recherche exploratoire consiste à trouver des structures dans les jeux de données qui en apportent de la connaissance. Le regroupement est un ensemble de techniques qui partitionnent les données en groupes homogènes et permettent ainsi de découvrir des classes d’éléments. Le regroupement par les  $k$ -moyennes cherche à minimiser la somme pondérée des variances intra-groupes en assignant à chaque groupe un centre : son prototype qui sert de “modèle” au groupe. On parle de regroupement par modèles. Le problème des  $k$ -moyennes est NP-dur et une heuristique utilisée consiste à initialiser les centres par l’algorithme  $k$ -moyennes++ qui garantit probabilistiquement une bonne initialisation, puis d’améliorer le regroupement itérativement par l’algorithme de Lloyd qui répète jusqu’à convergence ces deux étapes (1) affectation des points aux centres les plus proches, et (2) mise à jour des centres aux centroïdes des groupes. L’heuristique de Lloyd converge vers un minimum local qui est caractérisé par une partition de Voronoï induite par les centres. Puisqu’on ne connaît pas le nombre de groupes *a priori*, il faut pouvoir aussi l’estimer en faisant de la sélection de modèles : une méthode classique est de choisir le  $k$  qui minimise le rapport des variances intra-clusters pour  $k$  et  $k + 1$ , et qui s’interprète visuellement comme le coude, point d’inflexion dans le graphe qui dessine la fonction de coût en fonction de  $k$ . On parallélise les  $k$ -moyennes sur architecture à mémoire distribuée avec l’interface MPI en utilisant la propriété de composabilité des centroïdes : le centroïde d’un ensemble de données découpé en paquets est le centroïde des centroïdes des paquets.

## 7.12 Exercices

**Exercice 11 :** *Barycentre et variance avec des poids positifs non-normalisés*

- montrez que pour un vecteur de probabilité  $w = (w_1, \dots, w_n) \in \mathbb{R}_+^d$  non-normalisé sur  $X = \{x_1, \dots, x_n\}$ , on vérifie que le minimiseur de la somme pondérée des distances Euclidiennes au carré au centre  $\sum_{i=1}^n w_i \|x_i - c\|^2$  est obtenu en choisissant comme centre  $c$  le barycentre avec la formule :

$$\bar{x} = \sum_{i=1}^n \frac{w_i}{W} x_i,$$

où  $W = \sum_{i=1}^n w_i$  est la somme totale des poids et la variance non-normalisée s'écrit :

$$v(X, w) = \sum_{i=1}^n w_i \|x_i - \bar{x}\|^2 = \sum_{i=1}^n w_i \|x_i\|^2 - W \bar{x}^2.$$

- notez que cela revient à prendre les poids normalisés  $\tilde{w}_i = \frac{w_i}{W}$  et utiliser la formule usuelle.
- que se passe-t-il si le vecteur de poids a des composantes négatives ? Peut-on encore garantir l'unicité du minimiseur ?
- en déduire la formule de composabilité des barycentres : soit  $\{X_i\}_i$   $k$  jeux de données pondérés avec leurs sommes respectives des poids totaux  $W_i$ . Montrez que :

$$\bar{x}(\uplus_{i=1}^k X_i) = \sum_{i=1}^k \frac{W_i}{\sum_{j=1}^k W_j} \bar{x}(X_i),$$

où  $\bar{x}X_i$  sont les barycentres des  $X_i$ .

### Exercice 12 : Centre de masse pour les scalaires ( $d = 1$ )

Montrez qu'en 1D, la *moyenne arithmétique*  $c = \frac{1}{n} \sum_{i=1}^n x_i$  est aussi un *centre d'équilibre* puisque nous avons :

$$\sum_{x_i < c} (c - x_i) = \sum_{x_i \geq c} (x_i - c).$$

### Exercice 13 : Décomposition biais-variance

Soit  $v(X, z) = \sum_{x \in X} \|x - z\|^2$  et  $v(X) = v(X, \bar{x})$  avec  $\bar{x} = \frac{1}{n} \sum_i x_i$ .

- montrez que  $v(X, z) = v(X) + n \|\bar{x} - z\|^2$ . En déduire que le centre de masse  $\bar{x}$  minimise  $v(X, z)$ .
- généraliser aux éléments pondérés  $X = \{(x_i, w_i)\}_i$ .
- en interprétant  $X$  comme une variable aléatoire discrète, démontrez la propriété de biais-variance pour une variable aléatoire générale  $X$ .

### Exercice 14 : Les $k$ -médoides

On cherche à maximiser la fonction de coût des  $k$ -moyennes en restreignant les prototypes  $c_j$  aux données  $x_i$ .

- montrez en utilisant la décomposition biais-variance que le meilleur coût dans ce cas-là est au pire deux fois le meilleur coût quand les prototypes ne sont pas contraints.
- en déduire une heuristique pour les  $k$ -moyennes : les  $k$ -médoides où les prototypes figurent parmi les données.
- donnez une borne sur le nombre maximal d'itérations.

**Exercice 15 :** *Minimisation des distances intra-clusters et maximisation des distances inter-clusters*

Soit  $X = \{x_1, \dots, x_n\}$  un ensemble de  $n$  données (numériques ou catégorielles) et  $D(x_i, x_j) \geq 0$  une fonction de dissimilarité entre deux éléments quelconques de  $x_i \in X$  et  $x_j \in X$ . Montrez que pour une partition de  $X$  en  $k$  clusters  $C_1, \dots, C_k$  que minimiser la somme des distances intra-clusters  $\sum_{l=1}^k \sum_{x_i \in C_l} \sum_{x_j \in C_l} D(x_i, x_j)$  est équivalent à maximiser la somme des distances inter-clusters  $\sum_{l=1}^k \sum_{x_i \in C_l} \sum_{x_j \notin C_l} D(x_i, x_j)$ . Pour des données avec des attributs catégorielles, on pourra prendre la distance de Jaccard,  $D(x_i, x_j) = \frac{|x_i \cap x_j|}{|x_i \cup x_j|}$ , et regrouper avec la technique des  $k$ -médoides de l'exercice 7.12.

**Exercice 16 :** *L'heuristique locale itérative de MacQueen [64]*

L'heuristique incrémentale de MacQueen, pour regrouper avec les  $k$ -moyennes un ensemble de  $n$  points  $\{x_1, \dots, x_n\}$ , met à jour un point à la fois jusqu'à convergence :

- On initialise  $c_j = x_j$  pour  $j = 1, \dots, k$
- On ajoute *incrémentalement* les points  $x_1, \dots, x_n$  en cyclant jusqu'à convergence : On associe  $x_i$  à son plus proche centre  $c_j$  de  $C$  puis mise à jour de ce centre : on retire  $x_i$  du centre où il était auparavant affecté puis on l'affecte au nouveau centre.
- prouvez ces formules :

$$c_{l(x_i)} \leftarrow \frac{n_{l(x_i)} c_{l(x_i)} - x_i}{n_{l(x_i)} - 1}, \quad n_{l(x_i)} \leftarrow n_{l(x_i)} - 1$$

$$l(x_i) = \arg \min_j \|x_i - c_j\|^2$$

$$c_{l(x_i)} \leftarrow \frac{n_{l(x_i)} c_{l(x_i)} + x_i}{n_{l(x_i)} + 1}, \quad n_{l(x_i)} \leftarrow n_{l(x_i)} + 1$$

- montrez que les minima locaux correspondent aux minima locaux de l'heuristique de Lloyd.
- quelle est complexité de l'algorithme ?

**Exercice 17 :** *L'heuristique d'Hartigan : échange d'un point d'un cluster à un autre [90]*

On propose l'heuristique itérative suivante pour les  $k$ -moyennes : on considère cycliquement les points  $x_i$  un à un. Pour un  $x_i$  donné appartenant au groupe  $G_{l(x_i)}$  où  $l(x_i) = \arg \min_{j=1}^k \|x_i - c_j\|^2$  est son étiquette d'appartenance, on bouge  $x_i$  dans un groupe  $G_l$  ssi la fonction de coût des  $k$ -moyennes décroît.

1. écrire  $\Delta(x_i, l)$  le gain de la fonction de coût quand  $x_i$  passe de  $G_{l(x_i)}$  à  $G_l$ . Pour  $x_i$  passant d'un cluster source  $G_s$  à un cluster target  $G_t$ , prouvez que :

$$\Delta(x_i; s \rightarrow t) = \frac{n_t}{n_t + 1} \|c_t - x_i\|^2 - \frac{n_s}{n_s - 1} \|c_s - x_i\|^2$$

- montrez que les minima locaux de cette heuristique sont un sous-ensemble des minima locaux de l'heuristique de Lloyd
- écrire un algorithme qui minimise les  $k$ -moyennes avec cette heuristique. Quelle est la complexité de votre algorithme ?
- montrez que cette heuristique évite d'avoir des clusters vides (ce qui peut-être le cas pour l'heuristique de Lloyd)

**Exercice 18 :** *\*\* Les  $k$ -moyennes pour les divergences de Bregman [13]*

La fonction de coût des  $k$ -moyennes se généralise en remplaçant la distance Euclidienne au carré par une divergence de Bregman :  $e_k(X, G) = \sum_{i=1}^n \min_{j=1}^k D_F(x_i, c_j)$ . Les divergences de Bregman se définissent pour une fonction génératrice strictement différentiable et convexe  $F(x)$  par :

$$D_F(x, y) = F(x) - F(y) - (x - y)^\top \nabla F(y),$$

où  $\nabla F(y) = (\frac{d}{dy^1} F(y), \dots, \frac{d}{dy^d} F(y))$  est le vecteur gradient.

- montrez que la distance euclidienne au carré est une divergence de Bregman pour le générateur  $F(x) = x^\top x$  mais pas la distance Euclidienne.
- montrez que le minimiseur  $\min_c \sum_{i=1}^n w_i D_F(x_i, c)$  est le barycentre  $\bar{x} = \sum_{i=1}^n w_i x_i$ . (En fait, on peut montrer que les fonctions de distances qui donnent lieu aux barycentres sont uniquement les divergences de Bregman).
- en déduire une extension de l'algorithme des  $k$ -moyennes aux divergences de Bregman.
- démontrez la propriété de composabilité des barycentres pour les divergences de Bregman.

**Exercice 19 :** *\* Les  $k$ -modes [49]*

Pour regrouper des données catégorielles (c'est-à dire non-numériques), on utilise la distance de Hamming entre deux vecteurs attributs  $d$ -dimensionnels  $x$  et  $y$  :  $D_H(x, y) = \sum_{j=1}^d 1_{x^j \neq y^j}$  où  $1_{a \neq b} = 1$  si  $a \neq b$  et zéro autrement. La distance de Hamming est une métrique respectant l'inégalité triangulaire. Soit  $t_{l,m}$  la  $m$ -ième catégorie de la  $l$ -ième dimension d'un attribut.

- montrez que le mode  $m = (m^1, \dots, m^d)$  avec  $m^j = t_{j, m^*}$  avec  $m^* = \operatorname{argmax}_m \#\{x_i^j = t_{j,m}\}$  maximise  $\sum_{i=1}^n w_i D_H(x_i, m)$  où  $\#\{\cdot\}$  indique la cardinalité d'un ensemble. C'est à dire que pour chaque dimension, on choisit la catégorie dominante pour le mode.
- montrez que contrairement aux barycentres des  $k$ -moyennes, le mode n'est pas nécessairement unique.
- écrire l'algorithme  $k$ -modes en se calquant sur les  $k$ -moyennes, et montrez comment l'utiliser pour regrouper une collection de textes.
- montrez comment adapter les  $k$ -moyennes et les  $k$ -modes pour utiliser des attributs mixtes numériques et catégorielles.

**Exercice 20 :** \*\* *Barycentres pour une fonction de distance convexe quelconque*

Soit  $D(\cdot, \cdot)$  une fonction de distance strictement convexe et différentiable (pas nécessairement symétrique ni ne respectant l'inégalité triangulaire). On définit le barycentre  $\bar{x}$  d'un nuage de points pondérés  $X = \{(x_i, w_i)\}_i$  comme le minimiseur de  $\bar{x} = \arg \min_c \sum_{i=1}^n w_i D(x_i, c)$ .

- montrez que ce barycentre est unique.
- donnez une interprétation géométrique pour l'annulation du gradient  $\nabla_c(\sum_{i=1}^n w_i D(x_i, c))$  : le barycentre est l'unique point qui annule le champ vectoriel  $V(x) = \sum_{i=1}^n w_i \nabla_x D(x_i, x)$ .

**Exercice 21 :** \*\* *Les  $k$ -moyennes en 1D par programmation dynamique [75]*

Bien que les  $k$ -moyennes soit un problème NP-dur en général, en 1D on obtient un algorithme polynomial par programmation dynamique. D'abord, on tri les  $n$  scalaires  $X = \{x_1, \dots, x_n\}$  par ordre croissant en  $O(n \log n)$ . On suppose dans la suite que  $x_1 \leq \dots \leq x_n$ .

- on cherche une relation entre le clustering optimal pour  $k$  clusters et celui pour  $k-1$  clusters. En notant  $X_{i,j} = \{x_i, \dots, x_j\}$  les sous-séquences des scalaires, écrire l'équation du meilleur clustering  $e_k(X_{1,n})$  à l'aide de  $e_{k-1}(X_{1,j-1})$  et  $e_1(X_{j,n})$ .
- montrez comment retrouver la partition optimale à partir de la table de programmation dynamique par *backtracking*. Quelle est la complexité de cet algorithme ?
- montrez qu'en prétraitant les données en trois tableaux des sommes cumulatives  $\sum_{l=1}^j w_l$ ,  $\sum_{l=1}^j w_l x_l$  et  $\sum_{l=1}^j w_l x_l^2$ , on peut calculer  $v(X_{i,j}) = \sum_{l=i}^j w_l \|x_l - \bar{x}_{i,j}\|$  en temps constant où  $\bar{x}_{i,j} = \frac{1}{\sum_{l=i}^j w_l} \sum_{l=i}^j w_l x_l$ . En déduire que les  $k$ -moyennes peuvent se calculer en 1D en  $O(n^2 k)$ .



## Chapitre 8

# Le regroupement hiérarchique

Un résumé des points essentiels à retenir est donné dans §8.7.

### 8.1 Regroupement hiérarchique ascendant et descendant : représentations enracinées par dendrogrammes

Nous continuons notre excursion pour la recherche exploratoire dans les jeux de données par les techniques de regroupement (apprentissage non-supervisé). Dans le chapitre précédent, nous avons détaillé le regroupement des données de  $X = \{x_1, \dots, x_i\}$  en partitions ( $X = \uplus_{i=1}^k G_i$ ) avec la fonction de coût des  $k$ -moyennes : dans ce cas de techniques de regroupement, on parle encore de *regroupement plat* (*flat clustering*). Dans ce chapitre, nous présentons par opposition une autre technique de clustering toute aussi usuelle en pratique : le *regroupement hiérarchique* (*hierarchical clustering*, HC).

Le regroupement hiérarchique consiste à construire une structure d'*arbre binaire de fusion* en partant des feuilles qui représentent les éléments de  $X$  (ensembles singletons), et en remontant jusqu'à la racine en fusionnant deux à deux les sous-ensembles les plus proches définis en fonction d'une distance appropriée  $\Delta(X_i, X_j)$  entre deux sous-ensembles  $X_i \subseteq X$  et  $X_j \subseteq X$  quelconques. On parle de *regroupement hiérarchique ascendant* ou *regroupement agglomératif* qui produit un arbre de fusion où l'ensemble  $X$  contenant tous les éléments se trouve à la racine.

La *représentation* graphique de cet arbre binaire de fusion s'appelle un *dendrogramme* (étymologie provenant du grec : *dendron* = arbre, *gramma* = dessiner). Par exemple, pour dessiner un dendrogramme, on peut placer un nœud  $s(X')$  codant un sous-ensemble  $X' \subseteq X$  à la hauteur  $h(X') = |X'|$  (le nombre d'éléments de  $X'$ , sa cardinalité) et pour un nœud interne, relier ce nœud à ses deux fils. Les feuilles qui contiennent les éléments  $x_i$  de  $X$  se trouvent alors toutes au niveau 1 et la racine contenant l'ensemble complet  $X$  est à hauteur  $n = |X|$ . La figure 8.1 montre un exemple de dendrogramme.

Il existe plusieurs autres façons de visualiser ces structures hiérarchiques obtenues par le regroupement hiérarchique. Par exemple, on peut utiliser des diagrammes de Venne en utilisant des ensembles convexes concentriques. Voir la figure 8.2.

La figure 8.3 montre un exemple de dendrogramme créé à partir d'un jeu de données<sup>1</sup> sur les

---

1. Disponible dans le langage R (<http://www.r-project.org/>).

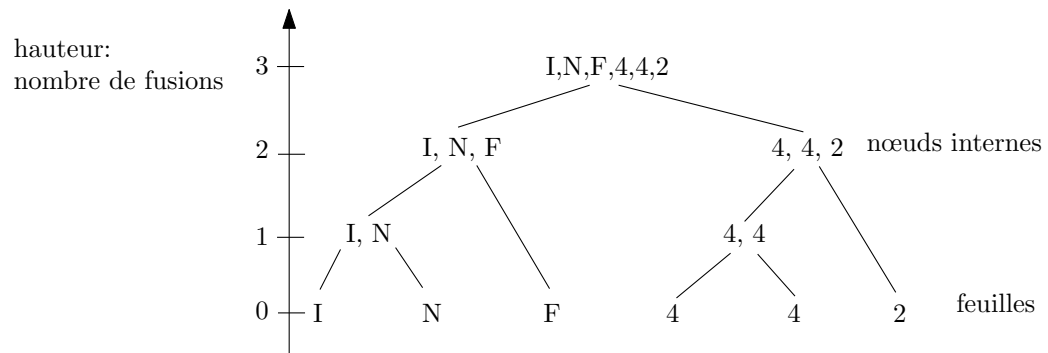


FIGURE 8.1 – Un dendrogramme est dessiné en plongeant les nœuds dans le plan grâce à une fonction de hauteur  $h(\cdot)$ .

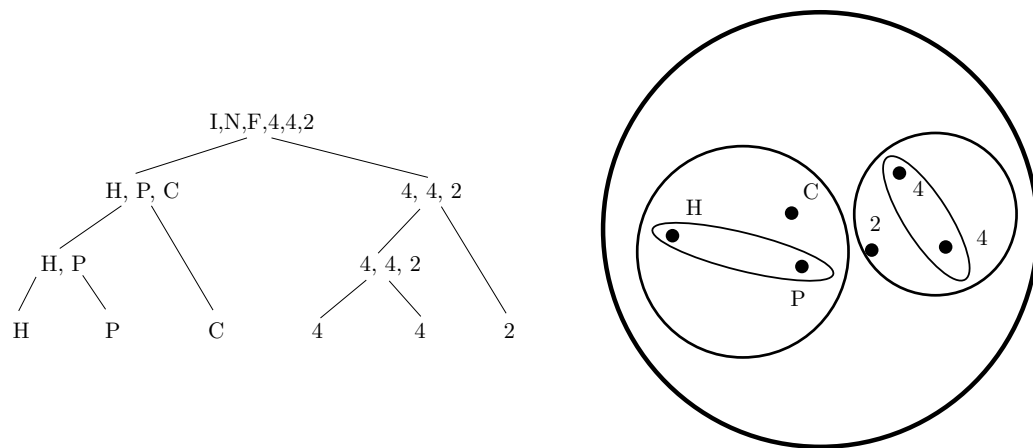


FIGURE 8.2 – Visualisation d'un dendrogramme (gauche) par un diagramme équivalent de Venn (droite) qui utilise des ellipses concentriques.

caractéristiques de voitures. On a simplement pris la distance Euclidienne sur ce jeu de données :

|                     | mpg  | cyl | displacement | horsepower | ratio | weight | quarter mile time | vs | acceleration | gear | carburetor |
|---------------------|------|-----|--------------|------------|-------|--------|-------------------|----|--------------|------|------------|
| Mazda RX4           | 21.0 | 6   | 160.0        | 110        | 3.90  | 2.620  | 16.46             | 0  | 1            | 4    | 4          |
| Mazda RX4 Wag       | 21.0 | 6   | 160.0        | 110        | 3.90  | 2.875  | 17.02             | 0  | 1            | 4    | 4          |
| Datsun 710          | 22.8 | 4   | 108.0        | 93         | 3.85  | 2.320  | 18.61             | 1  | 1            | 4    | 1          |
| Hornet 4 Drive      | 21.4 | 6   | 258.0        | 110        | 3.08  | 3.215  | 19.44             | 1  | 0            | 3    | 1          |
| Hornet Sportabout   | 18.7 | 8   | 360.0        | 175        | 3.15  | 3.440  | 17.02             | 0  | 0            | 3    | 2          |
| Valiant             | 18.1 | 6   | 225.0        | 105        | 2.76  | 3.460  | 20.22             | 1  | 0            | 3    | 1          |
| Duster 360          | 14.3 | 8   | 360.0        | 245        | 3.21  | 3.570  | 15.84             | 0  | 0            | 3    | 4          |
| Merc 240D           | 24.4 | 4   | 146.7        | 62         | 3.69  | 3.190  | 20.00             | 1  | 0            | 4    | 2          |
| Merc 230            | 22.8 | 4   | 140.8        | 95         | 3.92  | 3.150  | 22.90             | 1  | 0            | 4    | 2          |
| Merc 280            | 19.2 | 6   | 167.6        | 123        | 3.92  | 3.440  | 18.30             | 1  | 0            | 4    | 4          |
| Merc 280C           | 17.8 | 6   | 167.6        | 123        | 3.92  | 3.440  | 18.90             | 1  | 0            | 4    | 4          |
| Merc 450SE          | 16.4 | 8   | 275.8        | 180        | 3.07  | 4.070  | 17.40             | 0  | 0            | 3    | 3          |
| Merc 450SL          | 17.3 | 8   | 275.8        | 180        | 3.07  | 3.730  | 17.60             | 0  | 0            | 3    | 3          |
| Merc 450SLC         | 15.2 | 8   | 275.8        | 180        | 3.07  | 3.780  | 18.00             | 0  | 0            | 3    | 3          |
| Cadillac Fleetwood  | 10.4 | 8   | 472.0        | 205        | 2.93  | 5.250  | 17.98             | 0  | 0            | 3    | 4          |
| Lincoln Continental | 10.4 | 8   | 460.0        | 215        | 3.00  | 5.424  | 17.82             | 0  | 0            | 3    | 4          |
| Chrysler Imperial   | 14.7 | 8   | 440.0        | 230        | 3.23  | 5.345  | 17.42             | 0  | 0            | 3    | 4          |
| Fiat 128            | 32.4 | 4   | 78.7         | 66         | 4.08  | 2.200  | 19.47             | 1  | 1            | 4    | 1          |
| Honda Civic         | 30.4 | 4   | 75.7         | 52         | 4.93  | 1.615  | 18.52             | 1  | 1            | 4    | 2          |
| Toyota Corolla      | 33.9 | 4   | 71.1         | 65         | 4.22  | 1.835  | 19.90             | 1  | 1            | 4    | 1          |
| Toyota Corona       | 21.5 | 4   | 120.1        | 97         | 3.70  | 2.465  | 20.01             | 1  | 0            | 3    | 1          |
| Dodge Challenger    | 15.5 | 8   | 318.0        | 150        | 2.76  | 3.520  | 16.87             | 0  | 0            | 3    | 2          |
| AMC Javelin         | 15.2 | 8   | 304.0        | 150        | 3.15  | 3.435  | 17.30             | 0  | 0            | 3    | 2          |
| Camaro Z28          | 13.3 | 8   | 350.0        | 245        | 3.73  | 3.840  | 15.41             | 0  | 0            | 3    | 4          |
| Pontiac Firebird    | 19.2 | 8   | 400.0        | 175        | 3.08  | 3.845  | 17.05             | 0  | 0            | 3    | 2          |
| Fiat X1-9           | 27.3 | 4   | 79.0         | 66         | 4.08  | 1.935  | 18.90             | 1  | 1            | 4    | 1          |
| Porsche 914-2       | 26.0 | 4   | 120.3        | 91         | 4.43  | 2.140  | 16.70             | 0  | 1            | 5    | 2          |
| Lotus Europa        | 30.4 | 4   | 95.1         | 113        | 3.77  | 1.513  | 16.90             | 1  | 1            | 5    | 2          |
| Ford Pantera L      | 15.8 | 8   | 351.0        | 264        | 4.22  | 3.170  | 14.50             | 0  | 1            | 5    | 4          |
| Ferrari Dino        | 19.7 | 6   | 145.0        | 175        | 3.62  | 2.770  | 15.50             | 0  | 1            | 5    | 6          |
| Maserati Bora       | 15.0 | 8   | 301.0        | 335        | 3.54  | 3.570  | 14.60             | 0  | 1            | 5    | 8          |
| Volvo 142E          | 21.4 | 4   | 121.0        | 109        | 4.11  | 2.780  | 18.60             | 1  | 1            | 4    | 2          |

La représentation visuelle par dendrogramme des regroupements permet une analyse riche et qualitative par l'œil humain des informations contenues dans les jeux de données.

Le *regroupement hiérarchique divisif* ou *regroupement hiérarchique descendant* consiste, quant à lui, à partir de la racine contenant  $X$ , de trouver le meilleur découpage en deux sous-ensembles (*best split*), et de procéder ainsi de suite récursivement jusqu'aux feuilles de cardinalité unité. Nous ne nous intéressons dans la suite qu'au Regroupement Hiérarchique Agglomératif (RHA).

## 8.2 Stratégies pour définir une bonne distance de chaînage

Soit  $D(x_i, x_j)$  la distance élémentaire entre deux éléments de  $X$ . Afin de définir les deux sous-ensembles  $X_i \subseteq X$  et  $X_j \subseteq X$  les “plus proches” pour les fusionner, nous devons définir une distance (macroscopique)  $\Delta(X_i, X_j)$  entre eux qui repose sur la distance  $D(\cdot, \cdot)$ . Bien entendu, lorsque  $X_i = \{x_i\}$  et  $X_j = \{x_j\}$ , nous devons nécessairement avoir  $\Delta(X_i, X_j) = D(x_i, x_j)$ . On présente les trois fonctions les plus usuelles appelées aussi *fonctions de chaînage* (*linkage distance*) pour le critère de fusion :

1. stratégie du *saut minimum* (*Single Linkage*, SL) :

$$\Delta(X_i, X_j) = \min_{x_i \in X_i, x_j \in X_j} D(x_i, x_j)$$

2. stratégie du *saut maximum* ou *diamètre* (*Complete Linkage*, CL) :

$$\Delta(X_i, X_j) = \max_{x_i \in X_i, x_j \in X_j} D(x_i, x_j)$$

3. stratégie du saut qui minimise la *distance groupe moyenne* (*Group Average*, GA) :

$$\Delta(X_i, X_j) = \frac{1}{|X_i||X_j|} \sum_{x_i \in X_i} \sum_{x_j \in X_j} D(x_i, x_j)$$

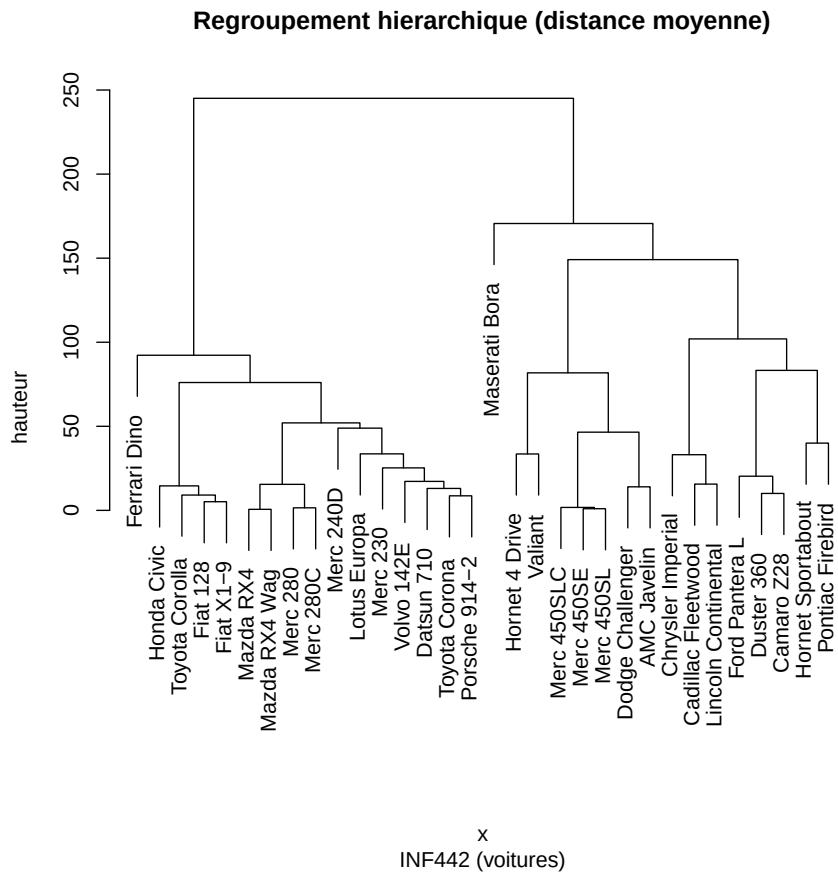


FIGURE 8.3 – Exemple de dendrogramme sur un jeu de données de voitures.

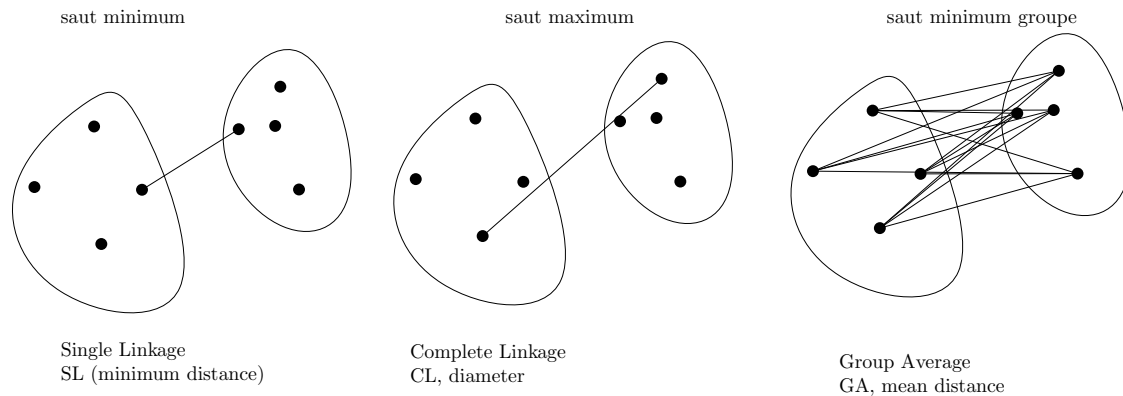


FIGURE 8.4 – Illustration pour les trois principales fonctions de chaînage : saut minimum (*single linkage*), saut maximum (*complete linkage*) et saut qui minimise la distance groupe moyenne (*group average*).

La figure 8.4 montre graphiquement la différence entre les trois principales fonctions de chaînage.

Il existe de nombreuses autres fonctions de chaînage appelées ainsi parce qu'elles chaînent littéralement les sous-arbres entre-eux dans la représentation visuelle du dendrogramme.

### 8.2.1 Algorithmes pour le regroupement hiérarchique agglomératif

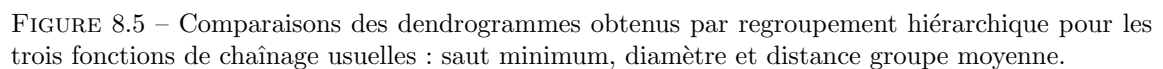
Nous résumons le principe de l'algorithme générique pour le regroupement hiérarchique agglomératif (RHA) pour une fonction de chaînage  $\Delta(\cdot, \cdot)$  donnée :

#### Algorithme **RHA**

- Initialiser pour chaque  $x_i$  un cluster singleton  $G_i = \{x_i\}$  dans une liste.
- Tant qu'il reste au moins deux clusters dans la liste :
  - Choisir  $G_i$  et  $G_j$  tel que  $\Delta(G_i, G_j)$  soit minimal,
  - Fusionner  $G_{i,j} = G_i \cup G_j$ , et (i) ajouter  $G_{i,j}$  et (ii) retirer  $G_i$  et  $G_j$  de la liste des sous-ensembles.
- Retourner le groupe restant de la liste comme la racine du dendrogramme

Puisque l'on part de  $n$  feuilles (la forêt des sous-ensembles singletons) pour arriver à la racine contenant  $X$ , nous faisons exactement  $n - 1$  opérations de fusion. Une implémentation directe de l'algorithme RHA a donc une complexité en temps cubique (en  $O(n^3)$ ). En fonction des distances de chaînage utilisées, on peut améliorer la complexité de cet algorithme naïf.

**Observation 4.** Remarquez qu'il n'y a pas unicité du dendrogramme pour une fonction de chaînage donnée (utilisant la distance entre deux éléments) : en effet, il peut y avoir plusieurs groupes les "plus proches" et on fusionne une paire et réitère. Autrement dit, si on fait une permutation sur les éléments de  $X$  et qu'on relance l'algorithme RHA, on peut obtenir un dendrogramme différent en sortie. Pour les données numériques, on peut toutefois légèrement perturber les données initiales en ajoutant sur chaque coordonnée un bruit aléatoire tiré uniformément dans  $(0, \epsilon)$  (pour un  $\epsilon > 0$  aussi petit que l'on désire) afin d'obtenir des distances distinctes.



L'algorithme optimisé standard pour le saut minimal (*single linkage*) est appelé SLINK [84] (1973) et a une complexité quadratique (en  $O(n^2)$ ). Un problème de cette fonction de chaînage à saut minimum est qu'elle engendre un phénomène de cascade des sous-arbres, qui est fort visible dans le dendrogramme. Ce phénomène est illustré à la figure 8.5. Le RHA avec le saut maximal (ou diamètre) est appelé CLINK [26] (1977) et peut être calculé en temps  $O(n^2 \log n)$ . Un problème du critère du saut maximum est qu'il est très sensible à la présence de bruit (données aberrantes appelées en anglais *outliers*) : cas de quelques points qui ne sont pas des données propres mais des artefacts de mesure et qui se trouvent à une distance éloignée des données propres. À première vue, le critère du saut moyen est coûteux à calculer, mais peut être également optimisé. On recommande souvent dans les applications ce critère de distance groupe moyenne (*group average linkage*) qui ne produit pas d'effet indésirable de chaînage et est de plus robuste au bruit.

### 8.2.2 Choix de la distance de base

La fonction de la distance de base  $D(\cdot, \cdot)$  est également très importante dans la détermination de la structure du dendrogramme. Cette fonction de distance code la *dissimilitude* (*dissimilarity*) entre deux éléments  $x_i$  et  $x_j$ . Bien qu'on utilise souvent la distance Euclidienne, on peut aussi choisir d'autres métriques comme la *distance de Manhattan* (*city block*) :

$$D_1(p, q) = \sum_{j=1}^d |p^j - q^j|,$$

(on rappelle la notation  $x = (x^1, \dots, x^j, \dots, x^d)$  pour un vecteur attribut  $x$  à  $d$  composantes : les coordonnées du vecteur  $d$ -dimensionnel  $x$ ), ou bien encore les distances de Minkowski qui généralisent à la fois la distance euclidienne ( $m = 2$ ) et la distance de Manhattan ( $m = 1$ ) :

$$D_m(p, q) = \left( \sum_{j=1}^d |p^j - q^j|^m \right)^{\frac{1}{m}}.$$

Lorsque les coordonnées des données ont des facteurs d'échelle différentes ou bien encore sont corrélées, on peut utiliser la *distance de Mahalanobis* :

$$D_\Sigma(p, q) = \sqrt{(p - q)^\top \Sigma^{-1} (p - q)} = D_2(L^\top p, L^\top q),$$

avec  $\Sigma^{-1} = L^\top L$  provenant de la factorisation de Cholesky<sup>2</sup>. C'est-à-dire que la distance de Mahalanobis revient à une distance Euclidienne classique après le changement affine :  $x \leftarrow L^\top x$ . La matrice  $\Sigma$  est la *matrice de covariance*, et son inverse  $\Sigma^{-1}$  est encore appelé *matrice de précision*. On peut l'estimer à partir d'un échantillon  $x_1, \dots, x_n$  en calculant :

$$\Sigma = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(x_i - \bar{x})^\top,$$

avec  $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$  est la moyenne empirique.

Pour les données catégorielles (non-numériques), on utilise souvent la *distance de Hamming* :

2. [http://fr.wikipedia.org/wiki/Factorisation\\_de\\_Cholesky](http://fr.wikipedia.org/wiki/Factorisation_de_Cholesky)

$$D_H(p, q) = \sum_{j=1}^d 1_{p^j \neq q^j}$$

où  $1_{a \neq b} = 1$  ssi  $a \neq b$  et zéro autrement. C'est-à-dire, la distance de Hamming compte le nombre d'attributs différents entre deux vecteurs à  $d$  attributs. Souvent, on peut relier les fonctions de *ressemblance* (*similarity*) aux fonctions de *dissimilitude* et vice-versa. Par exemple, pour la distance de Hamming sur des vecteurs binaires à  $d$  bits, une fonction de similarité sera  $S_H(p, q) = \frac{d - D_H(p, q)}{d}$  (avec  $0 \leq S_H(p, q) \leq 1$ ).

Il existe de très nombreuses autres fonctions de distances qui sont utilisées suivant les applications comme la distance de Jaccard  $D_J(A, B) = \frac{|A \cap B|}{|A \cup B|}$  sur les ensembles, la distance d'édition pour les structures combinatoires (comme les textes), la distance cosinus  $D_{\cos}(p, q) = 1 - \frac{p^\top q}{\|p\| \|q\|}$  (sur des corpus de texte), etc.

### 8.3 Critère de fusion de Ward et centroïdes

On peut également faire intervenir les *centroïdes* des groupes pour implémenter un critère qui minimise la somme des variances des sous-ensembles : c'est le chaînage par critère de Ward. Le critère de Ward pour fusionner  $X_i$  ( $n_i = |X_i|$ ) avec  $X_j$  ( $n_j = |X_j|$ ) :

$$\Delta(X_i, X_j) = \frac{n_i n_j}{n_i + n_j} \|c(X_i) - c(X_j)\|^2,$$

où  $c(X')$  est le centroïde du sous-ensemble  $X'$  :  $c(X') = \frac{1}{|X'|} \sum_{x \in X'} x$ . Notez que la distance entre deux éléments  $\Delta(\{x_i\}, \{x_j\}) = \frac{1}{2} \|x_i - x_j\|^2$  est la demi-distance Euclidienne au carré. La figure 8.7 illustre la différence des dendrogrammes obtenus pour la distance moyenne groupe et le critère de variance minimale de Ward.

On peut définir la similarité entre deux groupes  $S(X_i, X_j)$  comme  $S(X_i, X_j) = -\Delta(X_i, X_j)$ . L'opération de fusion est monotone si pour une séquence de fusion (chemin du dendrogramme) on a  $S_1 \geq S_2 \geq \dots \geq S_l$ . Un regroupement hiérarchique est dit *non-monotone* s'il existe au moins une inversion  $S_i < S_{i+1}$  sur un chemin du dendrogramme. La figure 8.6 montre un exemple d'inversion dans un dendrogramme.

L'algorithme de regroupement hiérarchique qui minimise la variance (Ward) n'est pas monotone car des inversions peuvent exister. Par contre, les critères du saut minimum (*single linkage*), du saut maximum (*complete linkage*) et de la distance groupe moyenne (*average group linkage*) sont garantis monotones. Si l'on dessine les nœuds du dendrogramme avec une hauteur en fonction de la distance ou similarité (en traçant une ligne hauteur), une inversion dans un dendrogramme se remarque par le fait qu'une ligne hauteur horizontale est plus basse qu'une ligne hauteur horizontale d'une précédente opération de fusion. Cela contredit aussi le fait que sur un chemin de fusion des feuilles à la racine, la similarité doit décroître monotonement.

### 8.4 Partitionnements à partir du dendrogramme

On peut à partir d'un regroupement hiérarchique stocké dans un dendrogramme obtenir des regroupements plats en partitions en choisissant un niveau pour obtenir la partition en sous-ensembles.

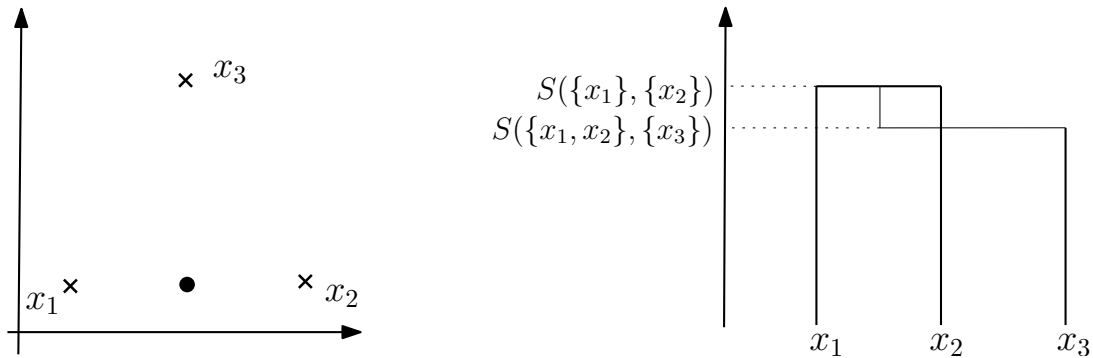


FIGURE 8.6 – Petit exemple montrant le phénomène d'inversion dans un dendrogramme obtenue par le regroupement hiérarchique en utilisant le critère de Ward.

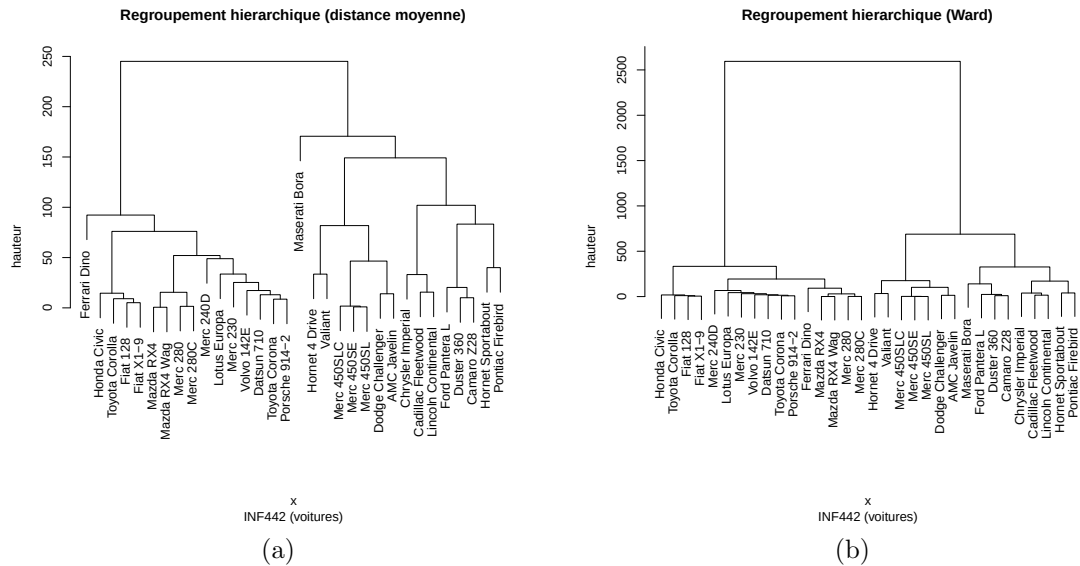


FIGURE 8.7 – Comparaisons des dendrogrammes obtenus pour (a) la distance moyenne groupe et le critère de variance minimale de Ward (b).

La figure 8.8 illustre ce point en montrant deux coupes possibles. Les niveaux de coupes ne sont pas obligatoirement constants (voir l'exercice 8.8).

## 8.5 Distances ultramétriques et arbres phylogénétiques

Une distance  $D(\cdot, \cdot)$  est dite *métrique* si elle satisfait ces trois axiomes :

**Loi des indiscernables.**  $D(x, y) \geq 0$  avec égalité ssi.  $x = y$ ,

**Symétrie.**  $D(x, y) = D(y, x)$

**Inégalité triangulaire.**  $D(x, y) \leq D(x, z) + D(z, y)$ ,

La distance Euclidienne et la distance de Hamming sont deux exemples de métriques. Par contre, la distance Euclidienne *au carré* bien que symétrique et respectant la propriété des indiscernables n'est pas une métrique car l'inégalité triangulaire n'est pas satisfaite. La loi des indiscernables est parfois décomposée en deux : loi de non-négativité ( $D(p, q) \geq 0$ ) et loi de réflexivité ( $D(p, q) = 0 \Leftrightarrow p = q$ ).

Le regroupement hiérarchique est lié à une classe de distances appelée *classe ultramétrique*. Une distance est ultramétrique ssi. elle est métrique et qu'en plus elle possède la propriété ultramétrique :

$$D(x, y) \leq \max_z (D(x, z), D(z, y)).$$

Attention : dans le jargon courant (notamment celui des journaux), on appelle malheureusement métrique un *indicateur numérique* qui permet de quantifier mais ne respecte pas forcément les propriétés mathématiques des métriques.

Dans la théorie de l'évolution décrite par les arbres phylogénétiques, la distance entre deux espèces impose des restrictions sur la fonction distance  $D(\cdot, \cdot)$ . On écrit compactement  $D_{i,j} = D(x_i, x_j)$ . Un arbre est dit *additif* (*additive tree*) ssi. on peut mettre un poids sur chaque arête tel que pour chaque paire de feuilles, la distance est la *somme des distances des arêtes les reliant*. Un arbre est *ultramétrique* (*ultrametric tree*) lorsque les distances entre deux feuilles  $i$  et  $j$  et leur *ancêtre commun*  $k$  sont égales :  $D_{i,k} = D_{j,k}$ . On peut dessiner le dendrogramme d'un arbre ultramétrique en choisissant la hauteur  $\frac{1}{2}D_{i,j}$  que l'on peut interpréter comme le temps écoulé : on a ainsi défini une *horloge globale* parmi tous les éléments de  $X$ .

L'algorithme de regroupement hiérarchique ascendant avec la fonction de chaînage de la moyenne garantit de produire un arbre ultramétrique. On nomme cet algorithme qui en plus associe à chaque nœud sa hauteur : l'algorithme UPGMA (*Unweighted Pair Group Method using arithmetic Averages*, UPGMA).

**Algorithme UPGMA :**

- Pour tout  $i$ , initialiser  $x_i$  à son cluster  $C_i = \{x_i\}$  et positionner ce nœud feuille à hauteur 0.
- Tant qu'il reste plus de deux clusters :
  - Trouver les clusters  $C_i$  et  $C_j$  qui ont la distance  $\Delta_{i,j}$  minimale
  - Définir un nouveau cluster  $C_k = C_i \cup C_j$  et calculer la distance  $\Delta_{k,l}$  pour tout  $l$
  - Ajouter un nœud  $k$  avec les fils  $C_i$  et  $C_j$  et positionner ce nœud à hauteur  $\frac{1}{2}\Delta(C_i, C_j)$
  - Retirer  $C_i$  et  $C_j$  de la liste des clusters, et continuer jusqu'à obtenir la racine.
- Pour les deux derniers clusters  $C_i$  et  $C_j$ , placer la racine de l'arbre de fusion à hauteur  $\frac{1}{2}\Delta(C_i, C_j)$ .

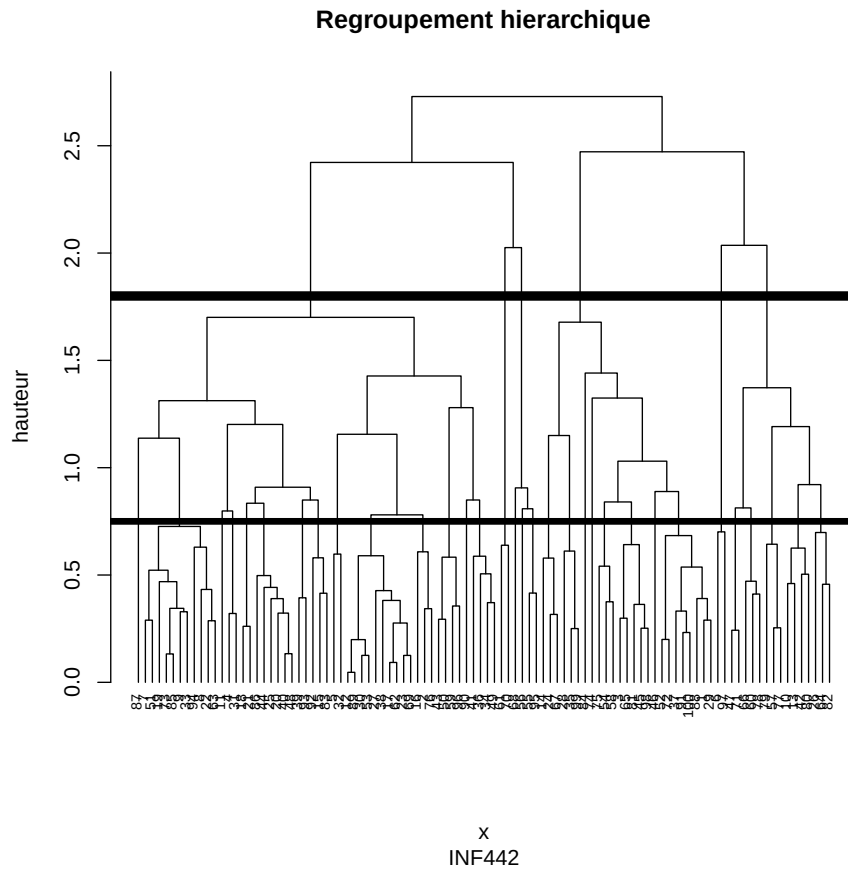


FIGURE 8.8 – Obtenir des partitions à partir d’un dendrogramme : choisir la hauteur de la coupe. À une hauteur donnée, on obtient un clustering plat (*cf.* les  $k$ -moyennes) en récupérant la partition induite par la coupe. Cette coupe ne doit pas nécessairement être à une hauteur constante. Un dendrogramme permet ainsi d’obtenir de nombreux regroupements par partitions. Ici, on montre deux coupes à hauteur constante pour  $h = 0,75$  et  $h = 1,8$ .

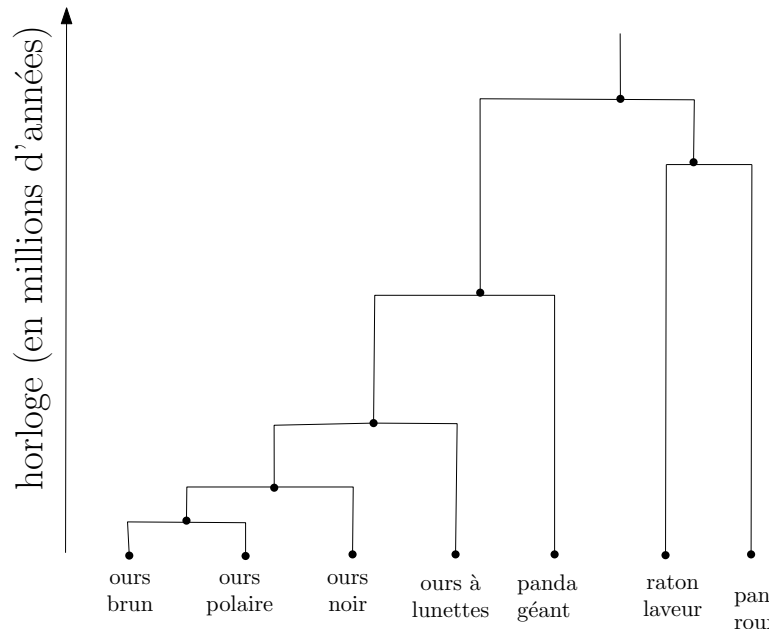


FIGURE 8.9 – Dendrogrammes et arbres phylogénétiques.

**Théorème 9.** *Si les données  $X$  sont accompagnées d'une matrice carrée des distances  $M = [D_{i,j}]_{i,j}$  avec  $D_{i,j} = D(x_i, x_j)$  qui satisfait les propriétés ultramétriques, alors il existe un unique arbre ultramétrique qui peut être construit par l'algorithme UPGMA.*

Les arbres philogénétiques utilisés dans la théorie de l'évolution des espèces sont des dendrogrammes pour lesquels on peut mettre sur l'axe vertical une horloge globale (cf. la figure 8.9). L'algorithme UPGMA permet donc la construction d'un arbre phylogénétique. Malheureusement en pratique les données ne sont pas souvent ultramétriques (car bruitées)! Une autre limitation pour les grandes données est le fait de considérer la matrice des distances qui requiert un espace mémoire quadratique.

## 8.6 \* Pour en savoir plus : notes, références et discussion

Les algorithmes de regroupements hiérarchiques depuis SLINK [84] (Single Linkage, 1973) et CLINK [26] (Complete Linkage, 1977) sont nombreux et bien étudiés [69] dans un formalisme général. Bien que le regroupement plat par les  $k$ -moyennes soit NP-dur, il a été démontré récemment (2012) qu'on pouvait à partir d'un regroupement hiérarchique à saut minimum (SLINK) retrouver la partition des  $k$ -moyennes optimale par programmation dynamique sous hypothèse d'avoir un regroupement "stable" [6] sous des critères de perturbation. L'algorithme d'agglomération hiérarchique qui minimise le critère de variance de Ward et ses variantes ont été étudiés dans [95, 70]. Ces différents algorithmes (SLINK, CLINK et Ward) peuvent être unifiés dans l'algorithme générique de Lance-Williams [57] (cf. exercice 8.8). L'unicité et la monotonie pour le regroupement hiérarchique sont détaillées dans [68]. Bien que les algorithmes de regroupements hiérarchiques se parallélisent a

*priori* moins bien que ceux des regroupements par partitions, mentionnons les travaux [76]. Pour le regroupement hiérarchique divisif, on peut aussi consulter les travaux [71] qui maximisent la notion de modularité.

## 8.7 En résumé : ce qu'il faut retenir !

Le regroupement hiérarchique se distingue du regroupement en partitions par le fait qu'il construit un arbre binaire de fusion en partant des feuilles qui contiennent les éléments et en remontant jusqu'à la racine qui renferme l'ensemble des données. La représentation graphique de cet arbre de fusion enraciné est appelé un dendrogramme. Pour implémenter un regroupement hiérarchique, on doit choisir une fonction de chaînage (saut minimum, saut maximum, distance groupe moyenne, critère de variance de Ward, etc.). Un regroupement hiérarchique est monotone ssi. pour tout chemin d'une feuille à la racine la similarité décroît sinon il existe au moins une inversion. Les critères de saut minimum, saut maximum, et de distance groupe moyenne garantissent la monotonie mais pas celui de la variance minimum de Ward. On peut extraire d'un dendrogramme de nombreuses partitions qui correspondent à des regroupements par partitions. Les arbres phyllogénétiques sont des arbres ultramétriques. L'algorithme de regroupement hiérarchique par chaînage en utilisant la distance groupe moyenne produit des arbres ultramétriques si la distance de base est ultramétrique.

## 8.8 Exercices

**Exercice 22 :** *Vérification de la propriété ultramétrique d'une matrice de distances*

On se donne une matrice  $M$  de taille  $n \times n$  qui stocke à l'entrée  $(i, j)$  la distance  $D(x_i, x_j)$ .

- construire un algorithme qui vérifie les propriétés ultramétriques de  $M$ .
- quelle est la complexité de votre algorithme ?

**Exercice 23 :** *Métrieue Euclidienne et métrieue de Hamming*

- démontrez que la distance Euclidienne est une métrieue mais pas son carré,
- prouvez que la distance de Hamming est une distance métrieue.

**Exercice 24 :** *Combiner regroupement hiérarchique et regroupement plat*

Soit  $X = \{x_1, \dots, x_n\}$   $n$  données à  $d$  attributs.

- proposez un algorithme qui regroupe hiérarchiquement les données et en déduit une partition en un nombre de paquets d'au plus  $l$  éléments (sur-regroupement) et ensuite utilise un algorithme des  $k$ -moyennes sur les centroïdes de ces paquets.
- quelle est la complexité de votre algorithme ? et son avantage par rapport à un regroupement hiérarchique simple ou à un regroupement par partition ?

**Exercice 25 :** *Le regroupement hiérarchique de Lance-Williams [57]*

- décrivez l'algorithme de regroupement hiérarchique ascendant avec les notations  $D_{ij}$  pour  $\Delta(C_i, C_j)$  et  $D_{(ij)k}$  pour  $\Delta(C_i \cup C_j, C_k)$  pour des groupes disjoints  $C_i, C_j$  et  $C_k$ .
- Un algorithme de regroupement hiérarchique appartient à la famille de Lance-Williams ssi. la distance peut s'écrire *canoniquement* comme :

$$D_{(ij)k} = \alpha_i D_{ik} + \alpha_j D_{jk} + \beta D_{ij} + \gamma |D_{ik} - D_{jk}|,$$

avec  $\alpha_i, \alpha_j, \beta$ , et  $\gamma$  des paramètres dépendants de la taille des clusters. Montrez que le critère de variance minimum de Ward ( $D(x_i, x_j) = \|x_i - x_j\|^2$ ) pour des groupes  $C_i, C_j$  et  $C_k$  disjoints donne la formule suivante :

$$D(C_i \cup C_j, C_k) = \frac{n_i + n_k}{n_i + n_j + n_k} D(C_i, C_k) + \frac{n_j + n_k}{n_i + n_j + n_k} D(C_j, C_k) - \frac{n_k}{n_i + n_j + n_k} D(C_i, C_j).$$

- en déduire que l'algorithme de Ward est un cas spécial de l'algorithme de Lance-Williams avec

$$\alpha_i = \frac{n_i + n_k}{n_i + n_j + n_k}, \quad \beta = \frac{-n_k}{n_i + n_j + n_k}, \quad \gamma = 0.$$

- montrez que l'algorithme de Lance-Williams permet d'unifier les fonctions de chaînage de saut minimum (*single linkage*), saut maximum (*complete linkage*) et distance groupe moyenne (*group average linkage*).

**Exercice 26 :** *Regroupement hiérarchique par centroïde pour une fonction de distance donnée*

Pour une distance convexe  $D(\cdot, \cdot)$ , on définit le centroïde de  $X$  comme l'unique minimiseur de  $\min_c \sum_{x \in X} D(x, c)$ . Montrez que le phénomène d'inversions qui peut se produire avec le critère de Ward pour la distance Euclidienne au carré n'apparaît plus pour la distance Euclidienne ou la distance de Manhattan.

**Exercice 27 :** \* *Calcul par programmation dynamique de la meilleure partition des  $k$ -moyennes à*

*partir d'un dendrogramme [6]*

Étant donné un dendrogramme, on peut en extraire de nombreuses partitions :

- combien de partitions peuvent-elles être encodées dans le dendrogramme ?
- pour un sous-ensemble  $X'$ , on note par  $c(X')$  le centroïde de  $X'$  et par  $v(X')$  sa variance :  $v(X') = \frac{1}{|X'|} \sum_{x \in X'} x^\top x - (c(X')^\top c(X'))^2$ . Proposez un algorithme de programmation dynamique qui extrait le meilleur clustering par partition d'un dendrogramme pour la fonction de coût des  $k$ -moyennes. Quelle est la complexité de votre algorithme ?

**Exercice 28 :** \* *Distance cosinus entre documents*

Soit  $p$  et  $q$  deux vecteurs à  $d$  attributs et considérons la distance cosinus :  $D(p, q) = \cos \theta_{p,q} = 1 - \frac{p^\top q}{\|p\| \|q\|}$ . La distance cosinus est une distance angulaire qui ne tient pas compte de la magnitude

des vecteurs. Pour des documents textes, on modélise un texte  $t$  par un vecteur  $f(t)$  qui compte les mots d'un dictionnaire apparaissant dans le texte.

- montrez que cette distance satisfait les axiomes d'une métrique.
- donnez un algorithme de clustering hiérarchique qui permet le regroupement de documents textes
- décrivez un algorithme de regroupement par partitionnement qui généralise l'algorithme des  $k$ -moyennes. On pourra considérer les vecteurs attributs des documents comme un nuage de points sur la sphère et montrer que le centroïde "local" est le centre de masse Euclidien reprojété sur la sphère.

**Exercice 29 :** \* *Regroupement hiérarchique pour les divergences de Bregman [89]*

Les divergences de Bregman se définissent pour une fonction génératrice strictement différentiable et convexe  $F(x)$  par :

$$D_F(x, y) = F(x) - F(y) - (x - y)^\top \nabla F(y),$$

où  $\nabla F(y) = (\frac{d}{dy^1} F(y), \dots, \frac{d}{dy^d} F(y))$  est le vecteur gradient.

- montrez que pour  $F(x) = x^\top x$ , la divergence de Bregman est la distance Euclidienne au carré.
- prouvez que les divergences de Bregman ne sont jamais des métriques.
- démontrez que le critère de fusion généralisant celui de Ward pour les divergences de Bregman est :

$$\Delta(X_i, X_j) = |X_i| D_F(c(X_i), c(X_i \cup X_j)) + |X_j| D_F(c(X_j), c(X_i \cup X_j)),$$

avec  $c(X_l)$  le centre de masse de  $X_l$ .

- en déduire un algorithme de regroupement hiérarchique basé sur un critère de centroïde à la Ward pour les divergences de Bregman.

**Exercice 30 :** \* *Regroupement hiérarchique par saut minimum et arbre recouvrant de poids minimal [39]*

Montrez que l'information contenue dans l'*arbre recouvrant de poids minimal* (*minimum spanning tree (MST)*) permet de retrouver facilement le dendrogramme obtenu par le regroupement hiérarchique par saut minimum. En déduire un algorithme en temps quadratique.



## Chapitre 9

# La classification supervisée par les $k$ -Plus Proches Voisins

Un résumé des points essentiels à retenir est donné dans §9.7.

### 9.1 L'apprentissage supervisé

La classification supervisée consiste à partir d'un jeu de données étiquetées  $Z = \{(x_i, y_i)\}_i$  avec  $y_i \in \pm 1$  (les étiquettes<sup>1</sup> pour les deux classes  $C_{-1}$  et  $C_{+1}$ ) à apprendre un *classifieur*  $l(\cdot)$  tel que pour de nouvelles requêtes  $Q = \{x'_i\}_i$  (des exemples pas encore classés), on cherche à déterminer leurs étiquettes :  $y'_i = l(x'_i)$  en utilisant la “fonction” classifieur. Le jeu de données étiquetées  $Z$  est appelé *jeu d'entraînement* (*training set*) et le jeu des requêtes  $Q$  (*query data-set*), le jeu de test (*testing set*). Nous allons voir dans ce chapitre un algorithme très simple et néanmoins performant qui consiste en une règle de classification par vote majoritaire sur les  $k$ -Plus Proches Voisins ( $k$ -PPVs, en anglais *k-Nearest Neighbors*,  $k$ -NNs). Dans le cas de deux classes, on parle de *classification binaire*, sinon de *classification multi-classes*.

### 9.2 La règle du proche voisin (PPV)

Notons  $X = \{x_i \mid (x_i, y_i) \in Z\}$  les données attributs du jeu de données étiquetées. La règle du Plus Proche Voisin (PPV, en anglais *nearest neighbor classification*) donne comme étiquette  $l(x)$  à  $x$  celle de son plus “proche” voisin dans le jeu d'entraînement  $Z$ . C'est-à-dire, on a  $l(x) = y_e$  pour  $e = \arg \min_{x_i \in X} D(x, x_i)$ . Notons qu'en cas de non-unicité de la fonction  $\arg \min$ , on choisit arbitrairement un index ayant donné lieu à la distance minimale : par exemple, l'index le plus petit. Le plus proche voisin est donc défini en fonction d'une *distance* appropriée  $D(\cdot, \cdot)$  entre deux éléments quelconques. Par exemple, nous avons déjà vu dans les chapitres précédents la distance euclidienne  $D(p, q) = \sqrt{\sum_{j=1}^d (p^j - q^j)^2}$  et les distances de Minkowski  $D_l(p, q) = \left(\sum_{j=1}^d |p^j - q^j|^l\right)^{\frac{1}{l}}$  (des métriques pour  $l \geq 1$ ) pour les attributs numériques, et la distance de Hamming  $D_H(p, q) =$

---

1. Suivant le contexte, on peut encore noter ces deux classes par  $C_1$  et  $C_2$  au lieu de  $C_{-1}$  et  $C_{+1}$ .

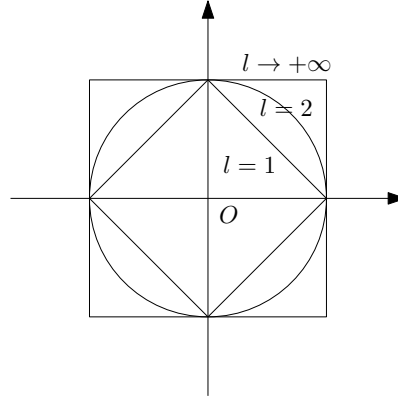


FIGURE 9.1 – Boules de Minkowski  $\{x \mid D_l(O, x) \leq 1\}$  avec  $D_l(p, q) = \left(\sum_{j=1}^d |p^j - q^j|^l\right)^{\frac{1}{l}}$  pour différentes valeurs de  $l \geq 1$ . Pour  $l = 2$ , on retrouve la boule euclidienne. Pour  $l = 1$ , on a la boule de Manhattan (de forme carrée) et quand  $l \rightarrow +\infty$ , on tend vers un carré, orienté à 45 degrés de celui de Manhattan.

$\sum_{j=1}^d (1 - \delta_{p^j}(q^j)) = \sum_{j=1}^d 1_{[p^j \neq q^j]}$ , pour les attributs catégoriques. La notation  $\delta_x(y) = 1$  est la fonction de Dirac qui vaut 1 ssi.  $y = x$ , et 0 autrement.

On classe ainsi  $t = |Q|$  nouvelles observations en faisant  $t$  requêtes de Plus Proche Voisins (PPVs) dans  $X$ . Le calcul rapide de PPVs est donc important. Cela peut se faire naïvement en parcourant tous les éléments de  $X$ , en temps linéaire. Il existe de nombreuses structures de données pour répondre aux requêtes de PPVs mais lorsque la dimension  $d$  augmente, il est difficile de battre l'algorithme naïf. On parle de la malédiction des grandes dimensions (*curse of dimensionality*) pour décrire ce phénomène !

### 9.2.1 Optimisation du calcul du plus proche voisin pour la distance euclidienne

Souvent nous choisissons la distance euclidienne pour laquelle les calculs peuvent-être optimisés en pratique : en effet, remarquons que la distance ou bien une fonction monotone de celle-ci, comme son carré, ne change pas l'ordre total des distances  $D(q, x_i)$  (utilisé par le  $\arg \min$ ) pour une requête  $q$  donnée et  $x \in X$ . Ainsi, nous avons  $l = \arg \min_{i=1}^n D(q, x_i) = \arg \min_{i=1}^n D^2(q, x_i)$ . Le calcul de la distance euclidienne entre deux vecteurs attributs à  $d$  dimensions revient à calculer  $d$  soustractions,  $d$  opérations carrés et  $d$  sommes, soit naïvement  $3d$  opérations arithmétiques. On peut aussi calculer  $D^2(q, x_i) = \langle q - x_i, q - x_i \rangle$  comme un produit scalaire en  $2d$  opérations. On trouve alors le plus proche voisin du jeu d'entraînement en temps  $2dn$ . Maintenant, si on pré-calcule les normes au carré  $\text{norm}^2(p) = \langle p, p \rangle = \sum_{j=1}^d (p^j)^2$  en temps linéaire  $2dn$ , alors on peut calculer plus rapidement  $D^2(p, q)$  comme  $D^2(p, q) = \text{norm}^2(p) + \text{norm}^2(q) - 2\langle p, q \rangle$  : c'est-à-dire que cela revient à faire  $n$  produits scalaires pour chaque requête du jeu de test  $Q$ . Une requête de plus proche voisin coûte au total  $d + (d + 1)n$  au lieu de  $3dn$ . Les cartes graphiques (*Graphical Processing Units*, GPUs) permettent aussi de calculer très rapidement les produits scalaires.

### 9.2.2 Règle du PPV et les diagrammes de Voronoï

Pour un jeu d'apprentissage  $Z$  de  $n = |Z|$  données, l'espace  $\mathbb{R}^d$  est partitionné en  $n$  *classes d'équivalence* pour la fonction d'étiquetage ( $\arg \min$  constant) : Ce sont les cellules de Voronoï qui décomposent le domaine en cellules de proximité. Cette notion a déjà été abordée dans le chapitre sur les  $k$ -moyennes.

Ici, puisque nous avons des générateurs de Voronoï coloriés en deux classes, le diagramme de Voronoï décompose l'espace en cellules bi-couleurs (diagramme bichromatique), et la frontière entre les changements de couleur indique la *frontière de décision* du classifieur par la règle du PPV. La figure 9.2 illustre ces aspects géométriques.

### 9.2.3 La règle des $k$ -Plus Proches Voisins

Afin de rendre plus robuste aux bruits le classifieur, on classe une nouvelle observation  $q$  en choisissant parmi ses  $k \in \mathbb{N}$  plus proches voisins dans  $X$ , la classe majoritaire (pour la classification binaire, on choisit  $k$  impair afin d'avoir un vote majoritaire). En pratique, augmenter  $k$  permet d'être tolérant aux données artefacts (*outliers*) mais les frontières de décision deviennent plus floues. Il existe de très nombreuses techniques qui permettent de choisir le bon  $k$  comme la méthode de validation croisée (*cross-validation*). Cette règle dite des  $k$ -Plus Proches Voisins ( $k$ -PPVs) généralise la règle du PPV (en prenant  $k = 1$ , PPV=1-PPV). Dans le cadre d'un apprentissage supervisé multi-classes à  $c$  classes, la règle consiste à choisir la classe majoritaire présente dans les  $k$ -PPVs.

## 9.3 Évaluation de la performance d'un classifieur

On a vu que suivant  $k$ , on peut construire une famille de classifieur  $l_k(\cdot)$ . Afin de choisir le meilleur classifieur dans cette famille, il faut pouvoir évaluer la performance d'un classifieur.

### 9.3.1 Taux d'erreur de classification (*misclassification*)

Le taux d'erreur sur un jeu de données (jeu de test  $Q$ ) comportant  $t$  nouvelles observations à classer (et donc différentes de celles du jeu d'apprentissage) est :

$$\tau_{\text{Erreur}} = \frac{\#\text{mal classé}}{t} = 1 - \frac{\#\text{bien classé}}{t} = \tau_{\text{misclassification}}.$$

Cet indicateur n'est pas très discriminant lorsque les proportions des étiquettes des classes sont très déséquilibrées entre elles (notamment dans le jeu d'entraînement). Par exemple, lorsque l'on classe des messages en  $C_{\text{spam}}$  (classe des pourriels) et  $C_{\text{ham}}$  (classe des courriels, non-spam), on a souvent beaucoup moins de spams que de courriels. Si l'on cherche à optimiser le taux d'erreur de classification (*misclassification*, les données mal classées), il serait alors préférable de classer le tout en non-spam, et avoir ainsi réalisé un bon taux d'erreur moyen ! Il faut donc pouvoir pondérer les erreurs de classification en fonction des proportions des classes.

### 9.3.2 Matrices de confusion et faux positifs/négatifs

La matrice de confusion  $M = [m_{i,j}]_{i,j}$  stocke dans ses entrées  $m_{i,j}$  le taux de réussite (c'est-à-dire, la valeur un moins le taux d'erreur) d'avoir classé  $x$  comme classe  $C_i$  (classe estimée) alors

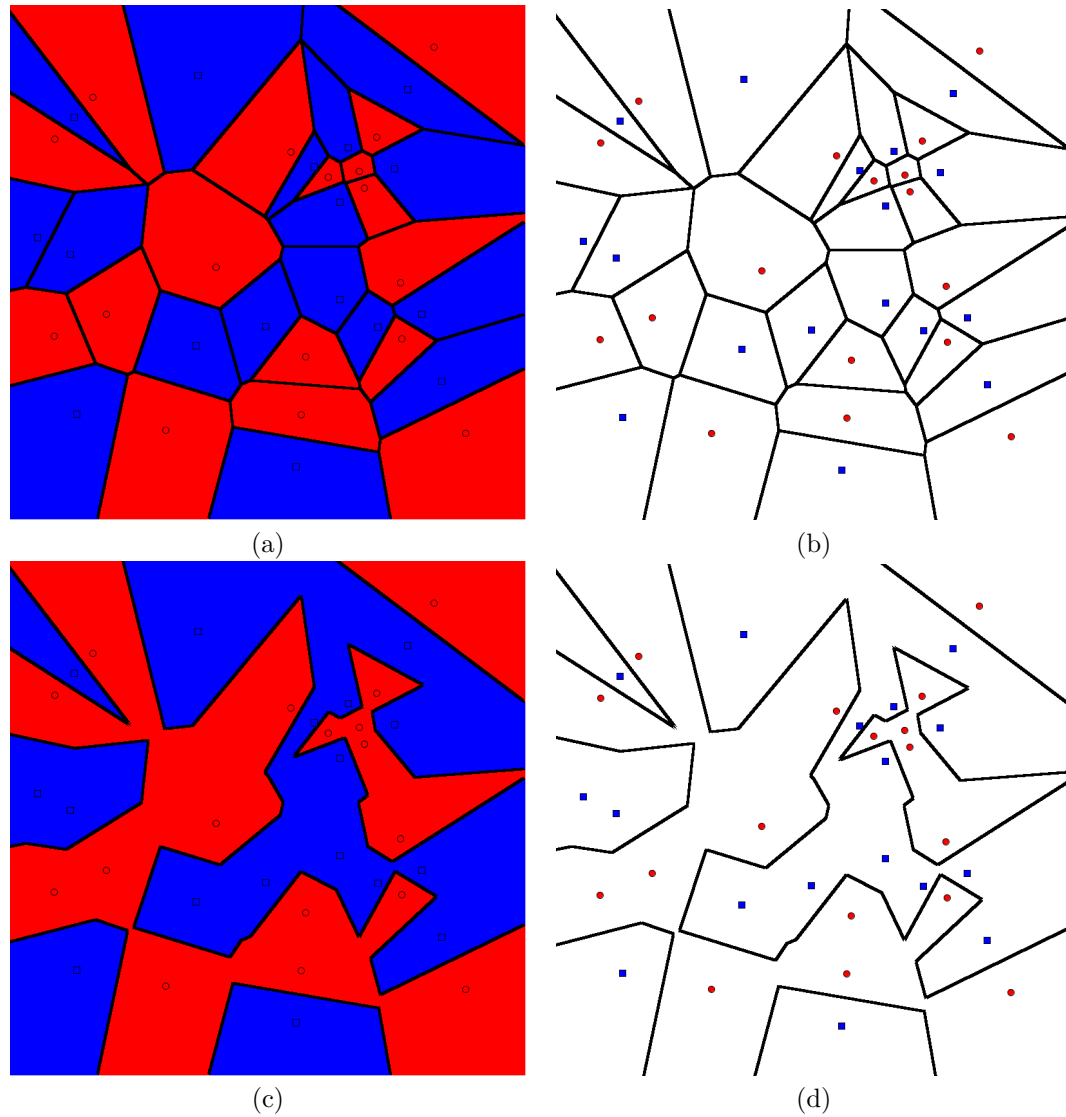


FIGURE 9.2 – Classifieur par la règle du Plus Proche Voisin (PPV) et le diagramme de Voronoï bichromatique : (a) diagramme de Voronoï, (b) frontières des cellules de Voronoï, (c) classifieur par le Plus Proche Voisin (les classes sont des unions monochromatiques de cellules de Voronoï), et (d) frontière du classifieur induite par l'union des cellules d'une même couleur en deux régions de classes.

que  $x$  appartient à la classe  $C_j$  (vraie classe) :

$$m_{i,j} = \tau_{(x \text{ prédit } C_i | x \in C_j)}$$

La diagonale de la matrice de confusion  $M$  montre ainsi le taux de réussite pour toutes les classes. Les cas mal-prédits sont appelés soit faux positifs (FP) soit faux négatifs (FN) :

- Un faux positif (FP, *False Positive*) est une observation  $x$  mal classée en  $C_{+1}$  alors qu'elle appartient en fait à  $C_{-1}$ . Ici, positif veut dire classe “+1”.
- Un faux négatif (FN, *False Negative*) est une observation  $x$  mal classée en  $C_{-1}$  alors qu'elle appartient à  $C_{+1}$ . Ici, négatif veut dire classe “-1”.

Les faux positifs sont encore appelés erreurs de type I, et les faux négatifs, erreurs de type II. De façon analogue, on définit les vrais négatifs (TN, *True Negative*) et les vrais positifs (TP, *True Positive*).

Ainsi, le taux d'erreur peut se réécrire comme :

$$\tau_{\text{erreur}} = \frac{\text{FP} + \text{FN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} = 1 - \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}},$$

puisque  $\text{TP} + \text{TN} + \text{FP} + \text{FN} = t = |Q|$ , le nombre de requêtes à classer du jeu de données test.

### 9.3.3 Précision, rappel et $F$ -score

La précision est la proportion de vraies (TP) dans les données classées “vraies” (vraiment “vrai” TP et faussement “vrai” FP)

$$\tau_{\text{Précision}} = \frac{\text{TP}}{\text{TP} + \text{FP}}.$$

On a bien entendu,  $0 \leq \tau_{\text{Précision}} \leq 1$ .

Le rappel (*Recall*) est la proportion de vrai “vrai” (TP) dans les données classées “vrai” (vraiment “vrai” TP et faussement “faux” FN = vrai) :

$$\tau_{\text{Rappel}} = \frac{\text{TP}}{\text{TP} + \text{FN}}.$$

Le taux de rappel indique la sensibilité.

Le  $F$ -score est un taux qui donne autant de poids aux faux positifs qu'aux faux négatifs. Il est défini comme la moyenne harmonique<sup>2</sup> de la précision et du rappel :

$$\tau_{\text{F-score}} = \frac{2 \times \tau_{\text{Précision}} \times \tau_{\text{Rappel}}}{\tau_{\text{Précision}} + \tau_{\text{Rappel}}}$$

En pratique, on choisit les classifieurs qui donnent les meilleurs  $F$ -scores. Par exemple, pour plusieurs valeurs (impaires) de  $k$ , on peut évaluer la performance des classifieurs par les  $k$ -PPVs en calculant leur  $F$ -score, et choisir la valeur de  $k$  qui a donné le meilleur  $F$ -score.

---

2. La moyenne harmonique  $h(x, y) = \frac{1}{\frac{1}{2} \frac{1}{x} + \frac{1}{2} \frac{1}{y}} = \frac{2xy}{x+y}$  est souvent utilisée pour moyenner des *rapports* de quantités.

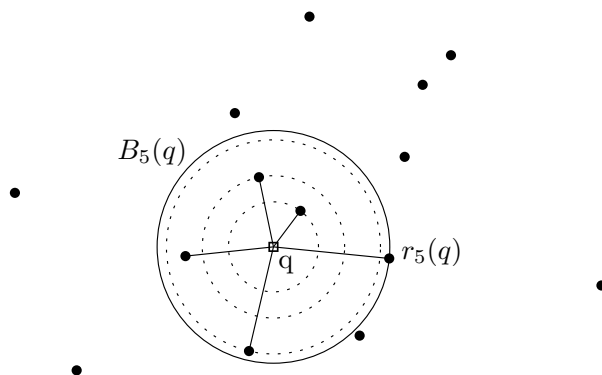


FIGURE 9.3 – Illustration d’une requête de  $k$ -PPVs pour  $k = 5$ . La boule englobant les  $k$ -PPVs  $B_k(q)$  a un rayon  $r_k(q)$  qui permet d’estimer localement la distribution sous-jacente de  $X$  par  $p(x) \approx \frac{k}{nV(B_k(x))} \approx \frac{k}{nr_k(x)^d}$ .

## 9.4 L’apprentissage statistique et l’erreur minimale bayésienne

Avec les grandes données (*big data*), il est raisonnable de modéliser les ensembles d’entraînements  $Z$  et de test  $Q$  par des *distributions statistiques* sous-jacentes. La performance des classifieurs peut alors être étudiée mathématiquement. On suppose que  $X$  (de  $Z = (X, Y)$ ) et  $Q$  sont des jeux de données, appelés observations, issues de *variables aléatoires* par des tirages *indépendamment et identiquement distribués* (*independently and identically distributed*, iid.). On note  $X \sim_{\text{iid}} \mathcal{D}$  pour décrire le fait que  $X$  est iid. de la loi de probabilité  $\mathcal{D}$  (par exemple, une loi gaussienne). Lorsque la loi a son support dans  $\mathbb{R}$ , on dit qu’on a une loi univariée, sinon on parle de lois multivariées (support dans  $\mathbb{R}^d$ ). On peut interpréter  $X$  comme un vecteur aléatoire de dimension  $n \times d$ . On rappelle que deux variables aléatoires  $X_1$  et  $X_2$  sont indépendantes ssi.  $\Pr(X_1 = x_1, X_2 = x_2) = \Pr(X_1 = x_1)\Pr(X_2 = x_2)$ . La modélisation statistique permet de considérer  $X$  comme un *modèle de mélange* (*statistical mixture*) en statistique avec la densité de probabilité de  $X$  qui s’écrit comme  $m(x) = w_1 p_1(x) + w_2 p_2(x)$  où  $w_1$  et  $w_2$  sont les probabilités *a priori* d’appartenir à la classe  $C_1$  et  $C_2$  ( $w_1 = 1 - w_2$ ), et  $p_1(x) = \Pr(X_1 | Y_1 = C_1)$  et  $p_2(x) = \Pr(X_2 | Y_2 = C_2)$  les *probabilités conditionnelles*. On recherche des classifieurs qui ont une bonne performance dans le cas limite de grands échantillons quand  $n \rightarrow +\infty$  (*large sample limit*).

### 9.4.1 Estimation non-paramétrique de densités

Étant donné un ensemble iid.  $X = \{x_1, \dots, x_n\}$  d’observations supposé échantillonné d’une densité  $p(x)$ , on cherche à modéliser la distribution sous-jacente. Pour une *loi paramétrique*  $p(x|\theta)$  (qui appartient à une famille de distributions indexée par un vecteur paramètre  $\theta$ ), cela revient à estimer le paramètre  $\theta$  de cette distribution. Par exemple, pour la loi gaussienne  $p(x|\theta = (\mu, \sigma^2))$ , on estime par le maximum de vraisemblance la moyenne  $\hat{\mu} = \frac{1}{n} \sum_{i=1}^n x_i$  et sa variance  $v = \sigma^2$  :  $\hat{\sigma}^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \hat{\mu})^2$  (non biaisée). Sinon, lorsque la densité ne dépend pas d’un vecteur de paramètres, on parle d’*estimation non-paramétrique*. Souvent, les distributions paramétriques utilisées

sont unimodales<sup>3</sup> et ces modèles manquent donc de flexibilité pour modéliser une densité multimodale. L'approche non-paramétrique est beaucoup plus flexible puisqu'elle permet de modéliser n'importe quel type de lois incluant toutes celles multi-modales.

**Théorème 10.** *L'estimateur "ballon" permet d'approximer une densité continue et lisse arbitraire  $p(x)$  à support dans  $\mathbb{R}^d$  par  $p(x) \approx \frac{k}{nV(B)}$ , avec  $k$  le nombre d'échantillons de  $X$  dans la boule  $B$  et  $V(B)$  son volume.*

*Démonstration.* Soit  $P_R$  la probabilité qu'un échantillon  $x$  tombe dans une région  $R$  :  $P_R = \int_{x \in R} p(x) dx$ . La probabilité que  $k$  des  $n$  échantillons tombent dans  $R$  est donnée par la loi binomiale :

$$P_R^{(k)} = \binom{n}{k} P_R^k (1 - P_R)^{n-k},$$

et l'espérance de  $k$  est  $\mathbb{E}[k] = nP_R$ , et l'estimateur  $\widehat{P}_R$  de maximum de vraisemblance pour  $P_R$  est  $\frac{k}{n}$ . Supposons la densité continue et la région  $R$  assez petite pour considérer  $p(x)$  constante dans  $R$ . Alors on a,

$$\int_{x \in R} p(x) dx \approx p(x) V_R,$$

avec  $V_R = \int_{x \in R} dx$  le volume de la région. L'estimateur de densité est donc  $p(x) \approx \frac{k}{nV}$ . □

On peut appliquer ce théorème de deux manières :

- on fixe le rayon de la boule  $B$  (et donc son volume  $V(B)$ ) et on compte le nombre de points dans  $B$  pour une position  $x$  (généralise la méthode d'histogramme),
- on fixe la valeur de  $k$  et on cherche la plus petite boule centrée sur  $x$  qui contienne exactement  $k$  points. Cette approche est communément appelée *estimation non-paramétrique par les  $k$ -PPVs*.

Soit  $r_k(x)$  le rayon de la boule englobante  $B_k(x)$ . On a le volume  $V_k(x)$  qui est proportionnel à  $r_k(x)^d$  à une constante multiplicative près qui dépend de la dimension<sup>4</sup> :  $V_k(x) = c_d r_k(x)^d \propto r_k(x)^d$ .

### 9.4.2 L'erreur minimale : la probabilité d'erreur et l'erreur de Bayes

Tout d'abord, notons que n'importe quel classifieur fera forcément un taux d'erreur non-nul puisque les distributions de  $X_{\pm 1}$  partagent le même support : on ne peut donc jamais être sûr à 100% d'avoir classé correctement un échantillon. On appelle *probabilité d'erreur* l'espérance de l'erreur minimale d'un classifieur en théorie de décision bayésienne (c'est-à-dire, sous les hypothèses d'apprentissages statistiques avec les probabilités *a priori* et les probabilités conditionnelles pour chaque classe) :

$$P_e = \Pr(\text{error}) = \int p(x) \Pr(\text{error}|x) dx,$$

avec

---

3. Les modes d'une densité  $p(x)$  sont ses maxima locaux.

4. La constante  $c_d$  est égale à  $\frac{\pi^{\frac{d}{2}}}{2!}$  pour les dimensions paires et à  $\frac{2(\frac{d-1}{2})!(4\pi)^{\frac{d-1}{2}}}{d!}$  pour les dimensions impaires.

On vérifie que  $c_1 = 2$ ,  $c_2 = \pi$ ,  $c_3 = \frac{4\pi}{3}$ , etc.

$$\Pr(\text{error}|x) = \begin{cases} \Pr(C_{+1}|x) & \text{la règle a décidé } C_{-1}, \\ \Pr(C_{-1}|x) & \text{la règle a décidé } C_{+1} \end{cases}$$

L'erreur bayésienne généralise cette probabilité d'erreur en prenant en compte une matrice de coûts  $[c_{i,j}]_{i,j}$  pour chaque cas de configuration : L'entrée de la matrice  $c_{i,j}$  est le coût de classer une nouvelle observation  $x$  dans la classe  $C_j$  sachant que  $x$  appartient à la classe  $C_i$ . L'erreur bayésienne minimise le risque en espérance, et coïncide avec la probabilité d'erreur  $P_e$  quand on choisit  $c_{i,i} = 0$  et  $c_{i,j} = 1$  pour  $j \neq i$ .

L'identité de Bayes stipule que  $\Pr(C_i|x) = \frac{\Pr(x|C_i)\Pr(C_i)}{\Pr(x)}$ . Cela se démontre très facilement en utilisant la propriété de chaînage des probabilités :

$$\Pr(A \wedge B) = \Pr(A)\Pr(B|A) = \Pr(B)\Pr(A|B) \Rightarrow \Pr(B|A) = \frac{\Pr(B)\Pr(B|A)}{\Pr(A)}.$$

La règle optimale pour la classification bayésienne qui minimise la probabilité d'erreur est celle qui classe les observations en utilisant la règle du *maximum a posteriori* (MAP) : on classe  $x$  dans la classe  $C_i$  ssi.

$$\Pr(C_i|x) \geq \Pr(C_j|x).$$

C'est-à-dire que l'on choisit la classe qui maximise la probabilité *a posteriori*. En utilisant l'identité de Bayes et supprimant le dénominateur commun  $\Pr(x)$ , cela revient donc à choisir la classe  $C_i$  telle que :

$$w_i \Pr(x|C_i) \geq w_j \Pr(x|C_j), \forall j \neq i.$$

Puisqu'on ne connaît ni les densités des lois conditionnelles  $\Pr(x|C_i)$  ni celles des lois *a priori*, on doit aussi estimer celles-ci à partir des observations. On peut estimer non-paramétriquement ces distributions en utilisant l'estimateur ballon qui se base sur les plus proches voisins comme suit : On se place dans le cas à deux classes  $C_{\pm 1}$ . Tout d'abord, on calcule les probabilités *a priori* (*prior probability*) à partir des fréquences des classes dans les observations :

$$\Pr(C_{\pm 1}) = w_{\pm 1} = \frac{n_{\pm 1}}{n}.$$

Puis on calcule les probabilités conditionnelles (*class-conditional probability*) comme :

$$\Pr(x|C_{\pm 1}) = \frac{k_{\pm 1}}{n_{\pm 1} V_k},$$

avec  $V_k$  le volume de la boule qui contient les  $k$ -PPVs de  $x$ .

De manière similaire, on a la densité non-conditionnelle qui peut être estimée à partir des  $k$ -PPVs par :

$$m(x) \approx \frac{k}{n V_k(x)}$$

On en déduit les probabilités *a posteriori* de la règle bayésienne MAP :

$$\Pr(C_{\pm 1}|x) = \frac{\Pr(x|C_{\pm 1})\Pr(C_{\pm 1})}{\Pr(x)} = \frac{\frac{k_{\pm 1}}{n_{\pm 1} V_k} \frac{n_{\pm 1}}{n}}{\frac{k}{n V_k}} = \frac{k_{\pm 1}}{k}.$$

On retrouve ainsi la règle du vote majoritaire pour les  $k$ -PPVs ! Nous allons maintenant quantifier la performance du classifieur  $k$ -PPVs en fonction de la probabilité d'erreur minimale  $P_e$ .

### 9.4.3 Probabilité d'erreur pour la règle du PPV

Quand la taille du jeu d'apprentissage et celle du jeu de test tendent vers l'infini ( $t, n \rightarrow +\infty$ ), l'erreur  $P_e(\text{PPV})$  de la règle PPV est au pire deux fois celle de la règle optimale (MAP si on connaissait exactement  $w_{\pm 1}$  et  $p_{\pm 1}(x)$ , voir la preuve dans [24]) :

$$P_e \leq \tau_{\text{erreur}}(\text{PPV}) \leq 2P_e.$$

Pour  $m \geq 2$  classes, et toujours la règle du PPV, on peut montrer [24] que :

$$P_e \leq \tau_{\text{erreur}}(\text{PPV}) \leq P_e \left( 2 - \frac{m}{m-1} P_e \right).$$

**Théorème 11.** *La règle optimale de classification bayésienne MAP est approximée par la classification par vote majoritaire des  $k$ -PPVs avec un facteur 2 d'erreur si l'on estime non-paramétriquement les probabilités des classes par l'estimateur ballon des PPVs.*

Notons que lorsque la dimension est grande, il faut vraiment beaucoup de données en pratique pour atteindre cette borne théorique (fléau de la dimension).

## 9.5 Requêtes des PPVs sur architecture parallèle à mémoire distribuée

Soit  $p$  unités de calcul (UCs) à mémoire distribuée. Notons  $\text{PPV}_k(x, X)$  les  $k$  plus proches voisins de  $x$  dans  $X$ . Pour classer une nouvelle observation requête  $q$ , nous allons utiliser la propriété de décomposition des  $k$ -PPVs pour cette requête : C'est-à-dire, que pour  $X = \biguplus_{l=1}^p X_l$  une partition de  $X$  en  $p$  sous-ensembles disjoints deux à deux, nous avons :

$$\text{PPV}_k(x, X) = \text{PPV}_k(x, \cup_{l=1}^p \text{PPV}_k(x, X_l)).$$

Pour  $p$  processeurs, nous partitionnons donc  $X$  en  $p$  paquets  $X_i$  de taille  $\frac{n}{p}$ , et faisons les requêtes  $\text{PPV}_k(x, X_i)$  sur chaque UC. Finalement, le processeur master reçoit  $kp$  éléments des UCs, et fait une requête des  $k$  plus proches voisins sur cet ensemble agrégé. On passe ainsi d'un algorithme séquentiel ( $p = 1$ ) en temps  $O(dnk)$  à un algorithme parallèle en temps  $O(dk\frac{n}{p}) + O(dk(kp))$ . Ainsi, pour  $kp \leq \frac{n}{p}$  (soit  $p \leq \sqrt{\frac{n}{k}}$ ), le speed-up réalisé est en  $O(p)$ .

## 9.6 \* Pour en savoir plus : notes, références et discussion

On recommande l'ouvrage [46] pour les détails concernant la règle des  $k$ -PPVs et l'apprentissage statistique. L'analyse de la performance statistique du classifieur par la règle du PPV a été étudiée en 1967 [24]. Les requêtes de  $k$ -Plus Proches Voisins sont un problème difficile et bien étudié en informatique [4]. Un algorithme sensible à la sortie<sup>5</sup> a été proposé pour calculer la frontière de décision entre deux classes de points dans le cas du plan [18]. En pratique, les cartes graphiques (GPUs) sont bien adaptées au calcul des  $k$ -PPVs [37]. Souvent, on peut relaxer la condition de trouver exactement les  $k$ -PPVs par celle de trouver les  $k$ -PPVs approchés à un facteur multiplicatif

5. C'est-à-dire dont la complexité dépend de la taille de la structure combinatoire de la sortie.

près de  $(1 + \epsilon)$  : les  $\epsilon$ -PPVs. Les avantages de la classification par la règle des  $k$ -PPVs sont que c'est simple à mettre en œuvre, cela garantit de bonnes performances asymptotiques (sous condition d'apprentissage statistique), et que c'est facile à paralléliser. Ses inconvénients sont que cela demande beaucoup de mémoire (le jeu d'apprentissage), et que les requêtes  $k$ -PPVs souffrent de la malédiction des grandes dimensions (*curse of dimensionality*) : on peut difficilement battre l'algorithme naïf pour des requêtes de  $k$ -PPVs. En pratique, il faut trouver le juste milieu pour le choix de  $k$  dans la règle des  $k$ -PPVs : pour des grandes valeurs de  $k$  qui donnent des frontières des classes plus lisses et une estimation non-paramétrique plus robuste des probabilités conditionnelles, mais cela devient plus coûteux pour les temps de requêtes et l'estimation non-paramétrique devient moins locale ! Calculer l'erreur de Bayes ou la probabilité d'erreur pour des modèles de distributions statistiques en formule close est souvent ardu, et on recherche alors plutôt à trouver des bornes supérieures en formule close [73].

Il existe de nombreuses extensions de cette règle des  $k$ -PPVs. On peut par exemple ajuster le vote majoritaire parmi les  $k$  voisins en apprenant un poids  $w_i$  pour chaque observation étiquetée  $z_i = (x_i, y_i)$  [78]. Pour des requêtes de classification, on choisit alors parmi les  $k$ -PPVs le voisin qui a le plus grand poids. La description de la frontière du classifieur en termes de diagrammes de Voronoï bichromatiques permet de démontrer la propriété de fonctions linéaires par morceaux des classifieurs PPVs. Notons qu'en pratique on ne peut pas calculer les diagrammes de Voronoï en grandes dimensions, car ceux-ci ont une complexité en  $O(n^{\lceil \frac{d}{2} \rceil})$  (déjà quadratique pour  $d = 3$ , cf. [16]), avec  $\lceil \cdot \rceil$  la fonction partie entière supérieure ( $\lceil x \rceil$  retourne l'entier immédiatement plus grand ou égal à  $x$ ).

Rappelons que le classifieur par les  $k$ -PPVs a l'avantage de garantir asymptotiquement un facteur d'erreur 2 par rapport à la meilleure règle bayésienne mais qu'un inconvénient majeur de ce classifieur est que l'on doit stocker *tous* les points de l'ensemble d'entraînement  $Z$  en mémoire. Une autre technique qui ne stocke que  $d + 1$  points en dimension  $d$  sont les *machines à vecteurs de support* (*support vector machines*, *SVMs*) qui demandent à ce que les classes soient séparables par un hyperplan. Cette dernière limitation est enlevée en utilisant l'astuce des noyaux [46] (*kernel tricks*).

En pratique, on observe une loi des rendements décroissants dans la classification utilisant des grands jeux de données (*law of diminishing returns*) : plus le jeu d'entraînement est grand, plus le gain de performance devient petit ! Voir la figure 9.4. Une explication simple provient du fait que les observations étiquetées ne sont pas identiquement et indépendamment distribuées dans les données réelles. L'erreur minimale de Bayes fournit une borne inférieure sur la performance d'un classifieur sous l'hypothèse d'apprentissage statistique. Comme il est souvent dur de calculer explicitement cette erreur de Bayes, on cherche à trouver des formules simples pour la borner supérieurement [73].

Pour conclure, discutons un peu de la complexité des modèles, du biais et de la variance des machines à apprendre, et de l'erreur de prédiction. Dans le chapitre 5, nous avons brièvement décrit la régression linéaire pour motiver l'utilisation de l'algèbre linéaire en science des données. Comparons maintenant les deux méthodes de classification usuelles : (1) la régression avec (2) la règle des  $k$ -PPVs :

- Modèle de régression : la complexité du modèle est égale au nombre de coefficients nécessaires pour définir l'hyperplan, soit  $d + 1$ . La classification par régression a une faible variance (ce qui veut dire que l'hyperplan est stable pour des petites perturbations des entrées) mais à un grand biais (ce qui veut dire que le séparateur hyperplan n'est pas proche de la frontière de décision).

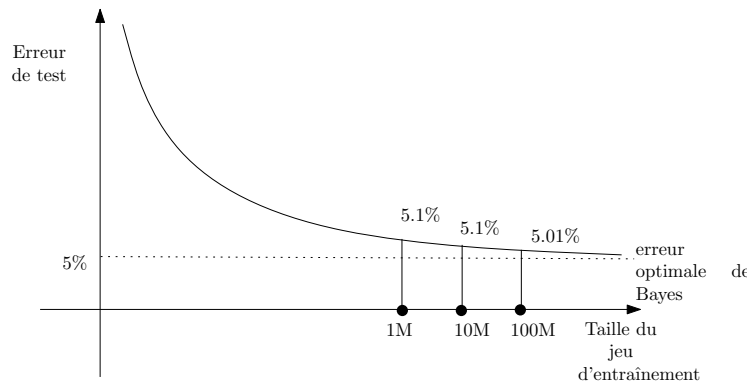


FIGURE 9.4 – En pratique, on observe une loi des rendements décroissants dans la classification utilisant des grands jeux de données (*law of diminishing returns*) : plus le jeu d’entraînement est grand, plus le gain de performance devient petit ! Une explication simple provient du fait que les observations étiquetées ne sont pas identiquement et indépendamment distribuées dans les données réelles. L’erreur minimale de Bayes fournit une borne inférieure sur la performance d’un classifieur sous l’hypothèse d’apprentissage statistique.

- Modèle par  $k$ -PPVs : la complexité du modèle est très grande,  $dn$ , puisque nous sommes obligés de garder tous les  $n$  points  $d$ -dimensionnels du jeu d’entraînement. La classification par  $k$ -PPVs a un faible biais puisque sa frontière de décision approxime bien la frontière des classes, mais a une grande variance puisqu’une simple perturbation par un point peut changer significativement la frontière de décision du classifieur par  $k$ -PPVs.

La figure 9.5 illustre ces deux propriétés de biais et variance pour les machines en fonction de la complexité des modèles. En pratique, on doit bien sûr choisir la complexité du modèle de la machine. Plus la complexité du modèle est grande, plus l’erreur de prédiction sera faible. Malheureusement, en augmentant la complexité des modèles, on rencontre un point critique où le classifieur généralise mal à de nouvelles données non étiquetées car il a sur-appris le jeu d’entraînement (*overfitting*). En pratique, on cherche à minimiser l’erreur de généralisation (et pas celle sur le jeu d’entraînement qui peut être 0 si on choisit  $k = n$  pour la règle du PPV).

## 9.7 En résumé : ce qu’il faut retenir !

La règle des  $k$ -Plus Proches Voisins ( $k$ -PPVs) classe une nouvelle observation requête  $q$  en choisissant la classe majoritaire parmi les  $k$  plus proches voisins de  $q$  dans le jeu étiqueté d’entraînement. On évalue la performance d’un classifieur en calculant son  $F$ -score qui est la moyenne harmonique de la précision et du rappel, et qui permet de tenir compte des quatre cas de figure d’une classification en false/true-positive/negative. En apprentissage statistique, un classifieur ne peut jamais battre asymptotiquement l’erreur de Bayes (ou probabilité d’erreur), et la règle du 1-PPV garantit asymptotiquement un facteur 2. Puisque les requêtes de Plus Proches Voisins sont décomposables ( $\text{PPV}_k(q, X_1 \cup X_2) = \text{PPV}_k(\text{PPV}_k(q, X_1), \text{PPV}_k(q, X_2))$ ), la règle de classification se parallélise bien sur architecture parallèle à mémoire distribuée. Un inconvénient de la règle est qu’elle doit stocker toute la base du jeu d’entraînement pour pouvoir classer.

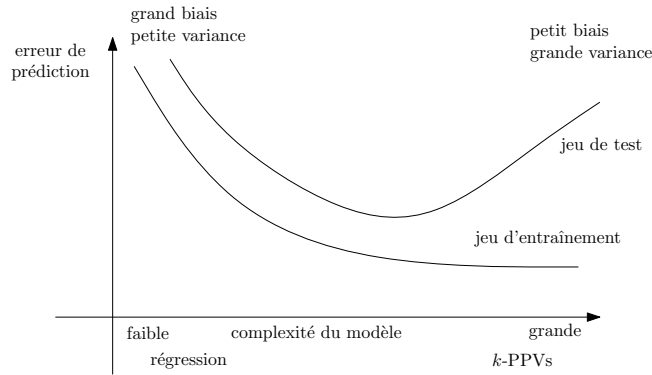


FIGURE 9.5 – Complexité du modèle, biais et variance des machines, et erreur de prédiction.

## 9.8 Exercices

### Exercice 31 : Élagage de la frontière de décision du classifieur PPV

Montrez que si pour une donnée du jeu d'entraînement  $x$  ses cellules de Voronoï avoisinantes (voisins naturels) sont de la même classe, alors cet échantillon peut être enlevé sans changer la frontière de décision du classifieur PPV.

### Exercice 32 : \* Probabilité d'erreur pour des lois conditionnelles normales

On considère les probabilités *a priori*  $w_1 = w_2 = \frac{1}{2}$  et les probabilités conditionnelles comme des lois normales  $X_1 \sim N(\mu_1, \sigma_1)$  et  $X_2 \sim N(\mu_2, \sigma_2)$ .

- calculer  $P_e$  exactement lorsque  $\sigma_1 = \sigma_2$ ,
- calculer  $P_e$  en utilisant la fonction standard cumulative de la loi normale  $\Phi(\cdot)$  lorsque  $\sigma_1 \neq \sigma_2$ .
- comme il est difficile, de calculer  $P_e$  en formule close, en utilisant l'astuce de réécriture  $\min(a, b) = \frac{a+b-|b-a|}{2}$  avec l'inégalité  $\min(a, b) \leq a^\alpha b^{1-\alpha}, \forall \alpha \in (0, 1)$  avec  $a, b > 0$ , en déduire une borne supérieure en formule close pour le cas des lois normales.

### Exercice 33 : \*\* La règles des $k$ -PPVs et les diagrammes de Voronoï d'ordre $k$ [59]

Pour un ensemble fini de points  $X = \{x_i\}_i$ , montrez que la décomposition de l'espace induite par les  $k$ -PPVs forment des cellules polyédrales convexes. On définit le diagramme de Voronoï d'ordre  $k$ , comme étant la partition de l'espace induite par toutes les  $\binom{n}{k}$  ensembles  $X_i \subset 2^X$  pour la fonction de distance  $D(X_i, x) = \min_{x' \in X_i} D(x', x)$ . C'est-à-dire que le diagramme de Voronoï d'ordre  $k$  est l'ensemble des cellules  $V_k(X_i)$  non-vides définies par  $V_k(X_i) = \{x \mid D(X_i, x) \leq D(X_j, x), \forall i \neq j\}$  (avec les  $|X_i| = k$ ). Comment simplifier la frontière de décision d'un classifieur  $k$ -PPV?

**Exercice 34 :** *\*\* Sensibilité de la règle des  $k$ -PPVs aux magnitudes des axes*

La performance du classifieur par la règle du PPV est très sensible aux changements d'échelle des axes car cela change la distance Euclidienne. En pratique, il faut trouver la bonne règle de poids sur les attributs (*feature weighting*) pour calibrer la distance Euclidienne :  $D_w(p, q) = \sqrt{\sum_{j=1}^d w_j (p^j - q^j)^2}$ . Étudiez les différentes méthodes de pondération des attributs [46].



## Chapitre 10

# Optimisation approchée par sous-ensembles noyaux et la réduction de dimension

Un résumé des points essentiels à retenir est donné dans §10.9.

### 10.1 Optimisation approchée pour les données volumineuses

Souvent lorsqu'on traite de grands volumes de données, on est intéressé par résoudre un problème d'optimisation sur celles-ci. Il peut être alors très avantageux de ne pas chercher la solution optimale (ou une solution optimale quand il en existe plusieurs) mais plutôt une solution approchée avec un facteur garantissant la qualité de l'approximation. Par exemple, dans un problème de regroupement par les  $k$ -moyennes, on cherche à minimiser la fonction de coût des  $k$ -moyennes (voir le chapitre 7) : c'est-à-dire la somme pondérée des variances des groupes. Il est clair que sous des hypothèses de stabilité du clustering<sup>1</sup>, une approximation de la minimisation de cette fonction objectif est suffisante.

L'optimisation approchée est d'autant plus intéressante en pratique que souvent ces problèmes deviennent de plus en plus dur lorsque la dimension des données augmente. Ce phénomène est connu sous l'appellation de *fléau de la dimension* (*curse of dimensionality*). En effet, lorsque la dimension  $d$  devient aussi un paramètre du problème à tenir en compte dans l'analyse de la complexité, alors nous obtenons souvent des algorithmes exponentiels en  $d$ , qui ne passent donc pas à l'échelle avec la dimension.

#### 10.1.1 Un exemple qui montre le besoin de traiter les grandes dimensions

En pratique, on manipule souvent des grandes dimensions. Par exemple, si on veut faire la recherche d'un motif de dimension  $s \times s$  d'une image source (disons, un *patch* d'une image source) dans une image cible, alors on peut représenter ce patch d'intensité de dimension  $s \times s$

---

1. Un bon regroupement doit aussi être un regroupement stable pour avoir du sens.

en un vecteur de dimension  $d = s^2$ . C'est la *vectorisation* (*vectorize*) ou encore la *linéarisation* (*linearize*). Idem pour l'image cible de taille  $w \times h$  qui est interprétée comme un nuage de  $n = w \times h$  points de dimension  $d = s^2$  (pour les pixels qui dépassent du bord, on prend la valeur de l'intensité du bord correspondant). Trouver le meilleur patch dans l'image cible qui minimise la somme des différences au carré (*sum of squared errors*, SSE) avec le patch revient alors à faire une requête de plus proches voisins en grande dimension  $d = s^2$ .

### 10.1.2 Phénomènes sur les distances en grandes dimensions

En grandes dimensions, nous avons des phénomènes qui peuvent sembler contre-intuitifs à première vue (fort heureusement, ils sont expliqués mathématiquement). Par exemple, le volume  $V_d(r)$  d'une boule de rayon  $r$  en dimension  $d$  est :

$$V_d(r) = \frac{\pi^{d/2}}{\Gamma(\frac{d}{2} + 1)} r^d,$$

avec  $\Gamma$  la fonction d'Euler qui généralise la fonction factorielle :  $\Gamma(t) = \int_0^\infty x^{t-1} e^{-x} dx$  (et  $\Gamma(k) = (k-1)!$  pour  $k \geq 2, k \in \mathbb{N}$ ). Le volume d'une boule de rayon unité tend donc vers zéro lorsque  $d \rightarrow \infty$  alors que cette boule unité centrée à l'origine touche bien les  $2d$  facettes du cube de côté 2 centré à l'origine. Avec la dimension qui augmente, la boule couvre donc de moins en moins de volume du cube<sup>2</sup> qui la contient !

### 10.1.3 Passer des grandes aux petites données !

Nous allons voir qu'il existe parfois pour des problèmes d'optimisation des sous-ensembles des données de petite taille (parfois aussi indépendants de la dimension  $d$ , et de la taille  $n$  du problème original) pour lesquels résoudre le problème d'optimisation sur ces sous-ensembles noyaux donne une solution approchée garantie. C'est le cas du regroupement par les  $k$ -moyennes.

L'existence de sous-ensembles noyaux permet de passer ainsi de données gigantesques à de petits volumes de données à traiter [32]. C'est donc une technique importante pour les *BigData* !

## 10.2 Définition des sous-ensembles noyaux (*coresets*)

On définit les *sous-ensembles noyaux* (*core-sets*, écrit aussi *coresets*) pour l'optimisation approchée (rapide). Soit un problème d'optimisation sur  $n$  données  $X = \{x_1, \dots, x_n\}$ . Pour fixer les idées, disons :

$$\min_{\theta \in \Theta} f(\theta | x_1, \dots, x_n),$$

avec  $\theta$  les paramètres du modèle à optimiser (par exemple, les  $k$  centres dans les  $k$ -moyennes) et  $\Theta$  l'espace des paramètres. La solution optimale  $\theta^*$  (ou une solution optimale lorsqu'il en existe plusieurs) de coût minimal  $c^*$  est obtenue par :

$$\begin{aligned} \theta^* &= \text{sol}(\theta | X) = \operatorname{argmin}_{\theta \in \Theta} f(\theta | x_1, \dots, x_n), \\ c^* &= \text{coût}(\theta | X) = \min_{\theta \in \Theta} f(\theta | x_1, \dots, x_n). \end{aligned}$$

---

2. appelé aussi hypercube quand  $d > 3$ .

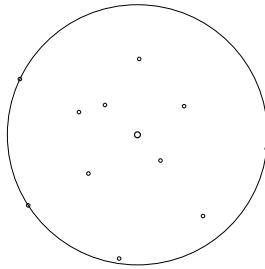


FIGURE 10.1 – La plus petite boule englobante dans le plan : un exemple.

Au lieu de résoudre ce problème d'optimisation sur l'ensemble  $X$  au complet, on cherche plutôt à trouver un *sous-ensemble noyau*  $C \subseteq X$  tel que :

$$\text{coût}(\theta|X) \leq \text{coût}(\theta|C) \leq (1 + \epsilon)\text{coût}(\theta|X).$$

De plus, on souhaiterait  $|C| \ll |S|$  (la cardinalité de  $C$  négligeable devant celle de  $S$ ) avec  $|C|$  qui dépend uniquement de  $\epsilon > 0$  (et pas de  $n = |X|$ , ni de  $d$  la dimension des  $x_i$ ).

### 10.3 Sous-ensembles noyaux pour la plus petite boule englobante

Le problème de la *plus petite boule englobante* (*Smallest Enclosing Ball*, SEB) consiste à trouver une boule  $B = \text{Boule}(c, r)$  de *rayon minimal* qui recouvre entièrement  $X$ . On modélise ce problème par cette optimisation :

$$c^* = \arg \min_{c \in \mathbb{R}^d} \max_{i=1}^n \|c - x_i\|.$$

La plus petite boule englobante est *unique* et son *centre circonscrit* (*circumcenter*) est noté  $c^*$ . Au lieu de la minimisation du rayon, on aurait pu prendre de façon équivalente le volume (une puissance du rayon), ou encore définir la minimalité par rapport à l'opérateur d'inclusion. La figure 10.1 montre un exemple de plus petite boule englobante pour un nuage de points du plan.

Soit  $c(X)$  le centre de la plus petite boule englobante de  $X$ ,  $\text{SEB}(X) = \text{Boule}(c(X), r(X))$ , et  $r(X)$  son rayon. Pour tout  $\epsilon > 0$ , un  $\epsilon$ -*noyau* ( $\epsilon$ -*coreset*) est un sous-ensemble  $C \subseteq X$  tel que :

$$X \subseteq \text{Boule}(c(C), (1 + \epsilon)r(C)).$$

C'est-à-dire qu'en élargissant la plus petite boule englobante  $\text{SEB}(C)$  par un facteur  $1 + \epsilon$ , on couvre complètement  $X$ . La figure 10.2 illustre un sous-ensemble noyau pour la boule englobante. Il a été démontré [9] qu'il existe des sous-ensembles noyaux de taille  $\lceil \frac{1}{\epsilon} \rceil$ , *indépendants de la dimension*  $d$ , et de la taille du problème original  $n$  pour ce problème ! En pratique, les sous-ensembles noyaux ont beaucoup d'applications en grandes dimensions (voire même quand  $d \gg n$ , c'est-à-dire quand le nombre de données est bien plus petit que la dimension de ces données).

Notons que dans le cas  $d \gg n$ , on pourrait *mathématiquement* considérer sans perte de généralité un espace affine de dimension  $d' = n - 1$  pour des points en *position générale*. Un ensemble

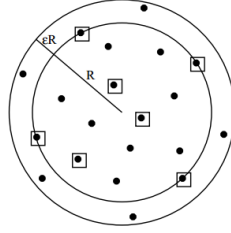


FIGURE 10.2 – Illustration d'un sous-ensemble noyau (*coreset*). Le sous-ensemble noyau  $C$  est défini par les points encadrés d'un carré. En élargissant la boule englobante  $\text{SEB}(C)$  par un facteur  $1 + \epsilon$ , on recouvre complètement  $X$  :  $X \subseteq \text{Boule}(c(C), (1 + \epsilon)r(C))$ .

de  $n$  points est en position générale ssi. il n'existe pas  $k + 1$  points dans un sous espace-affine de dimension  $k$  (pas trois points sur une même droite, etc.). Néanmoins, calculer ce sous-espace affine ferait intervenir des calculs de déterminants, non seulement très coûteux, mais aussi fort *instable* en pratique car on perdrait de la précision numérique<sup>3</sup>.

## 10.4 Une heuristique rapide pour approcher la boule englobante minimale

On présente ci-dessous une technique qui calcule une  $(1+\epsilon)$ -approximation de la boule englobante après  $\lceil \frac{1}{\epsilon^2} \rceil$  itérations. Comme on va le voir, cette heuristique construit aussi un ensemble noyau de taille  $\lceil \frac{1}{\epsilon^2} \rceil$ . Voici le pseudo-code de cet algorithme :

APPROXMINIBALL( $X, \epsilon > 0$ ) :

- Initialiser le centre  $c_1 \in X = \{x_1, \dots, x_n\}$  (on peut prendre aussi le centre de masse pour  $c_1$ ),
- Mettre à jour itérativement pour  $i = 2, \dots, \lceil \frac{1}{\epsilon^2} \rceil$  le centre avec la règle :

$$c_i \leftarrow c_{i-1} + \frac{f_{i-1} - c_{i-1}}{i}$$

où  $f_i$  est le plus lointain point de  $X$  pour le centre actuel  $c_i$  :

$$f_i = p_s, \quad s = \operatorname{argmax}_{j=1}^n \|c_i - x_j\|.$$

- Retourner  $\text{Boule}(c_{\lceil \frac{1}{\epsilon^2} \rceil}, \max_i \|x_i - c_{\lceil \frac{1}{\epsilon^2} \rceil}\|)$ .

3. On ne pourrait donc pas faire du calcul à virgule flottante (standard IEEE 754), et devrait donc plutôt utiliser une bibliothèque pour faire du *calcul multi-précision*.

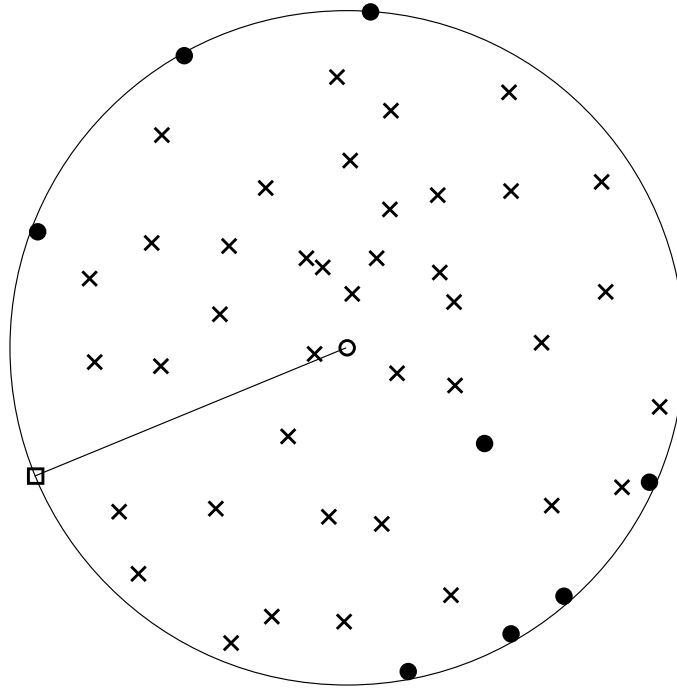


FIGURE 10.3 – Déroulement de l’algorithme qui approxime la plus petite boule englobante d’un nuage de points : le point cercle vide montre le centre courant, le point boîte vide le point le plus éloigné pour ce centre actuel, et les points disques pleins les points qui définissent le sous-ensemble noyau.

Cet algorithme<sup>4</sup> qui s’apparente à une méthode de descente de gradient coûte  $O(\frac{dn}{\epsilon^2})$  et produit par la même occasion un sous-ensemble noyau :  $f_1, \dots, f_l$  avec  $l = \lceil \frac{1}{\epsilon^2} \rceil$ . La figure 10.3 illustre la situation après quelques itérations de l’algorithme et visualise le sous-ensemble noyau correspondant. Notons que le sous-ensemble noyau calculé par cet algorithme est de taille  $\lceil \frac{1}{\epsilon^2} \rceil$ , non-optimale, puisqu’on sait qu’il existe un sous-ensemble noyau de taille optimale [9]  $\lceil \frac{1}{\epsilon} \rceil$ .

#### 10.4.1 \* Preuve de convergence

**Théorème 12.** *Le centre  $c^*$  de la plus petite boule englobante  $\text{Boule}(c^*, r^*)$  est approché par l’algorithme APPROXMINIBALL qui garantit qu’à l’itération  $i$ , nous ayons  $\|c_i - c^*\| \leq \frac{r^*}{\sqrt{i}}$ .*

Ce théorème garantit que nous obtenons une  $(1 + \frac{1}{\sqrt{i}})$ -approximation. En effet, pour tout  $x \in X$ , en utilisant l’inégalité triangulaire, on a :

$$\|x - c_i\| \leq \|x - c^*\| + \|c^* - c_i\| \leq r^* + \frac{r^*}{\sqrt{i}} = (1 + \frac{1}{\sqrt{i}})r^*.$$

4. Voir la démonstration en ligne à <http://kenclarkson.org/sga/t/t.xml>

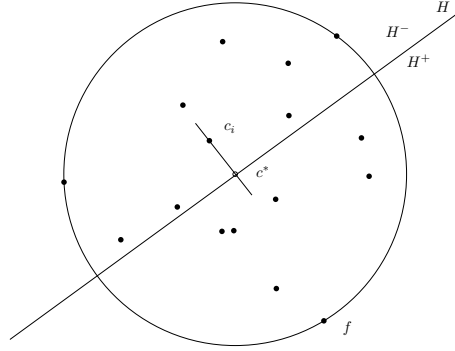


FIGURE 10.4 – Notations pour la preuve.

La preuve (technique) du théorème est donnée ci-dessous à titre indicatif. On procède par induction : Pour  $i = 1$ , on a  $\|c_1 - c^*\| \leq r^*$ . à l'étape  $i$  :

- soit  $c_i = c^*$  et alors on bouge d'au plus  $\frac{r^*}{i+1} \leq \frac{r^*}{\sqrt{i+1}}$  et finalement  $\|c^* - c_{i+1}\| \leq \frac{r^*}{\sqrt{i+1}}$ ,
- soit  $c_i \neq c^*$ , et on considère l'hyperplan orthogonal à  $[c^*c_i]$  qui contient  $c^*$ . On note  $H^+$  le demi-espace délimité par  $H$  qui ne contient pas  $c_i$  et  $H^-$  l'autre demi-espace complémentaire (figure 10.4). On peut montrer que le point  $f$  le plus éloigné se trouve dans  $X \cap H^+$ . Nous avons alors deux nouveaux cas de figure :
  - cas  $c_{i+1} \in H^+$  : la distance  $\|c_{i+1} - c^*\|$  est maximale quand  $c_i = c^*$  et donc  $\|c_{i+1} - c^*\| \leq \frac{r^*}{i+1} \leq \frac{r^*}{\sqrt{i+1}}$ ,
  - cas  $c_{i+1} \in H^-$  : en bougeant  $c_i$  le plus loin de  $c^*$  et en prenant  $f$  sur la sphère le plus proche de  $H^-$ , on augmente nécessairement  $\|c_{i+1} - c^*\|$ . Dans ce cas,  $[c^*c_{i+1}]$  est orthogonal à  $[c_i f]$ , et on a en utilisant Pythagore :

$$\|c_{i+1} - c^*\| = \frac{\left(\frac{r^*}{\sqrt{i}}\right)^2}{\frac{r^*}{\sqrt{1+\frac{1}{i}}}} = \frac{r^*}{\sqrt{i+1}}.$$

Pour garantir 1% de précision avec l'heuristique APPROXMINIBALL, il faut donc faire 10000 itérations.

#### 10.4.2 \* Boule englobante approchée et séparateur linéaire approché

Le calcul approché de la boule englobante peut se réécrire mathématiquement comme un exemple de *programmation quadratique (quadratic approximation)* où l'on cherche à minimiser le carré du rayon sous des contraintes linéaires [91].

Nous avons vu en apprentissage supervisé, la classification par la règle des  $k$  Plus Proches Voisins (PPVs) au chapitre 9. Une autre technique d'apprentissage très populaire en pratique sont les *machines à vecteurs de support (support vector machines, SVMs)*. Celles-ci distinguent deux classes de points étiquetés  $+1$  et  $-1$  par un séparateur linéaire : un hyperplan qui maximise la marge, c'est-à-dire la distance minimale entre cet hyperplan séparateur et un des points de la classe  $-1$ , et de la classe  $+1$ . On peut montrer que calculer ce meilleur hyperplan revient (dualement) au calcul de la boule englobante [91]. Les sous-ensembles noyaux de la boule peuvent ainsi donc servir

à définir un hyperplan séparateur approché : cela donne lieu au *Core Vector Machines* (CVMs) [91] (2007).

## 10.5 \* Sous-ensembles noyaux pour les $k$ -moyennes

Nous avons vu au chapitre 7 que regrouper les données par l'algorithme des  $k$ -moyennes est un problème NP-dur quand  $d > 1$ . On écrit le coût des  $k$ -moyennes pour des centres  $C$  par  $l_C(X) = \sum_{x \in X} D^2(x, C)$  où  $D^2(x, C)$  est la distance Euclidienne au carré minimale entre  $x$  et un des  $k$  centres de  $C$ . On dit que  $S$  est un  $(k, \epsilon)$ -sous-ensemble noyau (en anglais,  $(k, \epsilon)$ -coreset) pour  $X$  ssi.  $\forall C = (c_1, \dots, c_k)$ ,  $(1 - \epsilon)l_C(P) \leq l_C(S) \leq (1 + \epsilon)l_C(P)$ . En 1D, il existe un  $(k, \epsilon)$ -sous-ensemble noyau de taille  $O(\frac{k^2}{\epsilon^2})$ , et en dimension supérieure, on a prouvé [44] en 2007 l'existence de sous-ensembles noyaux de taille  $O(k^3/\epsilon^{d+1})$ . On peut donc faire du regroupement par les  $k$ -moyennes sur des données volumineuses.

## 10.6 Réduction rapide de la dimension des données

### 10.6.1 Le fléau des grandes dimensions

Le *fléau des grandes dimensions* où *curse of dimensionality* en anglais a été soulevé par M. Bellman, l'inventeur du paradigme de la programmation dynamique. En effet, M. Bellman a remarqué que l'efficacité des algorithmes dépend de la dimension des données, c'est-à-dire, du nombre d'attributs des données. Par exemple, calculer la distance (ou dissimilarité) entre deux vecteurs de dimension  $d$  demande un temps de calcul en  $\Omega(d)$ . Les algorithmes utilisant pour structures de données des partitionnements ont le plus souvent un temps de calcul dépendant exponentiellement de la dimension  $d$ . Cette dépendance exponentielle est le plus souvent cachée dans la notation de complexité big-Oh :  $O_d(1)$  (par exemple  $O(d^d) = O_d(1)$  et  $O(d^{d^d}) = O_d(1)$ ). De plus, il est extrêmement difficile de visualiser les données en grandes dimensions pour en comprendre leurs structures. Pourtant, il est aujourd'hui très courant de travailler avec des jeux de données en très grandes dimensions (par exemple,  $d = 1000$  pour les expressions de gènes observées pendant une série de  $t = 1000$  temps). Il arrive parfois aussi que la dimension extrinsèque  $d$  est bien plus grande que le nombre de données  $n$  :  $d \gg n$ .

On doit se méfier quand on travaille dans les grandes dimensions car on note un certain nombre de faits plutôt non intuitifs à première vue :

- Par exemple, le volume d'une balle de diamètre unité inscrite dans le cube unité tend vers zéro lorsque la dimension croît. Pourtant cette balle de rayon  $\frac{1}{2}$  touche bien le cube unité (de volume unité) en  $2d$  points de contact. En effet, on rappelle que le volume  $V_d$  d'une balle de rayon  $r = \frac{1}{2}$  est donnée par la formule :

$$V_d = \frac{\pi^{\frac{d}{2}}}{\Gamma(\frac{d}{2} + 1)} r^d, \quad r = \frac{1}{2},$$

où  $\Gamma(t) = \int_0^\infty x^{t-1} e^{-x} dx$  est la généralisation de la fonction factorielle, avec  $\Gamma(n) = (n-1)!$  pour  $n \in \mathbb{N}$ .

- La grille régulière de côté  $l \in \mathbb{N}$  dans  $\mathbb{R}^d$  est composé de  $l^d$  hypercubes élémentaires unités. Le nombre de ces hypercubes unités augmente donc exponentiellement avec la dimension

$d$  puisque nous avons  $l^d = e^{d \log l}$ . Les techniques de sous-divisions adaptatives (comme les *quad-trees*) bien que meilleures ne passent pas non plus à l'échelle en grandes dimensions.

- Les techniques d'intégration stochastique de Monte-Carlo ( $\int \approx \sum$ ) deviennent inefficaces en grandes dimensions puisque les échantillons aléatoires tirés suivant une loi uniforme sont de plus en plus rejetés dans le calcul approché d'intégrales multi-dimensionnelles.
- Citons aussi Beyer et ses collègues [15] en 1999 : "... the distance to the nearest data point approaches the distance to the farthest data point ...". C'est-à-dire qu'en grandes dimensions, les distances entre deux données deviennent de moins en moins discriminantes.

### 10.6.2 Deux exemples concrets pour illustrer le besoin des grandes dimensions

Pour notre premier exemple, considérons la tâche qui consiste à trouver les images quasiment semblables dans une base d'images (afin de détecter les duplications et nettoyer la base). Une image  $I[y][x]$  couleur RVB de taille  $w \times h$  est convertie en un vecteur  $v(I)$  de dimension  $\mathbb{R}^{3wh}$  en empilant tous les triplets de couleur des pixels en parcourant les pixels de l'image de haut en bas, et de gauche à droite. Ce procédé est appelé la vectorisation d'image (*image vectorization*).

La distance entre deux images  $I_1$  et  $I_2$  peut alors être définie comme la somme des carrés des différences (*sum of squared differences*, *SSD*) :

$$\begin{aligned} \text{SSD}(I_1, I_2) &= \sum_{i=1}^h \sum_{j=1}^w (I_1[i][j] - I_2[i][j])^2 \\ &= \|v(I_1) - v(I_2)\|^2 \end{aligned}$$

Une image  $I$  est visuellement très proche d'une autre image  $I'$  (disons, au bruit près) si le vecteur image  $v(I')$  est le plus proche voisin de  $v(I)$  et que leur distance passe sous un seuil fixé. Un algorithme naïf pour répondre aux requêtes d'un plus proche voisin de  $n$  vecteurs  $d$ -dimensionnels coûte  $O(dn)$  en temps de calcul : c'est prohibitif quand  $d$  est très grand. Cela pose donc la question de pouvoir calculer efficacement les requêtes de plus proches voisins (PPVs) en grandes dimensions, en temps sous-linéaire  $o(nd)$ .

Pour notre second exemple, considérons le problème de regrouper les images en catégories d'images semblables : c'est un problème de clustering en grandes dimensions. Pour se fixer les idées, prenons le jeu de données MNIST<sup>5</sup> qui consiste en  $n = 60000$  chiffres de codes postaux des États-Unis écrits à la main (du chiffre '0' au chiffre '9'). Chaque chiffre a été scanné dans une image en intensité de gris de taille  $28 \times 28$ . Ainsi, un bon algorithme de clustering doit trouver idéalement et rapidement les  $k = 10$  clusters, avec chaque cluster regroupant tous les mêmes chiffres. On ne peut pas utiliser naïvement le clustering par regroupement des  $k$ -moyennes car ça serait beaucoup trop lent. Il nous faut donc trouver une approche différente en se posant la question de comment réduire efficacement la dimension des données tout en préservant les paires de distances entre éléments ?

### 10.6.3 La réduction de dimension linéaire

Soit un jeu de données  $X$  qui consiste en un ensemble de  $n$  vecteurs de  $\mathbb{R}^d$ .  $X$  est interprété comme une matrice de dimension  $n \times d$  avec un point stocké sur chaque ligne de la matrice (ici, c'est

5. disponible à [https://en.wikipedia.org/wiki/MNIST\\_database](https://en.wikipedia.org/wiki/MNIST_database)

la convention donnée-ligne choisie). On peut penser à deux techniques pour réduire la dimension :

- sélectionner certaines dimensions et oublier (effacer) d'autres dimensions moins importantes : c'est ce qu'on appelle faire de la *sélection d'attributs* (*feature selection*).
- recomposer en combinant les dimensions en de nouvelles dimensions tout en préservant au mieux les informations de distance.

La réduction de dimension linéaire associe à chaque vecteur entrée  $x \in \mathbb{R}^d$  un autre vecteur  $y = y(x) \in \mathbb{R}^k$  (avec  $k < d$ ) grâce à une fonction linéaire  $A : \mathbb{R}^d \rightarrow \mathbb{R}^k$  qui peut s'écrire simplement sous notation matricielle comme suit :

$$y = x \times A, \quad Y = X \times A.$$

La matrice  $A$  est de taille  $d \times k$  ( $x$  et  $y$  sont tous les deux des vecteurs lignes). On cherche alors la matrice  $A$  qui préserve au mieux les paires de distances entre éléments de  $X$  (un plongement quasi-isométrique) :

$$\forall x, x' \in X, \quad \|y - y'\|^2 = \|xA - x'A\|^2 \approx \|x - x'\|^2$$

Une fois la matrice  $A$  calculée, on obtient une représentation compacte de  $X$  (en taille mémoire aussi) :  $X \times A$ . Le problème que l'on doit résoudre est donc de calculer efficacement une telle transformation matricielle  $A$  !

#### 10.6.4 Le théorème de Johnson-Lindenstrauss

Le théorème de Johnson-Lindenstrauss [51] (1984) a permis une avancée majeure dans la réduction de dimension :

**Théorème 13** (Théorème de Johnson-Lindenstrauss [51]). *Soit  $X$  un ensemble de  $n$  points de  $\mathbb{R}^d$  et  $\epsilon \in (0, 1)$  un seuil fixé. Alors il existe une transformation linéaire  $A : \mathbb{R}^d \rightarrow \mathbb{R}^k$  avec  $k = O(\frac{1}{\epsilon^2} \log n)$  telle que :*

$$\forall x, x' \in X, \quad (1 - \epsilon)\|x - x'\|^2 \leq \|xA - x'A\|^2 \leq (1 + \epsilon)\|x - x'\|^2$$

Le théorème de Johnson-Lindenstrauss nous indique donc qu'il existe mathématiquement un *plongement à faible distortion* (*low distortion embedding*) qui permette de considérer un ensemble de points de dimension  $k = O(\frac{1}{\epsilon^2} \log n)$  quasi-équivalent pour la notion de distance inter-point. Notons que  $k$  est indépendant de la dimension extrinsèque  $d$  des données mais dépend logarithmiquement de  $n$ .

#### 10.6.5 Matrices de projection aléatoire

Fort heureusement, il existe une recette très simple pour trouver une telle bonne transformation linéaire  $A$  :

- tirer aléatoirement de façon identique et indépendante (iid) les coefficients d'une matrices  $A'$  suivant une loi normale  $N(0, 1)$  (loi Gaussienne) :

$$A' = [a_{i,j}], \quad a_{i,j} \sim N(0, 1)$$

Un échantillon aléatoire  $x \sim N(0, 1)$  peut-être obtenu en utilisant la transformation dite de Box-Müller à partir de deux échantillons aléatoires indépendants  $U_1$  et  $U_2$  d'une loi uniforme :

$$x = \sqrt{-2 \log U_1} \cos(2\pi U_2)$$

— Ajuster l'échelle de la matrice aléatoire :

$$A = \frac{k}{d} A'$$

Mettons en pratique cette technique pour notre problème de regroupement d'images en grandes dimensions. Avec cette technique de réduction de dimension, on peut en pratique regrouper  $n$  images en couleur de taille  $300 \times 300$  ( $\mathbb{R}^{300 \times 300 \times 3} = \mathbb{R}^{270000}$ ) en réduisant les données en dimension  $k = 532$  ( $\mathbb{R}^{532}$ ) tout en trouvant des clusterings semblables au final. On obtient ainsi une accélération considérable de l'heuristique des  $k$ -moyennes de Lloyd qui demande un temps  $O(sdkn)$  (avec  $s$  le nombre d'itérations requises pour arriver à convergence) en dimension  $d$ , et un temps  $O_\epsilon(s'kn \log n)$  une fois le plongement calculé (le nombre d'itérations requises pour arriver à convergence est différent). Notons qu'il coûte un temps  $O_\epsilon(nd \log n)$  pour calculer les projections des données avant d'appeler l'algorithme des  $k$ -moyennes sur les nouvelles données projetées de plus petite dimension.

On omet la preuve technique qui montre qu'une telle matrice aléatoire donne un plongement valide avec forte probabilité. Le théorème de Johnson-Lindenstrauss a été très étudié et raffiné dans plusieurs directions. Tout d'abord, la dimension  $k = \Omega(\epsilon^{-2} \log n)$  est optimale lorsque l'on considère des transformations linéaires [58] (2014). Ensuite, il est suffisant et meilleur en pratique de considérer des matrices de projection aléatoire avec des coefficients rationnels [1] (2003) qui peuvent être obtenus en lançant un dé. Par exemple, on peut choisir aléatoirement pour  $A = [a_{i,j}]$  :

$$a_{i,j} = \begin{cases} 1 & \text{avec probabilité } \frac{1}{6} \\ 0 & \text{avec probabilité } \frac{2}{3} \\ -1 & \text{avec probabilité } \frac{1}{6} \end{cases}$$

De plus, on peut également considérer des matrices creuses et donc compactes en mémoire : il suffit d'avoir  $O(\frac{1}{\epsilon})$  coefficients de la matrice non-nuls [52] (2014).

## 10.7 Exercices

**Exercice 35 :** *Boule englobante et diagramme de Voronoï le plus lointain*

Montrez que le centre  $c^*$  de la plus petite boule englobante  $X = \{x_1, \dots, x_n\}$  se trouve nécessairement sur le diagramme de Voronoï des plus lointains sites. On définit les cellules de Voronoï des plus lointains sites comme :  $V(x_i) = \{x \mid \|x - x_j\| \leq \|x - x_i\|, \forall j \neq i\}$ . La cellule  $V(x_i)$  constitue donc l'ensemble de points pour lesquels  $x_i$  est le point le plus éloigné de  $X$ .

**Exercice 36 :** *Parallélisation de l'heuristique APPROXMINIBALL*

Montrez comment paralléliser l'heuristique APPROXMINIBALL sur  $P$  machines à mémoire distribuée. Quelle est la complexité de votre algorithme ?

## 10.8 \* Pour en savoir plus : notes, références et discussion

L'existence (surprenante) de sous-ensembles noyaux [9] de taille  $\lceil \frac{1}{\epsilon} \rceil$ , dépendant seulement de l'approximation  $1 + \epsilon$  et non de la taille des données  $n$  ni de la dimension de l'espace ambiant  $d$ , a permis d'étudier les propriétés de sous-ensembles noyaux pour divers problèmes d'optimisation. Calculer la plus petite boule englobante permet de résoudre le problème dual du calcul de l'hyperplan séparateur à marge maximale pour les machines à vecteurs de support [91]. La construction de sous-ensembles noyaux [44] pour le regroupement par les  $k$ -moyennes et pour d'autres fonctions objectives du clustering a permis de pouvoir calculer des regroupements pour les BigData. En effet, les sous-ensembles noyaux [32] permettent de transformer des problèmes de grande taille en des problèmes de petite taille lorsque l'on fixe la valeur d'approximation  $\epsilon$ . On recommande au lecteur intéressé cet article récent [30] pour une vision des défis de la *fouille des données* sur les BigData qui inclut les techniques des sous-ensembles noyaux. Il existe de nombreuses techniques pour la réduction de dimension linéaire (ou non-linéaire) : c'est un domaine de recherche vaste qu'on appelle plus communément dans sa généralité l'*apprentissage de variétés* (*manifold learning*).

## 10.9 En résumé : ce qu'il faut retenir !

Une grande classe de problèmes à résoudre consiste à optimiser une fonction paramétrique sur des données. Un sous-ensemble noyau est un sous-ensemble des données pour lequel le résultat exact de la minimisation sur ce sous-ensemble garantit une solution approchée pour la totalité des données. Il existe des sous-ensembles noyaux de taille  $\lceil \frac{1}{\epsilon} \rceil$  pour le calcul de la plus petite boule englobante, et des sous-ensembles noyaux de taille  $O(k^{\frac{3}{\epsilon}}/\epsilon^{d+1})$  pour calculer une approximation du regroupement par les  $k$ -moyennes qui garantissent une  $(1 + \epsilon)$ -approximation. Ces tailles de sous-ensembles noyaux sont indépendantes de la taille des données  $n$ , et dépendent uniquement de l'approximation  $1 + \epsilon$  recherchée. La dimension de réduction linéaire par matrice aléatoire de projection permet de transformer des données initialement en grande dimension extrinsèque  $d$  en des données plongées en petite dimension  $k = \Omega(\epsilon^{-2} \log n)$  pour une approximation à un facteur  $1 + \epsilon$  près des distances inter-éléments, indépendant de la dimension extrinsèque initiale !

Les sous-ensembles noyaux permettent ainsi de transformer des problèmes de grande taille en des problèmes de petite taille, et permettent de traiter rapidement les BigData voire les flots de données.



# Chapitre 11

## Algorithmique parallèle pour les graphes

Un résumé des points essentiels à retenir est donné dans §11.4.

### 11.1 Détection de sous-graphes denses dans les (grands) graphes

#### 11.1.1 Définition du problème

Soit  $G = (V, E)$  un graphe à  $|V| = n$  nœuds (ou sommets) et  $|E| = m$  arêtes. On cherche le sous-graphe  $V' \subseteq V$  qui maximise la *densité* :

$$\rho(V') = \frac{|E(V')|}{|V'|},$$

où  $E(V') = \{(u, v) \in E \mid (u, v) \in V' \times V'\}$ . On note  $G_{|V'} = (V', E(V'))$  ce sous-graphe restreint aux nœuds de  $V'$ . La densité est donc la moyenne des degrés du sous-graphe. Pour la clique  $G = K_n$  (le graphe complet), on a donc  $\rho_{\max} = \rho(V) = \frac{n(n-1)}{2n} = \frac{n-1}{2}$ .

Trouver le graphe le plus dense, de densité  $\rho^*$ , revient donc à ce problème d'optimisation :

$$\rho^* = \max_{V' \subseteq V} \rho(V').$$

Remarquons qu'il peut y avoir *plusieurs* sous-graphes  $V^*$  de densité optimale  $\rho^* = \rho(V^*)$ . La figure 11.1 montre un graphe source avec son sous-graphe le plus dense.

Calculer les sous-graphes les plus denses est une opération très utile en analyse de graphes, et notamment pour les grands graphes qui modélisent les connections dans les réseaux sociaux ! Cette primitive est aussi liée à la détection de communautés et à la compression de graphes.

De nos jours, les données modélisant les réseaux sont omniprésentes dans notre quotidien. Citons comme exemples, les réseaux de communications, les réseaux de citations scientifiques, les réseaux de collaborations, les réseaux d'interactions de protéines, les réseaux d'information des médias, les réseaux financiers, etc.

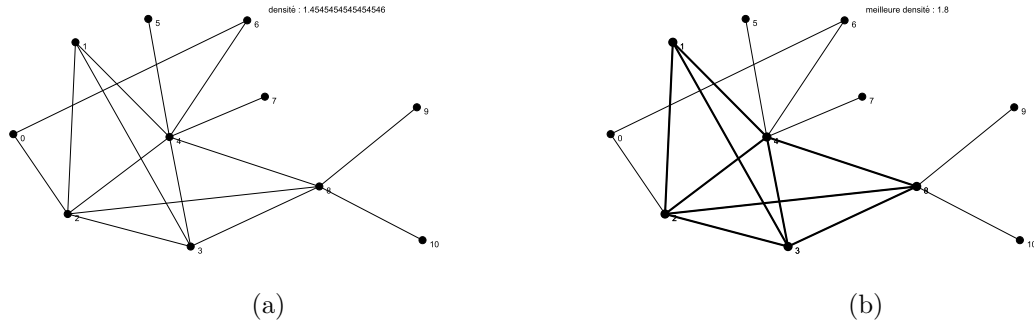


FIGURE 11.1 – Un exemple de sous-graphe le plus dense (b, sous-graphe épais) d'un graphe source (a) à 11 nœuds.

### 11.1.2 Complexité du problème et une heuristique gloutonne

En théorie, on peut résoudre ce problème en temps *polynomial* en utilisant une technique d'optimisation mathématique appelée la programmation linéaire [22]. Par contre, le problème devient *NP-difficile* lorsque l'on impose la contrainte  $|V'| = k$  : c'est-à-dire, quand nous contraignons la cardinalité du sous-graphe à avoir exactement  $k$  sommets.

Nous décrivons une heuristique qui calcule une *2-approximation* : c'est-à-dire, une méthode qui renvoie un sous-ensemble des sommets  $V'' \subseteq V$  tel que  $\rho(V'') \geq \frac{1}{2}\rho^*$ . Le sous-graphe  $G_{|V''}$  a un degré moyen au pire 50% du degré moyen d'un meilleur sous-graphe le plus dense  $G_{|V^*}$ .

Cette heuristique récente, proposée par Charikar<sup>1</sup> de l'université de Princeton en 2000, procède itérativement comme suit :

- enlever le nœud de plus faible degré ainsi que toutes ses arêtes incidentes, et mettre à jour les degrés des autres nœuds, puis recommencer jusqu'à épuisement des nœuds (c'est-à-dire, obtenir le graphe vide).
- garder le graphe intermédiaire le plus dense rencontré lors de ces  $n$  itérations.

Les figures 11.2 et 11.3 illustrent le déroulement de cette heuristique sur un graphe simple.

Une  $\alpha$ -approximation du sous-graphe le plus dense est un sous-graphe  $G_{|V'}$  (graphe  $G$  restreint aux nœuds de  $V'$ ) tel que  $\rho(V') \geq \frac{1}{\alpha}\rho(V^*)$ . Démontrons maintenant que cette heuristique qui consiste à enlever le sommet de plus faible degré itérativement et de garder le meilleur sous-graphe obtenu garantit bien une 2-approximation : notons  $V^*$  une solution optimale de densité  $\rho^* = \rho(V^*) = \frac{|E(V^*)|}{|V^*|}$ .

L'algorithme 7 ci-dessous décrit cette heuristique en pseudo-code :

1. <http://www.cs.princeton.edu/~moses/>

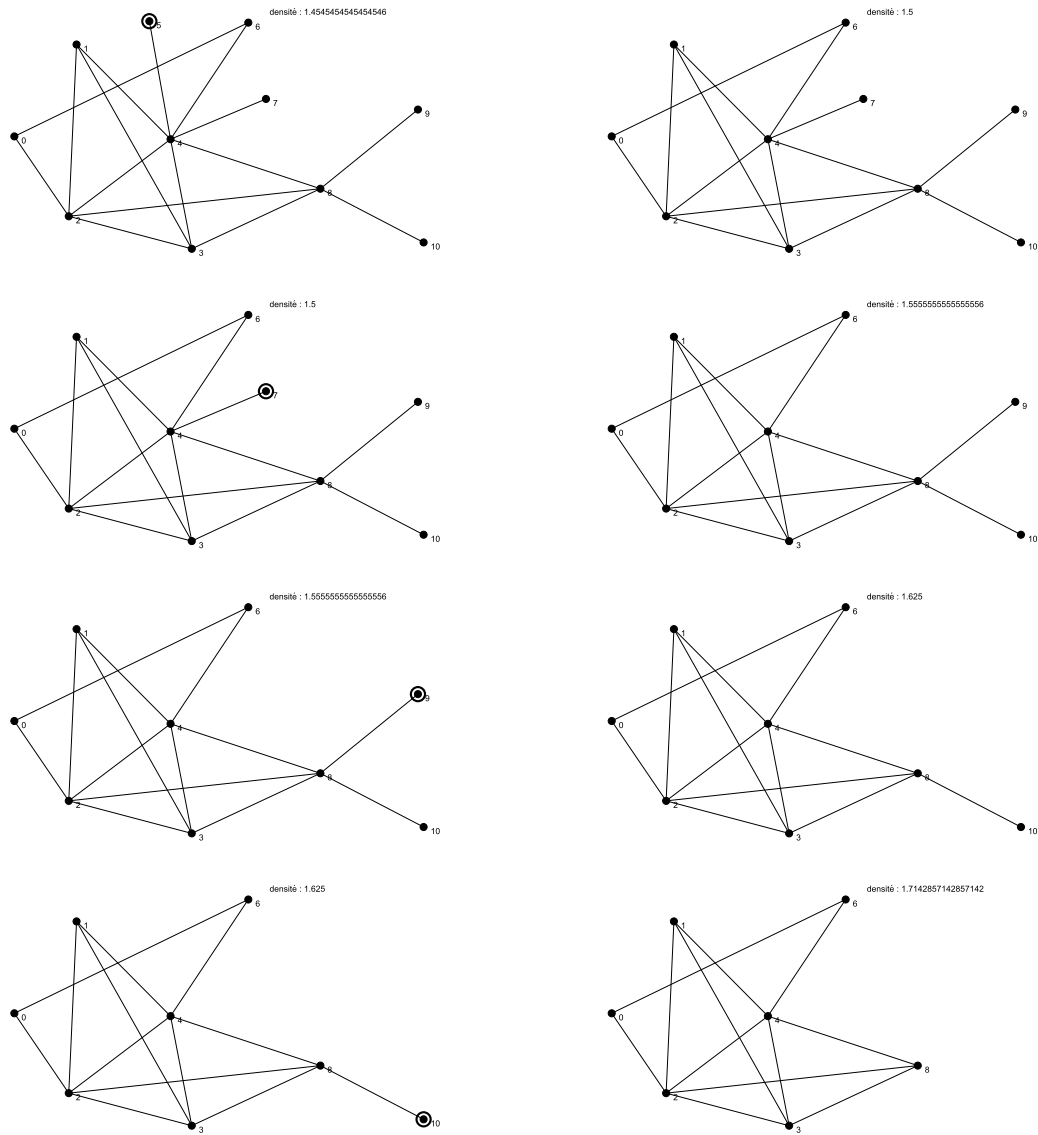


FIGURE 11.2 – Illustration de l'heuristique de Charikar pour trouver une 2-approximation du sous-graphe le plus dense : les différentes étapes de gauche à droite, et de haut en bas.

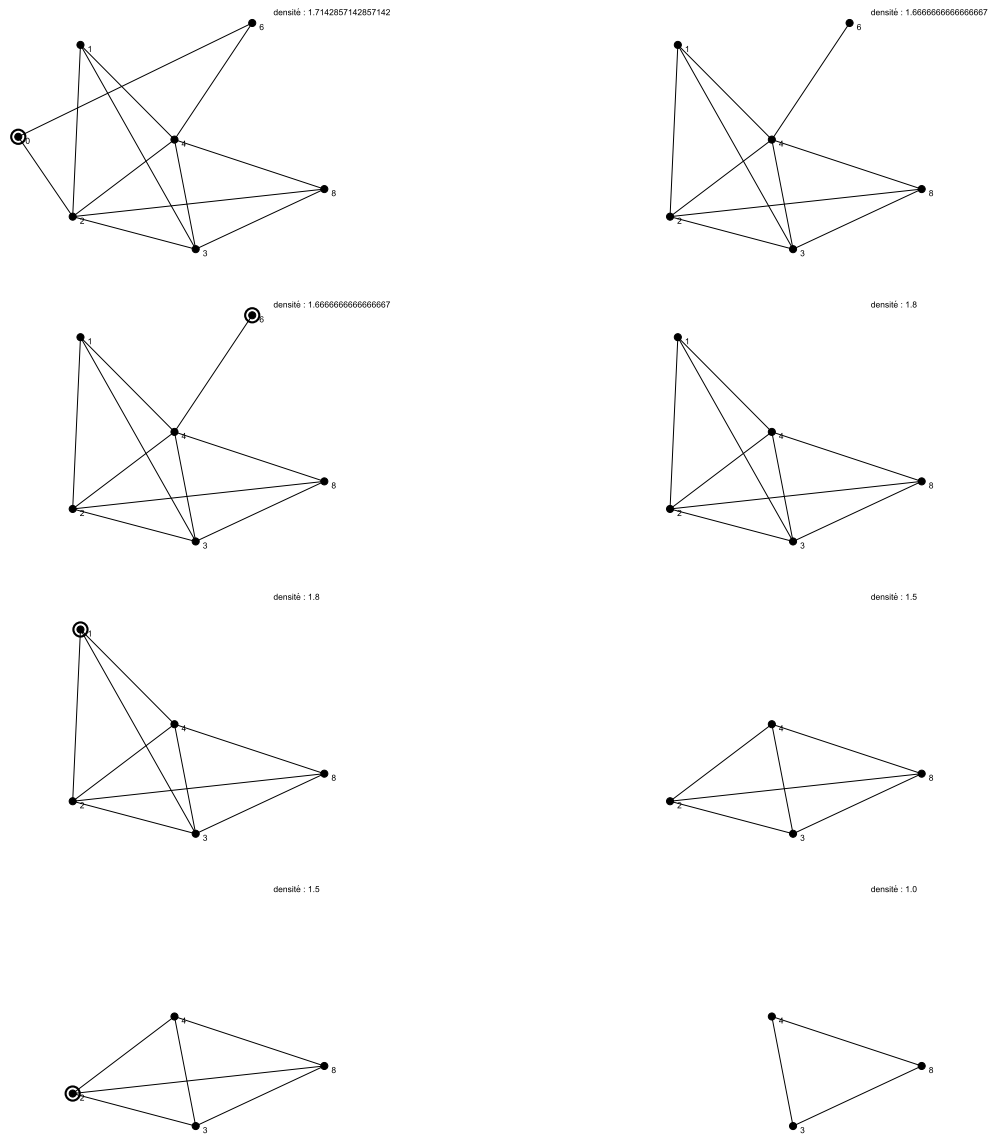


FIGURE 11.3 – Illustration de l’heuristique de Charikar pour trouver une 2-approximation du sous-graphe le plus dense : les différentes étapes de gauche à droite, et de haut en bas. La meilleure densité obtenue dans les graphes intermédiaires est  $\rho^* = \frac{9}{5} = 1,8$ . À chaque étape, le sommet entouré indique le prochain sommet sélectionné (de degré le plus petit) à être enlevé.

**Data :** Un graphe non-orienté  $G = (V, E)$   
**Result :** Retourne un sous-ensemble  $\tilde{S} \subseteq V$  des sommets qui induit le graphe restreint  $G|_{\tilde{S}}$  qui donne une 2-approximation du sous-graphe le plus dense de  $G$ .  
 $\tilde{S} \leftarrow V;$   
 $S \leftarrow V;$   
**while**  $S \neq \emptyset$  **do**  
     $s \leftarrow \arg \min_{s \in S} \deg_S(s);$   
     $S \leftarrow S \setminus \{s\};$   
    **if**  $\rho(S) > \rho(\tilde{S})$  **then**  
         $\tilde{S} \leftarrow S$   
    **end**  
**end**  
**return**  $\tilde{S}$

**Algorithme 7 :** Heuristique gloutonne de Charikar qui retourne une 2-approximation  $\tilde{S}$  du sous-graphe le plus dense de  $G = (V, E)$ .

On note que  $\sum_{s \in S} \deg_S(s) = 2|E(S)| = 2|S|\rho(S)$ . Prouvons d'abord simplement que la densité maximale  $\rho^* \leq d_{\max}$ , avec  $d_{\max}$  le degré maximum d'un nœud du graphe  $G$ . En effet, il y a au plus  $|V^*|d_{\max}$  arêtes dans  $E(V^*)$  (sinon le degré maximal ne serait pas  $d_{\max}$ !). On en déduit donc que  $E(V^*) \leq |V^*|d_{\max}$ , c'est-à-dire que :

$$\rho^* = \frac{E(V^*)}{|V^*|} \leq d_{\max}.$$

Considérons maintenant dans la séquence des itérations qui enlèvent les sommets un à un, la première fois qu'on enlève un nœud de  $V^*$  (on ne connaît pas explicitement  $V^*$ ). Alors chaque nœud de  $V^*$  a nécessairement un degré au moins égal à  $\rho^*$ , car sinon on pourrait augmenter la densité  $\rho^* = \frac{|E^*|}{|V^*|}$ . On en déduit que  $E(S) \geq \frac{1}{2}\rho^*|V^*|$ , et la densité de ce sous-graphe avec les sommets  $S$  est donc de :

$$\rho(S) = \frac{|E(S)|}{|V(S)|} \geq \frac{\rho^*|V^*|}{2|V^*|} = \frac{\rho^*}{2}.$$

Puisqu'on choisit le maximum des densités des graphes intermédiaires, on conclut que l'heuristique garantit une 2-approximation.

Sur le modèle real-RAM, on peut implémenter cette heuristique de plusieurs manières. Une implémentation directe coûte un temps quadratique  $O(n^2)$ . On peut aussi choisir une structure de données de type *tas* [72] (*heap*) pour sélectionner au fur et à mesure les sommets. On rappelle, qu'un tas est une structure de données abstraite représentée par un arbre binaire parfait (avec tous les niveaux pleins sauf éventuellement le dernier) qui vérifie que la clef d'un nœud est supérieure ou égale à la clef de ses fils (voir les cours INF311 [72]/321) pour tous les nœuds internes de l'arbre. En mettant à jour les clefs quand on enlève les arêtes, on arrive à un temps de calcul en  $O((n+m) \log n)$ . On peut implémenter cette heuristique en temps linéaire (en  $O(n+m)$ ) comme suit : on maintient les sommets dans au plus  $n+1$  listes  $L_i$  telles que chaque liste contient les sommets de degré  $i$ , pour  $i \in \{0, \dots, n\}$ . Lors d'une itération, on choisit un sommet  $s$  de la plus petite liste  $L_i$  non vide, puis on l'enlève de cette liste, et on met à jour les sommets adjacents. Entre deux itérations consécutives, la valeur du degré minimal ne peut changer que d'au plus un. Cette implémentation demande au total pour toutes les  $n$  itérations un temps global en  $O(n+m)$ .

Par contre, cette heuristique est difficilement parallélisable comme telle.

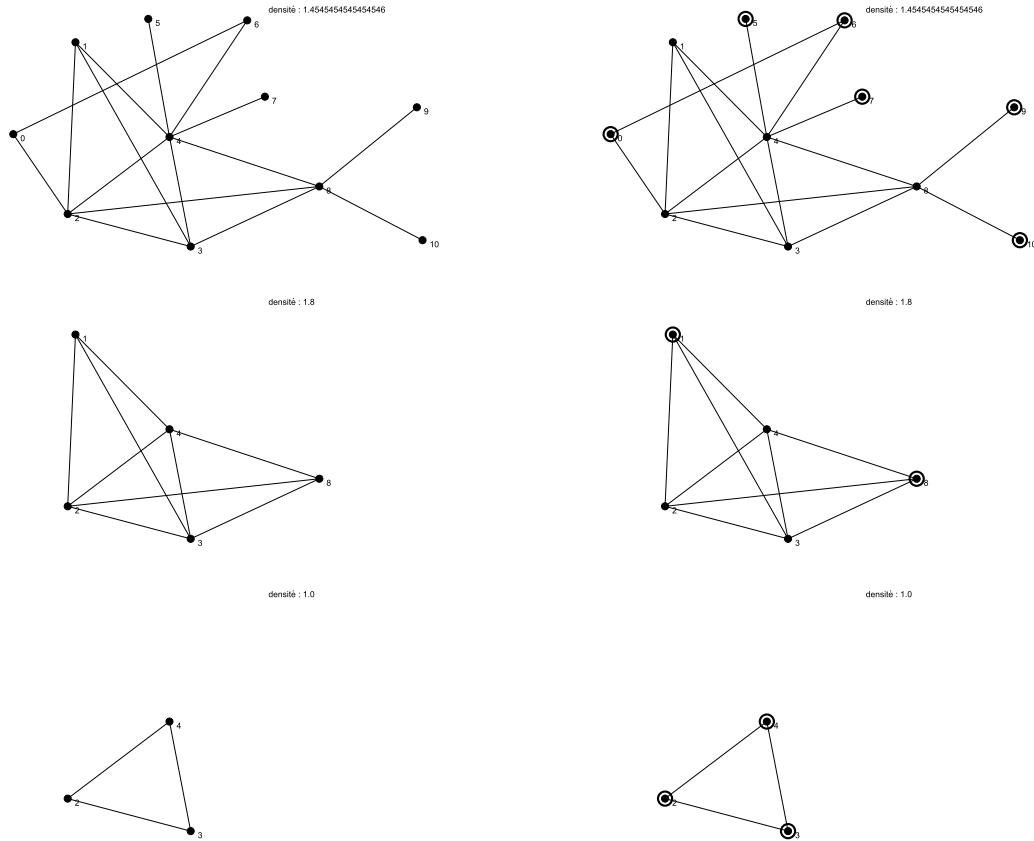


FIGURE 11.4 – Illustration de l’heuristique parallèle : on enlève simultanément tous les sommets de degré moins élevé que  $(1 + \epsilon)\bar{d}$ , où  $\epsilon > 0$  est fixé et  $\bar{d} = \rho(G)$  désigne la valeur du degré moyen du graphe  $G$ . Les sommets entourés montrent les sommets qui vont être enlevés à la prochaine étape.

**Théorème 14.** *On peut calculer une 2-approximation d’un sous-graphe le plus dense d’un graphe non-orienté à  $n$  sommets et  $m$  arêtes en temps linéaire  $O(n + m)$ .*

### 11.1.3 Une heuristique facilement parallélisable

On se donne  $\epsilon > 0$  (fixé) et on considère une nouvelle heuristique qui remplace la condition gloutonne du sommet de plus petit degré à enlever par : on enlève à chaque itération *tous* les nœuds de degré moins que  $2(1 + \epsilon)$  fois la moyenne des degrés  $\bar{d} = \rho(V)$ , et on calcule la densité  $\rho$  du graphe résultant  $V'$ , puis on réitère ainsi jusqu’à obtenir le graphe vide.

On parallélise donc par blocs ces deux étapes :

- étape 1 : compter les arêtes restantes, sommets et degrés des nœuds, afin de pouvoir calculer le degré moyen  $\bar{d}$ ,
- étape 2 : enlever tous les nœuds dont le degré est inférieur à  $2(1 + \epsilon)\bar{d}$  du graphe courant.

```

Data : Un graphe  $G = (V, E)$  et  $\epsilon > 0$ 
 $\tilde{S} \leftarrow V;$ 
 $S \leftarrow V;$ 
while  $S \neq \emptyset$  do
   $A(S) \leftarrow \{s \in S \mid \deg_S(s) \leq 2(1 + \epsilon)\rho(S)\};$ 
   $S \leftarrow S \setminus A(S);$ 
  if  $\rho(S) > \rho(\tilde{S})$  then
     $\tilde{S} \leftarrow S$ 
  end
end
return  $\tilde{S}$ 

```

**Algorithme 8 :** Heuristique gloutonne (parallèle) par blocs pour trouver une approximation  $\tilde{S}$  du sous-graphe le plus dense.

La figure 11.4 illustre le déroulement des étapes de ce nouvel algorithme d'approximation du sous-graphe le plus dense.

Quelle est la performance de cette heuristique ? On peut démontrer qu'on obtient une  $2(1 + \epsilon)$ -approximation comme suit : soit  $S$  l'ensemble des sommets du graphe restant lorsque pour la première fois un sommet de  $V^*$  se retrouve dans les sommets sélectionnés  $A(S)$ . Soit  $s = V^* \cap A(S)$  un sommet de la solution optimale. Alors on a  $\rho(V^*) \leq \deg_{V^*}(s)$  et comme  $V^* \subseteq S$ , on a  $\deg_{V^*}(s) \leq \deg_S(s)$ . Mais puisque  $s$  se trouve dans  $A(S)$  on a  $\deg_S(s) \leq (2 + 2\epsilon)\rho(S)$ . On en déduit donc par transitivité des  $\leq$  que :

$$\rho(S) \geq \frac{\rho(V^*)}{2 + 2\epsilon}.$$

Maintenant, puisqu'on choisit la meilleure densité lors des itérations, on a  $\rho(\tilde{S}) = \max_S \rho(S) \geq \frac{\rho(V^*)}{2 + 2\epsilon}$ . On peut définir  $\gamma = 2\epsilon$ , et avoir ainsi  $\forall \gamma > 0$  une  $(2 + \gamma)$ -approximation.

Analysons maintenant la complexité de cet algorithme en nombre d'étapes. On a

$$\begin{aligned} 2|E(S)| &= \sum_{s \in A(S)} \deg_S(s) + \sum_{s \in S \setminus \{A(S)\}} \deg_S(s), \\ &> 2(1 + \epsilon)(|S| - |A(S)|)\rho(S), \end{aligned}$$

en tenant compte que des termes dans la deuxième somme. Puisque  $\rho(S) = |E(S)|/|S|$ , on en déduit donc que :

$$\begin{aligned} |A(S)| &\geq \frac{\epsilon}{1 + \epsilon}|S|, \\ |S \setminus \{A(S)\}| &< \frac{1}{1 + \epsilon}|S|. \end{aligned}$$

On élimine donc une fraction des sommets à chaque itération, et on a au plus  $O(\log_{1+\epsilon} n)$  itérations (étapes parallèles).

**Théorème 15.** *Pour tout  $\epsilon > 0$ , l'heuristique garantit un facteur d'approximation  $2 + \epsilon$  après  $O(\log_{1+\epsilon} n)$  étapes.*

On décrit ci-dessous une implémentation parallèle de cette heuristique avec le formalisme MapReduce. On doit implémenter les trois primitives de base comme ceci :

- calcul de la densité  $\rho$  : opération triviale car on a seulement besoin de calculer le nombre total d'arêtes ainsi que le nombre total de nœuds présents à une itération donnée. On rappelle que les données sont stockées en MapReduce sous forme de paires (clef;valeur). On émet donc (“nœud”;“1”) pour chaque paire (clef;valeur) qui encode un nœud avec comme clef l'identificateur du nœud, et aussi (“arête”;“1”) pour chaque paire (clef;valeur) qui encode une arête avec comme clef l'identificateur de l'arête. Puis on réduit après regroupement des clefs intermédiaires similaires (donc soit “nœud” ou “arête”) en calculant les sommes cumulées pour ces deux clefs intermédiaires.
- calcul du degré de chaque nœud : on duplique chaque arête  $(u, v)$  en deux paires (clef;valeur) :  $(u; v)$  et  $(v; u)$ . On réduit en effectuant la somme cumulée sur les paires  $(u; v_1, v_2, \dots, v_d)$  et en produisant comme résultat la paire  $(u, \deg(u))$ .
- élimination des nœuds de degré plus petit qu'un seuil fixé  $t$  et de leurs arêtes incidentes : cela se fait en deux passes de MapReduce. Dans la première passe, on commence par marquer les nœuds  $v$  qui doivent être supprimés en émettant une paire  $(v; \$)$ . On associe aussi à chaque arête  $(u, v)$  la paire  $(u; v)$ . L'opération de réduction associée à  $u$  récupère toutes les arêtes dont une extrémité est  $u$  avec éventuellement le symbole spécial  $\$$ . Si le nœud  $u$  est marqué, le processus `reducer` ne retourne rien, sinon il recopie simplement son entrée. Puis dans la deuxième passe, on associe à chaque arête  $(u, v)$  la paire  $(v; u)$  et on recommence le procédé. Ainsi, seules les arêtes n'étant pas marquées vont survivre à ces deux passes de MapReduce.

En pratique, on note expérimentalement qu'on peut calculer une approximation  $G_{|V''}$  du sous-graphe le plus dense en une dizaine d'itérations pour des graphes ayant même un milliard de sommets. Cet algorithme est fort utile car il permet aussi la décomposition du graphe en sous-graphes denses en l'appliquant récursivement sur le graphe restant  $G' = G \setminus G_{|V''}$ , etc. Cet algorithme tournant sur de très grands graphes permet ainsi de découvrir des communautés.

## 11.2 Tester l'isomorphisme de (petits) graphes

Soient  $G_1 = (V_1, E_1)$  et  $G_2 = (V_2, E_2)$  deux graphes, avec  $n_i = |V_i|$  et  $m_i = |E_i|$ . On considère le problème qui consiste à déterminer si ces deux graphes sont identiques via une permutation  $\sigma$  sur la numérotation de leurs sommets : c'est le problème du test d'*isomorphisme de graphes*. Bien entendu, une condition nécessaire est que  $n_1 = n_2 = n$  et  $m_1 = m_2 = m$ . On note  $v_1^{(i)}$  et  $v_2^{(i)}$  les sommets de  $V_1$  et  $V_2$ , respectivement. La figure 11.5 illustre des graphes isomorphes. Ces graphes sont dessinés dans le plan, c'est-à-dire que leurs structures combinatoires sont plongées dans le plan : c'est l'art de dessiner les graphes (*graph drawing*)

Ce problème appartient à la classe NP mais on ne connaît toujours pas sa complexité. En effet, si on nous donne une permutation  $\sigma$ , alors on teste facilement en  $O(m)$  si les graphes sont bien isomorphes. On note  $G_1 \cong G_2$  lorsque ces graphes sont congruents, et on a donc  $G = G_1 \cong G_2$ . La permutation  $\sigma$  associe les nœuds  $v_1^{(i)}$  aux nœuds  $v_2^{(\sigma(i))}$ , pour  $i \in \{1, \dots, n\}$ . Notons que  $\sigma$  est définie à des sous-permutations près qui dépendent des symétries du sous-graphe. Par exemple, pour des cliques (des sous-graphes complets), on peut choisir n'importe quelle sous-permutation de façon équivalente. Il est aussi évident que le graphe complet  $K_n$  est équivalent à lui-même pour n'importe quelle permutation  $\sigma : K_n \cong K_n^{(\sigma)} \forall \sigma$ .

La difficulté provient du fait qu'il existe  $n!$  permutations possibles pour le choix de  $\sigma$  (voir

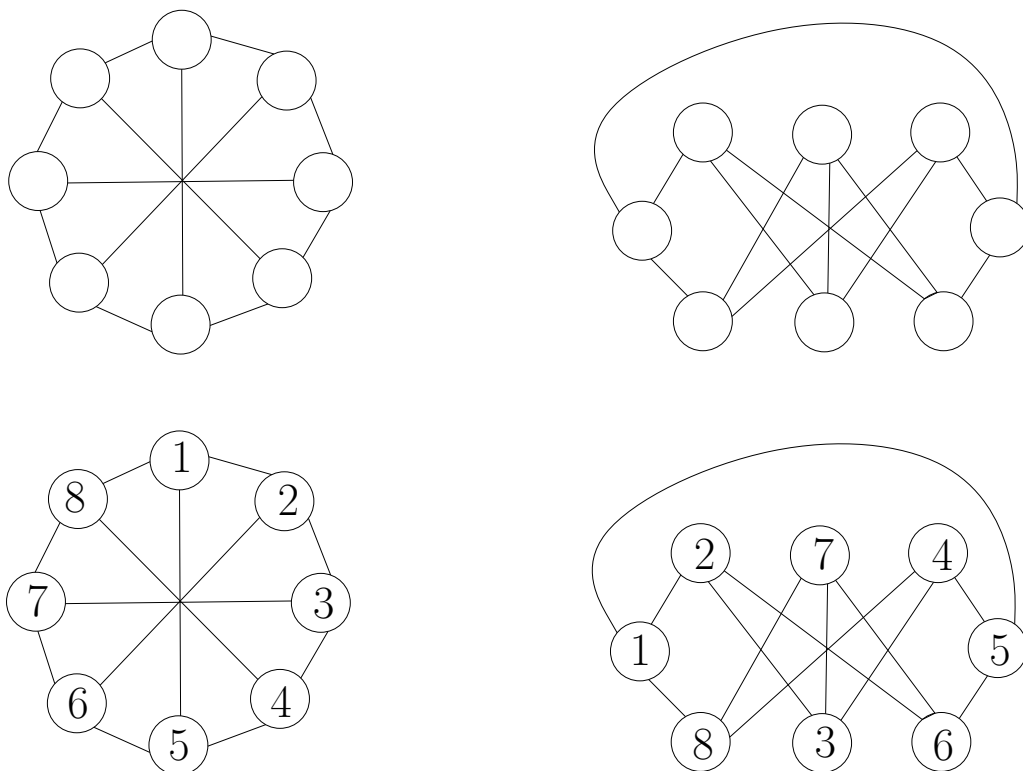


FIGURE 11.5 – Isomorphismes de graphes : graphes dessinés sans étiquette (en haut) et avec les numéros correspondants (en bas).

la figure 11.5). La permutation n'est pas nécessairement unique non plus car on doit tenir compte d'éventuelles symétries dans le graphe. Ce problème a été très étudié dans les années 1950 en chimie pour trouver si une molécule modélisée par son graphe se trouve déjà dans une base de données (autrement dit, si le graphe de la nouvelle molécule est isomorphe à un graphe de la base de données des graphes de molécules).

Par contre, le problème de tester s'il existe un sous-graphe  $G'_1$  de  $G$  tel que  $G'_1$  est isomorphe à  $G_2$ , est *NP-complet*. Un sous-graphe  $G'_1 \subseteq G_1$  est un graphe défini sur un sous-ensemble des sommets  $V'_1$  de  $V_1$  tel que les arêtes de  $G_1$  sont définies par  $E_1 = E_1 \cap (V'_1 \times V'_1)$  (on garde l'arête  $(u, v)$  ssi.  $u$  et  $v$  appartiennent à  $V'_1$ ).

Étant donné un graphe  $G = (V, E)$  avec  $|V| = n$  et  $|E| = m$ , on peut représenter combinatoirement ce graphe par une matrice binaire  $M$  (la *matrice d'adjacence*) telle qu'une arête  $(v_i, v_j) \in E$  ssi.  $M_{i,j} = 1$ , et 0 autrement. La matrice  $M$ , symétrique pour les graphes non-orientés a exactement  $2m$  entrées à 1. Si l'on considère la *matrice d'adjacence*  $M_1$  et  $M_2$  des graphes  $G_1$  et  $G_2$ , alors tester l'isomorphisme de graphes revient à tester s'il existe une permutation  $\sigma$  sur les indices telle que  $G_1 = G_2^{(\sigma)}$  avec  $G_2^{(\sigma)} = (\sigma(V_2), \sigma(E_2))$ .

### 11.2.1 Principes généraux des algorithmes d'énumération

La plupart des algorithmes qui ont été développés pour tester l'isomorphisme de graphes procèdent comme suit :

- on augmente itérativement un appariement partiel des sommets,
- les paires de sommets associés sont choisies de façon à respecter certaines conditions (par exemple, avoir le même degré),
- on élimine les chemins de recherche qui n'aboutissent pas à un appariement complet des sommets (élagage, *pruning*),
- lorsqu'on arrive à une impasse, on supprime la dernière hypothèse (faire marche arrière, *backtracking*),
- l'algorithme s'arrête lorsqu'il a trouvé une solution (avec un certificat  $\sigma$ ) qui prouve l'existence de l'isomorphisme ou lorsque tous les appariements possibles ont été testés de façon infructueuse.

La complexité de cet algorithme naïf (mais générique) est dans le pire des cas en  $O(n!)$ , un temps super-exponentiel<sup>2</sup> ! Cet algorithme permet aussi de résoudre le problème d'isomorphisme de sous-graphes. On rappelle que certains algorithmes optimaux ont des temps exponentiels (comme les fameuses tour de Hanoï à  $n$  disques qui demandent  $2^n - 1$  opérations). Par contre, pour le test d'isomorphisme la borne inférieure n'est toujours pas prouvée, et cela reste un défi en informatique théorique.

### 11.2.2 L'algorithme d'Ullmann pour tester l'isomorphisme

Il s'agit de l'un des plus vieux algorithmes (1976) pour tester l'isomorphisme de sous-graphes qui utilise les matrices d'adjacence. On introduit les *matrices de permutation* qui sont des matrices carrées avec des entrées binaires telles que sur chaque ligne et sur chaque colonne, on ait exactement un seul 1. Par exemple, la matrice :

---

2. En effet,  $\log n! = O(n \log n)$ .

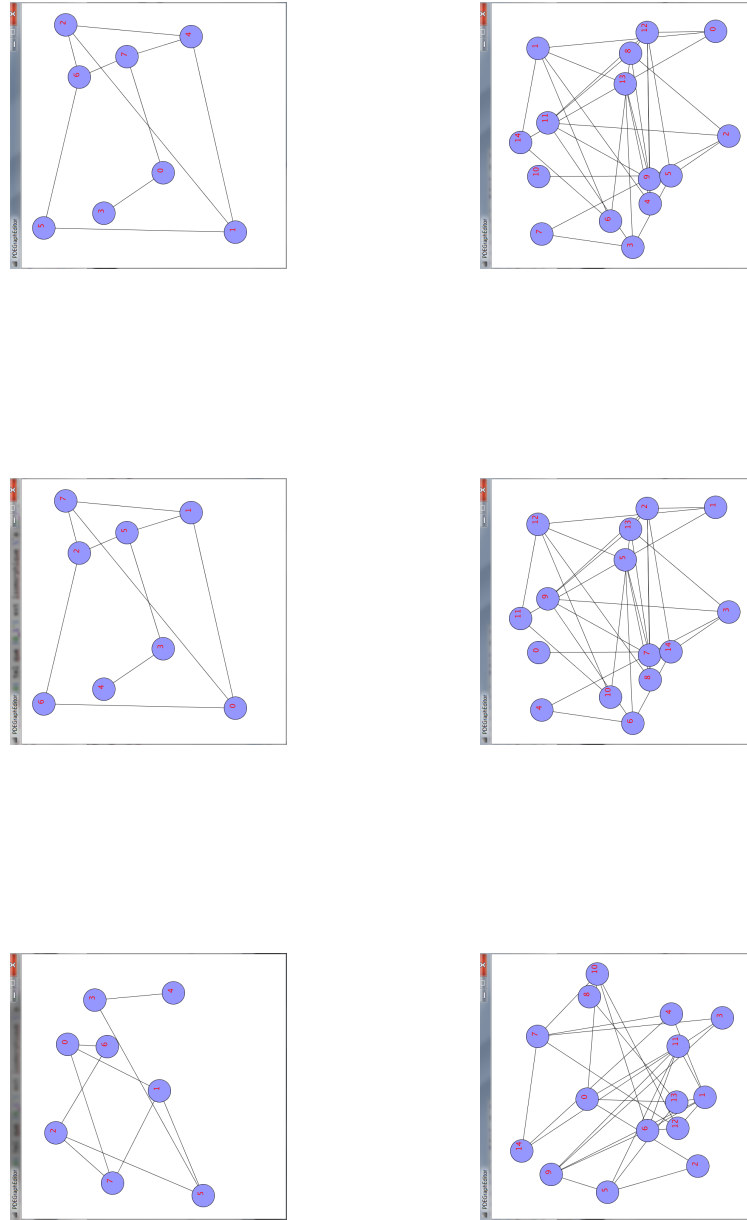


FIGURE 11.6 – Exemples de graphes isomorphes : les structures combinatoires des graphes sont dessinées en choisissant arbitrairement des positions  $(x, y)$  pour les nœuds. Haut (pour  $n = 8$  nœuds) : (a) graphe original, (b) même graphe dessiné avec une autre position  $(x, y)$  pour les sommets, et (c) après application d'une permutation sur les étiquettes. Il est plus difficile de comparer directement (a) avec (c). La permutation recherchée est  $\sigma = (1, 4, 6, 0, 3, 7, 5, 2)$  : elle prouve que les graphes (a) et (c) sont isomorphes. Bas : un autre exemple pour  $n = 15$  nœuds.

$$P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix},$$

est une matrice de permutation.

Ces matrices de permutation sont en bijection avec les permutations  $\sigma$  de  $(1, \dots, n)$ . La matrice de permutation  $P_\sigma$  correspondant à  $\sigma$  est :

$$[P_\sigma]_{i,j} = \begin{cases} 1 & \text{si } i = \sigma(j), \\ 0 & \text{sinon.} \end{cases}$$

On a de plus une structure de groupe puisque  $P_{\sigma_1} P_{\sigma_2} = P_{\sigma_1 \circ \sigma_2}$ , avec  $\sigma_1 \circ \sigma_2$  la composition des permutations  $\sigma_1$  et  $\sigma_2$ .

Une propriété essentielle est que deux graphes  $G_1$  et  $G_2$  sont isomorphes ssi. leurs matrices d'adjacence  $M_1$  et  $M_2$  satisfont :

$$M_2 = P \times M_1 \times P^\top, \quad (11.1)$$

avec  $P^\top$  la matrice transposée de la matrice  $P$ , pour une matrice de permutation  $P$ . Cette identité est équivalente à  $M_2 P = P M_1$ .

On peut donc énumérer toutes les matrices de permutation<sup>3</sup> et tester si on a l'égalité matricielle donnée à l'équation 11.1 pour au moins une de ces matrices.

L'algorithme d'Ullman [92] repose sur ce principe en effectuant un parcours en profondeur (*depth first search*, DFS)

La complexité de l'algorithme d'Ullmann pour tester l'isomorphisme de graphes à  $n$  sommets est en  $O(n^n n^2)$ . En pratique, on peut tester l'isomorphisme pour des graphes allant jusqu'à quelques centaines de nœuds grâce à l'élagage (*pruning*) et au backtracking.

### 11.2.3 Parallélisation de l'algorithme énumératif

Si on utilise  $P$  processeurs en parallèle, nous allons voir que nous pouvons réduire le temps de calcul d'un facteur  $P$ , et obtenir ainsi une accélération optimale. La parallélisation se fait trivialement en considérant initialement pour le sommet  $v_1^{(1)}$  tous les sommets  $v_2^{(j)}$  avec  $j \in \{1, \dots, P = n\}$  sur le processeur  $P_j$ . La complexité de l'algorithme en parallèle est donc en  $O(n^n n^2 / P)$ , soit  $O(n^n)$  quand  $P = n$ . Bien entendu, en pratique, on n'apparie  $v_1^{(1)}$  à  $v_2^{(j)}$  que si ces sommets ont le même degré. Hormis le cas de la clique, certains processeurs vont donc se retrouver sans travail en pratique. Il est donc judicieux de choisir un processeur maître et les  $P - 1$  autres processeurs esclaves. Le processeur maître donne ensuite les instructions de recherche aux processeurs esclaves en communiquant avec des messages. Quand un processeur esclave finit soit en arrivant dans une impasse (permutation incomplète ne donnant pas l'isomorphisme de graphes), soit en trouvant une permutation complète démontrant l'isomorphisme, il communique son résultat au processus maître. Le processus maître doit alors implémenter une routine pour faire l'équilibrage de charges (*load-balancing*) pour ses  $P - 1$  processeurs travailleurs.

3. Tout comme le problème des placements de  $n$  reines en positions libres sur un échiquier de taille  $n \times n$ . Voir le cours INF311 [72].

D'une manière générale, de nombreux algorithmes sur les graphes se retrouvent dans une situation similaire où il faut pouvoir équilibrer intelligemment les différentes tâches des processeurs. C'est le problème de l'*ordonnancement de tâches*.

### 11.3 \* Pour en savoir plus : notes, références et discussion

L'heuristique gloutonne pour approximer le sous-graphe le plus dense a été proposée par Moses Charikar [22] en 2000. L'algorithme parallèle (heuristique pour MapReduce ou pour un flot de données) pour calculer un sous-graphe dense est décrit dans ce papier [10] de 2012. Les figures illustrant ces heuristiques proviennent de la présentation de Sergei Vassilvitskii. Tester l'isomorphisme de graphes peut être réalisé en temps  $2^{O(\sqrt{n \log n})}$  [7]. Pour les graphes planaires (pour lesquels, il existe un plongement dans le plan tel que deux arêtes ne se coupent pas), on peut tester l'isomorphisme de graphes en temps linéaire. Notons que les arbres appartiennent à la classe des graphes planaires. L'algorithme d'Ullman [92] qui se base sur les matrices d'adjacence des graphes fut publié en 1976. L'algorithme plus performant VF2 (version améliorée de l'algorithme VF [23]) est implémenté dans la bibliothèque `Boost`<sup>4</sup> du C++. Une comparaison expérimentale de cinq algorithmes pour tester l'isomorphisme de graphes est décrite dans [33].

### 11.4 En résumé : ce qu'il faut retenir !

Les graphes permettent de modéliser structurellement les données en introduisant des relations entre elles (les arêtes). De nos jours, on trouve des graphes volumineux lorsqu'on fait l'analyse de réseaux de communications comme les réseaux sociaux (facebook, twitter, etc.). L'analyse de graphes devient un des sujets dominants des sciences des données. On a présenté une heuristique gloutonne simple pour trouver une approximation d'un sous-graphe le plus dense, qui peut représenter une communauté, et on a montré comment paralléliser efficacement cette heuristique. Puis nous avons présenté le problème de tester si deux graphes sont identiques à une permutation près de leurs étiquettes : le test d'isomorphisme de graphes. La complexité de ce problème n'est pas connue alors qu'il est facile de vérifier que les graphes sont isomorphes si la permutation le démontrant est donnée. Nous avons présenté un algorithme d'énumération simple qui utilise les techniques d'élagage (pruning) et de retour en arrière (backtracking), et montré comment paralléliser celui-ci. Cette dernière étape souligne l'importance d'ordonnancer dynamiquement les tâches à réaliser par les différents processeurs afin de faire un bon équilibrage de charges pour obtenir un bon temps de calcul parallèle : le *load-balancing*.

### 11.5 Exercices

**Exercice 37 :** *Sous-graphe le plus dense pour un graphe à poids*

Soit  $G = (V, E, w)$  un graphe avec les arêtes pondérées par une fonction positive  $w(\cdot)$ . Comment généraliser l'heuristique de Charikar pour trouver une approximation du sous-graphe le plus dense ?

---

4. <http://www.boost.org/>

**Exercice 38 :** *\* Sous-graphe le plus dense : implémentation rapide*

Montrez comment implémenter l'heuristique de Charikar en temps linéaire  $O(n + m)$  en maintenant des listes de sommets organisées par valeur du degré des sommets. Montrez comment utiliser les tas de Fibonacci pour déterminer efficacement le degré minimum des sommets à chaque itération de l'heuristique de Charikar.

**Exercice 39 :** *Détection de sous-graphe le plus dense en parallèle*

Comment implémenter sous MPI l'algorithme qui enlève par étape des sous-ensembles de sommets ? Quelle est l'accélération obtenue ?

# Index

- ( $k, \epsilon$ )-sous-ensemble noyau, 225
- 1-port, 68
- $\epsilon$ -coreset, 221
- $\epsilon$ -noyau, 221
- $k$ -port, 68
- 2D bloc ligne-colonne cyclique, 128
- BLAS, 127
- BigData, 220
- Calcul Haute Performance, 3
- Concurrent Read Exclusive Write, 22
- Cycle Connecté en Cube, 71
- E/Ss, 15
- Exclusive Read Exclusive Write, 22
- Hadoop, viii
- Most Significant Bit, 88
- NP-difficile, 232
- RSS, 125
- Residual Sum of Squares, 125
- SSD, 226
- SVMs, 214
- Super-Computing, ix
- échange total personnalisé, 80
- échange total, 80
- échelle, 78
- élément médian, 106
- équilibre de charge, 27
- étoile, 87
- accélérations super-linéaires, 12
- accélération, 11
- accumulateur, 131
- algèbre linéaire, 123
- algorithme déterministe, 104
- algorithme de diffusion, 88
- algorithme randomisé, 104
- algorithmes adaptatifs, 100
- anneau bidirectionnel, 115
- anneau, 70
- apprentissage de variétés, 229
- arêtes, 68
- arbre binaire de fusion, 189
- arbre binomial, 88
- arbre de décision, 100
- arbre recouvrant de poids minimal, 203
- attente, 25
- attributs, 161
- bande passante, 7
- barrière de synchronisation, 36
- barrières de synchronisation, 15
- barycentre, 164
- bidirectionnels, 68
- bit de poids fort, 77
- bloc-colonne cyclique, 128
- bloc-ligne cyclique, 128
- blocage, 80
- buffer des données, 29
- bus commun, 67
- bus partagé, 93
- calcul de préfixe, 150
- calcul local, 135
- calcul multi-précision, 222
- calcul vectoriel, 130
- calculs collaboratifs, 22
- centre circonscrit, 221
- centre de gravité, 175
- centroïde, 164
- chemin, 68
- circuit de comparateurs, 115
- classe ultramétrique, 198
- classification supervisée, 161
- clustering dur, 162
- code binaire réfléchi, 74
- code binaire, 74
- code de Gray, 74
- collision, 93

- comméragé, 80
- communicateurs, 35
- communication bi-directionnelle, 32
- communication uni-directionnelle, 32
- communications asynchrones, 29
- communications bufferisées, 29
- communications globales, 22
- communications synchrones, 25
- communicators, 35
- commutateur, 93
- confidentialité, 182
- connexité, 69
- cordes, 70
- curse of dimensionality, 225
- cycles d'horloge, 92
- débit, 80
- débordements, 87
- décale horizontalement, 137
- décale verticalement, 137
- damier, 128
- degré, 69
- dendrogramme, 189
- densité, 231
- diagramme de Voronoï, 174
- diagrammes de Voronoï, 175
- diamètre, 69
- diffusion personnalisée, 80, 84
- diffusion pipelinée, 86
- diffusion, 80
- dilatation, 88
- dimension, 69
- dispersion, 174
- dissimilitude, 195, 196
- distance de Hamming, 75, 195
- distance de Mahalanobis, 195
- distance de Manhattan, 195
- distance, 68
- distorsion, 174
- efficacité, 11
- erreur moyenne quadratique, 174
- espace des configurations, 53
- estimation non-paramétrique, 210
- expansion, 88
- extensibilité, 11
- feature selection, 227
- fichier de configuration initiale, 269
- fil de calcul, 22
- fils de calcul, 22
- fléau de la dimension, 219
- fléau des grandes dimensions, 225
- flux de données, 17
- fonction d'énergie, 163
- fonction objective, 163
- fonctions de chaînage, 191
- fouille de données, 155
- fouille des données, 229
- full-duplex, 68
- fusion récursive de listes bitoniques, 115
- garbage collector, 254
- gestionnaire de communications, 68
- grain fin, 5
- grain intermédiaire, 6
- granularité de parallélisme, 115
- granularité fine, 148
- granularité, 5
- graphe, 68
- grille, 71
- gros grain, 6
- half-duplex, 68
- hypercube, 73
- inertie, 175
- iso-efficacité, 14
- isobarycentre, 166
- isomorphisme de graphes, 238
- jeton, 93
- langage C, 251
- langage impératif, 251
- largeur de bisection, 69
- latence, 7, 92
- liens bidirectionnels, 68
- linéarisation, 220
- livre de codes, 174
- load-balancing, 243
- localité spatiale, 6
- loi de Gustafson, 13
- loi paramétrique, 210
- longueur d'un chemin, 68
- low distortion embedding, 227
- médiane, 104, 106
- mémoire distribuée, 6
- mémoire partagée, 6
- mémoire vive dynamique, 92

métrique, 198  
machine virtuelle, 157  
machines à vecteurs de support, 214, 224  
matrice d'adjacence, 240  
matrice de covariance, 195  
matrice de précision, 195  
matrice pseudo-inverse de Penrose-Moore, 125  
matrices de permutation, 240  
motif 2D bloc ligne-colonne, 128  
motif bloc-colonne, 128  
motif bloc-ligne, 128  
multi-port, 68  
multi-tâches, 22  
nids de boucles, ix  
nombre de liens, 69  
non-bloquantes, 22  
opérateur binaire, 149  
opérations bufferisées, 87  
opérations de préfixe, 34  
ordinateur parallèle, 67  
ordonnancement de tâches, 243  
ordonnanceur, 22  
ou exclusif, 74  
parcours en largeur, 71  
parcours en profondeur, 71  
partition bitonique, 115  
plongement à faible distortion, 227  
plongement, 67, 88  
plus petite boule englobante, 221  
poids faibles, 88  
poids forts, 88  
position générale, 221  
processus supérieur, 106  
produit d'Hadamard, 136  
produit de Kröneckers, 136  
produit scalaire, 127  
programmation fonctionnelle, 148  
programmation quadratique, 224  
propriété de clôture, 78  
pseudo-code, 138  
quality of service, 16  
réduction all-to-all, 47  
réduction généralisée, 47  
réseau d'interconnexion, 67  
réseau de tri, 112  
réseau logique, 67  
réseau oméga, 93, 95  
réseau physique, 67  
réseaux dynamiques, 67  
réseaux statiques, 67  
rang, 103  
rassemblement, 80  
recherche exploratoire, 161  
recouvrement, 84  
redondance par réplication, 154  
redondance, 16  
regroupement hiérarchique descendant, 191  
regroupement hiérarchique divisif, 191  
regroupement plat, 189  
regroupement, 161  
rendez-vous, 25  
ressemblance, 196  
rotations horizontales, 137  
rotations verticales, 137  
routage, 68  
sélection d'attributs, 227  
sélection de modèle, 176  
séquence bitonique, 115  
séquence unimodale, 115  
saut maximum, 191  
saut minimum, 191  
scalabilité, 11  
sciences des données, viii, 123  
shift circulaire, 49  
situations de blocage, 28  
somme préfixe, 34, 47  
sommets, 68  
sous-ensemble noyau, 221  
sous-ensembles noyaux, 220  
sous-graphe, 231  
speed-up, 11  
strictement convexe, 165  
sum of squared differences, 226  
sum of squared errors, 220  
super-étape, 39  
super-calculateurs vectoriels, 11  
super-ordinateurs, 3  
systèmes de fichiers parallèles, 15  
table de vérité, 74  
tags, 80  
tas, 235  
tenseurs, 127

- time-out, 28
- topologie régulière, 73, 91
- topologie virtuelle, 71
- topologie, 7, 68
- tore 2D, 135
- tore, 71
- torus, 71
- transposition, 49, 67, 88
- tri à bulles, 99
- tri par propagation, 99
- triangulation de Delaunay, 175
- un groupe de communication, 23
- unidirectionels, 68
- vérité terrain, 179
- vecteurs colonnes, 125
- vectorisation, 220
- vitesse de transmission, 80
- zones mémoires tampons, 29
- city block*, 195
- dissimilarity*, 195
- support vector machines*, 214

Boost, 243

Gene Amdahl, 11

Langage C

- NULL, 254
- malloc, 254
- struct, 255
- typedef, 255
- arithmétique sur les pointeurs, 257
- entrées et sorties, 257
- macros, 251
- pointeurs de structure, 255
- structures autoréférentielles, 255
- tableau statique multi-dimensionnel, 254
- tableau, 254
- fprintf, 255
- void \*, 255

Langage C++

- container, 265
- généricité, 264
- template, 264

# Troisième partie

## C/C++/Shell



## Annexe A

# Le langage C quand on connaît déjà Java

Le *langage C* a été conçu dans les années 1970 par Dennis Ritchie et Ken Thompson des Laboratoires AT&T Bell aux États-Unis. C'est un *langage impératif* (pas orienté objet) qui fut un précurseur dans le développement des langages orientés objets (OOs) comme C++ et Java. On utilise le compilateur GNU C<sup>1</sup> `gcc` pour compiler ce petit programme :

```
/* Premier programme */
#include <stdio.h>

void main(void)
{printf("Coucou INF442 !\n");}
```

On compile ce programme `premier.c` avec :

```
gcc premier.c -o premier.exe
```

et on exécute ce programme compilé avec :

```
./premier.exe
```

Le message est alors affiché dans la console du terminal.

On résume très succinctement les principaux concepts du C. Il est fortement conseillé de consulter l'ouvrage K&C [54] (pour Kernighan et Ritchie), ou de suivre les nombreux tutoriels en ligne sur Internet.

### A.1 Fichiers d'en-tête `.h`, fichiers de description `.c`, macros et le préprocesseur

Un programme peut être divisé en plusieurs fichiers afin de faciliter sa lisibilité et permettre aussi la modularité du code. On écrit les signatures des fonctions/procédures et les *macros* dans un

---

1. <https://gcc.gnu.org/>

fichier d'en-tête (*header file*) `programme.h` et le *corps de ces fonctions* dans un fichier de description `programme.c`

Le petit exemple ci-dessous illustre ces concepts.

Fichier `programme.h` :

```
#ifndef programme_h
#define programme_h

#define MAX(a,b) ((a)<(b) ? (b) : (a))

#endif
```

Fichier `programme.c` :

```
#include <stdio.h>
#include "programme.h"

int maxi(int a, int b)
{if (a>b) return a; else return b;}

double maxd(double a, double b)
{if (a>b) return a; else return b;}

void main(void){
    printf("Bonjour INF442 !\n");
    int a=442, b=2013;
    printf("Le maximum est : %d\n", maxi(a,b));
    double ad=442.0, bd=2013.0;
    printf("Le maximum est : %f\n", maxd(ad,bd));
    printf("avec la macro : %d %f\n", MAX(a,b), MAX(ad,bd));
}
```

On compile avec `gcc programme.c -o programme.exe` et on exécute avec `./programme.exe` pour obtenir à la console :

```
Bonjour INF442 !
Le maximum est : 2013
Le maximum est : 2013.000000
avec la macro : 2013 2013.000000
```

Lorsque l'on compile, le préprocesseur commence par remplacer les macros dans le code en donnant leurs expansions instanciées. Les macros sont très utiles par exemple pour fixer des constantes dans les programmes :

```
#define N 100
#define Dim 3
#define MonPI 3.14
```

Quand on écrit une macro, on parenthèse toujours les arguments afin d'éviter de faux sous-entendus comme celui-ci :

```
#define carre(x) x*x
```

En effet, si on a `carre(x+1)` dans le code, il sera remplacé par le processeur par `x+1*x+1` qui n'est pas  $(x+1)^2$ . Par contre, si on utilise la macro :

```
#define carre(x) ((x)*(x))
```

alors on aura `carre(x+1)` qui deviendra `((x+1)*(x+1))`, comme on le souhaite bien !

Puisqu'on ne peut pas inclure deux fois les mêmes définitions de fonctions, on garantit qu'on va lire une seule fois le fichier d'en-tête avec

```
#ifndef programme_h
#define programme_h
/* on declare les fonctions ici */
#endif
\end{listing}
```

Pour enlever la d\'efinition d'une macro, on utilise `{\tt undef identificateur}`.

Enfin, on affiche des messages sur la console avec la proc\'edure `\termdefC{\tt printf}` qui prend un nombre variable d'arguments : le premier est une cha\`i{}ne de caract\'eres pour le formatage qui peut inclure des `\%d`, `\%f`, `\%s`, et les arguments suivants correspondent aux valeurs qui seront remplac\'ees dans cette cha\`i{}ne de caract\'eres avant d'\^etre imprim\'ees sur la console (`'d'` pour le type entier `{\tt int}`, `'f'` pour le type `{\tt float}`, `'s'`, `'lf'` pour le type `{\tt double}`, pour une cha\`i{}ne de caract\'eres, etc.).

```
%%%%%
\section{Pointeurs et passage par valeur}
%%%%%
```

Un programme utilise de la m\'emoire qui est physiquement un tableau de cases m\'emoires cons\'ecutives num\'erot\'ees.

Ces num\'eros correspondent aux adresses des cases m\'emoires.

L'op\'erateur unaire `\termdefC{\tt \&}` retourne l'adresse d'une variable.

Par exemple, `{\tt \&v}` est un `\termdefC{pointeur}` sur la variable `{\tt v}`.

L'op\'erateur unaire `{\tt *}` est un op\'erateur de `\termdef{d\'ef\'erence}` qui s'applique sur des pointeurs pour donner acc\'es aux variables point\'ees par ces pointeurs.

Le passage des arguments dans les fonctions/proc\'edures se fait par `{\it valeur}` comme pour Java.

Ainsi, le programme ci-dessous est bien entendu faux :

```
\begin{lstlisting}[style=myc]
void echangerFaux(int x, int y)
{int tmp;
 tmp=x;
 x=y;
 y=tmp;
}
```

voir les cours INF311 (cours en anglais [72]) et INF321, et on doit plutôt faire :

```
void echanger(int *x, int *y)
{int tmp;
 tmp=*x;
 *x=*y;
 *y=tmp;
}
```

On appelle alors cette fonction sur deux variables `a` et `b` comme ceci : `echanger(&a,&b);`

## A.2 Allocation mémoire et destruction de tableaux

Pour déclarer un *tableau* en C, on utilise la syntaxe suivante :

```
int entT[442];
double reelT[2013];
```

Les tableaux de taille  $n$  sont indexés entre 0 et  $n-1$  comme en Java. Un pointeur sur la troisième case du tableau sera donc :

```
int *pa = &entT[2];
```

On peut définir un tableau dynamiquement avec `malloc` (*memory allocation*) :

```
int * tab = malloc(2015 * sizeof(int));
```

Notons que `sizeof` permet de connaître la taille mémoire occupée en octets d'une donnée de type `int`. Il faut inclure le fichier `stdlib.h`<sup>2</sup> avec `#include <stdlib.h>` pour avoir accès à `malloc`. Il se peut qu'au moment de la création du tableau nous n'ayons plus assez d'espace mémoire. Dans ce cas, `malloc` retourne le pointeur `NULL`. Donc en général, on écrit plutôt :

```
int * tab;
if ( NULL == (tab = malloc(2015 * sizeof(int))) )
{ /* si on ne prend pas cette pr\'e caution, alors on pourra
   avoir un memory fault */
  printf("pas assez de memoire\n");
  return(-1);
}
```

Lorsque l'on n'utilise plus le tableau `tab`, on doit le libérer avec `free` :

```
free(tab);
```

En C, il n'y a pas de ramasse-miettes (*garbage collector*) comme en Java!

On définit un *tableau statique multi-dimensionnel* comme suit :

```
static int tab[2][3]={1,2,3},{4,5,6};
```

Pour allouer dynamiquement un tableau bi-dimensionnel, on procède comme suit :

```
int **array;
array = malloc(nrows * sizeof(int *));
if(array == NULL)
{
  fprintf(stderr, "pas de memoire\n");
  return(-1);
}
for(i = 0; i < nrows; i++)
{
  array[i] = malloc(ncolumns * sizeof(int));
  if(array[i] == NULL)
  {
    fprintf(stderr, "pas de memoire\n");
    return(-1);
  }
}
```

---

2. `stdlib` est le raccourci pour *standard library*

Notez qu'ici on a remplacé `printf` par `fprintf` qui écrit sur un fichier, celui de la sortie erreur (à la Unix). On garantit ainsi que le message d'erreur est immédiatement écrit sur la console et non pas bufferisé comme pour `printf`.

Pour libérer l'espace mémoire, on doit maintenant faire :

```
for(i = 0; i < nrows; i++)
    free(array[i]);
free(array);
```

En C, les chaînes de caractères sont des tableaux de `char`. Le type `void *` (pointeur sur `void`) permet de définir un pointeur générique.

### A.3 Construction de structures de données avec struct

Le langage C n'est pas orienté objet mais possède une construction de structure avec le mot clef `struct` :

```
struct Point {
    double x;
    double y;
};

struct Couleur {float r, g, b};
```

On déclare alors des points comme suit :

```
struct Point pt;
struct Point pt2={0.5, 0.7};
```

On accède aux champs de ces structures avec le `."` : `pt.x`, `pt.y`, ... On peut créer des structures qui utilisent d'autres structures :

```
struct PointCouleur {
    struct Point pt;
    struct Couleur coul;
};
```

Les *pointeurs de structure* se définissent comme `struct Point *pp`; et on accède alors à leurs champs avec `"->"` : `pp->x`, `pp->y`, etc.

Pour calculer la taille mémoire occupée par une donnée construite à partir d'une structure, on utilise `sizeof` :

```
sizeof(PointCouleur);
```

Les *structures autoréférentielles* comme les nœuds d'un arbre se définissent comme suit :

```
struct Noeud {
    int valeur;
    struct Noeud *gauche;
    struct Noeud *droit;
};
```

On peut définir aussi un alias sur un type de données avec `typedef` :

```
typedef int Entier;
typedef double Reel;
```

C'est fort utile en pratique car par exemple on peut écrire le code avec le type `Reel` puis juste en changeant une ligne dans le fichier en-tête, passer de `double` à `float` pour comparer les performances de calcul, ou alors utiliser de plus grandes dimensions (disons dans des matrices, images volumétriques, etc.).

Pour définir un type chaîne qui est un pointeur de type `char`, on utilise la syntaxe :

```
typedef char *Chaine;
```

On peut ainsi définir un type autoréférentiel pour une structure d'arbre binaire :

```
typedef struct noeud *PtrArbre;

typedef struct noeud {
    int valeur;
    PtrArbre gauche, droit;
} Noeud;
```

## A.4 Déclaration de fonctions

En C, on peut définir les arguments d'une fonction ou d'une procédure (une fonction qui ne renvoie rien) comme suit :

```
#include <stdio.h>

int Carre(int x)
{return x*x;}

void PrintCarre(void)
{
    int i;
    for(i=1; i <=5; i++)
    {
        printf("%d ", i*i);
    }
}
```

Dans les vieux codes en C (à prohiber<sup>3</sup>!), on déclare les arguments après la fonction :

```
int Cube(x)
int x;
{return x*x*x;}


```

Ceci explique que l'on trouve en C, des programmes avec la fonction `main` définie comme suit :

```
int main(argc,argv)
int argc;
char *argv[];
{...}
```

---

3. Donné ici juste pour référence car on croise encore sur la toile de tels codes.

## A.5 Quelques autres spécificités du langage C

Le langage C est un langage de bas niveau, proche du système d'exploitation, conçu pour donner du code compilé rapide. Il offre de nombreuses autres possibilités pour écrire des programmes efficaces comme les variables registres, la gestion de signaux, les pointeurs de fonction, les champs de bits, les unions, etc. L'instruction `goto` est aussi disponible bien que fortement déconseillée.

Le C permet de faire de l'*arithmétique sur les pointeurs* : Par exemple, `p+i` est l'adresse de `p+i*taille` où `taille` est la taille en octets d'un élément de type `p`. De même, `p2-p1` signifie l'adresse de `p2` moins l'adresse de `p1` divisée par la taille des éléments. Le petit programme ci-dessous illustre cette arithmétique des pointeurs :

```
#include <stdio.h>
#include <stdlib.h>
#define N 100
typedef struct pointmasse {int x,y; double w;} PointMasse;

void main(void){
    printf("taille element : %d\n",sizeof(PointMasse));
    PointMasse *Tab=malloc(N*sizeof(PointMasse));
    printf("adresse du tableau : %p\n",Tab);
    PointMasse *Ptr1, *Ptr2;
    Ptr1=&(Tab[10]); Ptr2=&(Tab[20]);
    printf("adresses Ptr1=%p Ptr2=%p\n",Ptr1,Ptr2);
    int diff= Ptr2-Ptr1;
    printf("difference : %d\n",diff);
    free(Tab);
}
```

## A.6 Les entrées et sorties illustrées par le tri à bulles

Pour illustrer très rapidement le principe des *entrées et sorties* en C, on donne ci-dessous le code du tri à bulles en temps quadratique :

```
#include <stdio.h>

int main()
{
    int array[100], n, c, d, tmp;

    printf("Entrez n<100:\n");
    scanf("%d", &n);

    printf("entrez %d entiers\n", n);

    for (c = 0; c < n; c++)
        {scanf("%d", &array[c]);}

    for (c = 0 ; c < ( n - 1 ); c++)
    {
        for (d = 0 ; d < n - c - 1; d++)
        {
            if (array[d] > array[d+1])
            {
                tmp          = array[d];
                array[d]    = array[d+1];
                array[d+1]  = tmp;
            }
        }
    }
}
```

```
        array[d]    = array[d+1];
        array[d+1] = tmp;
    }
}

printf("Liste triee :\n");

for ( c = 0 ; c < n ; c++ )
    {printf("%d\n", array[c]);}

return 0;
}
```

## Annexe B

# Le langage C++ quand on connaît déjà le C

Le langage C++ [88] est un langage de programmation compilé qui fut développé au cours des années 1980 par Bjarne Stroustrup des laboratoires de recherche Bell d'AT&T . C'est un langage de programmation compilé (non interprété) orienté-objet et qui permet aisément la programmation générique. Bjarne Stroustrup voulait originellement améliorer le langage C lors de sa thèse, et avait au départ appelé cette extension : "C with classes."

Nous conseillons au lecteur de lire d'abord l'introduction au langage C, au chapitre A, avant de lire ce résumé. Ce chapitre ne constitue pas un cours de C++ mais aborde quelques notions principales qui seront suffisantes pour notre cours.

En quelques mots, les différences notables avec le C sont que les données sont encapsulées dans des classes, que les données objets peuvent être éventuellement **const**, qu'il existe un mécanisme pour la surcharge des opérateurs usuels (comme **\***, **+**, **-**, **/**) , et que les opérateurs **new** et **delete** permettent de gérer la mémoire dynamiquement (au lieu de **malloc** et **free** en C). De plus, les entrées et sorties sont traitées bien différemment par des fichiers flots.

Par convention, les classes sont définies (prototypées) dans des fichiers à en-tête avec l'extension **.h** et la description de ces classes figurent dans les fichiers correspondants avec l'extension **.cpp** ("pp" pour ++).

On utilise le compilateur **g++**<sup>1</sup> de GNU pour compiler les codes sources. Le petit programme ci-dessous illustre la syntaxe du C++ avec un exemple de lecture de données en entrée et d'affichage en sortie :

```
// programme bonjour.cpp
#include <iostream>
using namespace std;

main()
{
    cout <<"Bonjour INF442 !\n";
    cout <<"entre un entier : ";
    int x;
    cin >> x;
```

---

1. <https://gcc.gnu.org/>

```
cout <<"son carre vaut : "<<x*x<<endl;
}
```

On compile ce programme `bonjour.cpp` avec la ligne de commande :

```
g++ bonjour.cpp -o bonjour.exe
```

et on exécute le programme compilé avec :

```
bonjour.exe
```

Le résultat d’une exécution est :

```
Bonjour INF442 !
entre un entier : 10
son carre vaut : 100
```

Ce programme utilise les routines d’entrées/sorties (E/Ss) définies dans le fichier `iostream`. On écrit à la console en dirigeant les messages dans le flot `cout` avec la notation `<<`. On lit les entrées du flot `cin` avec `>>`. Le caractère `\n` peut aussi être ajouté dans le flot avec `endl` qui est équivalent à `'\n'`. Les commentaires peuvent soit être définis sur une ou plusieurs lignes grâce à la syntaxe `/* commentaire */` ou soient définies sur une seule ligne à la fois avec `// commentaire`.

## B.1 Appel de fonctions : passage par valeurs

Tout comme le C et Java, le passage des variables se fait par valeur. Toutefois, l’opérateur unaire `&` qui retourne l’adresse d’une variable peut être utilisé. Par exemple, la procédure `echange` sur le type entier en C++ s’écrit comme suit :

```
void swap(int a, int b) {int c=a; a=b;b=c;}
void swapReference(int &a, int &b) {int c=a; a=b;b=c;}

swap(a,b); // passage par valeur
cout <<a<<" "<<b<<endl; // 5 10
swapReference(a,b); //passage par reference
cout <<a<<" "<<b<<endl; //10 5
```

## B.2 Les classes et les objets

Les types fondamentaux sont `char`, `short`, `int`, `long`, `float` et `double`. Les constantes sont définies comme suit :

```
const double MonPI=3.14;
const int MonCours=442;
```

Les “pointeurs sur” et “référence” à sont comme en C manipulés avec `*` et `&`. On peut aussi définir un pointeur constant (invariant, qui ne change pas) par `*const`.

### B.2.1 Définition d'une classe

En C++, on n'utilise plus le `struct` de C, mais des classes pour définir les nouveaux types de données :

```
#include <iostream>
using namespace std;

class Boite
{
public:
    double horizontal; // dimension largeur
    double vertical; /* dimension hauteur */
};

int main( )
{
    Boite B1, B2;
    double surface = 0.0;
    B1.horizontal = 5.0; B1.vertical = 6.0;
    surface = B1.horizontal * B1.vertical;
    cout << "Surface de la boite B1 : " << surface << endl;
    return 0;
}
```

Notons qu'à la différence de Java, on met un ";" après la définition d'une classe.

Lors de la création des objets, une méthode spécifique de la classe est appelée : le constructeur. De même, lors de la destruction des objets, la méthode destructeur est appelée. L'exemple suivant décrit la classe suivi de la construction et destruction d'un objet de cette classe :

```
#include <iostream>
using namespace std;

class Donnee
{public:
    int d;
    double * attribut;
    Donnee(){d=3;attribut=new double[d];}
    Donnee(int dd){d=dd; attribut=new double[d]; }
    ~Donnee() {delete [] attribut; cout<<"Destructeur appele"<<endl;}
};

int main()
{
    int dim=500;
    Donnee *x=new Donnee(dim);
    delete x;
    return 0;
}
```

On peut définir la classe dans un fichier à en-tête et le corps des constructeurs/destructeurs et procédures/fonctions dans le fichier `.cpp` correspondant (en Java, cela n'est pas possible).

Par exemple, on définit le fichier `maboite.h` :

```
class maboite
{
public:
    double horizontal, vertical;
```

```

        maboite(double h, double v);
        double surface();
};

```

et le fichier `maboite.cpp` comme ceci :

```

#include <iostream>
using namespace std;

#include "maboite.h"

maboite::maboite(double h, double v)
    {horizontal=h;vertical=v;}

double maboite::surface()
    {return horizontal*vertical;}

main()
{
    maboite* b=new maboite(3,4);
    cout<<"surface="<<b->surface()<<endl;
    delete b;
}

```

## B.2.2 L'héritage et les hiérarchies de classe

Tout comme en Java, le C++ est un langage orienté objet qui permet de définir des classes qui héritent d'autres classes. Le mécanisme de base pour déclarer une classe qui hérite d'une autre classe est :

```

class ClasseDerivee : public ClasseDeBase {};

```

Par exemple, on définit une classe `eleve` et une classe `delegue` qui dérive de la classe `eleve` comme suit :

```

#include <iostream>
#include <string>
using namespace std;

class eleve
{
public:
    string nom;
    int age;

    eleve(string nomE, int ageE) {
        nom=nomE;
        age=ageE;
    }
    void print() {
        cout<<nom<<" age="<<age;
    }
}

;

class delegue :
public eleve

```

```

{
public:
    int groupe ; // delegue de ce groupe

    delegue(string nom, int age, int gr) :
    eleve(nom, age)
    {
        groupe=gr;
    }

    void print()
    {
        eleve::print();
        cout<<" groupe="<<groupe<<endl;
    }
};

main()
{
    delegue* etudiant=new delegue("Frank", 23, 1);
    etudiant->print();
    delete etudiant;
}

```

### B.3 Le mot clef const dans les méthodes

Le mot clef `const` indique que l'on ne peut *pas changer* les variables de l'objet `this`

```

void Foo()
{
    counter++; // ca marche
    std::cout << "Foo" << std::endl;
}

void Foo() const
{
    counter++; // cela ne compilera pas !
    std::cout << "Foo const" << std::endl;
}

```

### B.4 Construction et destruction de tableaux

Tout comme le C (et Java), les indices commencent à partir de 0. Quelques exemples de création de tableaux :

```

int nombrePremiers [4] = { 2, 3, 5, 7 };
int baz [442] = { }; // valeurs initialisees a zero
int matrice [3][5]; // 3 lignes 5 colonnes

void procedure (int tableau[]) {...}

```

L'allocation dynamique de tableau de fait avec les mots clefs `new` et `delete`. Tout comme en C, on doit gérer l'espace mémoire soi-même en C++ :

```
int taille=231271;
int *tab;
tab=new int[taille];
// ... utilisez ce tableau puis LIBERER le !
delete [] tab;
```

## B.5 Surcharge des opérateurs

Il est pratique de redéfinir certains *opérateurs* (comme +, /, =, etc.) en les surchargeant. Par exemple, le test d'égalité == s'il n'est pas surchargé testera l'égalité physique des adresses. C'est-à-dire, `a==b` sera vrai ssi. l'adresse de `a` coïncide avec l'adresse de `b`. On désire plutôt, un test d'égalité logique : c'est-à-dire, de vérifier que les contenus membres des objets `a` et `b` sont tous égaux. Bien entendu, si deux objets ont des pointeurs qui pointent sur la même adresse mémoire, alors on a aussi l'égalité logique par définition.

Prenons, le cas d'une classe `Point` :

```
class Point{
public : double x,y;
Point(double xx, double yy){x=xx;y=yy;}
};
```

On surcharge l'opérateur == comme suit :

```
friend bool operator== (Point &P1, Point &P2)
{
    if ((P1.x == P2.x) && (P1.y == P2.y))
        return true;
    else
        return false;
}
};
```

Le mot-clef `friend` indique qu'une fonction non-membre de la classe peut accéder les champs `private` d'une classe si celle-ci est déclarée comme étant `friend` de cette classe.

```
#include <iostream>
using namespace std;

main()
{
    Point P=Point(0.2,0.5);
    Point Q=Point(0.2,0.5);

    if (P==Q) {cout<<"egalite !"<<endl;}
    else
        {cout<<"different !"<<endl;}
}
```

## B.6 La généricité en C++

Le C++ possède un mécanisme permettant la *généricité* (appelé en anglais *template*). Par exemple, la procédure `swap` est donnée dans sa version générique comme suit :

```
template <class T> void swap ( T &a, T &b )
{T c(a); a=b; b=c;}
```

Notez que la classe T doit fournir un **constructeur**  $T(Tobject)$  que l'on utilise dans `T c(a)`.

## B.7 La bibliothèque STL

La bibliothèque *Standard Template Library* (STL) procure une architecture globale pour la programmation générique. Elle fut initialement développée par Alexander Stepanov des laboratoires AT&T Bell puis des laboratoires de recherche de Hewlett-Packard. La STL fournit une abstraction en quatre composants qui sont (1) les algorithmes de base, (2) les containers, (3) les fonctions, et (4) les itérateurs. Cette bibliothèque a à son tour influencé la bibliothèque standard de C++. Écrire du code C++-STL est devenu important pour diverses raisons mais demande quelques efforts initialement pour apprendre les concepts. Nous ne mentionnerons ici que les tableaux extensibles, et recommandons le lecteur l'ouvrage de référence [79].

On utilise souvent la classe `vector` de la STL : c'est un *container* qui permet de gérer des tableaux de taille **dynamique** aisément comme l'illustre ce programme :

```
#include <vector>
size_t size = 442;
// make room for 442 integers, initialized to 0
std::vector<int> array(size);
// on peut rajouter dynamiquement des elements
for(int i=0; i<2*size; ++i){
    array[i] = i;
}
// pas besoin de delete
```

## B.8 Les entrées-sorties en C++

La gestion des entrées-sorties se fait à partir de flots dans C++. Le petit programme suivant montre comment lire un fichier CSV (*Comma Separated Values*) dans un tableau :

```
//Code de conversion : passage CSV -> array 2D
double data[150][4];
ifstream file("Iris.csv");
for(int row = 0; row < 150; ++row)
{
    string line;
    getline(file, line);
    if ( !file.good() )
        break;

    stringstream iss(line);

    for (int col = 0; col < 4; ++col)
    {
        string val;
        getline(iss, val, ',');
        if ( !iss.good() )
            break;
    }
}
```

```

        stringstream convertor(val);
        convertor >> data[row][col];
    }
}

```

## B.9 La bibliothèque Boost pour les matrices (ublas)

Dans ce cours, on utilise la bibliothèque Boost<sup>2</sup> en C++ pour manipuler les matrices (la partie ublas). D'autres bibliothèques utilisent Boost comme la bibliothèque CGAL de géométrie algorithmique.<sup>3</sup>

```

#include <boost/numeric/ublas/matrix.hpp>
#include <boost/numeric/ublas/io.hpp>

using namespace boost::numeric::ublas;

int main () {
    matrix<double> m (3, 3);
    vector<double> v (3);

    for (unsigned i = 0; i < 3; ++ i) {
        for (unsigned j = 0; j < m.size2 (); ++ j)
            m (i, j) = 3 * i + j;
        v (i) = i;
    }

    std::cout << m<<std::endl;
    std::cout << v<<std::endl;
    std::cout << "Produit matrice vecteur : "<<std::endl;
    std::cout << prod (m, v) << std::endl;
}

```

Sous Cygwin<sup>4</sup>, on peut compiler ce code en indiquant le chemin où trouver les bibliothèques comme suit :

```
g++ matrixboost.cpp -o matrixboost.exe -I c:/local/boost_1_56_0
```

L'exécution de ce code donne à la console le résultat suivant :

```

nielsen@nielsen-PC ~/c++
$ ./matrixboost.exe
[3,3]((0,1,2),(3,4,5),(6,7,8))
[3](0,1,2)
Produit matrice vecteur :
[3](5,14,23)

```

Le code de Boost est générique et se trouve dans des fichiers avec l'extension en `.hpp`.

---

2. <http://www.boost.org/>

3. <http://www.cgal.org/>

4. Plateforme à la Unix sous Windows®. Voir <https://www.cygwin.com/>

## B.10 Quelques autres spécificités du C++

Le C++-STL d'aujourd'hui n'est pas une extension du langage C avec des classes mais un langage à part entière. Le C++ fournit de nombreux avantages en permettant une flexibilité de programmation et en délivrant un code compilé rapide. Il existe de nombreuses caractéristiques du langage que nous n'avons pas mentionnées comme les opérateurs `cast`, les mécanismes d'exceptions (avec les mots clefs `try` et `catch`), le type `String` pour les chaînes de caractère, les classes amies, le polymorphisme avec les fonctions virtuelles, etc.

Le mot clef `inline` permet de définir des fonctions qui doivent être compilées par le compilateur afin d'être "rapides" :

```
inline int max(int a, int b)
{return (a > b) ? a : b;}
```

Le compilateur évite alors de mettre les arguments sur la pile d'exécution et de récupérer le résultat sur la pile, etc. Il implémente directement l'appel de ces fonctions *inline* avec un code équivalent. L'inconvénient est d'avoir un code binaire compilé plus gros.

La somme préfixe<sup>5</sup> est disponible en C++ en appelant `partial_sum` (après avoir inclus `#include <numeric>`)

---

5. [http://www.cplusplus.com/reference/numeric/partial\\_sum/](http://www.cplusplus.com/reference/numeric/partial_sum/)



## Annexe C

# Commandes shell pour manipuler les processus et les E/Ss

On décrit très brièvement quelques commandes en **shell** pour manipuler les processus. Dans les salles informatiques, le **shell** utilisé est le **bash**<sup>1</sup> pour *Bourne Again Shell*. À tout moment, on peut faire **man cmd** pour obtenir le manuel de la commande **cmd** (par exemple, **man kill**).

### C.1 Fichier de configuration initiale .bashrc

Lorsqu'on ouvre une fenêtre de type terminal, le **shell** lit le *fichier de configuration initiale* de votre fichier **.bashrc** qui se situe dans votre répertoire "home" (`~`). On visualise le contenu de ce fichier comme ceci :

```
more .bashrc
```

On peut le modifier en utilisant un éditeur de texte (comme **vi**, **emacs**, ...). Une fois le fichier modifié, on peut relire la configuration à n'importe quel instant d'une session avec :

```
source .bashrc
```

Par exemple, voici à quoi ressemble un fichier **.bashrc** :

```
if [ -f /etc/bashrc ]; then
    . /etc/bashrc
fi
# Prompt
PS1="[\h \W]\\$ "

alias rm='rm -i'
alias cp='cp -i'
alias mv='mv -i'
```

---

1. [http://fr.wikipedia.org/wiki/Bourne-Again\\_shell](http://fr.wikipedia.org/wiki/Bourne-Again_shell)

```
alias mm='/usr/local/openmpi-1.8.3/bin/mpic++ -I/usr/local/boost
-1.56.0/include/
-L/usr/local/boost-1.56.0/lib/ -lboost_mpi -lboost_serialization'

export PATH=/usr/lib/openmpi/1.4-gcc/bin:${PATH}
export PATH=/usr/local/boost-1.39.0/include/boost-1_39:${PATH}

LS_COLORS='di=0;35' ; export LS_COLORS
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:/usr/local/openmpi-1.8.3/lib
:/usr/local
/boost-1.56.0/lib/
```

## C.2 Commandes Unix pipelinées et la redirection des entrées/sorties

Pour connaître son identité sous Unix, on tape `id` :

```
[france ~]$ id
uid=11234(frank.nielsen) gid=11000(profs) groups=11000(profs)
```

On peut lister, renommer et effacer des fichiers avec les commandes respectives : `ls`, `mv` (*move*) et `rm` (*remove*, option `-i` par défaut). On visualise et on concatène des fichiers avec les commandes `more` et `cat`.

Les Entrées/Sorties (E/Ss) peuvent être manipulées avec le *pipe* `|` comme suit :

```
[france ~]$ cat fichier1 fichier2 |wc
      26      68     591
```

Ici, `wc` est le programme qui compte les lignes, mots, et octets. On accède au manuel des utilitaires Unix souvent avec `programme -h` (pour *help*!) :

```
[france ~]$ wc --h
Usage: wc [OPTION]... [FILE]...
Print newline, word, and byte counts for each FILE, and a total line if
more than one FILE is specified.  With no FILE, or when FILE is -,
read standard input.
  -c, --bytes          print the byte counts
  -m, --chars          print the character counts
  -l, --lines          print the newline counts
  -L, --max-line-length print the length of the longest line
  -w, --words          print the word counts
  --help              display this help and exit
  --version            output version information and exit
```

Sous Unix, on peut rediriger les entrées et les sorties. Par exemple, un programme qui lit les entrées tapées dans la fenêtre du terminal pourra lire ces entrées à partir d'un fichier grâce à la commande `programme <input`. De même, on pourra rediriger les sorties de la console dans

un fichier avec `programme >output`. On peut aussi rediriger les messages d'erreur en faisant `programme 2>error.log`. On utilise ainsi souvent cette syntaxe pour la redirection des E/Ss :

```
programme <input >output 2>error.log
```

### C.3 Manipuler les tâches (*jobs*)

On liste les numéros des processus courants (leur `pid`) avec la commande `ps` (on peut passer des arguments en option comme `ps -a`, pour *all*). Pour suspendre un processus, on appuie en même temps sur les touches Contrôle (`Ctrl`) et `'Z'`. Par exemple, faisons un `Ctrl-Z` sur le processus qui dort 10000 secondes :

```
sleep 10000
Ctrl-Z
```

On place une tâche suspendue en processus de fond en appelant `bg` (*background*, une des commandes *built-in* du shell) :

```
bg
```

De même, on place une tâche suspendue ou en fond sur le devant en faisant `fg` (pour *foreground*) :

```
[france ~]$ sleep 2015 &
[1] 18767
[france ~]$ jobs
[1]+  Running                  sleep 2015 &
[france ~]$ fg %1
sleep 2015
```

On peut aussi tuer des processus (par exemple, en cas de situation de blocage) ou envoyer des signaux Unix aux `pids` avec la commande `kill` :

```
[france ~]$ sleep 5000 &
[1] 13728
[france ~]$ kill %1
[1]+  Terminated              sleep 5000
```

On peut visualiser son historique de commandes d'une session avec `history` et rappeler la dernière commande commençant par disons `cat` en faisant :

```
!cat
```

La dernière commande peut aussi être rappelée avec `!!` et on peut nettoyer l'écran de la console en faisant `clean`.



## Annexe D

# Liste des ordinateurs en salles machines

Voici ci-dessous la liste des 169 machines organisée par salle.

**Salle 30 : 19 machines (pays)** allemagne, angleterre, autrice, belgique, espagne, finlande, france, groenland, hollande, hongrie, irlande, islande, lituanie, malte, monaco, pologne, portugal, roumanie, suedes.

**Salle 31 : 25 machines (oiseaux)**. albatros, autruche.polytechnique, bengali, coucou, dindon, epervier, faisan, gelinotte, hibou, harpie, jabiru, kamiche, linotte, loriol, mouette, nandou, ombrette, perdrix, quetzal, quiscale, rouloul, sitelle, traquet.polytechnique.fr, urabu, verdier.

**Salle 32 : 25 machines (orchidées)**. aerides, barlia, calanthe, diuris, encyclia, epipactis, gennaria, habenaria, isotria, ipsea, liparis, lycaste, malaxis, neotinea, oncidium, ophrys, orchis, pleione, pogonia, serapias, telipogon, vanda, vanilla, xylobium, zeuxine.

**Salle 33 : 25 machines (départements)**. ain, allier, ardennes, carmor, charente, cher, creuse, dordogne, doubs, essonne, finistere, gironde, indre, jura, landes, loire, manche, marne, mayenne, morbihan, moselle, saone, somme, vendee, vosges.

**Salle 34 : 25 machines (poissons)**. ablette, anchois, anguille, barbeau, barbue, baudroie, brochet, carrelet, gardon, gymnote, labre, lieu, lotte, mulet, murene, piranha, raie, requin, rouget, roussette, saumon, silure, sole, thon, truite.

**Salle 35 : 25 machines (os)**. acromion, apophyse, astragale, atlas, axis, coccyx, cote, cubitus, cuboide, femur, frontal, humerus, malleole, metacarpe, parietal, perone, phalange, radius, rotule, sacrum, sternum, tarse, temporal, tibia, xiphoide.

**Salle 36 : 25 machines (voitures)**. bentley, bugatti, cadillac, chrysler, corvette, ferrari, fiat, ford, jaguar, lada, maserati, mazda, nissan, niva, peugeot, pontiac, porsche, renault, rolls, rover, royce, simca, skoda, venturi, volvo.



# Bibliographie

- [1] Dimitris Achlioptas. Database-friendly random projections : Johnson-Lindenstrauss with binary coins. *Journal of computer and System Sciences*, 66(4) :671–687, 2003.
- [2] Gene M. Amdahl. Validity of the single processor approach to achieving large scale computing capabilities. In *Proceedings of Spring Joint Computer Conference*, AFIPS '67 (Spring), pages 483–485, New York, NY, USA, 1967. ACM.
- [3] David Arthur and Sergei Vassilvitskii.  $k$ -means++ : The advantages of careful seeding. In *Proceedings of the eighteenth annual ACM-SIAM symposium on Discrete algorithms*, pages 1027–1035. Society for Industrial and Applied Mathematics, 2007.
- [4] Sunil Arya, David M Mount, Nathan S Netanyahu, Ruth Silverman, and Angela Y Wu. An optimal algorithm for approximate nearest neighbor searching fixed dimensions. *Journal of the ACM*, 45(6) :891–923, 1998.
- [5] Pranjali Awasthi, Avrim Blum, and Or Sheffet. Stability yields a PTAS for  $k$ -median and  $k$ -means clustering. In *FOCS*, pages 309–318. IEEE Computer Society, 2010.
- [6] Pranjali Awasthi, Avrim Blum, and Or Sheffet. Center-based clustering under perturbation stability. *Information Processing Letters*, 112(1 ?2) :49 – 54, 2012.
- [7] László Babai and Eugene M. Luks. Canonical labeling of graphs. In *Proceedings of the Fifteenth Annual ACM Symposium on Theory of Computing*, STOC '83, pages 171–183, New York, NY, USA, 1983. ACM.
- [8] Artem Babenko and Victor S. Lempitsky. Improving bilayer product quantization for billion-scale approximate nearest neighbors in high dimensions. *CoRR*, abs/1404.1831, 2014.
- [9] Mihai Badoiu and Kenneth L. Clarkson. Optimal core-sets for balls. *Comput. Geom.*, 40(1) :14–22, 2008.
- [10] Bahman Bahmani, Ravi Kumar, and Sergei Vassilvitskii. Densest subgraph in streaming and MapReduce. *Proc. VLDB Endow.*, 5(5) :454–465, January 2012.
- [11] Bahman Bahmani, Benjamin Moseley, Andrea Vattani, Ravi Kumar, and Sergei Vassilvitskii. Scalable  $k$ -means+. *Proceedings of the VLDB Endowment*, 5(7), 2012.
- [12] Maria-Florina Balcan, Steven Ehrlich, and Yingyu Liang. Distributed  $k$ -means and  $k$ -median clustering on general topologies. In *Advances in Neural Information Processing Systems*, pages 1995–2003, 2013.
- [13] Arindam Banerjee, Srujana Merugu, Inderjit S Dhillon, and Joydeep Ghosh. Clustering with Bregman divergences. *The Journal of Machine Learning Research*, 6 :1705–1749, 2005.

- [14] J  r  my Barbay and Gonzalo Navarro. On compressing permutations and adaptive sorting. *Theoretical Computer Science*, 513(0) :109 – 123, 2013.
- [15] Kevin Beyer, Jonathan Goldstein, Raghu Ramakrishnan, and Uri Shaft. When is “nearest neighbor” meaningful? In *Database Theory (ICDT)*, pages 217–235. Springer, 1999.
- [16] J. D. Boissonnat and M. Yvinec. *G  om  trie Algorithmique*. Ediscience International, 1995.
- [17] Olivier Bournez. *Fondements de l’Informatique : logique, mod  les, calculs (INF412)*.   cole Polytechnique, 2014.
- [18] David Bremner, Erik Demaine, Jeff Erickson, John Iacono, Stefan Langerman, Pat Morin, and Godfried Toussaint. Output-sensitive algorithms for computing nearest-neighbour decision boundaries. *Discrete & Computational Geometry*, 33(4) :593–604, 2005.
- [19] Lynn Elliot Cannon. *A Cellular Computer to Implement the Kalman Filter Algorithm*. PhD thesis, Montana State University, Bozeman, MT, USA, 1969. AAI7010025.
- [20] Gunnar Carlsson and Facundo M  moli. Characterization, stability and convergence of hierarchical clustering methods. *The Journal of Machine Learning Research (JMLR)*, 11 :1425–1470, 2010.
- [21] H. Casanova, A. Legrand, and Y. Robert. *Parallel algorithms*. Chapman & Hall/CRC numerical analysis and scientific computing. CRC Press, 2009.
- [22] Moses Charikar. Greedy approximation algorithms for finding dense components in a graph. In *Proceedings of the Third International Workshop on Approximation Algorithms for Combinatorial Optimization, APPROX ’00*, pages 84–95, London, UK, UK, 2000. Springer-Verlag.
- [23] Luigi P Cordella, Pasquale Foggia, Carlo Sansone, and Mario Vento. Performance evaluation of the VF graph matching algorithm. In *Image Analysis and Processing, 1999. Proceedings. International Conference on*, pages 1172–1177. IEEE, 1999.
- [24] Thomas M. Cover and Peter E. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1) :21–27, 1967.
- [25] Robert Cypher and C.Greg Plaxton. Deterministic sorting in nearly logarithmic time on the hypercube and related computers. *Journal of Computer and System Sciences*, 47(3) :501 – 548, 1993.
- [26] Daniel Defays. An efficient algorithm for a complete link method. *The Computer Journal*, 20(4) :364–366, 1977.
- [27] Inderjit S. Dhillon, Yuqiang Guan, and Brian Kulis. Kernel  $k$ -means : Spectral clustering and normalized cuts. In *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD ’04*, pages 551–556, New York, NY, USA, 2004. ACM.
- [28] Inderjit S. Dhillon and Dharmendra S. Modha. A data-clustering algorithm on distributed memory multiprocessors. In *Revised Papers from Large-Scale Parallel Data Mining, Workshop on Large-Scale Parallel KDD Systems, SIGKDD*, pages 245–260, London, UK, UK, 2000. Springer-Verlag.
- [29]   ric Goubault et al. *Programmation d’Applications Concurrentes et Distribu  es (INF431)*. Ecole Polytechnique, 2015.
- [30] Wei Fan and Albert Bifet. Mining big data : current status, and forecast to the future. *ACM SIGKDD Explorations Newsletter*, 14(2) :1–5, 2013.

- [31] Dan Feldman, Melanie Schmidt, and Christian Sohler. Turning big data into tiny data : Constant-size coresets for  $k$ -means, PCA and projective clustering. In *Symposium on Discrete Algorithms (SODA)*, pages 1434–1453, 2013.
- [32] Dan Feldman, Melanie Schmidt, and Christian Sohler. Turning big data into tiny data : Constant-size coresets for  $k$ -means, PCA and projective clustering. In *Proceedings of the Twenty-Fourth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1434–1453. SIAM, 2013.
- [33] P. Foggia, C. Sansone, and M. Vento. A performance comparison of five algorithms for graph isomorphism. *Proc. of the 3rd IAPR TC-15 Workshop on Graph-based Representations in Pattern Recognition*, pages 188–199, 2001.
- [34] Geoffrey C Fox, Steve W Otto, and Anthony JG Hey. Matrix algorithms on a hypercube I : Matrix multiplication. *Parallel computing*, 4(1) :17–31, 1987.
- [35] Brendan J. Frey and Delbert Dueck. Clustering by passing messages between data points. *Science*, 315 :972–976, 2007.
- [36] François Le Gall. Powers of tensors and fast matrix multiplication. *arXiv preprint arXiv :1401.7714*, 2014.
- [37] Vincent Garcia, Eric Debreuve, Frank Nielsen, and Michel Barlaud.  $k$ -nearest neighbor search : Fast GPU-based implementations and application to high-dimensional feature matching. In *Proceedings of the International Conference on Image Processing (ICIP)*, pages 3757–3760, 2010.
- [38] Gene G. Golub and Charles F. Van Loan. *Matrix Computations*. The Johns Hopkins University Press, 1996.
- [39] John C Gower and GJS Ross. Minimum spanning trees and single linkage cluster analysis. *Applied statistics*, pages 54–64, 1969.
- [40] W. Gropp, T. Hoefler, R. Thakur, and E. Lusk. *Using Advanced MPI : Modern Features of the Message-Passing Interface*. MIT Press, Nov. 2014.
- [41] William D. Gropp, Steven Huss-Lederman, Andrew Lumsdaine, and Inc netLibrary. *MPI : the complete reference. Vol. 2. , The MPI-2 extensions*. Scientific and engineering computation series. Cambridge, Mass. MIT Press, 1998.
- [42] John L. Gustafson. Reevaluating Amdahl’s law. *Communications of the ACM*, 31(5) :532–533, 1988.
- [43] Yijie Han. Deterministic sorting in  $O(n \log \log n)$  time and linear space. *Journal of Algorithms*, 50(1) :96–105, 2004.
- [44] Sariel Har-Peled and Akash Kushal. Smaller coresets for  $k$ -median and  $k$ -means clustering. *Discrete & Computational Geometry*, 37(1) :3–19, 2007.
- [45] Sariel Har-Peled and Bardia Sadri. How fast is the  $k$ -means method ? *Algorithmica*, 41(3) :185–202, 2005.
- [46] T Hastie, R Tibshirani, and R Friedman. *Elements of Statistical Learning Theory*. Springer-Verlag, 2002.
- [47] Mark D Hill and Michael R Marty. Amdahl’s law in the multicore era. *Computer*, 1(7) :33–38, 2008.

- [48] Torsten Hoefer, Andrew Lumsdaine, and Jack Dongarra. Towards efficient MapReduce using MPI. In Matti Ropo, Jan Westerholm, and Jack Dongarra, editors, *Recent Advances in Parallel Virtual Machine and Message Passing Interface*, volume 5759 of *Lecture Notes in Computer Science*, pages 240–249. Springer Berlin Heidelberg, 2009.
- [49] Zhexue Huang. Extensions to the  $k$ -means algorithm for clustering large data sets with categorical values. *Data mining and knowledge discovery*, 2(3) :283–304, 1998.
- [50] Kai Hwang. *Advanced Computer Architecture : Parallelism, Scalability, Programmability*. McGraw-Hill Higher Education, 1st edition, 1992.
- [51] William Johnson and Joram Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. In *Conference in modern analysis and probability*, volume 26 of *Contemporary Mathematics*, pages 189–206. American Mathematical Society, 1984.
- [52] Daniel M. Kane and Jelani Nelson. Sparser Johnson-Lindenstrauss transforms. *Journal of the ACM (JACM)*, 61(1) :4, 2014.
- [53] Sharanjit Kaur, Vasudha Bhatnagar, and Sharma Chakravarthy. Stream clustering algorithms : A primer. In Aboul Ella Hassanien, Ahmad Taher Azar, Vaclav Snasael, Janusz Kacprzyk, and Jemal H. Abawajy, editors, *Big Data in Complex Systems*, volume 9 of *Studies in Big Data*, pages 105–145. Springer International Publishing, 2015.
- [54] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall Professional Technical Reference, 2nd edition, 1988.
- [55] John Kleinberg. An impossibility theorem for clustering. In S. Becker, S. Thrun, and K. Obermayer, editors, *Advances in Neural Information Processing Systems*, pages 446–453. MIT Press, 2002.
- [56] Vipin Kumar, Ananth Grama, Anshul Gupta, and George Karypis. *Introduction to Parallel Computing : Design and Analysis of Algorithms*. Benjamin-Cummings Publishing Co., Inc., Redwood City, CA, USA, 1994.
- [57] Godfrey N Lance and William Thomas Williams. A general theory of classificatory sorting strategies. *The computer journal*, 10(3) :271–277, 1967.
- [58] Kasper Green Larsen and Jelani Nelson. The Johnson-Lindenstrauss lemma is optimal for linear dimensionality reduction. *arXiv preprint 1411.2404*, 2014.
- [59] Der-Tsai Lee. On  $k$ -nearest neighbor voronoi diagrams in the plane. *Computers, IEEE Transactions on*, C-31(6) :478–487, June 1982.
- [60] Arnaud Legrand and Yves Robert. *Algorithmique parallèle : cours et exercices corrigés*. Sciences sup. Dunod, Paris, 2003.
- [61] Calvin Lin and Lawrence Snyder. A matrix product algorithm and its comparative performance on hypercubes. In *Scalable High Performance Computing Conference, 1992. SHPCC-92, Proceedings.*, pages 190–194. IEEE, 1992.
- [62] Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, IT-28(2) :129–137, March 1982. paru initialement en rapport de recherche en 1957.
- [63] Ulrike Luxburg. A tutorial on spectral clustering. *Statistics and Computing*, 17 :395–416, December 2007.
- [64] James B. MacQueen. Some methods of classification and analysis of multivariate observations. In L. M. Le Cam and J. Neyman, editors, *Proceedings of the Fifth Berkeley Symposium on*

- Mathematical Statistics and Probability*. University of California Press, Berkeley, CA, USA, 1967.
- [65] F. Magoulès and F.X. Roux. *Calcul scientifique parallèle : Cours, exemples avec OpenMP et MPI , exercices corrigés*. Mathématiques appliquées pour le Master/SMAI. Dunod, 2013.
  - [66] Jiri Matousek. On approximate geometric  $k$ -clustering. *Discrete & Computational Geometry*, 24(1) :61–84, 2000.
  - [67] M. Miller and J. Siran. Moore graphs and beyond : A survey of the degree/diameter problem. *Electronic Journal of Combinatorics*, 1(DS14), 2005.
  - [68] Byron J. T. Morgan and Andrew P. G. Ray. Non-uniqueness and inversions in cluster analysis. *Applied statistics*, pages 117–134, 1995.
  - [69] Fionn Murtagh. A survey of recent advances in hierarchical clustering algorithms. *The Computer Journal*, 26(4) :354–359, 1983.
  - [70] Fionn Murtagh and Pierre Legendre. Ward’s hierarchical agglomerative clustering method : Which algorithms implement Ward’s criterion? *J. Classification*, 31(3) :274–295, 2014.
  - [71] Mark EJ Newman. Modularity and community structure in networks. *Proceedings of the National Academy of Sciences*, 103(23) :8577–8582, 2006.
  - [72] Frank Nielsen. *A Concise and Practical Introduction to Programming Algorithms in Java*. Undergraduate Topics in Computer Science (UTiCS). Springer Verlag, 2009. <http://www.springer.com/computer/programming/book/978-1-84882-338-9>.
  - [73] Frank Nielsen. Generalized Bhattacharyya and Chernoff upper bounds on Bayes error using quasi-arithmetic means. *Pattern Recognition Letters*, pages 25–34, 2014.
  - [74] Frank Nielsen. *Introduction to HPC with MPI for Data Science*. Undergraduate Topics in Computer Science. Springer International Publishing, 2016.
  - [75] Frank Nielsen and Richard Nock. Optimal interval clustering : Application to Bregman clustering and statistical mixture learning. *IEEE Signal Processing Letters*, 21(10) :1289–1292, 2014.
  - [76] Clark F. Olson. Parallel algorithms for hierarchical clustering. *Parallel Computing*, 21(8) :1313 – 1325, 1995.
  - [77] Peter S. Pacheco. *Parallel Programming with MPI*. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1996.
  - [78] Paolo Piro, Richard Nock, Frank Nielsen, and Michel Barlaud. Leveraging  $k$ -NN for generic classification boosting. *Neurocomputing*, 80 :3–9, 2012.
  - [79] P.J. Plauger. *The C++ Standard Template Library*. Prentice Hall, 2001.
  - [80] Steven J Plimpton and Karen D Devine. Mapreduce in MPI for large-scale graph algorithms. *Parallel Computing*, 37(9) :610–632, 2011.
  - [81] Arnon Rotem-Gal-Oz, James Gosling, and L. Peter Deutsch. Fallacies of distributed computing explained, September 2006.
  - [82] Peter Sanders and Jesper Larsson Träff. Parallel prefix (scan) algorithms for MPI. In *Recent Advances in Parallel Virtual Machine and Message Passing Interface*, pages 49–57. Springer, 2006.
  - [83] D Sculley. Web-scale  $k$ -means clustering. In *Proceedings of the 19th international conference on World wide web*, pages 1177–1178. ACM, 2010.

- [84] Robin Sibson. SLINK : An optimally efficient algorithm for the single-link cluster method. *The Computer Journal*, 16(1) :30–34, January 1973.
- [85] Noam Slonim, Ehud Aharoni, and Koby Crammer. Hartigan’s  $k$ -means versus lloyd’s  $k$ -means - is it time for a change? In Francesca Rossi, editor, *IJCAI. IJCAI/AAAI*, 2013.
- [86] Marc Snir, Steve Otto, Steven Huss-Lederman, David Walker, and Jack Dongarra. *MPI-The Complete Reference, Volume 1 : The MPI Core*. MIT Press, Cambridge, MA, USA, 2nd. (revised) edition, 1998.
- [87] Hugo Steinhaus. Sur la division des corps matériels en parties. *Bull. Acad. Polon. Sci. Cl. III.* 4, pages 801–804, 1956.
- [88] Bjarne Stroustrup. *The C++ Programming Language*. Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, 3rd edition, 2000.
- [89] Matus Telgarsky and Sanjoy Dasgupta. Agglomerative Bregman clustering. In *ICML. icml.cc / Omnipress*, 2012.
- [90] Matus Telgarsky and Andrea Vattani. Hartigan’s method :  $k$ -means clustering without Voronoi. In *International Conference on Artificial Intelligence and Statistics*, pages 820–827, 2010.
- [91] Ivor W Tsang, Andras Kocsor, and James T Kwok. Simpler core vector machines with enclosing balls. In *Proceedings of the 24th international conference on Machine learning*, pages 911–918. ACM, 2007.
- [92] J. R. Ullmann. An algorithm for subgraph isomorphism. *J. ACM*, 23(1) :31–42, January 1976.
- [93] Jaideep Vaidya and Chris Clifton. Privacy-preserving  $k$ -means clustering over vertically partitioned data. In *Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 206–215. ACM, 2003.
- [94] Andrea Vattani.  $k$ -means requires exponentially many iterations even in the plane. *Discrete & Computational Geometry*, 45(4) :596–616, 2011.
- [95] Joe H. Ward. Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, 58(301) :236–244, 1963.
- [96] Dong Hyuk Woo and Hsien-Hsin S Lee. Extending Amdahl’s law for energy-efficient computing in the many-core era. *Computer*, 1(12) :24–31, 2008.
- [97] Reza Zadeh and Shai Ben-David. A uniqueness theorem for clustering. In Jeff Bilmes and Andrew Y. Ng, editors, *Uncertainty in Artificial Intelligence (UAI)*, pages 639–646. Association for Uncertainty in Artificial Intelligence (AUAI) Press, 2009.





© 2015 Frank Nielsen. Tous droits réservés.

5793b870

25 mars 2016